INTEGRATED VISUAL NOTES · JAVA · THIRD SEMESTER

Java Concepts Integrated Visual Notes

Streams, files, serialization, threads, JDBC, SQLite, Swing, Maven and Spring — taught as one connected system.

Most Java material teaches each topic in isolation. This book does the opposite. It follows **one real application** — **BasinWatch**, an offline floodplain operations console — from the moment a sensor observation enters a piped stream to the moment a restored session reappears on screen. Every technology is explained on its own terms first, then placed in the architecture that gives it a purpose.

Built for examination preparation, for coding reference, and for the viva in which you must explain how your application actually works.

Java I/O streams · File handling · Serialization · Piped streams · Threads & monitors · JDBC & SQL · Transactions · SQLite · Swing & events · Maven · XML & XSD · Annotations · JavaBeans · Spring JdbcTemplate

21 chapters · 6 parts · 1 connected system · BasinWatch case study

29 original diagrams

Table of Contents

One book, one chapter sequence, one glossary, one revision section.

PART I — UNDERSTANDING THE SYSTEM

1 The Complete Java System: From Data Input to a Working Application 10
- The problem BasinWatch solves 10
- The verified runtime architecture 10
- The Java Concepts Connection Map 10
- How to read this book 10

2 Java Application Structure and Core Object Concepts 19
- Classes, objects, state and behaviour 19
- Interfaces and polymorphism 19
- Inheritance where it is justified 19
- Enums, records and validation 19
- POJO versus JavaBean 19
- How BasinWatch divides responsibility 19

3 How Java Components Communicate 28
- Six distinct communication mechanisms 28
- Method calls and shared objects 28
- Streams, events, queues and threads 28
- Database operations as communication 28
- Choosing the right mechanism 28

PART II — JAVA I/O AND PERSISTENT FILES

4 What Is a Stream? 36
- Source, destination and direction 36
- Byte streams versus character streams 36
- The four I/O hierarchies 36
- The decorator idea 36

5 Reading and Writing Files 43
- File streams and character streams 43
- Constructors and methods 43
- Complete read and write workflows 43
- Resource management and exceptions 43

6 Buffering, Data Streams and Stream Wrappers 51
- Why buffering exists 51
- Layering wrappers 51
- Typed binary records 51
- Reading a typed record back 51

7 In-Memory and Piped Streams 58
- Byte-array streams 58
- Piped streams and threads 58
- The producer–consumer model 58

Where BasinWatch uses each 58

8 Serialization, Deserialization and Session Persistence 65

Object serialization and deserialization 65

serialVersionUID and transient 65

Validation, versioning and safety 65

How BasinWatch restores a session 65

9 How BasinWatch Uses Java I/O 73

The operational log 73

The typed sensor archive 73

Session saves and the save catalogue 73

Reports and file metadata 73

PART III — DATABASES AND JDBC

10 Why Databases and JDBC Exist 81

Memory, files and structured storage 81

Tables, rows and SQL 81

Why an application uses more than one store 81

Where JDBC fits 81

11 JDBC Architecture and Drivers 86

The JDBC architecture 86

The four driver types 86

Connections and the driver's job 86

BasinWatch's driver 86

12 SQL Execution Through JDBC 91

Connection, Statement and PreparedStatement 91

ResultSet and row mapping 91

CRUD operations 91

The complete execution lifecycle 91

SQLException and resource cleanup 91

13 Database Transactions and Reliable Persistence 99

Transaction boundaries 99

commit and rollback 99

Consistency between related rows 99

BasinWatch's mission transaction 99

14 How BasinWatch Uses JDBC and SQLite 105

The database architecture 105

The schema 105

Repositories and SQL 105

Tracing a history record 105

Why the database is optional 105

PART IV — SWING AND THE USER INTERFACE

15 Swing Fundamentals and Event-Driven Programming 114

Containers and components 114

Layout managers 114

Listeners and events 114
The lecture-note login application 114
Custom painting 114
16 Connecting Application Logic, JDBC and Swing 123
From a click to a command 123
Futures and the event-dispatch thread 123
Showing database results safely 123
Background work and repainting 123

PART V — CONFIGURATION AND RELATED TECHNIQUES

17 Maven, pom.xml and Project Dependencies 130
What a build tool does 130
The pom.xml structure 130
The build lifecycle 130
Packaging a runnable jar 130
How the JDBC driver reaches the classpath 130
18 XML, Schemas, Namespaces, Annotations and JavaBeans 136
XML as a document tree 136
Namespaces 136
XML Schema 136
Annotations as metadata 136
POJO, JavaBean and framework wiring 136
19 Raw JDBC versus Spring JdbcTemplate 143
The repetitive JDBC pattern 143
JdbcTemplate and RowMapper 143
DriverManagerDataSource and the DataSource idea 143
Side-by-side comparison with BasinWatch 143

PART VI — PUTTING EVERYTHING TOGETHER

20 End-to-End BasinWatch Architecture Walkthrough 149
Scenario A — an observation, fully traced 149
Scenario B — a dispatch and its transaction 149
Scenario C — save, exit and restore 149
Scenario D — reading history into the table 149
The dependency graph 149
21 Final Integration Exercise 157
The eight questions 157
Guided walkthrough of the code 157
Where each syllabus concept lives 157
Design alternatives 157

REVISION, GLOSSARY AND REFERENCE

Revision Comparison Tables and Chapter Summaries 164
Twelve essential comparison tables 164
One-page chapter summaries 164

Viva Viva and Theory Questions 172
Definitions and distinctions 172
Mechanisms and code 172
Integration and architecture questions 172
Exercises Programming and Tracing Exercises 178
Output prediction 178
Debugging and repair 178
Tracing and architecture interpretation 178
Glossary High-Yield Revision, Glossary and Reference 187
Key definitions 187
Essential classes and method signatures 187
Common mistakes 187
Glossary 187
Source coverage matrix 187

BEFORE YOU BEGIN

How to use this book

> A short guide to the structure, the labels, and the fastest route through the material for a specific purpose.

1 The book has one spine, not two

This is a single volume. It is not an I/O book and a database book sharing a cover. Part I establishes the architecture of one application. Parts II to V teach the technologies that make each part of that architecture work. Part VI returns to the whole system and traces real data through it. The revision section at the end is a single coordinated set of comparison tables, questions and exercises covering the entire scope.

2 What every chapter assumes

Chapters are written for a third-semester Computer Science student who can already declare a class, write a loop and call a method. No prior exposure to streams, JDBC or Swing is assumed. Where a chapter depends on an earlier one, a **CONNECTION TO ANOTHER CONCEPT** box says so explicitly and tells you where to look.

3 The eight labels you will see

Every explanatory box in this book carries one of a small set of labels. The label is a promise about the kind of claim being made, and the difference between them matters.

Table 0.1 — *The explanatory labels used throughout the book.*

Label	What it means	Trust level
FROM THE LECTURE NOTES	Terminology, definitions and examples that come from the source lecture material.	Source content
CONCEPT EXPLAINED FURTHER	Additional explanation added by this book to make the concept complete or precise.	Supplementary
BASINWATCH IN PRACTICE	A claim verified against the actual source code of the BasinWatch repository.	Verified in code
RELATED TECHNIQUE — NOT USED IN BASINWATCH	A real technique from the syllabus or the wider ecosystem that this particular project does not implement.	Taught, not used here
CONNECTION TO ANOTHER CONCEPT	An explicit link between this topic and another topic in the book.	Cross-reference
IMPORTANT DISTINCTION	Two things that are often confused, stated side by side.	Clarification

Label	What it means	Trust level
COMMON MISTAKE	An error students and programmers actually make, and how to avoid it.	Warning
TECHNICAL CLARIFICATION	Where a lecture-note statement is dated, incomplete or questionable, with the modern position stated fairly.	Correction

Two further labels appear inside code walkthroughs: **EXAM TIP** marks something that is frequently asked, and **VISUAL WALKTHROUGH** introduces a diagram reading order.

4 Where the code comes from

Java listings fall into three clearly separated groups, and each listing says which group it belongs to:

- **Verified BasinWatch code** — copied from the repository at `github.com/Rishu07368/basinwatch` and quoted as it actually appears, sometimes trimmed to the relevant lines. Captions say "*from BasinWatch*" and name the file.
- **Lecture-note style examples** — small programs in the shape the notes use, including the Swing login application. Captions say "*lecture-note style*".
- **Illustrative examples** — short programs written for this book to demonstrate a syllabus concept the project does not itself exercise. Captions say "*illustrative*".

> **COMMON MISTAKE**
>
> Nothing in this book is presented as verified project behaviour unless the source file was read. Where a plausible-sounding claim could not be confirmed — for example, that BasinWatch uses Spring, or that it stores passwords — the book says so plainly rather than inventing it. If you are reading this to prepare for a viva, that discipline is itself worth copying.

5 Four routes through the book

Table 0.2 — *Four reading routes.*

If your goal is…	Read	Then
Pass an I/O and file-handling examination	Chapters 4, 5, 6, 7, 8, then 9	Revision comparison tables and the I/O viva questions
Pass a JDBC and database examination	Chapters 10, 11, 12, 13, then 14	The JDBC comparison tables, transaction questions and SQL exercises
Explain your project in a viva	Chapter 1, then 20 and 21	Chapter 2 for object concepts, Chapter 16 for the UI path
Understand how it all fits together	Read straight through, Parts I to VI	Finish with Chapter 21 and the high-yield revision section

6 The BasinWatch repository

BasinWatch is an offline Java desktop simulation that monitors a fictional flood basin, processes incoming observations, coordinates limited response resources, keeps operational history, and presents everything through a Swing interface. It is a *planning simulation*: it does not forecast real floods, control real equipment, or connect to real services. It is a deliberately bounded teaching artefact that nevertheless uses each technology for a genuine reason.

The project is built with Java 17 and Maven, uses only standard `java.io`, `java.util.concurrent`, Swing and raw JDBC, and stores its optional history in a local SQLite file. That last sentence is worth remembering, because it determines which chapters can say "*used in BasinWatch*" and which must say "*related technique*".

BASINWATCH IN PRACTICE

The repository files referenced throughout are real: `src/main/java/com/basinwatch/{app,domain,engine,io,db,learning}`. When a chapter quotes a method, you can open the file and find it. That is the standard this book holds itself to.

7 What this book deliberately does not claim

- It does not claim that BasinWatch uses Spring, JPA, Hibernate, JavaFX, a web framework, or a remote database server. It uses none of them.
- It does not claim that the lecture notes contained every sentence here. Lecture content and supplementary explanation are labelled differently, on purpose.
- It does not present illustrative code as tested project code.
- It does not treat thread priority, serialization, or SQL string building as safe by default; each is discussed with its real limitations.

START HERE

- Read Chapter 1 before anything else. It gives you the mental model every later chapter refers back to.
- Keep Diagram 1 (the architecture) and Diagram 2 (the connection map) nearby.
- When a chapter says **CONNECTION TO ANOTHER CONCEPT**, follow the link — that is where the integrated understanding lives.

PART I

Understanding the System

Before studying a single API, you need to see the machine as a whole. This part introduces BasinWatch, the verified architecture of its runtime, the object concepts that hold it together, and the six different ways its components talk to one another. Everything later in the book hangs on this frame.

1 The Complete Java System: From Data Input to a Working Application

2 Java Application Structure and Core Object Concepts

3 How Java Components Communicate

CHAPTER 1

The Complete Java System: From Data Input to a Working Application

> One application, many technologies. This chapter builds the mental model that every later chapter refers back to.

1.1 The question this book answers

Here is the question. It is short, and it is the reason this book exists rather than a stack of separate revision sheets:

THE CENTRAL QUESTION

How do separate Java technologies work together to turn incoming data into a usable application, store information persistently, and display useful results to a user?

A student who has just finished a semester of Java can usually answer each part of that question alone. Streams read and write data. JDBC talks to databases. Swing draws windows. Maven builds projects. Each answer is correct and each is nearly useless on its own, because in a real program these things are not separate. A button press causes a query; a query returns rows that become objects; those objects are rendered; the same data is also written to a log file; a thread produced the data in the first place. The technologies are a system.

So this chapter does not explain any API in depth. It does something more important: it shows you the system, names every component in it, and explains why each technology is present. By the end you should be able to sketch the whole application from memory. Every later chapter fills in one box of that sketch.

CORE CONCEPT

The organising idea of this book is **one connected Java application**. Every technology you will study is introduced twice: first on its own terms, then in the place where the application actually needs it. If you ever lose the thread, come back to Diagram 1 and find your current topic in it.

1.2 What BasinWatch is trying to accomplish

BasinWatch is an offline floodplain operations console. A basin is a river catchment area; when rain falls and water levels rise, some districts flood before others, and a control room has to decide where to send limited equipment. BasinWatch simulates that situation on a desktop so a student or planner can practise the decisions without any real risk.

The operator's job, in the project's own words, is to watch a schematic basin map as rain and water levels change, review severity-ranked zone alerts, dispatch pumps, levee crews and evacuation teams to concurrent missions, see what resources remain available, then save the session, close the application, and restore the model and active operations later.

What enters the system

- Simulated station observations: rainfall in millimetres and observed water rise in centimetres, for six named zones.
- Simulation pulses marking the passage of a time interval.
- Operator commands: start, pause, step, inject a storm cell, dispatch, save, restore, export a report.
- Selection events: which zone the operator is inspecting.

What leaves the system

- A live map of six zones coloured by risk level.
- A ranked activity timeline and a list of active field operations.
- A resource board showing available and committed units.
- Persistent artefacts: an operational log, a typed binary archive, session saves and readable reports.
- Optional queryable history in a local SQLite database.

Six zones are monitored. Each has a position on a two-by-three schematic grid, an initial water level in metres, and a number of exposed residents. Risk is derived from the water level by fixed thresholds: below 1.7 m is NORMAL, 1.7 m and above is WATCH, 2.6 m and above is WARNING, and 3.6 m and above is CRITICAL.

BASINWATCH IN PRACTICE

Those thresholds are real code, not invented for the book. They appear in `BasinZone.assessRisk(double)`:

```
static RiskLevel assessRisk(double waterLevelMeters) {
    if (waterLevelMeters >= 3.6) return RiskLevel.CRITICAL;
    if (waterLevelMeters >= 2.6) return RiskLevel.WARNING;
    if (waterLevelMeters >= 1.7) return RiskLevel.WATCH;
    return RiskLevel.NORMAL;
}
```

Listing 1.1 — *from BasinWatch, `domain/BasinZone.java`*

1.3 The verified runtime architecture

The single most useful thing you can learn about a program is which component owns which responsibility and which thread runs which code. BasinWatch's architecture is documented in its own `ARCHITECTURE.md`, and — importantly — the documentation matches the code. Here is the verified picture.

VISUAL WALKTHROUGH

Read Diagram 1 from top to bottom. Follow the arrow from the operator into the Swing interface, down through the application commands into the engine, then along the green sensor path from left to right, and finally down into the two persistence columns. Every box is a real class; every arrow is a real call, stream write or thread hand-off.

Diagram 1 — BasinWatch: the complete Java system

FULL PAGE

Verified runtime architecture. Every box is a real class in the repository.

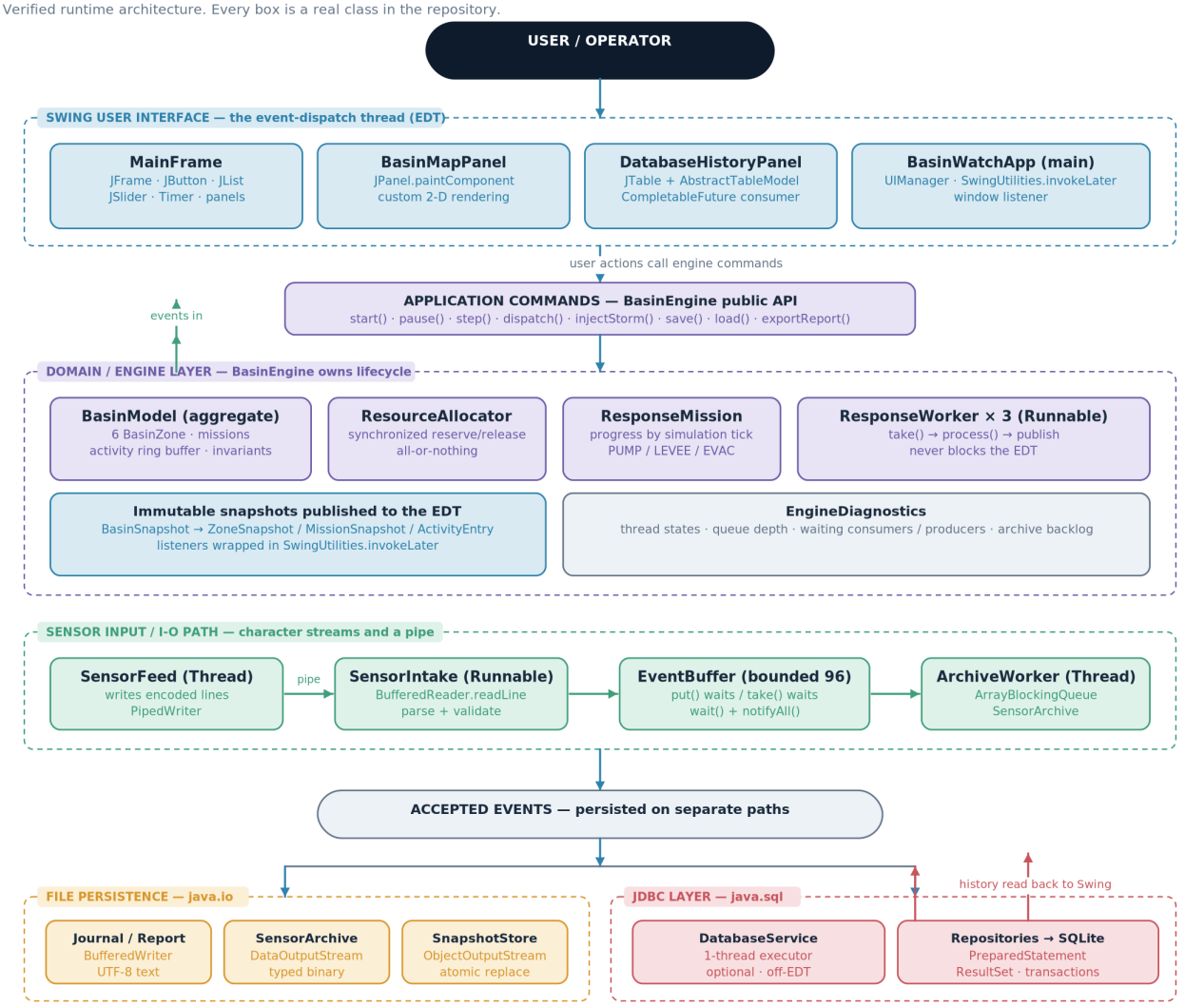
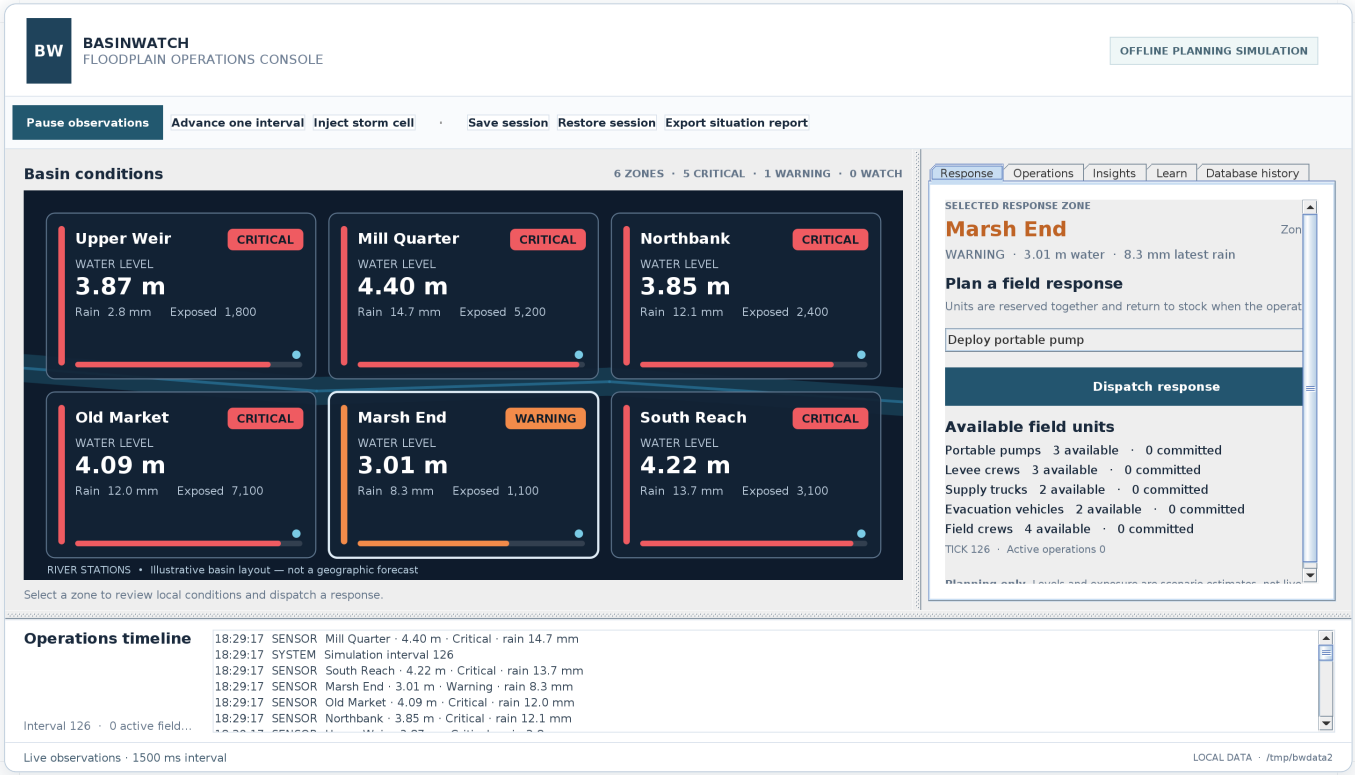


Diagram 1 — The complete BasinWatch system. Six layers: the operator, the Swing interface on the event-dispatch thread, the engine's command API, the domain and engine layer, the sensor input path with its piped streams, and the two independent persistence columns for files and JDBC.

What that architecture actually looks like on screen

Every diagram in this book describes code that runs. Before going any further, look at what the operator actually sees. The window below is the real BasinWatch application, launched from the shaded JAR built by Maven, with a simulation in progress: observations streaming, zones escalating to `CRITICAL`, and a field response dispatched against Marsh End.



Screenshot S1 — The running BasinWatch application, Response tab. Captured from the real application (`java -jar target/basinwatch-1.0.0.jar`). The header carries the brand, the title bar of controls (*Pause observations*, *Advance one interval*, *Inject storm cell*, *Save session*, *Restore session*, *Export situation report*) and the risk summary reading `6 ZONES · 5 CRITICAL · 1 WARNING · 0 WATCH`. The basin map below shows one card per zone with its live water level, rainfall and exposed population. The five tabs along the top of the panel — Response, Operations, Insights, Learn and Database history — are the subjects of Chapters 15 to 16, and each one is a real `JPanel` added to a `JTabbedPane`.

Two details in that screenshot are worth noticing now, because the whole book returns to them. First, the risk summary is not painted by the engine: the engine publishes an immutable `BasinSnapshot`, and a Swing listener copies numbers out of it. Second, the button labelled *Pause observations* means the feed is currently running — the label is derived from the engine's state, so the interface and the model cannot disagree.

> **VISUAL WALKTHROUGH**
>
> Read Diagram 1 from top to bottom. Follow the arrow from the operator into the Swing interface, down through the application commands into the engine, then along the green sensor path from left to right, and finally down into the two persistence columns. Every box is a real class; every arrow is a real call, stream write or thread hand-off.

The six layers, in the order data meets them

1. **The operator and the interface.** `BasinWatchApp.main` sets the system look and feel and then hands control to the event-dispatch thread with `SwingUtilities.invokeLater`. From that moment the `MainFrame` owns the window, and only the event-dispatch thread may touch Swing components.
2. **Application commands.** The interface never manipulates the model directly. It calls a narrow set of methods on `BasinEngine`: `start()`, `pause()`, `step()`, `dispatch(...)`, `injectStorm(...)`, `save(...)`, `load(...)`, `exportReport()`. Most return a `CompletableFuture`, so the interface can ask for work and keep painting.

3. **The domain and engine layer.** `BasinModel` is the aggregate root: it owns the six zones, the mission registry, the recent-activity ring buffer and the resource allocator. It enforces invariants and produces immutable snapshots. Three `ResponseWorker` tasks consume events and apply them.
4. **The sensor input path.** A dedicated `SensorFeed` thread encodes observations as text lines and writes them into a `PipedWriter`. A separate `SensorIntake` task reads them through a `BufferedReader` over a `PipedReader`, validates each record, and places accepted events into a bounded `EventBuffer`.
5. **File persistence.** Four independent file responsibilities: a UTF-8 append-only operational log, a typed binary sensor archive, versioned session snapshots written through an atomic replace, and human-readable reports.
6. **The JDBC layer.** An optional, single-threaded `DatabaseService` that writes accepted readings, mission lifecycles, resource allocations and operational events into a local SQLite file through parameterized SQL, and reads recent history back for the interface.

> **CONNECTION TO ANOTHER CONCEPT**
>
> Notice that the file column and the database column both hang off the *same* point — accepted events — but they are not alternatives. Chapter 9 shows what each file is for; Chapter 14 shows what the database adds; Chapter 10 explains why an application would deliberately choose to have both.

1.4 The seven questions the architecture must answer

An architecture is only useful if it answers concrete questions. These are the seven that matter for this application, with the verified answers.

Which component processes incoming data?

Observations arrive as text lines from the pipe. `SensorIntake` parses each line with `SensorReading.parse`, which validates the field count, the record type marker and every numeric range. A malformed record is reported and discarded — it does not stop the intake loop. Accepted events go into the bounded `EventBuffer`, and one of the three response workers takes them from there.

Where is Java I/O needed, and why?

In four genuinely different places, for four genuinely different reasons. The pipe carries observations between threads. The journal and reports give humans a readable audit trail. The archive keeps typed numeric measurements for later diagnosis. The snapshot store captures the whole session so it can be restored. Chapter 9 treats this as a design question rather than a list.

Why does JDBC exist in this application?

Because a file answers a different question than a database. A log file is a sequence of lines; asking "show me the 200 most recent readings for zone MIL, newest first" means reading and filtering the whole file. A database answers that with an index and a `LIMIT`. BasinWatch therefore keeps both, and the database is optional precisely because it is an addition rather than a requirement.

What does the Swing interface do?

Three things: it renders immutable snapshots, it collects operator intent, and it reports outcomes. It does no domain logic and no persistence. That separation is what makes the rest of the application testable without a window.

Which work happens on background threads?

Observation production (`SensorFeed`), intake (`basin-sensor-intake`), response processing (three `basin-response-N` threads), archiving (`basin-sensor-archive`), control commands (`basin-control`) and database writes (`basin-database`). Only rendering and event handling happen on the event-dispatch thread.

How does data survive the application closing?

Five ways, listed in Table 1.1. On a clean shutdown the engine also writes an automatic `last-session.bws` snapshot, so the next launch can restore the basin.

Table 1.1 — *The five persistence artefacts and what each one is for.*

Artefact	Written by	Contains	Restored on next launch?
`data/logs/operations.log`	OperationalJournal	Timestamped operational events as UTF-8 text	No — it is an audit trail
`data/archive/sensors.bin`	SensorArchive (via ArchiveWorker)	Typed tick, station, zone, rainfall, rise	No — it is a measurement record
`data/saves/*.bws`	SnapshotStore	Versioned session snapshot: tick, zones, stock, missions, activity	**Yes** — on request, or automatically for `last-session.bws`
`data/reports/*.txt`	ReportWriter	A human-readable situation report	No — it is a document
`data/basinwatch.db`	DatabaseService + repositories	Sensor readings, missions, resource usage, events in four tables	Read back as history, not as live state

Where do the technologies interact, and where do they stay separate?

They interact at exactly two boundaries. The first is the *accepted event*: after the model has processed an event, the engine writes to the log, submits to the archive worker, and enqueues a database operation. The second is the *published snapshot*: the engine hands an immutable value to the interface, the only channel by which background work becomes visible. Everywhere else the layers are deliberately ignorant of one another: the model does not know about files, the repositories do not know about Swing, and the interface does not know about SQL.

> **IMPORTANT DISTINCTION**
>
> The engine publishes **immutable snapshots**, not live objects. A `BasinSnapshot` is a record whose constructor copies every list and map with `List.copyOf` and `Map.copyOf`. That single decision is why the interface can render data on the event-dispatch thread while workers continue to mutate the model — the snapshot cannot change underneath it.

1.5 The Java Concepts Connection Map

Every syllabus topic in this book has a home. Diagram 2 places them all at once, grouped into four columns. Use it as a map rather than a summary: when a later chapter introduces a concept, you should be able to point at the column it belongs to.

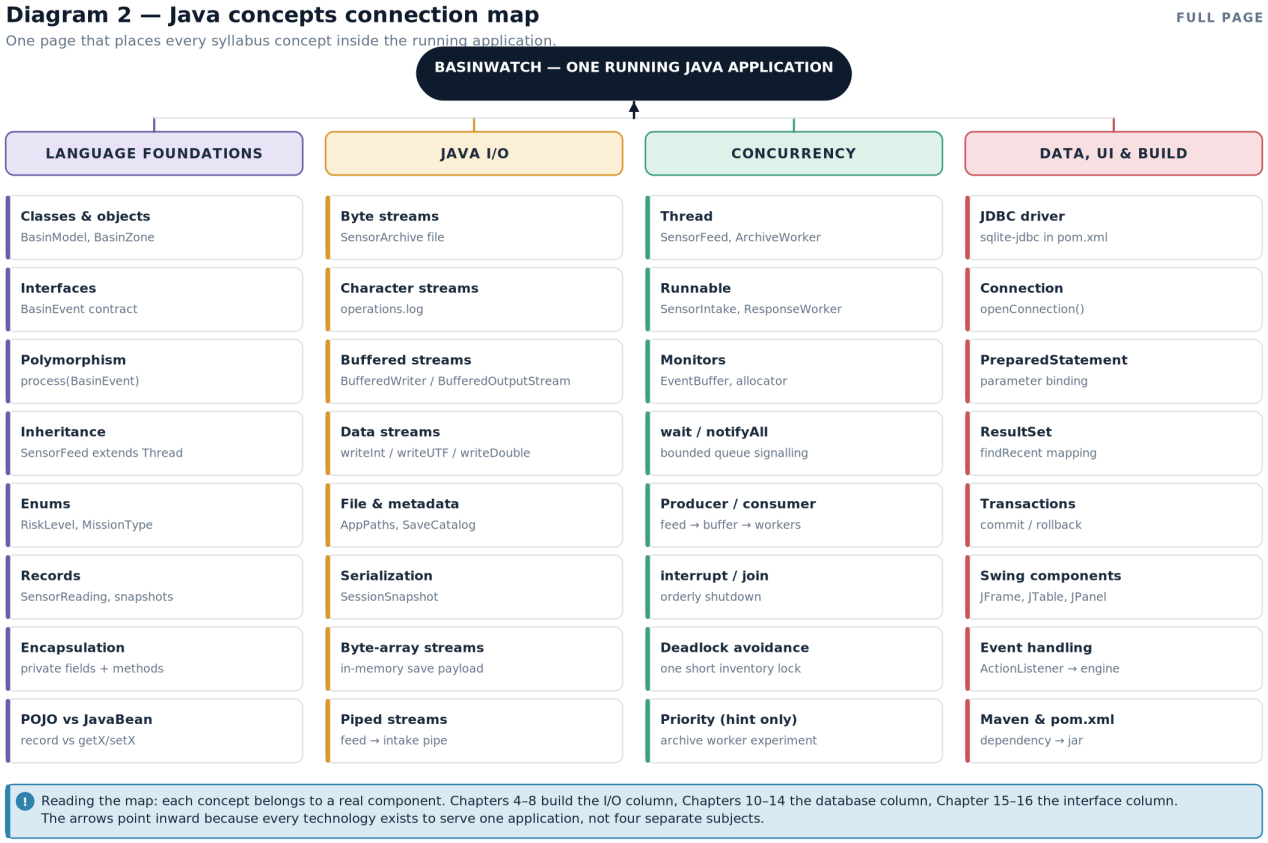


Diagram 2 — The Java Concepts Connection Map. Language foundations, Java I/O, concurrency, and the combined data/UI/build column. Each entry names a real class or file in the project.

Read the map in four movements:

- **Language foundations** (Chapter 2) explain the object concepts that make the rest possible: classes and objects, interfaces as contracts, polymorphism, justified inheritance, enums for bounded vocabularies, records for immutable values, encapsulation, and the POJO-versus-JavaBean distinction.
- **Java I/O** (Chapters 4 to 9) covers byte and character streams, buffering, data streams, byte-array streams, piped streams, serialization and file metadata.
- **Concurrency** (Chapters 3 and 7, then used everywhere) covers threads, runnables, monitors, condition waiting, the producer–consumer pattern, interruption and joining, and deadlock avoidance.
- **Data, UI and build** (Chapters 10 to 19) covers JDBC from driver to transaction, Swing from frame to repaint, and the configuration and packaging that make the application run at all.

1.6 What is genuinely difficult here

It is worth being honest about which parts of this application are hard, because those are the parts worth understanding deeply rather than memorising.

Concurrency is hard because correctness is invisible. If two dispatches both read the same resource count as available, the bug appears only under load and only sometimes. BasinWatch's answer

is a single short synchronized reservation that checks every required resource before decrementing any of them — an all-or-nothing operation. There is no way to observe the intermediate state, because there is no intermediate state.

Shutdown is hard because it is where resources leak. Stopping a feed thread, closing a pipe, waking waiting consumers, draining an archive queue, flushing a journal and writing a final snapshot must happen in the right order. The engine's `closeEngine()` does exactly this, and reports a timeout rather than pretending success.

Persistence is hard because the easy version is unsafe. Writing a file with `ObjectOutputStream` is two lines; making it safe means bounding the input size, filtering the classes that may be constructed, checking a format version, and validating the restored domain state. Chapter 8 walks through all four.

> **COMMON MISTAKE**
>
> The most common architectural mistake in student projects is letting the interface do the work. A button handler that opens a database connection, queries, and updates a label feels natural because it is one function in one file. It also freezes the window for the duration of the query, makes the logic untestable without a screen, and spreads SQL across the codebase. BasinWatch's rule — the interface asks, the engine decides, a worker executes, a snapshot reports — is the fix.

1.7 How the rest of the book is organised

Table 1.2 — *The book's structure and what each part delivers.*

Part	Chapters	What you will be able to do afterwards
I — Understanding the System	1–3	Sketch the architecture, explain the object concepts, and distinguish six forms of component communication.
II — Java I/O and Persistent Files	4–9	Choose and layer the right stream for a task, manage resources safely, and explain four different persistence mechanisms.
III — Databases and JDBC	10–14	Open a connection, execute parameterized SQL, map result sets, use transactions, and explain why a database is optional here.
IV — Swing and the User Interface	15–16	Build a component tree, handle events, keep the event-dispatch thread free, and show database results safely.
V — Configuration and Related Techniques	17–19	Explain how dependencies reach the classpath, what XML and annotations are for, and how JdbcTemplate abstracts raw JDBC.
VI — Putting Everything Together	20–21	Trace four real scenarios end to end and answer the eight integration questions.
Revision	R, V, E, G	

Part	Chapters	What you will be able to do afterwards
		Revise with comparison tables, answer viva questions, and work through tracing exercises.

EXAM TIP

If you are asked "describe your project architecture" in a viva, do not start with the code. Start with the data. Say what enters the system, which component validates it, where it is processed, what is persisted, and how the result reaches the screen. Then name the classes. Examiners reward the flow; the class names follow naturally.

CHAPTER SUMMARY

- BasinWatch is an offline floodplain operations console: a single-process Swing desktop application with no server, no network and no external service.
- Six layers: operator and interface, application commands, domain and engine, sensor input path, file persistence, and the optional JDBC layer.
- The event-dispatch thread owns all Swing components; five other threads do background work and publish immutable snapshots back to it.
- Java I/O appears in four distinct roles: a pipe between threads, a readable log, a typed binary archive, and a session snapshot.
- JDBC exists because selective queries over indexed rows are a different job from reading a text file; the database is an addition, not a replacement.
- Data survives shutdown in five artefacts, of which only the session snapshot restores live state.
- Technologies meet at exactly two boundaries: the accepted event and the published snapshot.

KEY TERMS

- **Aggregate root** — the single object that owns a group of related state and enforces its invariants. Here, `BasinModel`.
- **Immutable snapshot** — a value object copied out of the model so another thread can read it safely.
- **Event-dispatch thread (EDT)** — the single Swing thread that runs event handling and painting.
- **Bounded buffer** — a queue with a maximum size that makes producers wait when it is full.
- **Optional subsystem** — a component whose failure does not stop the application. Here, the SQLite history.

CHAPTER 2

Java Application Structure and Core Object Concepts

> Objects, interfaces, inheritance, enums, records and validation — the vocabulary the rest of the book assumes.

2.1 Why start with objects?

Every later chapter is about a mechanism — a stream, a query, a listener. Each mechanism operates on objects, and each is designed around an object concept. If a method takes an interface, the interface's design determines what the method can do. If a class is a record, it cannot be mutated. So the object model has to come first.

This chapter is not an introduction to writing classes. It is a tour of the specific object decisions BasinWatch makes, with the reason for each, plus the POJO-versus-JavaBean distinction that appears in the syllabus and matters the moment you meet a framework.

2.2 Classes, objects, state and behaviour

A class is a template; an object is an instance of it. An object has state (its fields) and behaviour (its methods), and good design keeps state private so behaviour is the only route in.

DEFINITION

Encapsulation — keeping fields private and exposing only the operations that make sense, so an object can never be placed in an invalid state from outside.

BasinWatch applies this at several levels. `BasinZone` is package-private: only classes in `com.basinwatch.domain` may even refer to it. Its fields are private, and it exposes a small set of package-private methods — `apply`, `pump`, `reinforceLevee`, `evacuate` — plus `snapshot()`, which returns an immutable copy.

```
final class BasinZone {
    private final String id;
    private final String name;
    private double waterLevelMeters;
    private double lastRainMillimeters;
    private int exposedResidents;
    private boolean leveeReinforced;
    private long lastReadingTick = -1;
    private RiskLevel risk;

    boolean apply(SensorReading reading) {
        if (reading.tick() <= lastReadingTick) {
            return false;                    // a stale reading is rejected, not applied
        }
        lastReadingTick = reading.tick();
        lastRainMillimeters = reading.rainfallMillimeters();
        double runoffFactor = leveeReinforced ? 0.58 : 1.0;
        double rainfallContribution = reading.rainfallMillimeters() * 0.0014;
        double observedRise = reading.observedRiseCentimeters() / 100.0;
        waterLevelMeters = Math.min(8.0,
                Math.max(0.0, waterLevelMeters + (rainfallContribution + observedRise) * runoffFactor));
```

```
        risk = assessRisk(waterLevelMeters);
        return true;
    }
    // pump, reinforceLevee, evacuate, snapshot, restore, assessRisk ...
}
```

from BasinWatch, domain/BasinZone.java — state is private; behaviour is the only route in.

Three things in that listing deserve attention. First, the stale-reading guard: if the incoming tick is not newer than the last one applied, the reading is ignored and the method returns `false`. That is an invariant, not a nicety — without it, an out-of-order observation could reduce a water level. Second, the reinforced levee multiplies runoff by 0.58, which is a real domain consequence of an earlier operator decision. Third, the water level is clamped to the range 0 to 8 metres on every update, so the field can never hold a physically impossible value.

CONNECTION TO ANOTHER CONCEPT

That guard is why Chapter 20's trace can say "the observation was accepted" and mean something specific. A reading is accepted only if it is newer than the last one for that zone *and* the model changed as a result. The engine uses exactly that test when deciding whether to write the reading to the database.

2.3 Interfaces as contracts

An interface declares what a type can do without saying how. In BasinWatch, the whole event system is built on one interface with four methods.

```
public interface BasinEvent {
    long tick();
    String zoneId();
    String category();
    String describe();
}
```

from BasinWatch, domain/BasinEvent.java

Three records implement it: `SensorReading`, `SimulationPulse` and `DispatchRequest`. They are radically different in content — a measurement, a time marker, an operator command — yet they all satisfy the same contract, so a single queue and a single `process` method can handle all of them.

2.4 Polymorphism: one call, several behaviours

Polymorphism means the same call site behaves according to the actual type of the object. `BasinModel.process` is the clearest example in the project.

```
public synchronized List<String> process(BasinEvent event) {
    if (event == null) {
        throw new IllegalArgumentException("Event is required.");
    }
    ArrayList<String> messages = new ArrayList<>();
    if (event instanceof SensorReading reading) {
        applyReading(reading, messages);
    } else if (event instanceof SimulationPulse pulse) {
        applyPulse(pulse, messages);
    } else if (event instanceof DispatchRequest request) {
        applyDispatch(request, messages);
    } else {
```

```
            throw new IllegalArgumentException("Unsupported basin event: " + event.getClass().getName());
        }
        return List.copyOf(messages);
    }
```

from BasinWatch, domain/BasinModel.java — pattern matching on the event type.

This is polymorphism expressed with pattern matching rather than with a method override. Both are valid; the choice here is deliberate. The model wants *one* place where the complete set of event types is visible, so that adding a fourth event type produces a compile-time-visible gap rather than a silently ignored class. The `else` branch throws, so an unknown event type is a loud failure rather than a silent no-op.

> **IMPORTANT DISTINCTION**
>
> **Interface polymorphism** (what the queue relies on) and **dispatch polymorphism** (what the model performs) are different things. The interface lets the buffer store any event without knowing its type. The `instanceof` chain is how the model decides which behaviour to run. You need both: the first for type safety, the second for behaviour selection.

2.5 Inheritance: used narrowly and for a reason

Inheritance is the most over-used feature in student Java. BasinWatch uses it exactly twice, and both times because the superclass is a real framework contract rather than a convenience.

- `SensorFeed extends Thread`. A feed has a genuine lifecycle to own: it must be started, it has a long-running loop, it holds a blocking pipe endpoint, and it must be interruptible and joinable. Extending `Thread` lets it carry a name (`"basin-sensor-feed"`), a priority, and a state that diagnostics can report.
- `MainFrame extends JFrame`, `BasinMapPanel extends JPanel`, `DatabaseHistoryPanel extends JPanel`. Swing is an inheritance-based framework: to be a component you must be a component. Overriding `paintComponent` is how custom drawing is done.

Everywhere else, composition is preferred. `BasinEngine` does not extend anything; it *has* an `EventBuffer`, a `DatabaseService`, a list of listener lists and an `ExecutorService`.

```
public final class SensorFeed extends Thread {
    public SensorFeed(Writer destination, List<String> zoneIds, SimulationSettings settings,
                      AtomicLong tickSequence, Consumer<String> failureReporter) {
        super("basin-sensor-feed");           // a named thread is a diagnosable thread
        this.destination = destination;
        this.zoneIds = List.copyOf(zoneIds);
        this.settings = settings;
        this.tickSequence = tickSequence;
        this.failureReporter = failureReporter;
        setPriority(Thread.NORM_PRIORITY);
    }
    @Override
    public void run() { /* the timed emission loop */ }
    public void requestStop() throws IOException { /* cooperative shutdown */ }
}
```

from BasinWatch, engine/SensorFeed.java — the one justified Thread subclass.

IMPORTANT DISTINCTION

Compare with `SensorIntake`, which `implements Runnable` instead. Why the difference? Because intake is a *task* — a unit of work — not a lifecycle owner. The engine creates a plain `Thread`, names it `"basin-sensor-intake"`, and hands it the task. This is the "prefer `Runnable` unless you must own the thread" rule, applied literally.

2.6 Enums: a fixed vocabulary, typed

When a value comes from a small closed set, an enum beats a `String` or an `int`, because the compiler then knows every legal value. BasinWatch has four domain enums, and each carries data with it.

```
public enum RiskLevel {
    NORMAL("Routine", 0), WATCH("Watch", 1), WARNING("Warning", 2), CRITICAL("Critical", 3);

    private final String label;
    private final int rank;
    RiskLevel(String label, int rank) { this.label = label; this.rank = rank; }
    public String label() { return label; }
    public int rank() { return rank; }
}
```

from BasinWatch, domain/RiskLevel.java — each constant carries a label and a rank.

The `rank()` method is what makes risk escalation detectable: `applyReading` compares `currentRisk.rank() > previousRisk.rank()` and prefixes the activity message with "Risk increased:" when it is true. That comparison is a domain rule expressed as ordinary code, and it is why the database can be told `riskEscalated` as a boolean rather than being asked to work it out.

`MissionType` goes further and carries the whole definition of a mission:

```
public enum MissionType {
    PUMP_DEPLOYMENT("Deploy portable pump", 8, 0.62,
            Map.of(ResourceType.PUMP, 1, ResourceType.FIELD_CREW, 1)),
    LEVEE_REINFORCEMENT("Reinforce levee", 10, 0.0,
            Map.of(ResourceType.LEVEE_CREW, 1, ResourceType.SUPPLY_TRUCK, 1)),
    EVACUATION("Evacuate exposed residents", 6, 0.0,
            Map.of(ResourceType.EVAC_VEHICLE, 1, ResourceType.FIELD_CREW, 1));

    private final String label;
    private final int durationTicks;
    private final double waterEffectMeters;
    private final Map<ResourceType, Integer> requirements;
    // constructor, accessors, toString ...
}
```

from BasinWatch, domain/MissionType.java — duration, effect and resource cost live with the type.

CORE CONCEPT

Putting a mission's duration, its effect on water level, and the resources it requires inside the enum constant means there is exactly one place to look when a mission type changes. The allocator, the scheduler and the database all read the same definition. This is the single most useful habit in the whole chapter: **make the data live where the knowledge lives**.

2.7 Records: immutable values with no boilerplate

A record declares immutable data in one line. The compiler generates a canonical constructor, accessors named after the components, and correct `equals`, `hashCode` and `toString`.

```
public record SensorReading(
        long tick, String stationId, String zoneId,
        double rainfallMillimeters, double observedRiseCentimeters) implements BasinEvent {

    public SensorReading {                      // compact constructor: validation
        if (tick < 0 || !validIdentifier(stationId) || !validIdentifier(zoneId)
                || !Double.isFinite(rainfallMillimeters) || rainfallMillimeters < 0
                || rainfallMillimeters > 500
                || !Double.isFinite(observedRiseCentimeters) || observedRiseCentimeters < 0
                || observedRiseCentimeters > 200) {
            throw new IllegalArgumentException("Invalid sensor reading.");
        }
    }

    @Override public String category() { return "OBSERVATION"; }

    public String encode() {
        return String.format(Locale.ROOT, "READ|%d|%s|%s|%.3f|%.3f",
                tick, stationId, zoneId, rainfallMillimeters, observedRiseCentimeters);
    }

    public static SensorReading parse(String line) {
        String[] fields = line.split("\\|", -1);
        if (fields.length != 6 || !"READ".equals(fields[0])) {
            throw new IllegalArgumentException("Malformed sensor observation.");
        }
        try {
            return new SensorReading(Long.parseLong(fields[1]), fields[2], fields[3],
                    Double.parseDouble(fields[4]), Double.parseDouble(fields[5]));
        } catch (NumberFormatException ex) {
            throw new IllegalArgumentException("Sensor reading contains an invalid number.", ex);
        }
    }
}
```

from BasinWatch, domain/SensorReading.java — a record that validates on construction.

Three patterns in that listing are worth naming, because they recur throughout the book.

The compact constructor validates. A record's canonical constructor can be written without repeating the parameter list, and any exception thrown there prevents the object from existing. This is why `parse` can be trusted: if it returns a `SensorReading`, the ranges are already correct. Validation happens at the boundary, once, instead of being re-checked by every consumer.

encode and parse are a matched pair. `encode()` produces a pipe-delimited line; `parse()` is its inverse and rejects anything that is not exactly six fields beginning with `READ`. Keeping them next to each other in the same class is what makes the wire format maintainable. Chapter 7 follows that format through the pipe.

Immutability makes the object safe to share. A `SensorReading` can be handed from the feed thread to the intake thread to a worker thread with no synchronization at all, because no thread can change it.

CONNECTION TO ANOTHER CONCEPT

Records and serialization meet in Chapter 8. A record is an excellent in-memory value, but records are serializable only if every component type is serializable and the record has no non-serializable superclass. BasinWatch's snapshot DTOs are therefore plain final classes with a no-argument constructor rather than records — a deliberate choice explained in that chapter.

2.8 POJO versus JavaBean

DEFINITION

POJO — a Plain Old Java Object — any ordinary class with fields and methods, subject to no framework convention at all.

DEFINITION

JavaBean — a POJO that follows a convention: a public no-argument constructor, private fields, and public getX/setX accessors, usually implementing Serializable, so that a framework can create and populate it by reflection.

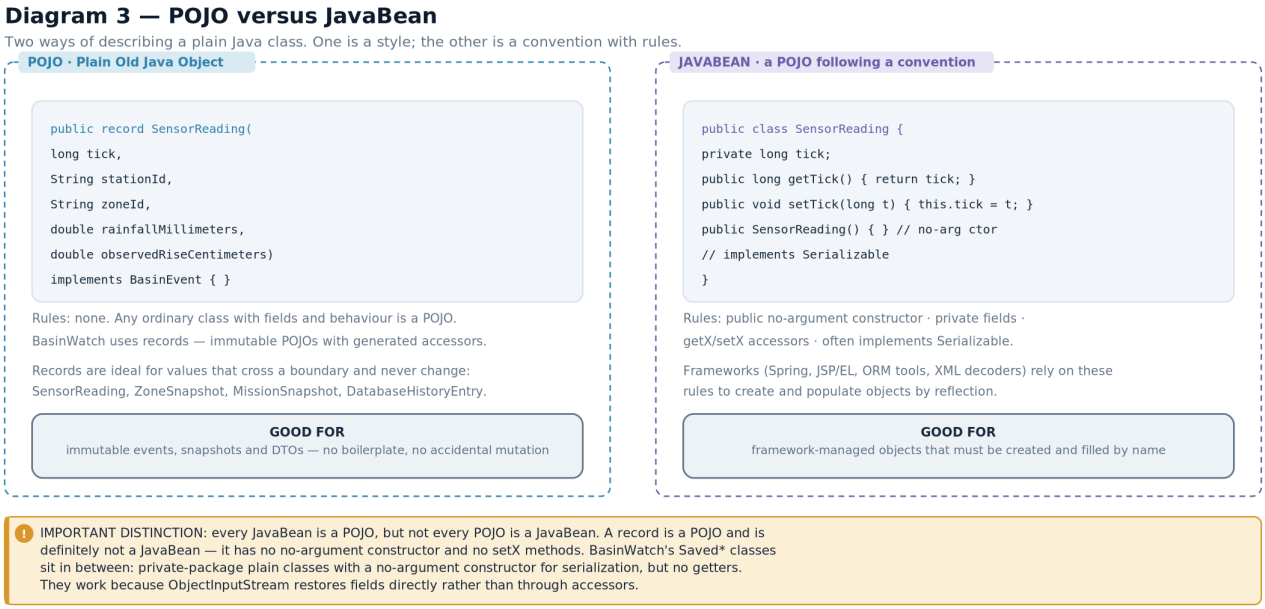


Diagram 3 — POJO versus JavaBean, side by side. Left: BasinWatch's SensorReading record — a POJO with no convention. Right: the same data as a JavaBean with a no-argument constructor and accessors.

The distinction matters practically. A record cannot be a JavaBean: it has no no-argument constructor and no `setX` methods, so a framework that expects to instantiate a bean and then call setters cannot use it. Conversely, nothing requires a JavaBean to be mutable in practice — it only has to *look* that way to the framework.

BasinWatch contains three shapes, and each is deliberate:

Table 2.1 — *The three object shapes and why BasinWatch uses each.*

Shape	Example	Why that shape
Record (immutable POJO)	`SensorReading`, `ZoneSnapshot`, `DatabaseHistoryEntry`	Values that cross a boundary must not change while in flight; the compiler generates accessors and equality.
Plain final class with mutable fields	`SavedZone`, `SavedMission`, `SessionSnapshot`	Serialization restores fields directly, so accessors are unnecessary. A no-argument constructor is required by the serialization mechanism.
Framework-style JavaBean	None in BasinWatch	No framework in this project needs to instantiate objects by name. Chapter 19 shows where a JavaBean would be required.

RELATED TECHNIQUE — NOT USED IN BASINWATCH

If BasinWatch used Spring's component scanning, its services would become JavaBeans or annotated components: a no-argument constructor plus `@Autowired` setters, or a constructor annotated for injection. It does not. Instead, wiring is explicit and visible: `BasinEngine` constructs the `DatabaseService`, which constructs the `DatabaseManager` and passes it to each repository constructor. That is constructor injection performed by hand. Diagram 27 contrasts the two.

2.9 Access modifiers as a design tool

Modifiers are not decoration; they are how a design is enforced. BasinWatch's choices are systematic.

Table 2.2 — *Access modifiers, their use in the project, and what they enforce.*

Modifier	Where it is used	What it prevents
`public`	Engine commands, domain value types, repository methods called from the service	Nothing — but the set is deliberately small and stable.
`package-private`	`BasinZone`, `ResponseMission`, `SavedZone`, the connection-taking overloads in repositories	Any code outside the package reaching into the aggregate's internals. Only `BasinModel` can mutate a zone.
`private`	Fields, helper methods, the schema array, the filter function	Everything except the class's own methods.
`final` (class)	`BasinModel`, `BasinEngine`, `SnapshotStore`, `OperationalJournal`	Subclassing a class that was not designed for it.
`static final` (constants)	`MAGIC`, `FORMAT_VERSION`, `MAX_SAVE_BYTES`, `WORKER_COUNT`	Magic numbers scattered through the code, and accidental modification.

BASINWATCH IN PRACTICE

The connection-taking overloads are a particularly neat example. Each repository has a public method that opens its own connection and a package-private method that accepts an existing one:

```
public void insert(SensorReading reading, ZoneSnapshot zone, Instant timestamp)
        throws SQLException {
    try (Connection connection = database.openConnection()) {
        insert(connection, reading, zone, timestamp);      // the package-private overload
    }
}

void insert(Connection connection, SensorReading reading, ZoneSnapshot zone,
            Instant timestamp) throws SQLException { /* binds and executes */ }
```

Listing 2.2 — *from BasinWatch, `db/SensorReadingRepository.java`*

The public overload is for ordinary use. The package-private overload exists so that `MissionRepository` can enlist several writes in *one* transaction on *one* connection — the subject of Chapter 13. Visibility encodes the rule: only a class in the same package may participate in someone else's transaction.

2.10 How BasinWatch divides responsibility

Six packages, six responsibilities. This division is the reason later chapters can be organised by technology without becoming a pile of unrelated facts.

Table 2.3 — *Package responsibilities and the boundaries between them.*

Package	Responsibility	Knows about	Must not know about
`app`	The Swing window, panels, listeners and entry point	engine commands, snapshots	SQL, streams, files
`domain`	Zones, events, missions, resources, invariants	nothing outside itself	files, SQL, Swing, threads
`engine`	Lifecycle, threading, queues, coordination	domain, io, db	Swing components
`io`	File paths, streams, snapshots, reports	domain values	SQL, Swing
`db`	Connections, schema, repositories, transactions	domain values	Swing, files
`learning`	Optional in-application architecture cards	nothing	everything else

EXAM TIP

A reliable viva answer to "how is your project structured?" is: "*by responsibility, and dependencies point one way — the interface depends on the engine, the engine depends on the domain, file and database layers, and none of those depend back on the interface.*" Then give one concrete example: the model has no import of `javax.swing`, which is why it can be tested without a window.

CHAPTER SUMMARY

- Encapsulation is enforced by modifiers: `BasinZone` is package-private and its fields are private, so only the aggregate can change zone state.
- One interface, `BasinEvent`, unifies three unrelated event records so a single queue and a single process method can handle them.
- Polymorphism appears as pattern matching in `BasinModel.process`, with an explicit failure branch for unknown types.
- Inheritance is used twice and only where a framework contract requires it: `Thread` for the feed, Swing components for the UI.
- Enums carry their own data — labels, ranks, durations, water effects and resource requirements.
- Records provide validated immutable values; the compact constructor is the validation point.
- A POJO has no convention; a JavaBean adds a no-argument constructor, accessors and often Serializable. Every bean is a POJO, not vice versa.

KEY TERMS

- **Aggregate** — a cluster of objects treated as one unit for change; `BasinModel` with its zones and missions.
- **Compact constructor** — a record constructor written without the parameter list, used here purely for validation.
- **Pattern matching** — testing and binding an object's type in one expression, as in `event instanceof SensorReading reading`.
- **Stale guard** — a check that rejects input older than the state already applied.
- **Constructor injection** — supplying an object's dependencies as constructor arguments, here done by hand rather than by a framework.

CHAPTER 3

How Java Components Communicate

> Method calls, shared objects, streams, events, queues and database operations are six different mechanisms. This chapter separates them.

3.1 The problem with saying "they communicate"

When you describe an application it is tempting to say that components "talk to each other" and leave it there. That sentence hides six distinct mechanisms with completely different properties. A method call is synchronous and immediate. A stream carries bytes or characters in one direction and can block. A queue decouples two threads in time. An event listener inverts control. A database operation crosses an execution boundary. An interrupt is a cooperative request, not a command.

Choosing the wrong one produces the classic failures: a frozen interface, a race condition, a lost update, a deadlock, a silent data loss. So this chapter names each mechanism, states its properties, and shows where BasinWatch uses it — and, just as importantly, where it does not.

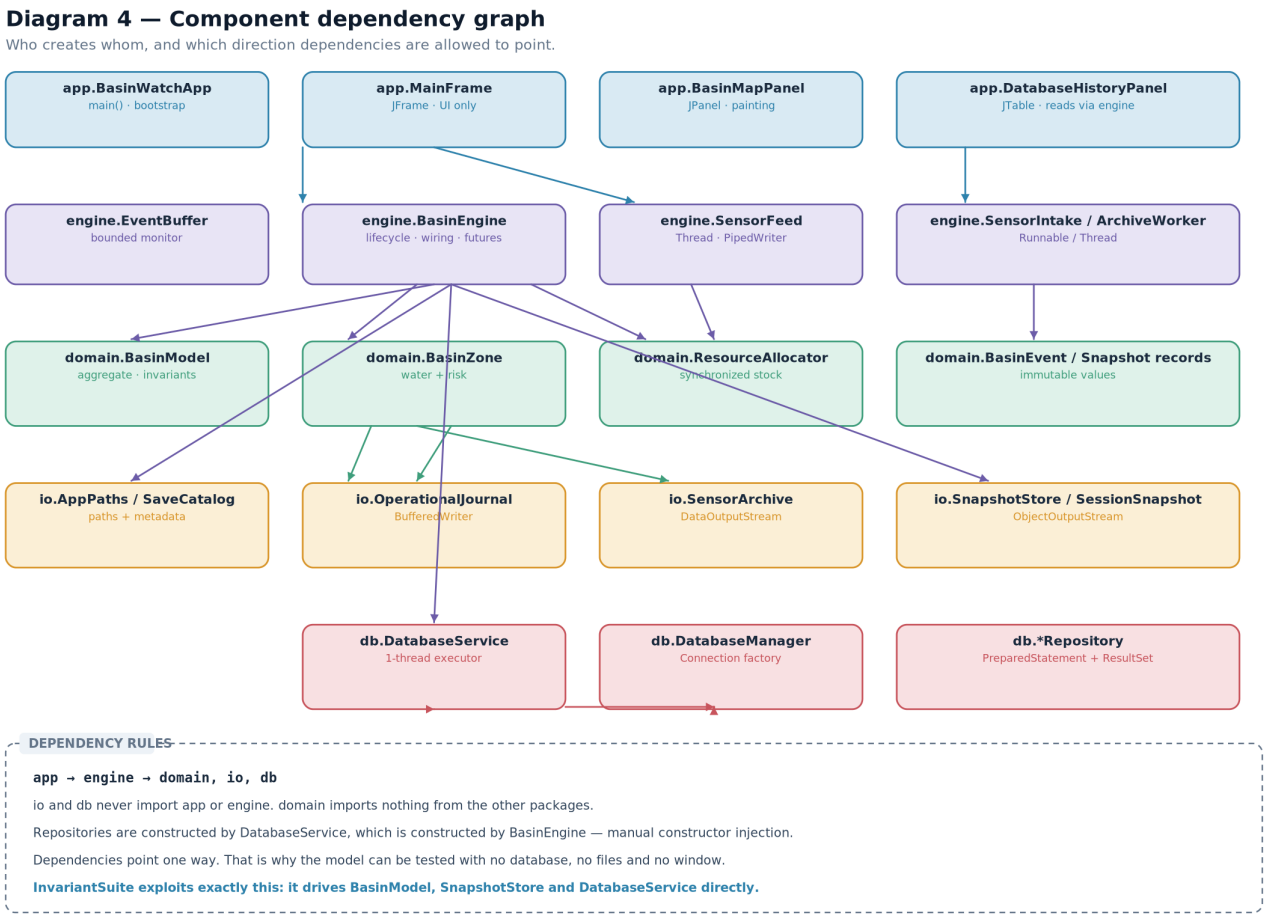


Diagram 4 — Component dependency graph. Who creates whom, and the one-way direction of dependencies: app depends on engine, engine depends on domain, io and db, and nothing depends back on app.

3.2 Mechanism one: the method call

The ordinary method call is synchronous, in-process, typed and immediate. The caller waits, the callee runs on the caller's thread, and arguments and return values are checked by the compiler.

BasinWatch uses method calls for everything that is fast and deterministic. `BasinModel.process(event)` is a method call. `BasinZone.assessRisk(waterLevel)` is a method call. `SessionSnapshot.from(snapshot)` is a method call. None of them touches a disk, a network or a thread.

CORE CONCEPT

The design rule the project follows is: **a method call may not block on external work.** Anything that reads a disk, waits for another thread, or talks to a database is moved behind a different mechanism — a future, a queue, or a background thread. That is why the model's methods are short and why the engine's file and database work is submitted rather than performed inline.

3.3 Mechanism two: shared objects under a monitor

Two threads can communicate simply by reading and writing the same object. The cost is that every such access needs protection, and the protection is what creates the bugs.

`ResourceAllocator` is the canonical example: a shared, finite inventory read and written by several response workers.

```
public synchronized boolean tryReserve(Map<ResourceType, Integer> request) {
    validateRequest(request);
    for (Map.Entry<ResourceType, Integer> entry : request.entrySet()) {
        if (available.get(entry.getKey()) < entry.getValue()) {
            return false;                       // nothing has been changed yet
        }
    }
    request.forEach((type, amount) -> available.put(type, available.get(type) - amount));
    return true;
}
```

from BasinWatch, domain/ResourceAllocator.java — check every requirement, then decrement.

Read that method twice. The first loop only *checks*. If any single requirement cannot be met, the method returns `false` having changed nothing at all. Only after every requirement has been confirmed does the second loop decrement. That is what makes the reservation all-or-nothing: there is no moment at which a caller could observe a partially reserved state, because the entire method holds the object's monitor from start to finish.

CONNECTION TO ANOTHER CONCEPT

This is also the project's deadlock-avoidance strategy, which its own architecture notes describe directly: one short inventory monitor, no I/O inside it, no nested acquisition of another lock, and a stable policy. A lock-ordering cycle requires two locks acquired in opposite orders by two threads; with one lock, that cycle cannot form.

3.4 Mechanism three: streams

A stream is a one-directional channel that carries a sequence of bytes or characters, with its own buffering and blocking behaviour. It is the subject of all of Part II, so here we only need its role in the architecture: it is how the feed thread hands data to the intake thread without either one knowing the other's class.

```
// BasinEngine.startFeed() — creating the connected pair
PipedWriter writer = new PipedWriter();
PipedReader reader = new PipedReader(writer, 8_192);
feedWriter = writer;
intakeThread = new Thread(new SensorIntake(
        new BufferedReader(reader), eventBuffer, this::reportFailure,
        ignored -> publishDiagnosticsStatus()), "basin-sensor-intake");
sensorFeed = new SensorFeed(writer, List.of("UPR", "MIL", "NOR", "OLD", "MAR", "SOU"),
        settings, tickSequence, this::reportFailure);
intakeThread.start();
sensorFeed.start();
```

from BasinWatch, engine/BasinEngine.java — the writer is handed to the feed, the reader to intake.

Note what is *not* shared: the feed never sees the `EventBuffer`, and intake never sees the `SensorFeed`. The only object they have in common is the pipe. This is a much narrower contract than a shared list, and it is why the two threads can be reasoned about separately.

3.5 Mechanism four: the queue with condition waiting

A queue decouples producer and consumer in time. The producer does not wait for the consumer to be ready; the consumer does not poll. The mechanism that makes this work is `wait()` and `notifyAll()` under a monitor, and it must be written carefully.

```
public synchronized boolean put(BasinEvent event) throws InterruptedException {
    if (event == null) {
        throw new IllegalArgumentException("Event is required.");
    }
    while (!closed && events.size() == capacity) {     // condition re-checked in a loop
        waitingProducers++;
        try {
            wait();                                    // releases the monitor while waiting
        } finally {
            waitingProducers--;
        }
    }
    if (closed) {
        return false;
    }
    events.addLast(event);
    notifyAll();                                       // wake every waiter to re-check
    return true;
}
```

from BasinWatch, engine/EventBuffer.java — a correct condition-wait loop.

Four properties of that code are the whole lesson, and they appear in every correct condition-wait implementation:

1. **Wait inside a loop, never inside an `if`.** A thread that wakes up is not guaranteed that its condition is now true — another consumer may have taken the item first. So the test is repeated.

2. **Hold the monitor.** `wait()` may only be called by a thread that owns the object's monitor. The `synchronized` keyword on the method provides it.
3. **Signal with `notifyAll()`, not `notify()`.** Producers and consumers wait on the same monitor, so waking one arbitrary thread might wake a producer when only consumers can progress. `notifyAll()` lets every waiter re-check, and the ones that cannot proceed go back to waiting.
4. **Count the waiters.** `waitingProducers` and `waitingConsumers` are incremented and decremented around the wait in a `finally` block, so the diagnostic view stays accurate even if the thread is interrupted. These counters are diagnostics, not synchronisation — an important distinction this chapter returns to.

IMPORTANT DISTINCTION

`wait()` releases the monitor; `sleep()` does not. This single sentence explains most of the confusion around Java threading. A waiting thread lets another thread enter the synchronized region; a sleeping thread keeps the lock and blocks everyone else. BasinWatch uses `wait()` for coordination and `Thread.sleep(...)` only for cadence — the feed's interval and the shutdown quiescence check — never as a signalling device.

COMMON MISTAKE

Using `Thread.sleep()` to wait for another thread's work, and using a flag variable polled in a loop, are the two most common substitutes for condition waiting. Both are wrong for the same reason: they guess at timing instead of reacting to state. A sleeping consumer may sleep too long (wasted latency) or too little (busy spinning), and a polled flag cannot wake anybody at the moment the state changes.

3.6 Mechanism five: events and listeners

An event listener inverts control: instead of a component asking for data, the component declares interest and is called when something happens. In BasinWatch the interface registers listeners on the engine.

```
engine.addSnapshotListener(snapshot ->
        SwingUtilities.invokeLater(() -> updateSnapshot(snapshot)));
engine.addStatusListener(message ->
        SwingUtilities.invokeLater(() -> statusLabel.setText(message)));
engine.addDatabaseStatusListener(message ->
        SwingUtilities.invokeLater(() -> databaseStatus.setText(message)));
```

from BasinWatch, app/MainFrame.java and app/DatabaseHistoryPanel.java — three listener registrations.

The engine holds these in `CopyOnWriteArrayList` instances, which is the right collection when reads vastly outnumber writes and iteration happens concurrently: publication iterates a stable snapshot of the list while a registration safely creates a new array.

The important part of this mechanism is the wrapper. The listener is invoked on a worker thread, and the very first thing it does is post the update to the event-dispatch thread. The listener contract is therefore: *I may be called from any thread; I will not touch Swing directly.* Chapter 16 examines that hand-off in detail.

3.7 Mechanism six: database operations

A database operation is the only mechanism in this list that leaves the Java execution model. Even with an embedded SQLite file, the operation involves a different execution path: SQL is parsed and planned, rows are read through a cursor, and failures arrive as `SQLException` rather than as ordinary Java exceptions.

BasinWatch wraps this in two different shapes depending on direction:

- **Writes are fire-and-forget.** `recordSensor`, `recordDispatch` and `recordCompletion` return `void`. They enqueue a lambda and return immediately. The caller learns about failure through the status listener, not through a return value.
- **Reads return a future.** `loadHistory` returns a `CompletableFuture<List<DatabaseHistoryEntry>>`, so the interface can attach a callback without blocking.

```
public void recordSensor(SensorReading reading, ZoneSnapshot zone, Instant timestamp,
                         boolean riskEscalated) {
    enqueue(() -> {
        sensors.insert(reading, zone, timestamp);
        if (riskEscalated) {
            operationalEvents.insert(timestamp, "RISK ESCALATION", zone.id(),
                    "Risk increased to " + zone.risk().name() + " at " + zone.name());
        }
    });
}
```

from BasinWatch, db/DatabaseService.java — a write is a queued lambda, not a blocking call.

IMPORTANT DISTINCTION

A `CompletableFuture` is not a thread. It is a handle to a result that may not exist yet. `whenComplete` registers work to run when the result arrives, and that work runs on whichever thread completed the future — which is why every UI callback in this project immediately calls `SwingUtilities.invokeLater`.

3.8 Choosing between the six

Table 3.1 is the decision table this chapter has been building towards. It is worth memorising, because examiners like asking "which mechanism would you use here?"

Table 3.1 — *The six communication mechanisms and where each belongs.*

Mechanism	Direction	Blocking?	Thread boundary?	Used in BasinWatch for
Method call	two-way	yes, briefly	no — same thread	All domain logic; every fast, deterministic operation
Shared object under a monitor	two-way	may block on the lock	yes	The finite resource inventory
Stream	one-way	yes when full or empty	usually yes	Observation text from feed to intake; all file access

Mechanism	Direction	Blocking?	Thread boundary?	Used in BasinWatch for
Queue with wait/ notifyAll	one-way	yes when full or empty	yes	The bounded event buffer between intake and workers
Listener / event	one-way, inverted	no	callback may arrive on any thread	Snapshot, status and database-status publication to the UI
Database operation	request and response	yes unless offloaded	yes — moved to its own thread	Optional SQLite history: writes queued, reads via futures

EXAM TIP

If you are asked why a pipe is used when a shared list would work, the answer is about contract width. A shared list gives both threads access to every element and to the list's own methods, so any bug in either thread can corrupt it. A pipe gives them one narrow operation each: write a character sequence, read a character sequence. Narrower contracts are easier to reason about, and the pipe also provides back-pressure for free — the writer blocks when the buffer is full.

3.9 The two boundaries, restated

Chapter 1 claimed that the technologies meet at exactly two boundaries. With the six mechanisms named, that claim can be made precise.

Boundary one — the accepted event

A worker has just applied an event to the model. At this single point the engine fans out: it appends to the journal, submits to the archive worker, and enqueues a database operation. Everything downstream of that fan-out is persistence, and none of it can change the model.

Boundary two — the published snapshot

The engine calls every registered snapshot listener with an immutable value. That call crosses from background threads to the event-dispatch thread and is the only channel by which background work becomes visible to a human.

CONNECTION TO ANOTHER CONCEPT

Chapter 20 traces four complete scenarios across these boundaries: an observation, a dispatch with its transaction, a save-and-restore cycle, and a history read. Each scenario exercises a different subset of the six mechanisms, which is exactly why tracing them is a better revision exercise than reciting the table.

CHAPTER SUMMARY

- Six distinct communication mechanisms: method call, shared object under a monitor, stream, queue with condition waiting, listener event, and database operation.
- A method call may not block on external work; anything slow moves behind a queue, a future or a background thread.
- Shared state requires a monitor, and the reservation is all-or-nothing: check everything, then change everything.
- A stream is a narrow one-directional contract with built-in back-pressure — that is why a pipe beats a shared list.
- A condition wait must be a loop, must hold the monitor, must use notifyAll, and must re-check its condition.
- wait() releases the monitor; sleep() does not. sleep() is for cadence, never for coordination.
- Writes to the database are queued and non-blocking; reads return CompletableFutures whose callbacks hop back to the EDT.

KEY TERMS

- **Monitor** — the lock associated with an object; entered by `synchronized`.
- **Condition wait** — `wait()` inside a loop that re-tests a predicate while holding a monitor.
- **Back-pressure** — a bounded channel making a fast producer slow down instead of consuming unbounded memory.
- **Inversion of control** — a component declaring interest and being called, rather than polling for change.
- **Fire-and-forget** — a call that queues work and returns immediately, reporting failure out of band.

PART II

Java I/O and Persistent Files

Everything an application knows arrives through a stream and everything it remembers leaves through one. This part builds the stream model from the ground up, layers wrappers on top of it, adds in-memory and piped variants, and finishes with object serialization — then shows all four mechanisms doing four different jobs in one program.

4 What Is a Stream?

5 Reading and Writing Files

6 Buffering, Data Streams and Stream Wrappers

7 In-Memory and Piped Streams

8 Serialization, Deserialization and Session Persistence

9 How BasinWatch Uses Java I/O

CHAPTER 4

What Is a Stream?

> The stream abstraction, source and destination, direction, and the four hierarchies that organise the whole java.io package.

4.1 The problem a stream solves

Suppose you want to copy a large file. You could read the whole thing into memory as a `byte[]` and then write it out. That works for a small file and fails for a large one, because the file must fit in memory. It also fails for data that has no end yet — a network socket, a pipe from another process, or a sensor feed that runs for hours.

A stream is the abstraction that solves this. Instead of handling all the data at once, you handle it in pieces: read some, process it, read some more. The stream hides whether the bytes come from a file, memory, a socket or another thread. That is why one `read()` method can serve a disk file and a pipe.

DEFINITION

Stream — an ordered, one-directional sequence of data with a source or a destination, consumed or produced incrementally rather than all at once.

CORE CONCEPT

Three properties define every stream in `java.io`, and every question about streams reduces to them:

1. **Direction.** Input or output — read or write, never both.
2. **Unit.** Bytes (8-bit) or characters (16-bit, with an encoding).
3. **Source or destination.** A file, memory, a pipe, or another stream.

Name those three and you have named the class you need.

4.2 Source, destination and direction

Direction is the first thing to fix, because it determines which of two class families you are in. A **source** is where data comes from: for an input stream, the source is the file or pipe you read. A **destination** is where data goes: for an output stream, the destination is the file or pipe you write.

The naming is always from the *program's* point of view, not the device's. A `FileInputStream` reads *into* the program from a file. A `FileOutputStream` writes *out of* the program to a file. This is the single most common source of confusion for beginners, and it disappears once you say the direction aloud.

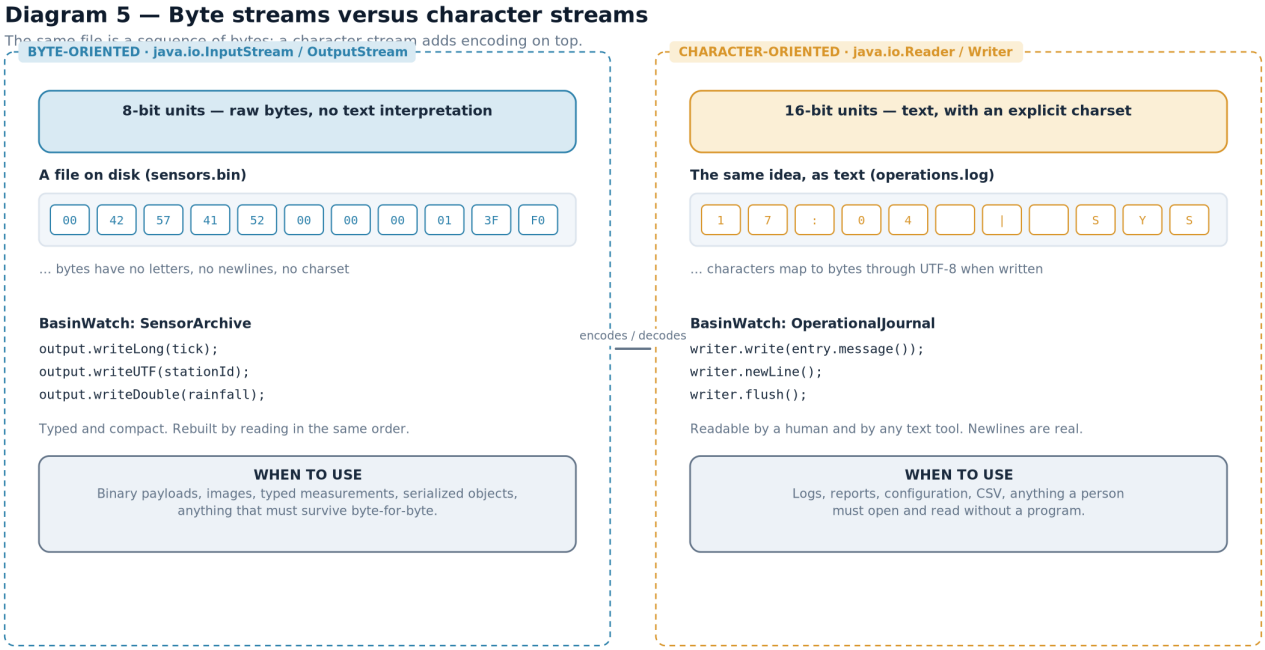


Diagram 5 — Byte streams versus character streams. The same file seen two ways: as raw 8-bit units that only a program can interpret, and as 16-bit characters encoded through a charset such as UTF-8.

4.3 Byte streams versus character streams

This is the first real decision you make with any stream, and it is a decision about meaning rather than performance.

Table 4.1 — *The byte/character decision, stated as a comparison.*

	Byte stream	Character stream
Abstract roots	`InputStream` / `OutputStream`	`Reader` / `Writer`
Unit	one byte (8 bits)	one character (16 bits)
Encoding	none — bytes are just bytes	an explicit charset converts characters to bytes, and back
Main methods	`read()`, `read(byte[])`, `write(int)`, `write(byte[])`	`read()`, `read(char[])`, `write(int)`, `write(String)`
Convenience	none — you assemble values yourself	`readLine()`, `newLine()`, `append()`
Correct for	images, audio, compressed data, typed measurements, serialized objects	logs, reports, configuration, CSV, HTML, any text a person reads
Wrong for	text you intend a human to read	binary data — encoding will corrupt it

COMMON MISTAKE

Using a character stream on binary data. Reading a PNG through a `FileReader` does not merely look wrong — it silently replaces byte sequences that are not valid in the current charset with a substitution character, and the file is destroyed. The rule is simple: if a human is meant to read it, use characters; otherwise use bytes.

CONCEPT EXPLAINED FURTHER

Why are characters 16-bit when bytes are 8-bit? Because Java's `char` is a UTF-16 code unit, chosen so that the common characters of the world's writing systems fit in one unit. The bridge between a 16-bit character and 8-bit storage is the *charset* — UTF-8, UTF-16, ISO-8859-1 and so on. A character stream always uses one, either explicitly or through the platform default.

IMPORTANT DISTINCTION

Encoding is not **encryption** and it is not compression. Encoding is a reversible mapping from characters to bytes. Nothing is hidden and nothing is made smaller — UTF-8 often makes text larger than a single-byte encoding would.

BASINWATCH IN PRACTICE

BasinWatch uses character streams where a human reads the output and byte streams where a machine must reproduce values exactly:

```
// character: a person can open this file and read it
writer = Files.newBufferedWriter(path, StandardCharsets.UTF_8,
        StandardOpenOption.CREATE, StandardOpenOption.APPEND);

// bytes: values keep their exact type and size
output = new DataOutputStream(new BufferedOutputStream(Files.newOutputStream(path,
        StandardOpenOption.CREATE, StandardOpenOption.APPEND)));
```

Listing 4.1 — *from BasinWatch, `io/OperationalJournal.java` and `io/SensorArchive.java`*

Both files store rainfall and water level, and both are correct. The difference is who reads them. `operations.log` is for an operator; the archive is for the program.

4.4 The four hierarchies

The entire `java.io` package is organised around four abstract roots, and every concrete class descends from exactly one of them.

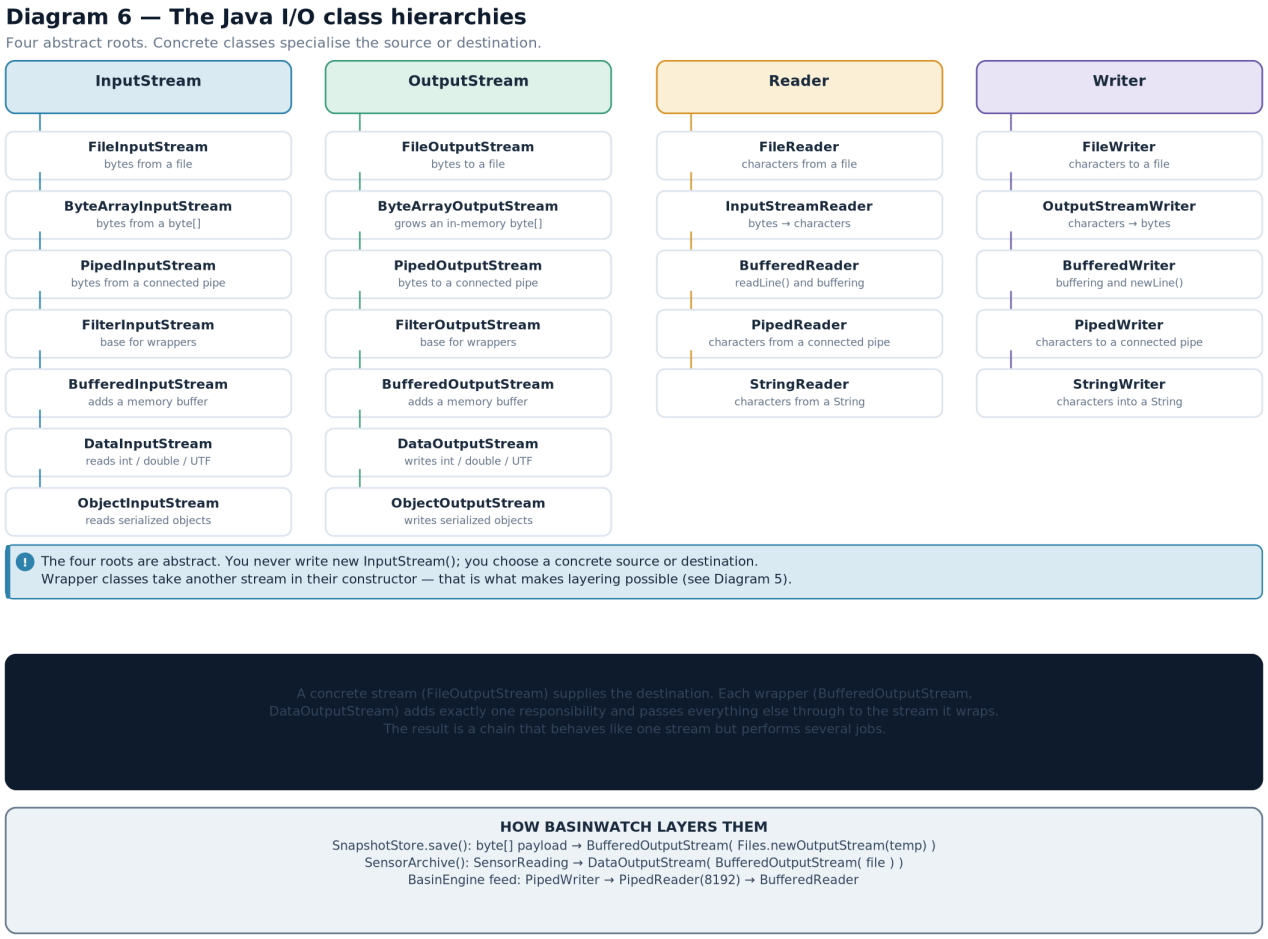


Diagram 6 — The Java I/O class hierarchies. Four abstract roots — InputStream, OutputStream, Reader, Writer — and the concrete classes that specialise the source or destination. The wrapper classes take another stream in their constructor, which is what makes layering possible.

Two things are worth noticing in that diagram.

There is a symmetry. Almost every capability exists in four forms. File access is `FileInputStream`, `FileOutputStream`, `FileReader`, `FileWriter`. In-memory access is `ByteArrayInputStream`, `ByteArrayOutputStream`, `StringReader`, `StringWriter`. Pipe access is `PipedInputStream`, `PipedOutputStream`, `PipedReader`, `PipedWriter`. Learn one family and you have learned the others.

Wrapper classes break the pattern deliberately. `BufferedInputStream`, `DataInputStream` and `ObjectInputStream` are not sources at all — they are *filters* that take another stream and add a capability to it. They sit under `FilterInputStream`, which is the base class for exactly this purpose.

CONNECTION TO ANOTHER CONCEPT

Chapter 6 is entirely about those wrappers. The idea they implement — add a capability by wrapping, without changing the underlying source — is the **decorator pattern**, and it is why a single chain of objects can read a typed `double` efficiently from a file while the file stream itself knows nothing about doubles.

4.5 The methods that actually matter

Java's stream API is large, but a small number of methods carry almost all the work. These are the ones to know by name and return type.

Table 4.2 — *The core stream methods. Note that `read()` returns `int`, not `byte`.*

Method	Class	Returns	Meaning
`read()`	InputStream / Reader	int	The next byte or character as an `int` in 0–255 (or 0–65535), or **-1** at end of stream.
`read(byte[] b)`	InputStream	int	How many bytes were read into the array, or -1 at end.
`read(char[] c)`	Reader	int	How many characters were read, or -1 at end.
`write(int b)`	OutputStream / Writer	void	Writes the low byte, or the character.
`write(byte[] b)`	OutputStream	void	Writes the whole array.
`write(String s)`	Writer	void	Writes a string, encoded.
`flush()`	both output families	void	Pushes buffered data to the destination. Does not close.
`close()`	all four roots	void	Flushes if applicable and releases the underlying resource. Idempotent.
`readLine()`	BufferedReader	String	One line without its terminator, or **null** at end of stream.
`newLine()`	BufferedWriter	void	Writes the platform line separator.

IMPORTANT DISTINCTION

Why does `read()` return an `int` instead of a `byte`? Because a byte cannot represent the end-of-stream marker. Every byte value from 0 to 255 is a legal data byte, so a separate signal is needed: `-1`. Widening the return type to `int` makes room for it. This is why the idiom is `int b; while ((b = in.read()) != -1)` and why casting to `byte` too early loses the end marker.

COMMON MISTAKE

Forgetting that `readLine()` returns `null` at end of stream while `read()` returns `-1`. Mixing them up produces an infinite loop that appears to work until the input ends. BasinWatch's intake loop is written correctly:

```
String line;
while ((line = source.readLine()) != null) { ... }
```

The assignment happens inside the condition, so the loop stops the moment the writer closes the pipe.

4.6 What happens when things go wrong

Almost every stream method declares a checked exception. There are only a few you meet in practice, and they mean different things.

Table 4.3 — *Stream exceptions and what they indicate.*

Exception	Thrown when	Typical cause
`IOException`	Any I/O failure	Disk full, file locked, pipe closed, permission denied
`FileNotFoundException`	A file cannot be opened for reading	The path does not exist, or the directory does
`EOFException`	A read hits the end before a full value was read	A truncated archive or an incomplete typed record — a *subclass* of IOException
`UnsupportedEncodingException`	The named charset is unavailable	A typo in a charset name
`NullPointerException`	A null stream or null array is passed	Not an IOException, and not checked — usually a programming error

EXAM TIP

Two facts about stream exceptions are asked constantly. First, `IOException` is *checked*, so the compiler forces you to handle or declare it — that is deliberate, because I/O failure is normal rather than exceptional. Second, `EOFException` *extends* `IOException`, so catching the parent also catches it. BasinWatch relies on that relationship when it validates an archive header:

```
catch (java.io.EOFException ex) {
    throw new IOException("Sensor archive header is incomplete.", ex);
}
```

4.7 The one rule about closing

Every stream holds an operating-system resource: a file handle, a pipe buffer, a socket. Those are finite. Failing to release them leaks them, and a long-running application will eventually fail to open anything at all.

Java gives you two ways to guarantee release, and both are used in BasinWatch.

```
// (a) try-with-resources — the preferred form
try (BufferedWriter writer = Files.newBufferedWriter(report, StandardCharsets.UTF_8)) {
    writer.write("BASINWATCH · SITUATION REPORT");
    writer.newLine();
    // ... more writes
}   // close() runs here, even if an exception was thrown

// (b) implementing AutoCloseable on a component that owns a stream
public final class OperationalJournal implements AutoCloseable {
    private final BufferedWriter writer;
    private int pendingLines;

    public OperationalJournal(Path path) throws IOException {
        writer = Files.newBufferedWriter(path, StandardCharsets.UTF_8,
                StandardOpenOption.CREATE, StandardOpenOption.APPEND);
    }

    @Override
    public synchronized void close() throws IOException {
        writer.flush();
        writer.close();
    }
}
```

from BasinWatch, io/ReportWriter.java and io/OperationalJournal.java — both release patterns.

CORE CONCEPT

Resources are released in **reverse order of acquisition**. In `try (BufferedInputStream in = new BufferedInputStream(Files.newInputStream(p)))` the buffered stream is closed first, which flushes it into the file stream, which is then closed. That order is what makes the last bytes reach the disk.

CHAPTER SUMMARY

- A stream is an ordered, one-directional sequence of data, consumed or produced incrementally rather than all at once.
- Every stream is described by three properties: direction, unit (byte or character), and source or destination.
- Four abstract roots: `InputStream`, `OutputStream`, `Reader`, `Writer`.
- Byte streams carry raw 8-bit units and preserve data exactly; character streams carry 16-bit characters through a charset and are correct for text.
- `read()` returns `int` so that -1 can mark end of stream; `readLine()` returns `null` instead.
- Stream failures are checked `IOException`s; `EOFException` is a subclass indicating a truncated read.
- Streams are released by try-with-resources or by implementing `AutoCloseable`, in reverse order of acquisition.

KEY TERMS

- **Stream** — an incremental, one-directional sequence of data.
- **Charset** — the reversible mapping between characters and bytes, for example UTF-8.
- **Filter stream** — a stream that wraps another stream and adds a capability.
- **try-with-resources** — a statement that closes every declared resource automatically, in reverse order.
- **End of stream** — the signal that no more data will arrive: -1 from `read()`, null from `readLine()`.

CHAPTER 5

Reading and Writing Files

> The file streams and character streams, their constructors and methods, complete read and write workflows, and the resource rules that keep them safe.

5.1 From a stream to an actual file

Chapter 4 described streams in the abstract. To make one real you attach it to a file, and that requires two different objects in modern Java: a `Path` that names the file, and a stream that reads or writes its contents.

IMPORTANT DISTINCTION

A `File` or `Path` is not the file's contents. It is a description of where the file is. You can ask it whether the file exists, what it is called, how large it is and when it was modified — all without opening it. Only when you create a stream do you touch the data. BasinWatch uses this distinction deliberately: it lists saves by inspecting metadata, and reads contents only when a save is actually restored.

```
public static List<Path> list(Path directory) throws IOException {
    File folder = directory.toFile();
    File[] files = folder.listFiles((parent, name) -> name.toLowerCase().endsWith(".bws"));
    if (files == null) {
        throw new IOException("Unable to list session saves in " + directory.toAbsolutePath());
    }
    return Arrays.stream(files)
            .filter(File::isFile)
            .sorted(Comparator.comparingLong(File::lastModified).reversed())
            .map(File::toPath)
            .toList();
}
```

from BasinWatch, io/SaveCatalog.java — listing saves needs metadata only, no stream at all.

Three details in that method repay attention. The filter runs on the directory listing, so non-matching names are never even considered. The sort is by last-modified time, newest first, which is what an operator expects from a restore dialog. And the null check matters: an unreadable or non-existent directory makes `listFiles` return `null` rather than an empty array, and a `NullPointerException` here would be far less useful than the explicit `IOException`.

5.2 The file-stream classes

Java has two generations of file-stream API, and both are still in use. Knowing which you are reading matters.

Table 5.1 — *The file-stream classes and how each is constructed.*

Class	Family	How it is built	Notes
`FileInputStream`	byte input	`new FileInputStream(path)` or `new FileInputStream(file)`	The classic class. Throws `FileNotFoundException`, a subclass of IOException.

Class	Family	How it is built	Notes
`FileOutputStream`	byte output	`new FileOutputStream(path, true)` for append	Without the boolean it truncates the file. The second argument is easy to forget.
`FileReader`	character input	`new FileReader(path, StandardCharsets.UTF_8)`	Without a charset it uses the platform default, which differs between machines.
`FileWriter`	character output	`new FileWriter(path, StandardCharsets.UTF_8, true)`	Same truncate/append trap, and the same platform-default trap.
`Files.newInputStream`	byte input	`Files.newInputStream(path, StandardOpenOption.READ)`	The modern equivalent. Options are explicit; failures are plain `IOException`.
`Files.newBufferedWriter`	character output	`Files.newBufferedWriter(path, UTF_8, CREATE, APPEND)`	Buffered and charset-explicit in one call. This is what BasinWatch uses.

COMMON MISTAKE

Two classic bugs live in that table. First, forgetting the append flag: `new FileOutputStream(path)` *truncates* the file to zero length, so a log that was supposed to grow silently restarts. Second, omitting the charset: `new FileWriter(path)` uses the platform default encoding, so a file written on one machine can be unreadable on another. BasinWatch never omits either — it uses `StandardCharsets.UTF_8` and `StandardOpenOption.APPEND` explicitly at every site.

5.3 A complete write workflow

Here is the full sequence for writing text to a file, using the modern API. Read it as a template; the same five steps appear in every write in this book.

```
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class WriteWorkflow {
    public static void main(String[] args) throws IOException {
        Path target = Path.of("demo", "notes.txt");
        Files.createDirectories(target.getParent());          // 1. ensure the folder exists

        try (BufferedWriter writer =                          // 2. open in try-with-resources
                     Files.newBufferedWriter(target, StandardCharsets.UTF_8)) {
            writer.write("first line");                       // 3. write content
            writer.newLine();                                 //    a real line separator
            writer.write("second line");
            writer.newLine();
            writer.flush();                                   // 4. push buffered bytes out
        }                                                     // 5. close() runs automatically
        System.out.println("Wrote " + Files.size(target) + " bytes");
```

```
    }
}
```

illustrative — the canonical text-write workflow, mirroring ReportWriter.write

The five steps are: create the parent directory, open inside try-with-resources, write the content, flush, and let the block close the stream. The flush at step four is technically optional here because `close()` flushes, but writing it explicitly documents the intent and matters when a stream is kept open across many writes.

BASINWATCH IN PRACTICE

BasinWatch's report writer is the same five steps with real content. Note that the whole file is produced inside one try-with-resources block, so a failure halfway through still closes the stream:

```java
public static Path write(Path directory, BasinSnapshot snapshot) throws IOException {
    String filename = "basin-status-" + FILE_TIME.format(Instant.now())
            + "-" + UUID.randomUUID().toString().substring(0, 8) + ".txt";
    Path report = directory.resolve(filename);
    try (BufferedWriter writer = Files.newBufferedWriter(report, StandardCharsets.UTF_8)) {
        writer.write("BASINWATCH · SITUATION REPORT");
        writer.newLine();
        writer.write("Generated: " + Instant.now());
        writer.newLine();
        // ... zone conditions, resources, missions, recent activity
    }
    return report;
}
```

Listing 5.1 — *from BasinWatch, `io/ReportWriter.java`*

The filename carries a timestamp and eight characters of a UUID. That combination means two reports generated in the same second still get distinct names, and the operator can tell when each was produced without opening it.

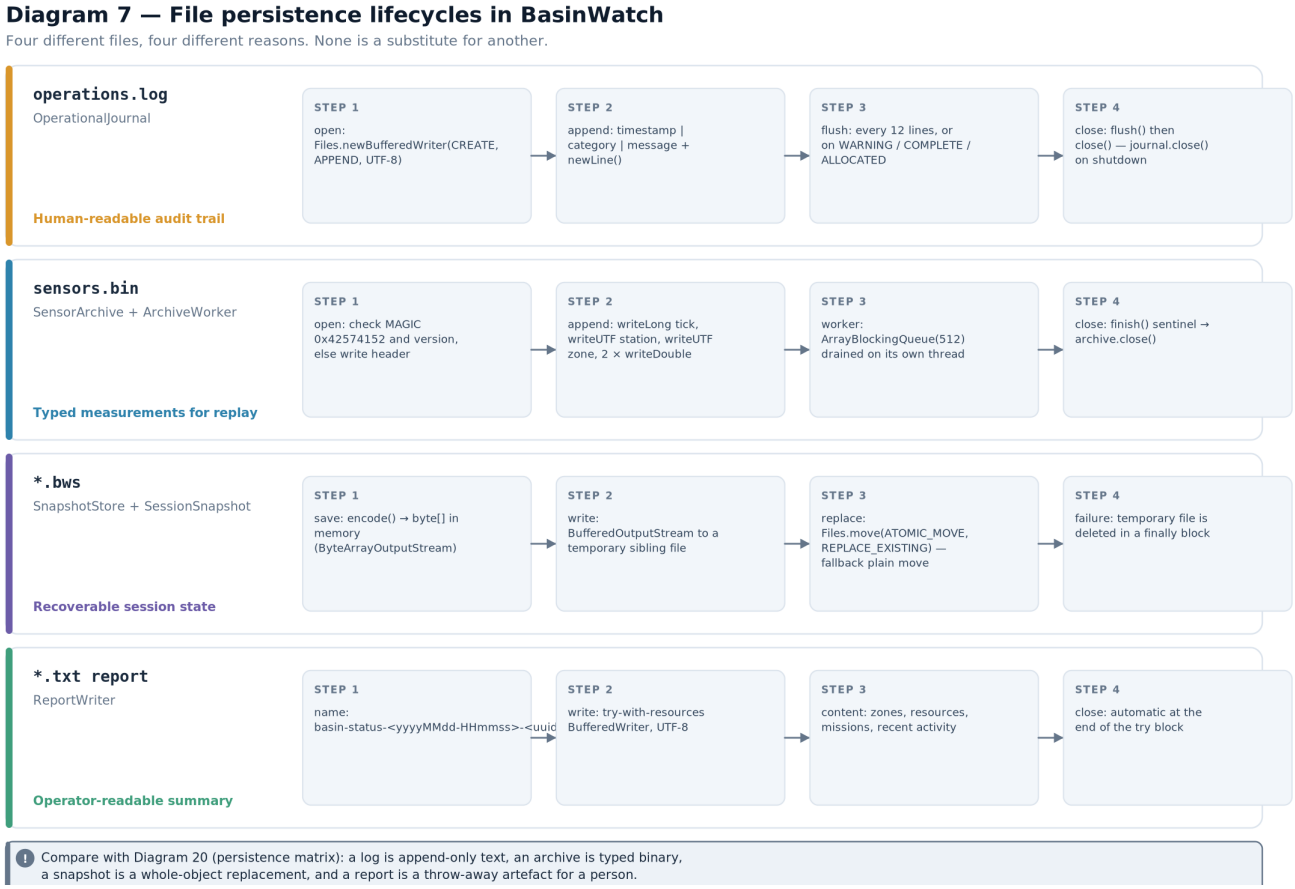


Diagram 7 — File persistence lifecycles. Each file is opened, written, flushed and closed on a different schedule. The log appends for the lifetime of the session; the archive is opened once and appended to per observation; the snapshot is written once and replaced atomically; a report is written once and never reopened.

5.4 A complete read workflow

Reading has one extra consideration: you do not know how many lines there are, so the loop must be driven by the read itself.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class ReadWorkflow {
    public static void main(String[] args) throws IOException {
        Path source = Path.of("demo", "notes.txt");
        if (!Files.isRegularFile(source)) {
            throw new IOException("No such file: " + source.toAbsolutePath());
        }
        int lines = 0;
        try (BufferedReader reader =
                     Files.newBufferedReader(source, StandardCharsets.UTF_8)) {
            String line;
            while ((line = reader.readLine()) != null) {   // null marks end of stream
                lines++;
                System.out.println(lines + ": " + line);
            }
        }
        System.out.println("Read " + lines + " line(s)");
```

```
    }
}
```

illustrative — the canonical text-read workflow, mirroring SensorIntake's loop

Three habits are worth copying. The existence check produces a clearer message than a raw `FileNotFoundException`. The charset is explicit, so the file reads identically everywhere. And the loop condition performs the read, so there is no possibility of testing for end-of-stream and then reading anyway — a bug that would skip or duplicate the last line.

BASINWATCH IN PRACTICE

`SensorIntake.run` is this workflow applied to a pipe rather than a file, with validation folded in:

```
try (source) {                                  // source is the BufferedReader
    String line;
    while ((line = source.readLine()) != null) {
        BasinEvent event;
        try {
            if (line.startsWith("PULSE|")) {
                event = new SimulationPulse(Long.parseLong(line.substring(6)));
            } else {
                event = SensorReading.parse(line);
            }
        } catch (IllegalArgumentException ex) {
            warningReporter.accept("Discarded malformed sensor record: " + ex.getMessage());
            continue;                           // skip, keep reading
        }
        // ... put(event) into the buffer
    }
} catch (IOException ex) {
    warningReporter.accept("Sensor intake ended with an I/O error: " + ex.getMessage());
}
```

Listing 5.2 — *from BasinWatch, `engine/SensorIntake.java`*

Note the two different failure responses. A *malformed record* is reported and skipped, because one bad line should not stop the stream. An *I/O error* ends the loop, because the stream itself is broken and there is nothing left to read. Distinguishing recoverable bad data from an unrecoverable channel failure is a design decision, not a detail.

5.5 Appending versus replacing

Two very different intentions share the same class, and the difference is one boolean or one option.

Table 5.2 — *Open options and their effect on existing data.*

Intention	How to open	Effect on existing content
Replace the file	`Files.newBufferedWriter(p, UTF_8)`	Truncated to zero length immediately on open
Add to the file	`Files.newBufferedWriter(p, UTF_8, CREATE, APPEND)`	Existing content is preserved; new writes go to the end
Create if absent	`CREATE`	A missing file is created
Fail if it already exists	`CREATE_NEW`	Throws `FileAlreadyExistsException` — useful for safety

BASINWATCH IN PRACTICE

BasinWatch makes both choices, and each is right for its artefact. The operational journal appends, because it is a running record of everything that has happened:

```
// OperationalJournal — append
writer = Files.newBufferedWriter(path, StandardCharsets.UTF_8,
        StandardOpenOption.CREATE, StandardOpenOption.APPEND);
```

The report writer replaces, because each report is a fresh snapshot of the current situation and carries a unique name anyway:

```
// ReportWriter — create a new file
try (BufferedWriter writer = Files.newBufferedWriter(report, StandardCharsets.UTF_8)) { ... }
```

Getting this backwards would be a serious defect: a truncating journal would erase history on every restart, and an appending report would mix yesterday's situation into today's document.

5.6 Flushing: when data actually reaches the disk

A buffered stream keeps data in memory until the buffer fills or you flush. This means that "the write succeeded" and "the bytes are on disk" are different statements.

BASINWATCH IN PRACTICE

The journal manages this explicitly, with a policy rather than a reflex. It flushes when twelve lines have accumulated, or immediately when the message is operationally significant:

```
public synchronized void append(ActivityEntry entry) throws IOException {
    writer.write(DateTimeFormatter.ISO_INSTANT.format(entry.at()));
    writer.write(" | ");
    writer.write(entry.category().replace('\n', ' '));
    writer.write(" | ");
    writer.write(entry.message().replace('\n', ' '));
    writer.newLine();
    pendingLines++;
    if (pendingLines >= 12 || entry.category().contains("WARNING")
            || entry.category().contains("COMPLETE")
            || entry.category().contains("ALLOCATED")) {
        writer.flush();
        pendingLines = 0;
    }
}
```

Listing 5.3 — *from BasinWatch, `io/OperationalJournal.java`*

This is a real engineering trade-off. Flushing every line would guarantee durability but cost a system call per line. Never flushing would be fast but could lose a burst of activity in a crash. Flushing on the important categories means that an allocation, a completion or a warning is on disk immediately, while routine readings can wait a few lines.

CONCEPT EXPLAINED FURTHER

The two `replace('\n', ' ')` calls are not cosmetic. An activity message could contain a newline — from an exception message, for example — and a newline inside a log line would break the file's one-line-per-entry structure. Replacing it with a space preserves the format without losing information.

IMPORTANT DISTINCTION

`flush()` and `close()` are not the same. `flush()` pushes buffered data to the operating system but leaves the stream usable. `close()` flushes *and* releases the file handle, after which any further write throws `IOException`. Calling `close()` twice is safe and has no additional effect.

5.7 What a real write failure looks like

I/O failures are ordinary events, not exotic ones, and the code should read as though it expects them. BasinWatch's engine treats a journal failure as something to report but not to propagate into the event-processing path:

```
IOException journalFailure = null;
for (String message : messages) {
    ActivityEntry entry = new ActivityEntry(Instant.now(), event.category(), message);
    try {
        journal.append(entry);
    } catch (IOException ex) {
        if (journalFailure == null) {
            journalFailure = ex;
        } else {
            journalFailure.addSuppressed(ex);      // keep every failure, lose none
        }
    }
}
// ... archive and database work still happen
if (journalFailure != null) {
    throw journalFailure;                          // reported after the rest of the work
}
```

from BasinWatch, engine/BasinEngine.java — collect failures, continue the work, report at the end.

The pattern is worth learning because it appears throughout professional code: attempt every independent piece of work, accumulate the failures, and report them once at the end. It is the opposite of stopping at the first problem, and it matters here because a log failure should not prevent the archive and database writes that follow.

EXAM TIP

A very common exam question asks you to write a method that copies a text file line by line and to say what happens if the destination cannot be opened. The expected answer mentions: try-with-resources so both streams close; an explicit charset; the `readLine() != null` loop; and that an `IOException` from the *opening* of the destination means the source is never read. Say the order of operations explicitly — examiners look for it.

CHAPTER SUMMARY

- A `Path` or `File` describes where data is; only a stream reads or writes the data itself.
- Four classic file classes exist for each direction and unit; the modern `Files.newXxx` methods make options and charsets explicit.
- Opening a file for output truncates it unless an append flag or option is supplied.
- A complete write workflow is: create the parent, open in try-with-resources, write, flush, close.
- A complete read workflow is: check existence, open with an explicit charset, loop on `readLine() != null`, close.
- Recoverable bad data should be skipped and reported; an unrecoverable stream failure should end the loop.
- `flush()` pushes data onward and keeps the stream open; `close()` flushes and releases the handle.

KEY TERMS

- **Append mode** — opening an output stream so new writes follow existing content.
- **Truncate** — reducing a file to zero length when it is opened for output.
- **Flush** — forcing buffered data out to the destination without closing the stream.
- **Suppressed exception** — a secondary failure attached to a primary one so neither is lost.
- **Metadata** — information about a file (name, size, modification time) as opposed to its contents.

CHAPTER 6

Buffering, Data Streams and Stream Wrappers

> How a wrapper adds a capability without changing the source, why buffering changes performance by orders of magnitude, and how typed values are written to a binary file.

6.1 The problem with one byte at a time

Every `read()` call on a raw file stream is a system call. The operating system has to leave user code, consult its file-system layer, copy a byte, and return. That overhead is roughly the same whether you asked for one byte or eight thousand.

So reading a megabyte one byte at a time means a million system calls to move data that would fit in 128 calls of 8 KB. The CPU spends its time in the kernel boundary rather than on your program. Buffering fixes this by adding an intermediate array.

CORE CONCEPT

A **buffer** is a memory array that sits between your code and the underlying resource. On the first read the buffer is filled with one large request; subsequent reads are served from memory until the buffer is exhausted, at which point it is refilled. Writing works in reverse: bytes accumulate in the buffer and are written out in one large operation when the buffer fills, when you flush, or when you close.

6.2 Layering: the decorator pattern

The classes that add buffering, typing and object support are not sources. They take another stream in their constructor and wrap it. That single design choice is what makes stream code composable.

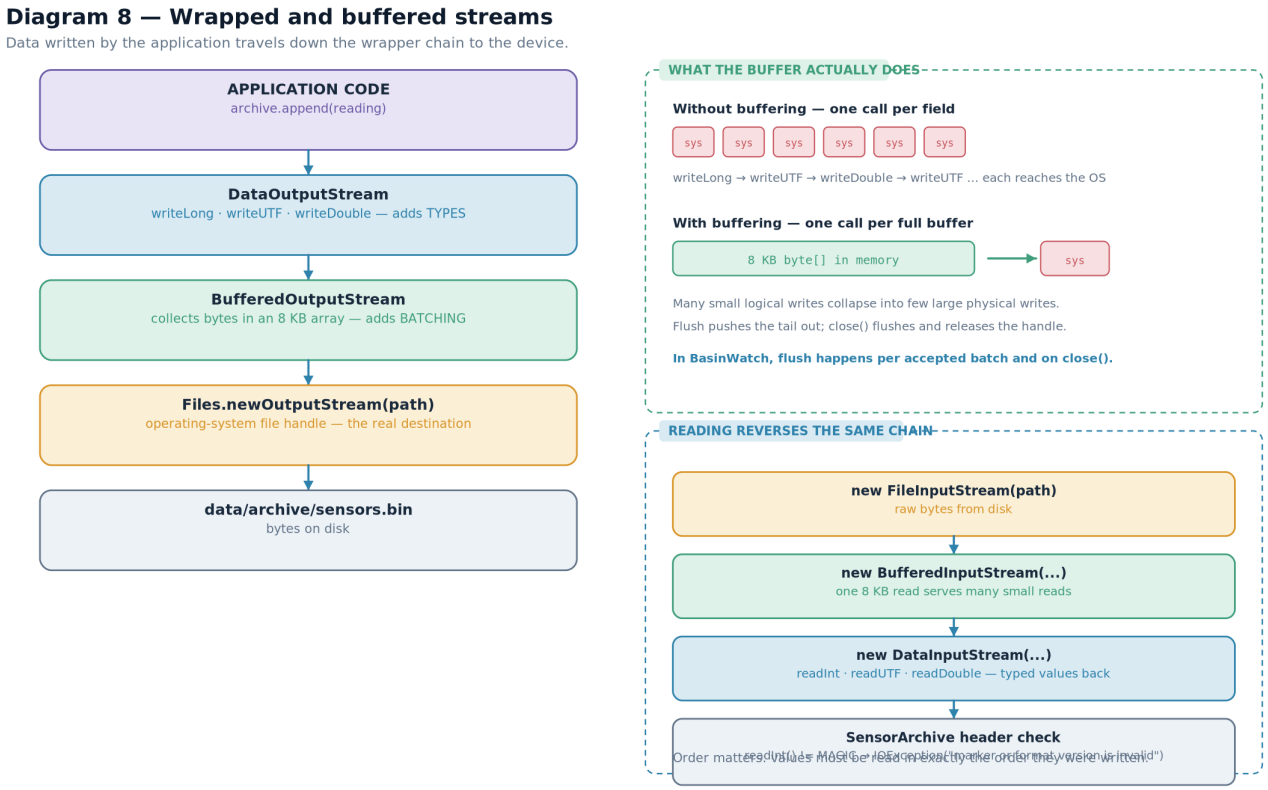


Diagram 8 — Wrapped and buffered streams. Reading a file through three layers. Each wrapper delegates downward and adds exactly one capability: the buffered stream adds an array, the data stream adds type interpretation. Data flows upward through the same layers.

Read that diagram from the bottom up, because that is the direction data travels on a read. The file stream delivers raw bytes. The buffered stream collects them into an array so the file stream is called rarely. The data stream interprets the next four bytes as a `double` and the next four as an `int`. Each layer knows only the layer beneath it.

The same chain on a write goes the other way: your code calls `writeDouble`, the data stream converts the value to eight bytes, the buffered stream collects those bytes, and the file stream eventually writes them out.

BASINWATCH IN PRACTICE

BasinWatch's archive writer is exactly this chain, built in one statement:

```
private static DataOutputStream openArchive(Path path) throws IOException {
    boolean fresh = !Files.isRegularFile(path) || Files.size(path) == 0;
    DataOutputStream output = new DataOutputStream(new BufferedOutputStream(
            Files.newOutputStream(path, StandardOpenOption.CREATE, StandardOpenOption.APPEND)));
    if (fresh) {
        output.writeInt(MAGIC);
        output.writeInt(FORMAT_VERSION);
        output.flush();
    }
    return output;
}
```

Listing 6.1 — *from BasinWatch, `io/SensorArchive.java`*

Three layers, three capabilities: `Files.newOutputStream` talks to the file, `BufferedOutputStream` batches the bytes, and `DataOutputStream` converts typed values to bytes. The `fresh` test decides whether the header must be written — an archive that already contains data must not get a second header.

6.3 Buffered streams and buffered readers

Buffering exists in both families, and the character version adds something the byte version cannot have: line awareness.

Table 6.1 — *The four buffering classes and what each adds.*

Class	Wraps	Adds	Characteristic method
`BufferedInputStream`	any `InputStream`	an internal byte array, default 8192 bytes	`mark()` / `reset()` become usable
`BufferedOutputStream`	any `OutputStream`	an internal byte array	`flush()` actually matters now
`BufferedReader`	any `Reader`	a character array *and* line splitting	`readLine()`
`BufferedWriter`	any `Writer`	a character array and platform line separators	`newLine()`

IMPORTANT DISTINCTION

`readLine()` exists only on `BufferedReader`. It is not on `Reader`, not on `FileReader`, and not on `InputStream`. This is why the standard idiom is `new BufferedReader(new FileReader(path))` — the outer class supplies the convenience method, the inner one supplies the source. BasinWatch's intake wraps its `PipedReader` in a `BufferedReader` for exactly this reason.

6.4 Data streams: typed values in a binary file

A character stream can store the text `"12.5"`. A data stream can store the *number* 12.5 — eight bytes in a defined format that any Java program can read back without parsing.

DEFINITION

DataInputStream / DataOutputStream — filter streams that read and write Java's primitive types and UTF-8 strings in a fixed binary format, independent of locale and of how the value would be printed.

Table 6.2 — *Data-stream methods and their exact sizes. These sizes are what makes the format self-describing.*

Method	Bytes written	Notes
`writeBoolean(boolean)`	1	A single byte, 0 or 1
`writeInt(int)`	4	Big-endian, most significant byte first
`writeLong(long)`	8	Big-endian
`writeDouble(double)`	8	IEEE 754 double-precision, big-endian
`writeUTF(String)`	2 + n	A two-byte length, then modified UTF-8 bytes — maximum 65535 bytes
`writeChars(String)`	2 × n	Two bytes per character, no length prefix

CORE CONCEPT

Every data-stream write has exactly one matching read, and the **order must be identical**. A file written as `writeInt` then `writeDouble` must be read as `readInt` then `readDouble`. There is no self-describing metadata inside a data stream — the sequence of calls *is* the schema. Change the order in one place and every subsequent read returns garbage, usually without an exception.

COMMON MISTAKE

Reading a data stream in the wrong order is the classic silent corruption. Because every method simply consumes the next n bytes, a `readDouble()` where a `readInt()` was expected does not throw — it returns a perfectly valid but completely wrong number. The only defence is to keep the write order and the read order adjacent in the codebase and to version the format explicitly, which BasinWatch does with `FORMAT_VERSION`.

6.5 A complete typed-record workflow

Here is the write side and the read side of one record, side by side. BasinWatch writes a header once and then one record per accepted observation.

```
// --- the header, written once when the archive is created ---
static final int MAGIC = 0x42574152;          // "BWAR"
static final int FORMAT_VERSION = 1;

output.writeInt(MAGIC);                       // 4 bytes: identifies the file
output.writeInt(FORMAT_VERSION);              // 4 bytes: which layout follows

// --- one record per accepted observation ---
output.writeLong(reading.tick());             // 8 bytes
output.writeUTF(reading.stationId());         // 2 + n bytes
output.writeUTF(reading.zoneId());            // 2 + n bytes
```

```
output.writeDouble(reading.rainfallMillimeters());    // 8 bytes
output.writeDouble(reading.observedRiseCentimeters()); // 8 bytes
```

from BasinWatch, io/SensorArchive.java — the write order defines the format.

```
// --- reading the header back, and validating it ---
int magic = input.readInt();
if (magic != MAGIC) {
    throw new IOException("Sensor archive has an unexpected signature.");
}
int version = input.readInt();
if (version != FORMAT_VERSION) {
    throw new IOException("Unsupported sensor archive version: " + version);
}

// --- reading records until the file ends ---
while (true) {
    try {
        long tick = input.readLong();
        String stationId = input.readUTF();
        String zoneId = input.readUTF();
        double rainfall = input.readDouble();
        double rise = input.readDouble();
        // ... build a SensorReading and report it
    } catch (EOFException end) {
        break;                                  // a clean end of file, not an error
    }
}
```

from BasinWatch, io/SensorArchive.java — the matching read order, with EOF as the terminator.

Four things in those listings are worth naming, because they are the difference between a toy binary format and a usable one.

A magic number identifies the file. The constant 0x42574152 spells "BWAR" in ASCII. If someone points the reader at a different file, the first read fails immediately with a clear message rather than producing nonsense.

A version number allows the format to change. When the layout gains a field, the version is incremented and the reader can decide what to do. Without it, an old reader would silently misinterpret a new file.

End of file is the terminator. There is no record count, so the loop reads until `EOFException` and treats that as the normal end. This is why the record must be written completely: a half-written record produces an `EOFException` in the middle of a record, which the reader cannot distinguish from a clean end unless it tracks how far it got.

The strings use writeUTF. Each string carries its own length, so there is no ambiguity about where one ends and the next begins. That is precisely what `writeChars` cannot offer.

> **CONNECTION TO ANOTHER CONCEPT**
>
> Compare this with Chapter 8's session snapshot, which stores whole objects rather than typed primitives. Both are binary, both are byte streams — but the data stream gives you a format you control field by field, while object serialization gives you a format the JDK controls and that you must actively secure. Neither is universally better; the archive chose control, the snapshot chose convenience plus validation.

6.6 Where each wrapper belongs

Choosing a chain is a small design decision you make constantly. The pattern is always the same: start with the source or destination, then add one wrapper per capability you need.

Table 6.3 — *Common wrapper chains and the reason for each layer.*

Task	Chain	Why
Read a large file quickly, byte by byte	`BufferedInputStream` → `FileInputStream`	Buffering removes the per-byte system call.
Read a text file line by line	`BufferedReader` → `FileReader`	`readLine()` lives on the buffered reader.
Read typed numbers from a binary file quickly	`DataInputStream` → `BufferedInputStream` → `FileInputStream`	Typing on the outside, buffering in the middle, the file at the core.
Write a log line by line	`BufferedWriter` → `FileWriter`	Line methods plus batching, with an explicit charset.
Build bytes in memory then hand them to a socket	`DataOutputStream` → `ByteArrayOutputStream`	Chapter 7 covers the in-memory destination.

EXAM TIP

If asked "why wrap a `FileInputStream` in a `BufferedInputStream`?", the precise answer is about call frequency, not speed in the abstract: buffering reduces the number of system calls by roughly the ratio of buffer size to read size, and each system call has a fixed cost that dominates for small reads. Saying "it's faster" without the mechanism usually loses marks.

CHAPTER SUMMARY

- A buffer is a memory array between your code and the resource, turning many small operations into few large ones.
- Wrapper classes take another stream in their constructor and add exactly one capability — the decorator pattern.
- Reading travels upward through the chain; writing travels downward. Each layer knows only the layer beneath it.
- `readLine()` exists only on `BufferedReader`, which is why buffering is not optional for line-based text.
- Data streams write primitives in a fixed binary format whose write order defines the schema; the matching read order must be identical.
- A usable binary format has a magic number, a version, self-describing strings via `writeUTF`, and a defined end condition.
- A `BufferedOutputStream` makes `flush()` significant — without it, bytes may sit in memory.

KEY TERMS

- **Decorator pattern** — adding behaviour by wrapping an object in another object of the same interface.
- **Filter stream** — a wrapper stream; its constructor takes the stream it wraps.
- **Magic number** — a constant at the start of a file identifying its format.
- **Format version** — a number that lets a reader detect an incompatible layout.
- **writeUTF** — a data-stream method that writes a two-byte length followed by modified UTF-8 bytes.

CHAPTER 7

In-Memory and Piped Streams

> Streams whose source is memory, and streams that connect two threads. This chapter covers byte-array streams, piped streams, and the producer–consumer pattern.

7.1 Why a stream with no file?

Not every stream is attached to a device. Two important sources and destinations are entirely inside the program: a byte array in memory, and a pipe connecting two threads.

The in-memory case is useful because it lets you reuse the whole stream API for something that never touches a disk. Building a payload with `DataOutputStream` gives you the same typed writes as a file, and the result is a `byte[]` you can hand to a network call, a cache or a test assertion.

The piped case is useful because it solves a real problem: two threads that must exchange a stream of data without sharing a data structure.

7.2 Byte-array streams

A `ByteArrayOutputStream` collects everything written to it in an internal growable array; `toByteArray()` hands you the contents. A `ByteArrayInputStream` reads from an array you already have.

```
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.IOException;

public class InMemoryPayload {
    public static void main(String[] args) throws IOException {
        byte[] payload;

        // write typed values into memory
        try (ByteArrayOutputStream buffer = new ByteArrayOutputStream();
             DataOutputStream out = new DataOutputStream(buffer)) {
            out.writeInt(7);                // a tick
            out.writeUTF("MIL");            // a zone
            out.writeDouble(42.5);          // rainfall
            out.flush();
            payload = buffer.toByteArray();
        }

        System.out.println("payload size = " + payload.length + " bytes");

        // read them back, with no file anywhere
        try (DataInputStream in = new DataInputStream(new ByteArrayInputStream(payload))) {
            System.out.println("tick   = " + in.readInt());
            System.out.println("zone   = " + in.readUTF());
            System.out.println("rain   = " + in.readDouble());
        }
    }
}
```

illustrative — the same typed format, entirely in memory

Expected output, and the arithmetic behind the size:

```
payload size = 17 bytes
tick   = 7
zone   = MIL
rain   = 42.5
```

VISUAL WALKTHROUGH

Seventeen bytes is not a coincidence. `writeInt` contributes 4, `writeUTF("MIL")` contributes 2 for the length plus 3 for the characters, and `writeDouble` contributes 8. Four plus five plus eight is seventeen. If you can predict that number before running the program, you understand the format.

IMPORTANT DISTINCTION

Closing a byte-array stream has no effect on the array. There is no operating-system resource to release, so `close()` is a no-op. It is still worth calling — through try-with-resources — because it means the same code shape works when the destination later becomes a file or a socket. BasinWatch's archive code is written so that only the innermost source changes.

7.3 Piped streams

A pipe is a matched pair: a writer at one end, a reader at the other, with a bounded buffer between them. What the writer writes, the reader reads, in order.

DEFINITION

Piped stream — a connected pair of streams, `PipedWriter`/`PipedReader` or `PipedOutputStream`/`PipedInputStream`, that transfers data from one thread to another through an internal buffer.

Diagram 9 — Piped streams: a producer, a consumer, and a thread boundary

SensorFeed and SensorIntake run on different threads and never call each other directly.

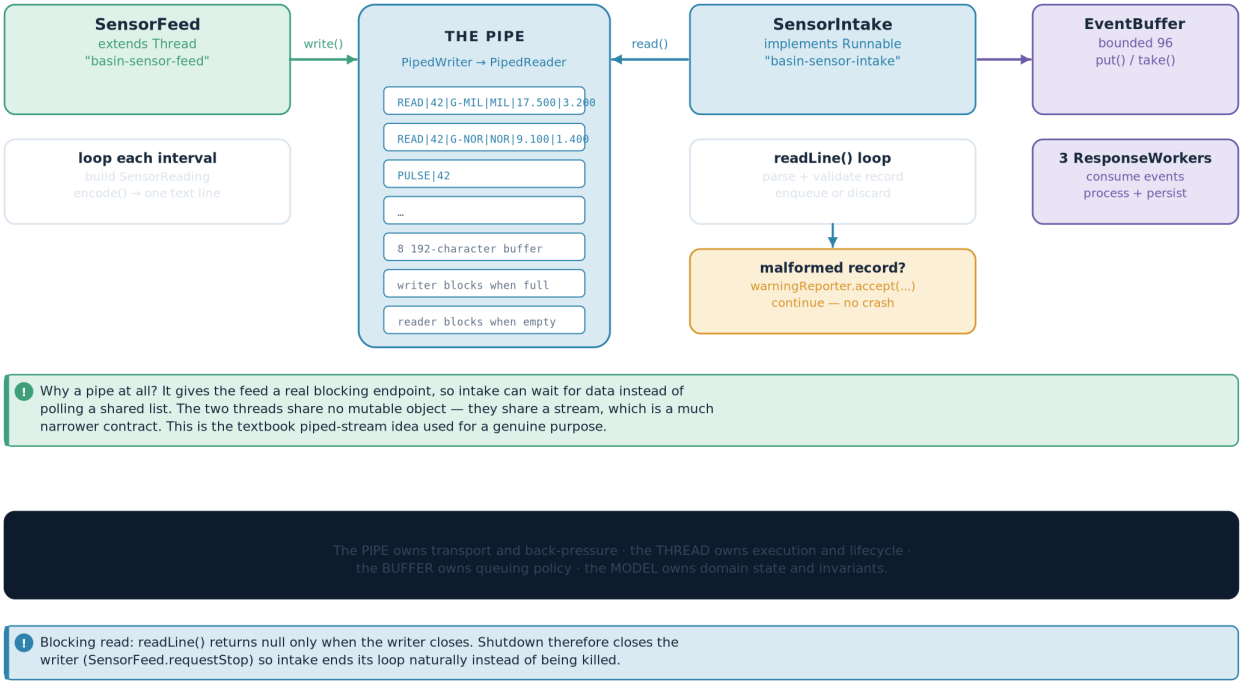


Diagram 9 — The piped-stream producer and consumer. SensorFeed writes encoded observation lines into a PipedWriter. SensorIntake reads them from a PipedReader wrapped in a BufferedReader, validates each line, and places accepted events into the bounded EventBuffer. The pipe provides the thread boundary; the buffer provides the queue.

Three properties of that arrangement matter, and each is a design decision.

The pipe is the only shared object. The producer never sees the buffer, the queue or the model. The consumer never sees the producer. Each thread's dependencies are visible in its constructor.

The pipe is bounded, so it provides back-pressure. BasinWatch constructs it with an 8,192-character buffer. If the producer runs faster than the consumer, the producer blocks on `write` instead of filling memory. This is the same property a bounded queue has, obtained for free.

Both ends must be closed for the reader to finish cleanly. The writer closing is what turns the reader's next `readLine()` into a `null`, ending the loop. If the writer never closes, the reader waits forever.

> COMMON MISTAKE
>
> **A pipe is not automatically thread-safe in the sense people assume.** A `PipedReader` and its `PipedWriter` are intended to be used by *different* threads. Using both ends in one thread is a classic deadlock: the reader blocks waiting for data that only the same thread could write. If you need a stream in one thread, use a byte-array stream.

7.4 The producer side

The producer is a thread with a loop, a cadence and a stop flag. Its job is to encode a value and write it as one line.

```
public final class SensorFeed extends Thread {
    private final Writer destination;
    private final List<String> zoneIds;
    private final SimulationSettings settings;
    private final AtomicLong tickSequence;
    private final Consumer<String> failureReporter;
    private volatile boolean running = true;

    @Override
    public void run() {
        Random random = new Random();
        try {
            while (running && !isInterrupted()) {
                long tick = tickSequence.incrementAndGet();
                for (String zoneId : zoneIds) {
                    SensorReading reading = new SensorReading(tick, stationId(zoneId), zoneId,
                            round1(random.nextDouble() * settings.rainfallMillimeters()),
                            round1(random.nextDouble() * settings.riseCentimeters()));
                    destination.write(reading.encode());
                    destination.write(System.lineSeparator());
                }
                destination.flush();
                Thread.sleep(settings.tickIntervalMillis());
            }
        } catch (InterruptedException ex) {
            Thread.currentThread().interrupt();      // restore the flag and stop
        } catch (IOException ex) {
            failureReporter.accept("Sensor feed stopped: " + ex.getMessage());
        } finally {
            closeQuietly();
        }
    }
}
```

from BasinWatch, engine/SensorFeed.java — encode one line per observation, then flush.

Five details in that loop are worth studying, because each one is a concurrency decision.

`running` is volatile. It is written by one thread (the caller of `requestStop()`) and read by another (this loop). Without `volatile`, the reader could cache a stale value and never stop.

The loop checks both the flag and the interrupt. Either mechanism alone is enough, but checking both means a stop request works whether it arrives between iterations or during the sleep.

`InterruptedException` restores the flag. Catching the exception clears the thread's interrupt status, so `Thread.currentThread().interrupt()` puts it back. This is the correct handling and it is easy to get wrong.

The line separator is platform-independent at the source. Using `System.lineSeparator()` means the encoding matches what `readLine()` expects on the same machine.

The writer is closed in a finally block. That is what makes the consumer's loop terminate, even if the producer stopped because of an error.

7.5 The consumer side

The consumer reads lines, validates them and passes accepted events onward. It is a `Runnable` rather than a thread, because the engine owns the thread.

```
public final class SensorIntake implements Runnable {
    private final BufferedReader source;
    private final EventBuffer buffer;
    private final Consumer<String> warningReporter;
```

```
    private final Consumer<String> statusReporter;

    @Override
    public void run() {
        statusReporter.accept("Sensor intake online");
        try (source) {
            String line;
            while ((line = source.readLine()) != null) {
                BasinEvent event;
                try {
                    if (line.startsWith("PULSE|")) {
                        event = new SimulationPulse(Long.parseLong(line.substring(6)));
                    } else {
                        event = SensorReading.parse(line);
                    }
                } catch (IllegalArgumentException ex) {
                    warningReporter.accept("Discarded malformed sensor record: " + ex.getMessage());
                    continue;                        // one bad line must not stop the stream
                }
                if (!buffer.put(event)) {
                    warningReporter.accept("Event buffer closed; intake stopping.");
                    break;
                }
            }
        } catch (IOException ex) {
            warningReporter.accept("Sensor intake ended with an I/O error: " + ex.getMessage());
        } catch (InterruptedException ex) {
            Thread.currentThread().interrupt();
        } finally {
            statusReporter.accept("Sensor intake offline");
        }
    }
}
```

from BasinWatch, engine/SensorIntake.java — read, parse, report, enqueue.

CONNECTION TO ANOTHER CONCEPT

Note the interaction of three mechanisms in twelve lines. The pipe supplies characters (`readLine()`). Parsing supplies validation (`SensorReading.parse` throws on bad data). The bounded queue supplies hand-off to the workers (`buffer.put` blocks when full). Chapter 3 named these as three separate mechanisms; here they are one loop.

7.6 Wiring the pair, and shutting it down

Creating a connected pipe requires the writer to exist before the reader, and the reader's constructor to receive the writer. That ordering is why the engine builds both in one method.

```
private void startFeed() throws IOException {
    PipedWriter writer = new PipedWriter();
    PipedReader reader = new PipedReader(writer, 8_192);   // explicit buffer size
    feedWriter = writer;
    intakeThread = new Thread(new SensorIntake(new BufferedReader(reader), eventBuffer,
            this::reportFailure, ignored -> publishDiagnosticsStatus()), "basin-sensor-intake");
    sensorFeed = new SensorFeed(writer, List.of("UPR", "MIL", "NOR", "OLD", "MAR", "SOU"),
            settings, tickSequence, this::reportFailure);
    intakeThread.start();
    sensorFeed.start();
}

// --- and the shutdown, in the order that lets the consumer finish ---
public synchronized void closeEngine() {
```

```
    try {
        if (sensorFeed != null) {
            sensorFeed.requestStop();          // 1. stop producing
        }
        if (feedWriter != null) {
            feedWriter.close();                // 2. closing the writer ends readLine()
        }
        if (intakeThread != null) {
            intakeThread.join(2_000);          // 3. wait for the consumer to drain
        }
        eventBuffer.close();                   // 4. wake any waiting workers
        // ... archive drain, journal flush, final snapshot
    } catch (IOException | InterruptedException ex) {
        Thread.currentThread().interrupt();
    }
}
```

from BasinWatch, engine/BasinEngine.java — creation order and shutdown order are both deliberate.

The shutdown sequence is the part students most often get wrong, so it is worth stating as a rule. The four steps are ordered by dependency: you must stop the producer before closing the pipe, close the pipe before waiting for the consumer (otherwise the consumer blocks forever), and close the queue after the consumer has drained it (otherwise accepted events are lost).

Shutdown ordering, and what breaks if it is wrong

1. **Stop the producer.** `requestStop()` sets the flag; the feed loop exits at the next check.
2. **Close the writer.** This flushes any buffered characters into the pipe and signals end of stream.
3. **Join the intake thread.** The consumer's `readLine()` now returns null, the loop ends, and the thread finishes. A bounded join with a timeout means shutdown cannot hang forever.
4. **Close the buffer.** Waiting workers are woken by `notifyAll` and see the closed flag, so they exit instead of waiting for events that will never come.
5. *If you close the writer before stopping the producer:* the producer throws IOException on its next write and reports a spurious failure.
6. *If you close the buffer before joining the consumer:* events already read from the pipe are discarded, so the log and database lose the last few observations.
7. *If you never close the writer:* the consumer blocks in `readLine()` forever and the application will not exit.

7.7 Reading a file, an in-memory payload, and a pipe

The three cases look almost identical at the call site, which is the point of the stream abstraction. Only the constructor differs.

Table 7.1 — *The same read loop, three different sources.*

	From a file	From memory	Through a pipe
Source	`Files.newBufferedReader(path, UTF_8)`	`new ByteArrayInputStream(bytes)`	`new BufferedReader(new PipedReader(writer))`
Blocking behaviour	blocks on disk read	never blocks — data is already present	blocks until the writer produces data
End of stream	when the file ends	when the array is exhausted	when the writer closes
Resource to release	an OS file handle	nothing	

	From a file	From memory	Through a pipe
			a thread waiting at the other end
Thread safety	one reader per stream	safe if the array is not mutated	designed for two different threads
Used in BasinWatch	restoring a session snapshot, listing reports	building a payload before writing	the live sensor feed

EXAM TIP

A frequently asked question is "why does Java need a separate `PipedReader` when a shared queue would do?" The strongest answer covers three points: the pipe enforces a one-directional contract so neither thread can reach the other's state; the pipe's buffer gives back-pressure without extra code; and the pipe's close protocol gives a natural end-of-stream signal, which a queue must simulate with a sentinel value.

CHAPTER SUMMARY

- A byte-array stream uses the whole stream API with no device attached; closing it releases nothing.
- A pipe is a matched writer/reader pair with a bounded internal buffer, intended for two different threads.
- The pipe is the only object the producer and consumer share, which narrows both contracts.
- A bounded pipe provides back-pressure: a fast producer blocks instead of exhausting memory.
- The producer must close the writer for the consumer's read loop to terminate.
- Shutdown order is stop producer, close writer, join consumer, close queue — each step depends on the previous.
- The same read loop works for a file, an in-memory array and a pipe; only the constructor changes.

KEY TERMS

- **Back-pressure** — a bounded channel making a fast producer wait rather than buffer without limit.
- **End of stream** — in a pipe, the signal produced by closing the writer.
- **volatile** — a field modifier guaranteeing that a write by one thread becomes visible to another.
- **Interrupt** — a cooperative request for a thread to stop; it does not force termination.
- **Bounded join** — waiting for a thread with a timeout so shutdown cannot hang indefinitely.

CHAPTER 8

Serialization, Deserialization and Session Persistence

> How objects are turned into bytes and back, why that is dangerous by default, and how a real application makes it safe.

8.1 The problem: saving an object graph

A data stream can save a `double` because a double has a fixed size and layout. But what about an object? An object has fields of different types, may contain other objects, may contain collections of objects, and may have a class hierarchy. Writing it out field by field is tedious and breaks whenever a field is added.

Object serialization solves this by letting the runtime do the traversal. You declare that a class is serializable, and the JVM writes the object's class identity and its non-transient field values, recursing into any objects those fields reference.

DEFINITION

Serialization — the process of converting an object into a byte sequence that can be written to a stream and later reconstructed, preserving the object's class and field values.

DEFINITION

Deserialization — the reverse process: reading that byte sequence and reconstructing an object graph, including the classes it contains.

8.2 How it works

Two classes do the work, and they are both filter streams, so they follow exactly the pattern from Chapter 6.

```
import java.io.*;

public class SerializeDemo {
    public static void main(String[] args) throws IOException, ClassNotFoundException {
        Path path = Path.of("demo", "sample.ser");

        // ---- serialization: object -> bytes -> file ----
        try (ObjectOutputStream out = new ObjectOutputStream(
                new BufferedOutputStream(Files.newOutputStream(path,
                        StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING)))) {
            out.writeObject(new Sample("MIL", 2.45, 12));
            out.flush();
        }

        // ---- deserialization: file -> bytes -> object ----
        try (ObjectInputStream in = new ObjectInputStream(
                new BufferedInputStream(Files.newInputStream(path)))) {
            Object restored = in.readObject();
            System.out.println("restored = " + restored);
        }
    }
```

```
    static final class Sample implements Serializable {
        private static final long serialVersionUID = 1L;
        private final String zone;
        private final double level;
        private final int residents;
        Sample(String zone, double level, int residents) {
            this.zone = zone; this.level = level; this.residents = residents;
        }
        @Override public String toString() {
            return zone + " " + level + " m / " + residents + " residents";
        }
    }
}
```

illustrative — the minimal serialization round trip

Expected output:

```
restored = MIL 2.45 m / 12 residents
```

Diagram 10 — Serialization, validation and session restoration

How a live object graph becomes a safe file and comes back as a validated model.

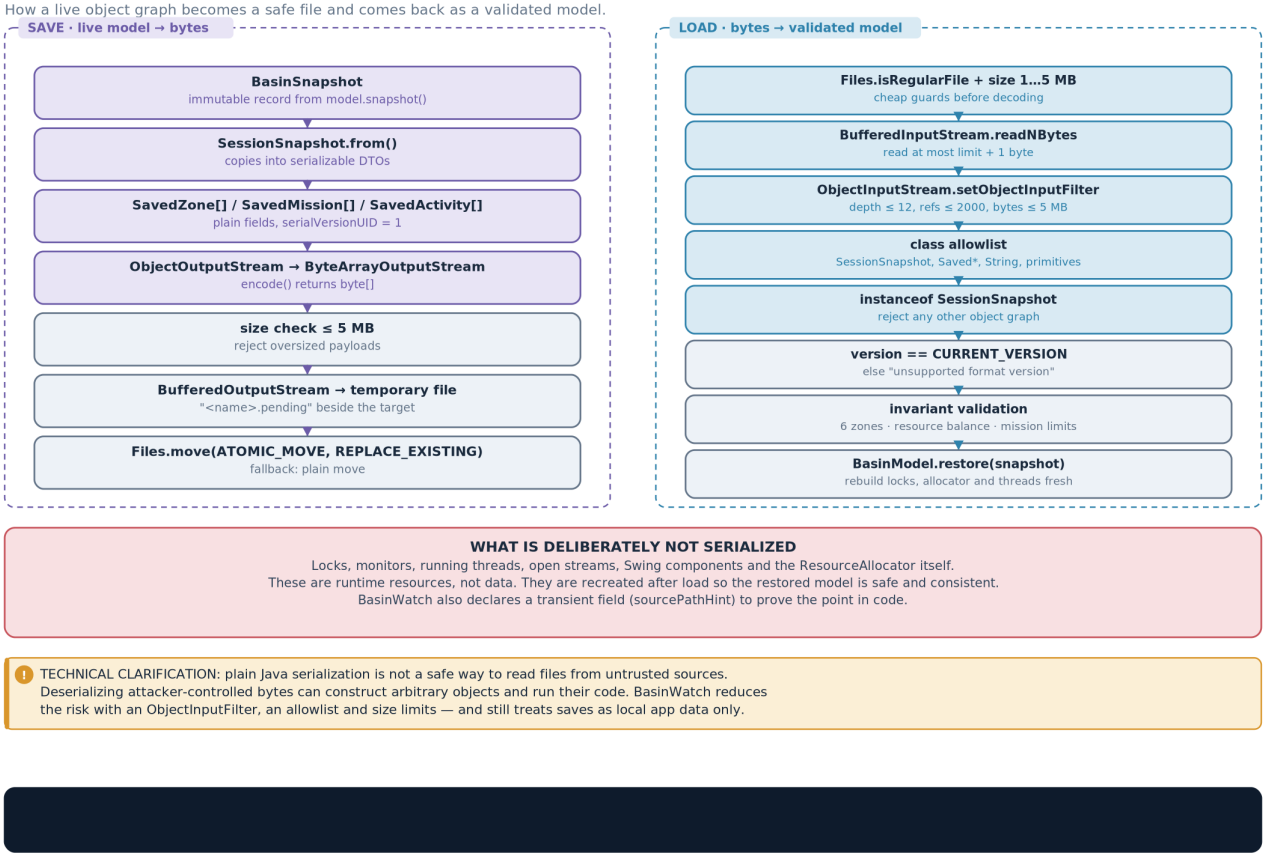


Diagram 10 — Serialization and deserialization. An object graph becomes a byte stream headed by the class descriptor and the serialVersionUID, then a field-value sequence. Deserialization reverses it: read the descriptor, load and check the class, then allocate and populate the object without calling its constructor.

8.3 The five facts that matter

1. A class must implement `Serializable`. It is a marker interface: it declares no methods. Its only purpose is to permit the runtime to write the object. Any class that does not implement it, and is not an ancestor, causes `NotSerializableException` at write time.

2. Fields must be serializable too, or marked transient. Serialization recurses, so every non-transient field must itself be serializable or a primitive. A `Thread`, an open `Stream`, a `Connection` — none of these can be serialized, and trying produces an exception naming the offending field.

3. `transient` excludes a field. A transient field is skipped entirely and, on deserialization, receives its default value: `null`, 0 or `false`. This is how you exclude a cache, a lock or a derived value.

4. `static` fields are not serialized. They belong to the class, not the object, so they are not part of the instance state being written.

5. The constructor is not called on deserialization. The object is allocated and its fields are set directly by the runtime. This is why the no-argument constructor that serialization conventionally requires need not do anything, and why validation placed in a constructor will *not* run when an object is restored.

CORE CONCEPT

Fact five is the one that surprises people, and it is the reason a real application cannot simply trust the objects it reads. If a constructor enforces "water level must be between 0 and 8", that rule is bypassed entirely by deserialization. BasinWatch therefore validates the restored state explicitly after reading it, rather than assuming the invariants held.

8.4 serialVersionUID and version compatibility

Every serializable class has an identity that the stream records. If you declare it yourself, the compiler uses your value. If you do not, the compiler derives one from the class's structure — its name, its modifiers, its interfaces and every method and field signature.

Table 8.1 — *Why an explicit `serialVersionUID` is worth declaring.*

Situation	Without an explicit serialVersionUID	With an explicit serialVersionUID
Add a harmless method	The derived UID changes	Nothing changes
Change a field's type	The derived UID changes	Deserialization fails with a field-type error
Add a new field	The derived UID changes	The new field receives its default value — old files still load
Remove a field	The derived UID changes	The removed field is ignored — old files still load
Recompile the same source	The derived UID is stable for the same compiler	Stable by construction

COMMON MISTAKE

Relying on the computed `serialVersionUID`. Two builds of the "same" class on different JDK versions or with different compilers can produce different values, and then every saved file becomes unreadable with `InvalidClassException: local class incompatible`. Declaring the constant explicitly is one line and removes a whole class of support problems.

8.5 Why default serialization is unsafe

Deserialization reconstructs an object graph from bytes that may come from anywhere. The runtime will happily instantiate whatever classes the stream names. That is a security problem, not a theoretical one: a crafted stream can instantiate classes you never intended, and the "gadget chains" that made several widely reported vulnerabilities possible all relied on exactly this.

DEFINITION

Deserialization vulnerability — the risk that a malicious byte stream causes the runtime to instantiate and initialise classes of the attacker's choosing, because deserialization bypasses constructors and honours the class names in the stream.

There are four independent defences, and a robust implementation uses all of them rather than picking one.

Table 8.2 — *The four defences, all of which BasinWatch implements.*

Defence	What it prevents	How it is expressed
Bound the input size	Denial of service by an enormous or deeply nested stream	Check the file length against a constant before opening the stream
Filter the classes that may be constructed	Instantiating an attacker-chosen class	`ObjectInputFilter` via `setObjectInputFilter`
Validate after reading	Accepting a structurally valid but semantically impossible object	Explicit range and consistency checks on the restored values
Check a format version	Reading a file written by an incompatible build	A version field written in the header and compared on read

8.6 How BasinWatch saves and restores a session

The project's snapshot store is a careful implementation of the four defences, and reading it is the best way to see them in practice. It writes to a temporary file and then replaces the real file atomically, so a crash during a save cannot leave a half-written snapshot in place.

```
public final class SnapshotStore {
    static final int FORMAT_VERSION = 1;
    static final long MAX_SAVE_BYTES = 4L * 1024 * 1024;   // 4 MB is generous for this model

    public void save(Path target, SessionSnapshot snapshot) throws IOException {
        Files.createDirectories(target.toAbsolutePath().getParent());
        Path temporary = target.resolveSibling(target.getFileName() + ".tmp");
        try (ObjectOutputStream out = new ObjectOutputStream(new BufferedOutputStream(
                Files.newOutputStream(temporary, StandardOpenOption.CREATE,
                        StandardOpenOption.TRUNCATE_EXISTING)))) {
            out.writeInt(FORMAT_VERSION);      // written first, so a reader can check it early
            out.writeObject(snapshot);
            out.flush();
        }
        try {
            Files.move(temporary, target, StandardCopyOption.REPLACE_EXISTING,
                    StandardCopyOption.ATOMIC_MOVE);       // atomic replace
        } catch (AtomicMoveNotSupportedException ex) {
            Files.move(temporary, target, StandardCopyOption.REPLACE_EXISTING);
        }
```

```
    }
}
```

from BasinWatch, io/SnapshotStore.java — write to a temporary file, then move it into place.

Two things in the write path are deliberate. The version is written *before* the object, so a reader can reject an incompatible file without attempting to interpret it. And the save goes to `name.bws.tmp` first, with the real file replaced only when the write has succeeded. If the process dies mid-save, the previous good snapshot is still intact.

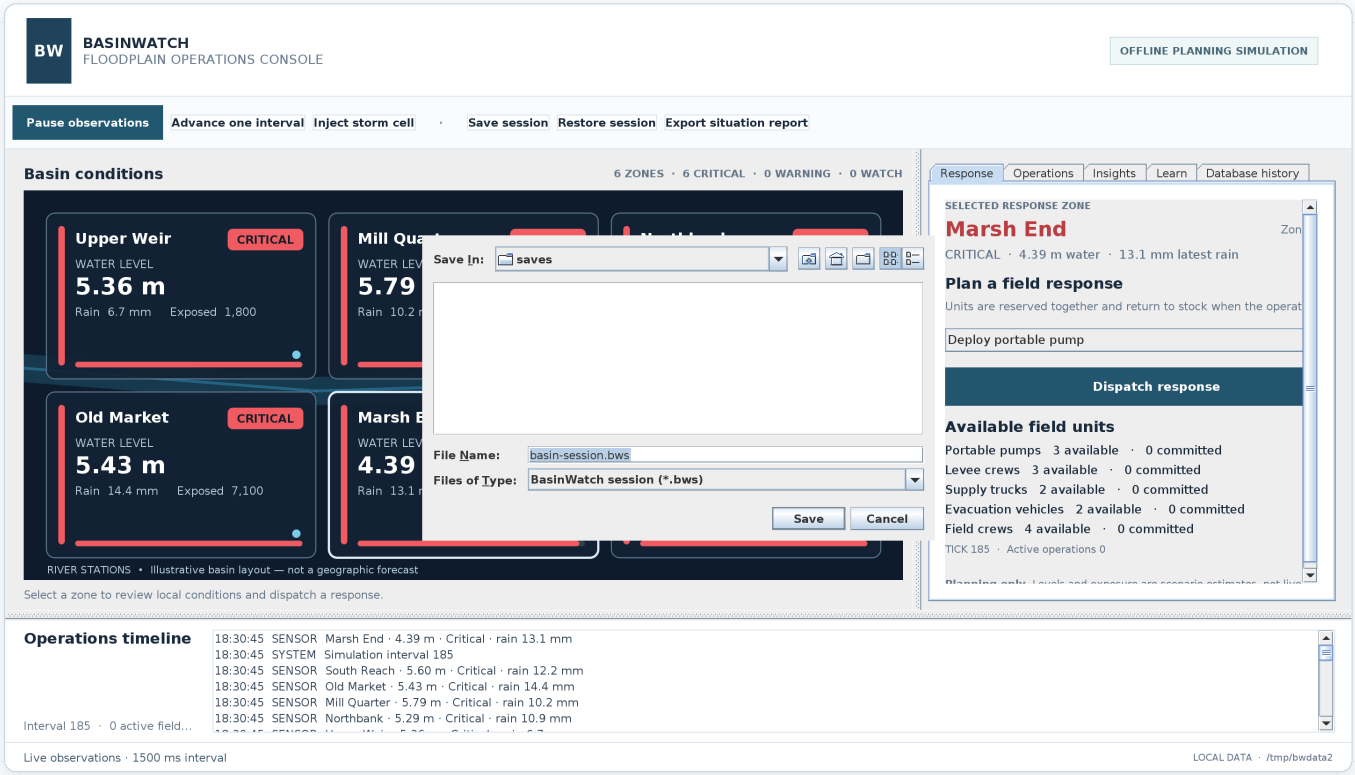
```
public SessionSnapshot load(Path source) throws IOException, ClassNotFoundException {
    if (!Files.isRegularFile(source)) {
        throw new IOException("Session save not found: " + source.toAbsolutePath());
    }
    if (Files.size(source) > MAX_SAVE_BYTES) {              // defence 1: bound the input
        throw new IOException("Session save is unexpectedly large: " + Files.size(source) + " bytes");
    }
    try (ObjectInputStream in = new ObjectInputStream(new BufferedInputStream(
            Files.newInputStream(source)))) {
        in.setObjectInputFilter(SnapshotStore::allowOnlySafeClasses);   // defence 2: filter classes
        int version = in.readInt();
        if (version != FORMAT_VERSION) {                    // defence 4: version check
            throw new IOException("Unsupported session format version: " + version);
        }
        Object payload = in.readObject();
        if (!(payload instanceof SessionSnapshot snapshot)) {
            throw new IOException("Session save does not contain a session snapshot.");
        }
        snapshot.validate();                                // defence 3: validate the domain state
        return snapshot;
    }
}

private static ObjectInputFilter.Status allowOnlySafeClasses(ObjectInputFilter.FilterInfo info) {
    Class<?> type = info.serialClass();
    if (type == null || type.isPrimitive()) {
        return ObjectInputFilter.Status.ALLOWED;
    }
    String name = type.getName();
    if (name.startsWith("com.basinwatch.") || name.startsWith("java.lang.")
            || name.startsWith("java.util.") || name.startsWith("java.time.")) {
        return ObjectInputFilter.Status.ALLOWED;
    }
    return ObjectInputFilter.Status.REJECTED;               // everything else is refused
}
```

from BasinWatch, io/SnapshotStore.java — all four defences in one method.

VISUAL WALKTHROUGH

Read the `load` method as four gates. Gate one is the size check, before the stream is even opened. Gate two is the filter, which runs on every class the stream tries to construct. Gate three is the version check, immediately after the header. Gate four is `snapshot.validate()`, after the object exists. A file must pass all four to be accepted.



Screenshot S4 — Choosing a destination for a serialized session. The *Save session* button opens a real `JFileChooser` whose file filter is `BasinWatch session (*.bws)` and whose suggested name is `basin-session.bws`. Confirming writes through `SnapshotStore.save`: the bytes go to a temporary file first and the real file is replaced only after a successful close, which is the atomic-replacement pattern described above. The resulting file begins with the bytes `AC ED 00 05` — the standard Java serialization magic number — followed by the stream header and the class descriptor for `com.basinwatch.io.SessionSnapshot`.

You can verify the format claim yourself, and it is a good habit. Running `xxd basin-session.bws | head -3` on a real save produced by the application shows:

```
00000000: aced 0005 7372 0021 636f 6d2e 6261 7369  ....sr.!com.basi
00000010: 6e77 6174 6368 2e69 6f2e 5365 7373 696f  nwatch.io.Sessio
00000020: 6e53 6e61 7073 686f 7400 0000 0000 0000  nSnapshot.......
```

output of xxd on a real BasinWatch session save — the aced0005 magic number and the SessionSnapshot class descriptor.

> **CONNECTION TO ANOTHER CONCEPT**
>
> The same `AC ED 00 05` header appears in every Java serialization stream, from a student exercise to a production cache. It is also why serialization is a security topic: any process that deserializes an untrusted stream is willing to construct whatever classes that stream names. The `ObjectInputFilter` above is the modern defence, and it is why the book shows the filter rather than treating serialization as a purely mechanical topic.

8.7 What validation has to check

Because deserialization skips constructors, the restored object can be structurally valid and semantically impossible. The validation method is where the invariants are re-established.

```
public void validate() {
    if (tick < 0) {
        throw new IllegalStateException("Session tick cannot be negative.");
    }
    if (zones == null || zones.size() != 6) {
```

```
            throw new IllegalStateException("A session must describe exactly six zones.");
        }
        for (SavedZone zone : zones) {
            if (zone == null || zone.waterLevelMeters() < 0.0 || zone.waterLevelMeters() > 8.0) {
                throw new IllegalStateException("Session contains an impossible water level.");
            }
        }
        if (missions == null || resources == null || recentActivity == null) {
            throw new IllegalStateException("Session is missing a required section.");
        }
        for (Map.Entry<ResourceType, Integer> entry : resources.entrySet()) {
            if (entry.getValue() == null || entry.getValue() < 0) {
                throw new IllegalStateException("Session contains a negative resource count.");
            }
        }
    }
```

from BasinWatch, io/SessionSnapshot.java — the invariants the constructors would have enforced.

Notice that every check here corresponds to a rule that a constructor would normally apply: a non-negative tick, exactly six zones, water levels inside the physical range, a non-negative inventory. That correspondence is not a coincidence — it is the whole point. Validation after deserialization exists to re-apply the rules that deserialization bypassed.

> **CONNECTION TO ANOTHER CONCEPT**
>
> Compare this with the sensor archive in Chapter 6. The archive also has a magic number and a version, but it never deserializes objects, so it needs no class filter — the format is a fixed sequence of primitives that cannot name a class. That is a real advantage of the data-stream approach and a real reason to prefer it where the shape is known.

8.8 Serialization versus the alternatives

Object serialization is convenient and, as the previous section showed, needs care. Knowing when it is the wrong tool is part of understanding it.

Table 8.3 — *Five persistence formats and the question each answers.*

Approach	Stores	Readable by	Best for	Used in BasinWatch for
Text with a character stream	Characters	a human, and any text tool	Logs, reports, configuration	The operational journal and reports
Data streams	Typed primitives in a fixed order	a Java program that knows the format	Measurement records, wire formats	The sensor archive
Object serialization	An object graph with class descriptors	a Java program with the same classes	Whole-state save and restore	Session snapshots
A database	Rows and columns	any SQL client	Selective queries over many records	Optional SQLite history
JSON or XML	A structured text document	almost anything	Interchange with other languages	**Not used** — see Chapter 18

EXAM TIP

Questions on serialization cluster around four points: what makes a class serializable; what `transient` and `static` do; why the constructor is bypassed; and what `serialVersionUID` is for. The constructor point is the one that separates a memorised answer from an understood one — if you can explain that the runtime allocates the object without invoking a constructor, and therefore that validation must be repeated explicitly, you have answered all four coherently.

CHAPTER SUMMARY

- Serialization converts an object graph into bytes; deserialization reconstructs it.
- A class must implement `Serializable`, and every non-transient field must be serializable.
- `transient` and `static` fields are not written; a transient field restores to its default value.
- Deserialization bypasses constructors, so constructor validation does not run on restore.
- Declaring `serialVersionUID` explicitly prevents unrelated recompiles from invalidating saved files.
- Default deserialization is unsafe: bound the size, filter the permitted classes, check a version, and validate the restored state.
- BasinWatch writes to a temporary file and moves it into place, so an interrupted save cannot corrupt an existing snapshot.

KEY TERMS

- **Serializable** — a marker interface permitting an object to be serialized.
- **serialVersionUID** — the explicit version identity of a serializable class.
- **transient** — a modifier excluding a field from serialization.
- **ObjectInputFilter** — a hook that approves or rejects each class a deserialization stream tries to construct.
- **Atomic move** — replacing a file in a single step, so no intermediate state is observable.

CHAPTER 9

How BasinWatch Uses Java I/O

> Four file mechanisms, four different purposes. This chapter consolidates Part II into a verified case study of the project's file layer.

9.1 The design question, stated properly

An application that needs to keep data could write everything to one file. BasinWatch uses four different file mechanisms, and the reason is that each answers a different question.

Diagram 11 — In-memory state, persistent files and a database

Three different answers to "where does the data live?" — and why a real application wants all three.

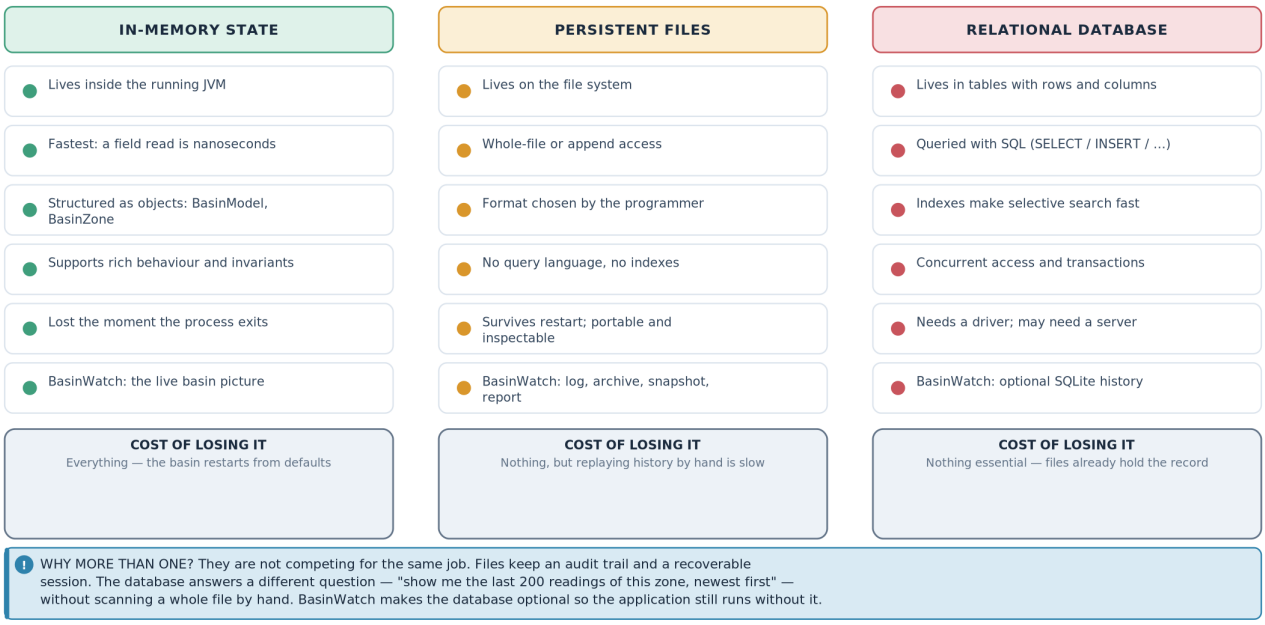


Diagram 11 — Three persistence mechanisms, three questions. File persistence split by the question it answers: a readable log for humans, a typed archive for measurements, and a versioned snapshot for whole-state recovery. The database column is the subject of Part III.

Table 9.1 — *The four file artefacts and the question each one answers.*

Artefact	The question it answers	Mechanism	Who reads it
`operations.log`	“What has the application been doing?”	Character stream, UTF-8, append-only, line per event	An operator, or a text editor
`sensors.bin`	“What exactly did the stations report?”	Byte stream with data-stream typing, header plus fixed records	The program, during diagnosis
`*.bws`	“How do I get the whole situation back?”	Object serialization of a versioned snapshot DTO, atomic replace	The program, on restore

Artefact	The question it answers	Mechanism	Who reads it
`*.txt` reports	“What is the situation right now?”	Character stream, UTF-8, one document per request	A person, outside the application

CORE CONCEPT

None of these four is interchangeable with the others, and that is the design lesson of Part II. A log cannot restore a session because it records events, not state. A snapshot cannot answer "what happened at 14:03" because it records only the present. An archive cannot be read by a person because it is binary. A report cannot be queried because it is prose. Choosing the format is choosing which questions the file will be able to answer.

9.2 Where the files live

All paths are derived from one place, so the layout is defined once and cannot drift.

```
public final class AppPaths {
    private static final Path ROOT = Path.of("data");

    public static Path logsDirectory() { return ROOT.resolve("logs"); }
    public static Path logFile() { return logsDirectory().resolve("operations.log"); }
    public static Path archiveDirectory() { return ROOT.resolve("archive"); }
    public static Path archiveFile() { return archiveDirectory().resolve("sensors.bin"); }
    public static Path savesDirectory() { return ROOT.resolve("saves"); }
    public static Path lastSessionFile() { return savesDirectory().resolve("last-session.bws"); }
    public static Path reportsDirectory() { return ROOT.resolve("reports"); }

    public static void ensureDirectories() throws IOException {
        for (Path directory : List.of(logsDirectory(), archiveDirectory(),
                savesDirectory(), reportsDirectory())) {
            Files.createDirectories(directory);
        }
    }
}
```

from BasinWatch, io/AppPaths.java — one place that defines the on-disk layout.

This small class is more important than it looks. Because every path comes from a method, there are no string literals scattered through the codebase, and a test can reason about the layout without touching the file system. It is the file-system equivalent of a schema.

9.3 The operational log

This is the human-facing record. It is a character stream, it appends, and it flushes on a policy.

```
data/logs/operations.log
2026-10-18T18:25:45.551894012Z | SYSTEM | BasinWatch opened
2026-10-18T18:26:29.394373823Z | SYSTEM | Sensor network connected
2026-10-18T18:26:29.396289783Z | OBSERVATION | Mill Quarter · 1.37 m · Routine · rain 7.5 mm
2026-10-18T18:26:32.401882145Z | OBSERVATION | Upper Weir · 2.04 m · Watch · rain 11.2 mm
2026-10-18T18:28:37.333076452Z | DISPATCH REQUEST | BW-0001 dispatched · Deploy portable pump · Marsh End
2026-10-18T18:37:10.011962040Z | OBSERVATION | Upper Weir · 8.00 m · Critical · rain 15.8 mm
```

from BasinWatch — the actual line format written by OperationalJournal. This is verbatim output from a real session, not a reconstruction. The three categories the project uses are SYSTEM, OBSERVATION and DISPATCH REQUEST.

Four properties make this file useful, and each was a decision.

The timestamp is ISO-8601 with a Z suffix. That is `DateTimeFormatter.ISO_INSTANT`, which sorts lexicographically in the same order as it sorts chronologically. A log that sorts correctly as text is a log you can grep.

The category is a separate field. Because it is delimited by `" | "`, an operator can filter for `RISK` or `ALLOCATED` with ordinary text tools.

Newlines inside a message are replaced. Chapter 5 covered this: it keeps one entry on one line, which is what makes the file greppable in the first place.

Flushing is category-aware. Routine observations can wait; warnings, completions and allocations are written through immediately, because those are the entries an investigation would want after a crash.

9.4 The typed sensor archive

Where the log is for reading, the archive is for measurement. It stores exactly what each station reported, in a format that cannot drift.

```
data/archive/sensors.bin
[ 4 bytes ] magic     0x42574152  ("BWAR")
[ 4 bytes ] version   1
   --- one record per accepted observation ---
[ 8 bytes ] tick                    long
[ 2+n     ] stationId               UTF
[ 2+n     ] zoneId                  UTF
[ 8 bytes ] rainfallMillimeters     double
[ 8 bytes ] observedRiseCentimeters double
```

from BasinWatch — the sensor archive layout, as written by SensorArchive.

VISUAL WALKTHROUGH

Count the bytes of a real record and you get a feel for why the format is compact. A tick is 8 bytes. The two identifiers the simulated feed uses are `G-UPR` and `UPR`, costing 2+5 and 2+3 bytes, so 12. Two doubles cost 16. The whole record is 36 bytes, plus the 8-byte header once at the start of the file. A text version of the same record would be roughly 45 characters and would need parsing and locale-independent number formatting to read back — and it could not distinguish a `double` from an `int` without a convention.

9.5 The session snapshot

The snapshot is the only artefact that can bring the application back to where it was. It is written on request, and automatically on a clean shutdown, into `last-session.bws`.

```
public static SessionSnapshot from(BasinSnapshot snapshot) {
    ArrayList<SavedZone> zones = new ArrayList<>();
    for (ZoneSnapshot zone : snapshot.zones()) {
        zones.add(new SavedZone(zone.id(), zone.name(), zone.waterLevelMeters(),
                zone.lastRainMillimeters(), zone.exposedResidents(),
                zone.leveeReinforced(), zone.lastReadingTick()));
    }
    ArrayList<SavedMission> missions = new ArrayList<>();
    for (MissionSnapshot mission : snapshot.activeMissions()) {
        missions.add(new SavedMission(mission.id(), mission.type().name(), mission.zoneId(),
```

```
                mission.startedTick(), mission.completesAtTick(), mission.status().name()));
    }
    return new SessionSnapshot(snapshot.tick(), List.copyOf(zones),
            Map.copyOf(snapshot.resources()), List.copyOf(missions),
            List.copyOf(snapshot.recentActivity()));
}
```

from BasinWatch, io/SessionSnapshot.java — a deliberate copy from immutable snapshots into serializable DTOs.

IMPORTANT DISTINCTION

Why a separate `SessionSnapshot` rather than serializing `BasinSnapshot` directly? Three reasons, and all three matter. First, the domain types may contain things that cannot or should not be serialized. Second, the on-disk format should be a deliberate contract rather than an accident of the current field list — a DTO lets the model change without changing the file. Third, records and enums carry serialization constraints that plain final classes avoid. The DTO is a boundary object, and boundary objects should be explicit.

9.6 Reports

A report is a document, not a record. It is written once, read once, and never parsed.

```
BASINWATCH · SITUATION REPORT
Generated: 2026-10-18T18:30:50.581292677Z
Simulation interval: 188

ZONE CONDITIONS
UPR | Upper Weir | 5.44 m | 12.9 mm rain | Critical | 1800 exposed
MIL | Mill Quarter | 5.83 m | 3.0 mm rain | Critical | 5200 exposed
NOR | Northbank | 5.37 m | 9.2 mm rain | Critical | 2400 exposed
OLD | Old Market | 5.52 m | 14.9 mm rain | Critical | 7100 exposed
MAR | Marsh End | 4.45 m | 4.8 mm rain | Critical | 1100 exposed
SOU | South Reach | 5.66 m | 8.5 mm rain | Critical | 3100 exposed

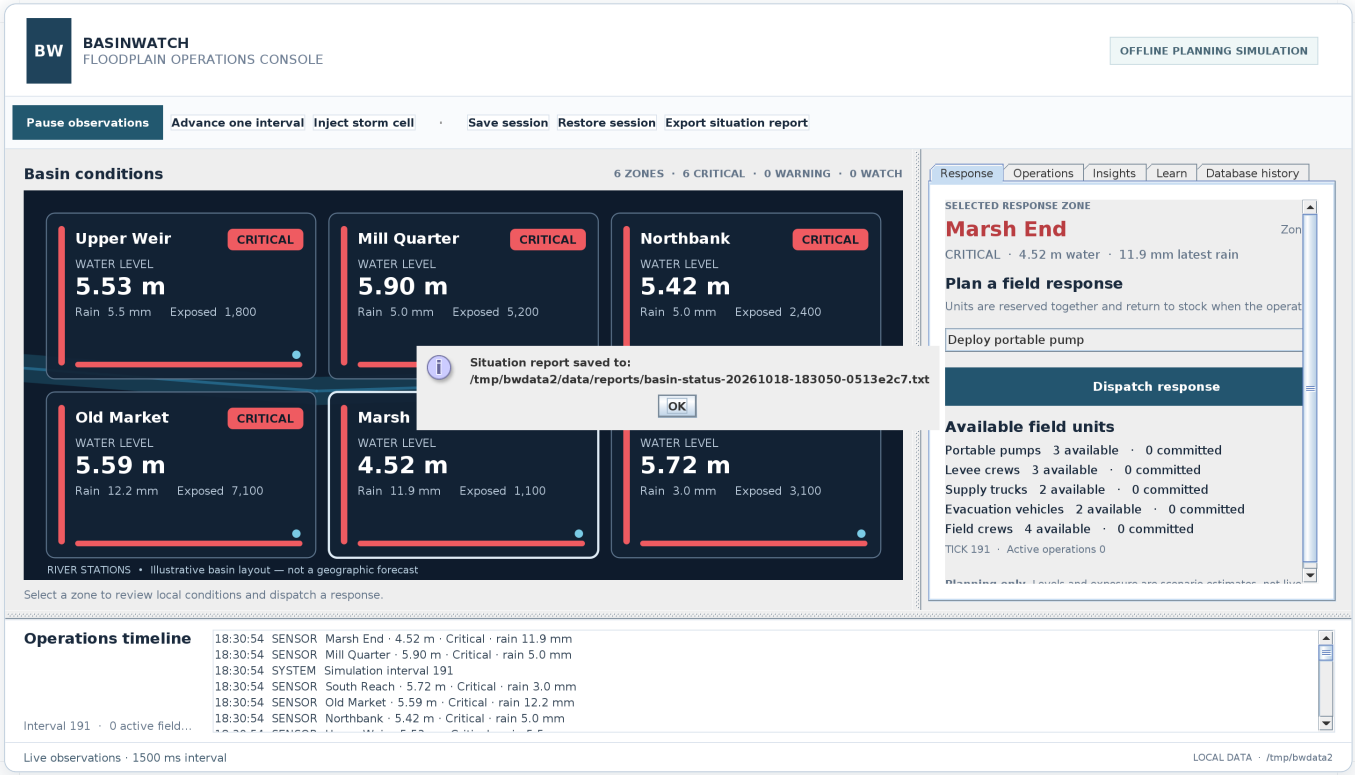
AVAILABLE RESPONSE RESOURCES
Portable pumps: 3
Levee crews: 3
Supply trucks: 2
Evacuation vehicles: 2
Field crews: 4

RESPONSE OPERATIONS
BW-0001 | Deploy portable pump | MAR | Completed | 100%

RECENT ACTIVITY
18:30:17  SENSOR  Northbank · 4.82 m · Critical · rain 3.0 mm
18:30:17  SENSOR  Old Market · 4.96 m · Critical · rain 14.6 mm
18:30:17  SENSOR  Marsh End · 3.97 m · Critical · rain 10.3 mm
18:30:17  SYSTEM  Simulation interval 166
18:30:18  SENSOR  Mill Quarter · 5.38 m · Critical · rain 4.7 mm
```

from BasinWatch — the exact structure produced by ReportWriter.write. This is verbatim output, not a reconstruction.

The report duplicates information that exists elsewhere, and that is correct. Its value is that it is complete, self-contained and readable without the application. An operator can attach it to an incident note; a reviewer can read it a year later. A format optimised for machine reading would be worse at exactly that job.



Screenshot S5 — Exporting a situation report from the running application. Clicking *Export situation report* calls `engine.exportReport()`, which returns a `CompletableFuture<Path>`. The write happens off the event-dispatch thread and the confirmation above appears only once the future completes, showing the real path `/tmp/bwdata2/data/reports/basin-status-20261018-183050-0513e2c7.txt`. The filename embeds the timestamp and a short random suffix, which is why two reports generated in the same second still get distinct names. Notice that the dialog is modal but the application was never frozen: the interface stayed responsive while the file was written, because the work was submitted rather than performed on the UI thread.

Diagram 12 — Which BasinWatch class uses which stream, and why

Nine I/O classes, one purpose each. Choosing the wrong one changes the meaning of the data.

Class	Stream family	What it stores	Why that family
AppPaths	java.nio.file.Path / Files	directory layout and the database path	Paths are metadata, not content
SaveCatalog	java.io.File	list of *.bws saves, newest first	Listing needs metadata only
OperationalJournal	BufferedWriter (Writer)	timestamped audit lines, UTF-8	Humans must read it
ReportWriter	BufferedWriter (Writer)	a formatted situation report	A report is text by definition
SensorArchive	DataOutputStream	typed tick / station / rain / rise	Values must keep their types
ArchiveWorker	ArrayBlockingQueue + Thread	off-thread archive writes	Disk latency must not block intake
SnapshotStore	ObjectOutputStream	the whole session object graph	Whole-object state capture
SensorFeed	PipedWriter (Writer)	encoded observation lines	Blocking hand-off to intake
SensorIntake	BufferedReader (Reader)	parsed and validated events	readLine() matches the format

Notice the deliberate split: readable text for people, typed binary for machines, object serialization for state, and metadata for discovery. This is the practical answer to "which persistence mechanism should I use?"

Diagram 12 — Which class uses which stream, and why. Every stream class in the project mapped to the class that uses it and the reason that stream was the right choice. The mapping is the argument of this chapter in one picture.

9.7 The four mechanisms side by side

Table 9.2 — *The four file mechanisms compared on the properties that matter.*

	Operational log	Sensor archive	Session snapshot	Report
Stream family	character (Writer)	byte (DataOutputStream)	byte (ObjectOutputStream)	character (Writer)
Unit of data	one line per event	one fixed record per observation	one object graph	one document
Open mode	append	append, header if fresh	replace, via temporary file	replace, new name
Reader	a person	the program	the program	a person
Can restore state?	no	no	yes	no
Can be queried?	with text tools	only by the program	no	no
Versioned?	no — append-only	yes, in the header	yes, before the object	no
Failure policy	collect and report at the end	worker thread reports	propagate; the temp file is discarded	propagate to the caller

9.8 Why not one file for everything

It is worth answering the obvious objection directly. Suppose you wrote everything to the log as text. You would then face four problems.

1. **Restoring state would mean replaying.** To recover the model you would have to re-apply every event ever logged, in order, and hope the replay logic still matches the original.
2. **Numbers would lose precision.** A `double` printed as text and parsed back may not be bit-identical, and any aggregation over thousands of readings would accumulate error.
3. **Diagnosis would mean parsing.** A tool wanting "every reading for MIL above 20 mm" would have to parse the whole file rather than seek.
4. **One corrupt line would break everything.** A truncated final line in an append-only log is normal after a crash; if that log were also the only save file, the session would be unrecoverable.

Conversely, if everything were one binary snapshot, you would lose the audit trail entirely. The four-mechanism design exists because the questions are genuinely different.

> **EXAM TIP**
>
> This chapter's comparison table is one of the highest-yield things in the book for a viva. If asked "why does your project have both a log file and a database?", the answer follows directly: the log records what happened, in order, for a human; the database records structured rows that can be queried selectively. The log is an audit trail; the database is an index. Chapter 10 makes that distinction precise.

CHAPTER SUMMARY

- BasinWatch has four file artefacts because each answers a different question: audit, measurement, recovery, communication.
- All paths derive from `AppPaths`, so the on-disk layout is defined in exactly one place.
- The operational log is UTF-8 text, appended, with ISO-8601 timestamps and category-aware flushing.
- The sensor archive is binary with a magic number, a version and fixed typed records — compact and unambiguous, but unreadable by a person.
- The session snapshot serializes a deliberate DTO, not the live model, and writes through a temporary file.
- Reports are complete, self-contained documents that duplicate other data on purpose.
- None of the four mechanisms can substitute for another, because the format determines which questions the file can answer.

KEY TERMS

- **Audit trail** — an ordered record of what happened, for later inspection.
- **DTO (data transfer object)** — a deliberately separate object used at a boundary, such as persistence or the network.
- **Atomic replace** — writing to a temporary file then moving it into place so no partial state is visible.
- **Header** — metadata at the start of a binary file identifying its format and version.
- **Append-only** — a file opened so that existing content is never modified, only added to.

PART III

Databases and JDBC

A file records what happened; a database answers questions about it. This part explains why an application keeps both, how JDBC carries SQL from Java to a database and rows back, what a transaction guarantees, and how BasinWatch's optional SQLite history is actually built.

10 Why Databases and JDBC Exist

11 JDBC Architecture and Drivers

12 SQL Execution Through JDBC

13 Database Transactions and Reliable Persistence

14 How BasinWatch Uses JDBC and SQLite

CHAPTER 10

Why Databases and JDBC Exist

> Memory, files and structured storage. Why an application chooses more than one persistence mechanism, and where JDBC fits between Java code and a database.

10.1 Three places data can live

Data in a running Java program lives in three quite different places, and each has a different lifetime and a different set of capabilities.

Table 10.1 — *Three homes for data, compared.*

	In-memory state	Persistent files	A relational database
Lifetime	Until the process ends or the object is discarded	Until the file is deleted	Until the rows are deleted
Access speed	Nanoseconds — a field read	Microseconds to milliseconds	Microseconds to milliseconds, plus query planning
Can be searched selectively?	Only by writing code to iterate	Only by reading and filtering yourself	**Yes** — indexes and `WHERE` clauses
Concurrent access	Requires explicit synchronisation	Effectively one writer	Built-in locking and isolation
Partial update	A field assignment	Rewrite the region or the file	An `UPDATE` touching one row
Guarantees	None beyond the language	None beyond what you write	Atomicity, consistency, isolation, durability
Structure	Objects and references	Whatever format you invent	Tables, columns, types, constraints

CORE CONCEPT

The decisive difference is not speed. It is **selective access**. Reading one record out of ten million is trivial in a database and expensive in a file, because a database maintains an index and a file does not. Everything else — transactions, constraints, concurrency — follows from that.

10.2 Why BasinWatch uses more than one mechanism

BasinWatch keeps in-memory state, four kinds of file, and an optional SQLite database. That is not indecision; it is five different questions being asked of the same data.

Table 10.2 — *Six questions, and the mechanism that answers each best.*

The question being asked	Best mechanism	Why not the others
What is the water level in MIL right now?	In-memory model	

The question being asked	Best mechanism	Why not the others
		A file read or a query would be slower and would need synchronising.
What has the application been doing today?	Operational log file	A database would work, but a text file is readable without any tooling.
What exactly did station MIL-ST-2 report at 09:15?	Sensor archive	The log does not carry every field; a snapshot keeps only the present.
Restore the situation from yesterday's save.	Session snapshot	Replaying a log is fragile; a snapshot restores state directly.
Show me the 200 most recent readings for MIL, newest first.	**Database**	A log requires reading and filtering everything; an index and a `LIMIT` answer it directly.
How many missions completed in the last hour?	**Database**	Aggregation is one SQL expression over indexed rows; over a text file it is a full scan in application code.

IMPORTANT DISTINCTION

A database is not simply "a better file". For a single-user desktop application, adding a database adds a dependency, a driver, a schema, a migration concern and a failure mode. BasinWatch treats its database as *optional* precisely because the file layer already satisfies the essential requirements. The database earns its place by enabling queries, not by being more modern.

10.3 Relational structure in one page

A relational database stores data in **tables**. A table has named **columns** with declared **types** and zero or more **rows**. A **primary key** uniquely identifies a row; a **foreign key** references a row in another table, which is how relationships are expressed.

```
CREATE TABLE sensor_readings (
    id               INTEGER PRIMARY KEY AUTOINCREMENT,
    reading_tick     INTEGER NOT NULL,
    station_id       TEXT    NOT NULL,
    zone_id          TEXT    NOT NULL,
    rainfall_mm      REAL    NOT NULL,
    rise_cm          REAL    NOT NULL,
    water_level_m    REAL    NOT NULL,
    risk_level       TEXT    NOT NULL,
    risk_escalated   INTEGER NOT NULL,
    recorded_at      TEXT    NOT NULL
);

CREATE INDEX idx_sensor_readings_zone_time
    ON sensor_readings (zone_id, recorded_at DESC);
```

from BasinWatch, db/DatabaseInitializer.java — one table and the index that makes the history query fast.

Read that definition as a set of promises the database will enforce for you.

`PRIMARY KEY AUTOINCREMENT` gives every row a unique identity without the application inventing one. `NOT NULL` on every column means a missing value is rejected rather than silently stored as null. The types are declared, so `rainfall_mm` cannot hold the text "twelve".

The index is the important line. Without it, a query for one zone's recent readings must examine every row. With it, the database walks the index directly to the matching rows in the right order — which is exactly the query the interface makes.

10.4 SQL in four verbs

Almost everything an application does with a database is one of four operations, and the acronym CRUD names them.

Table 10.3 — *CRUD mapped to SQL, with the project's use of each.*

Operation	SQL	Returns	BasinWatch example
Create	`INSERT INTO table (columns) VALUES (?, ?)`	an update count	Store one accepted sensor reading
Read	`SELECT columns FROM table WHERE condition`	a result set	The 200 most recent history entries
Update	`UPDATE table SET column = ? WHERE condition`	an update count	Mark a mission as completed
Delete	`DELETE FROM table WHERE condition`	an update count	Not used — history is retained by design

RELATED TECHNIQUE — NOT USED IN BASINWATCH

BasinWatch never deletes from its history tables. For an audit log that is the right choice — you want to be able to explain what happened last month. Real systems handle retention with a time-based purge, an archival export, or partitioning. If you are asked in a viva "what about the database growing forever?", that is the honest answer: the current design trades disk space for a complete record, and a production system would add a retention policy.

10.5 Where JDBC fits

Java cannot speak SQL to a database directly. There is no shared memory, no common protocol, and each database vendor has its own wire format. JDBC is the layer that bridges the gap.

DEFINITION

JDBC — the Java Database Connectivity API: a standard set of interfaces in `java.sql` that let Java code open a connection, send SQL, and read results without depending on a particular database product.

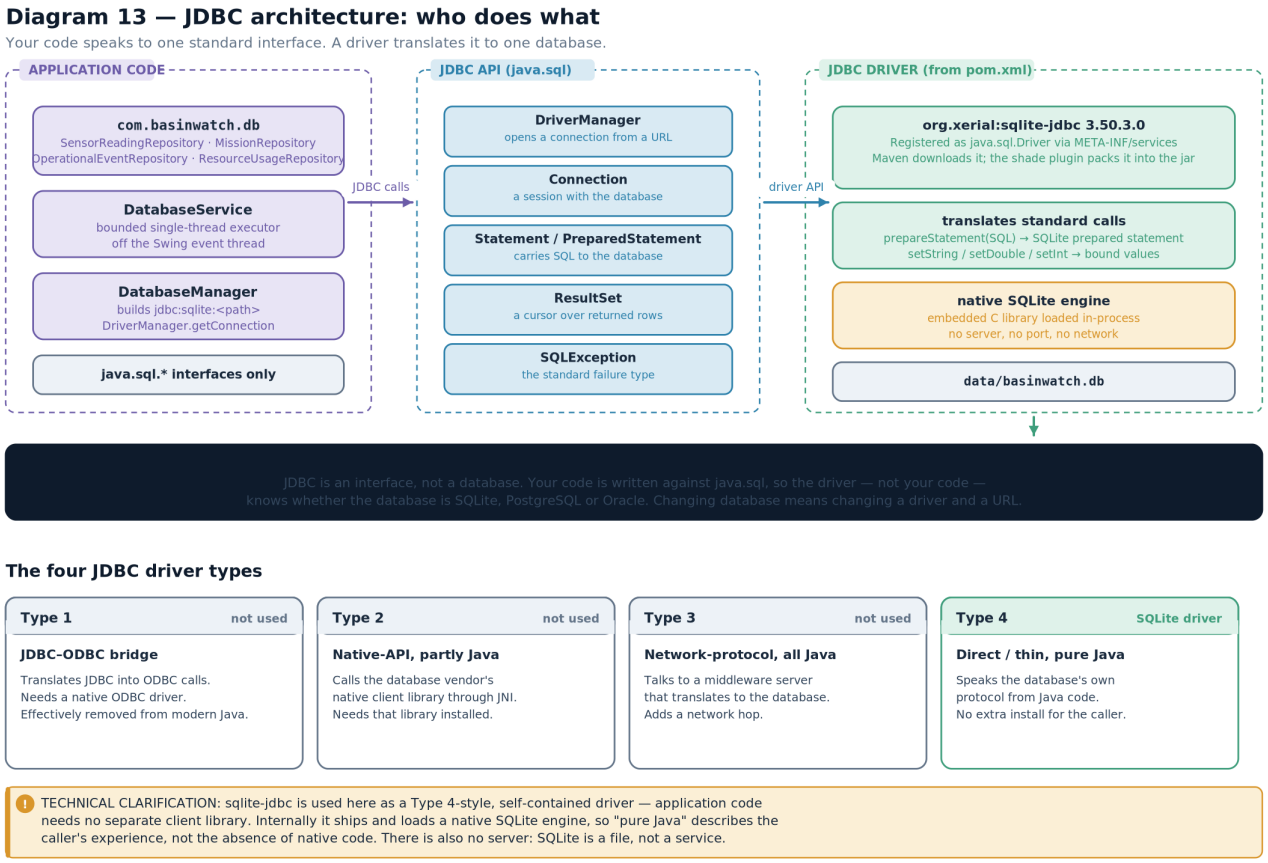


Diagram 13 — The JDBC architecture. Java application code calls only the java.sql interfaces. DriverManager selects a registered driver, the driver converts the calls into the database's own protocol, and the database executes the SQL. Results travel back through the same layers.

The key idea is that the application depends on *interfaces*, not on the database. Change from SQLite to PostgreSQL and the application code does not change — only the driver on the classpath and the connection URL do.

> **CONNECTION TO ANOTHER CONCEPT**
>
> Chapter 17 shows how that driver actually arrives on the classpath: as a Maven dependency. And Chapter 19 shows what happens when a framework like Spring wraps these same interfaces in a higher-level abstraction. JDBC is the foundation both approaches share.

10.6 Why SQLite specifically

SQLite is a complete relational database implemented as a library, not as a server. The database is a single file on disk, and the "connection" is a handle to that file. For an offline desktop application that is an ideal fit.

Table 10.4 — *Server databases and SQLite, and why the latter suits a desktop application.*

Property	Server database (PostgreSQL, MySQL)	SQLite
Process	A separate server process	Linked into your program as a library

Property	Server database (PostgreSQL, MySQL)	SQLite
Installation	Install, configure and start the server	Add one jar to the classpath
Connection	Network connection, host, port, credentials	A path to a file
Concurrent writers	Many, with row and table locking	One at a time — the whole file is locked for a write
Ideal for	Multi-user, network, high concurrency	Embedded, single-user, desktop, testing
Used by BasinWatch?	**No**	**Yes** — the only database the project uses

EXAM TIP

A common question asks why JDBC is described as a "thin" API. The answer is that it does very little itself: it defines interfaces and a driver-registration mechanism, and delegates almost all real work to the driver. That thinness is exactly why it works with any database and why frameworks can build on it.

CHAPTER SUMMARY

- Data can live in memory, in files, or in a database; the decisive difference is selective access via indexes.
- BasinWatch keeps all three because different questions need different mechanisms, not because one is superior.
- A relational table declares columns, types and constraints, and a primary key gives every row an identity.
- An index is what makes a query fast; without one, the database examines every row.
- CRUD maps to INSERT, SELECT, UPDATE and DELETE, and the project deliberately never deletes.
- JDBC is a set of interfaces plus a driver mechanism; the application depends on the interfaces, so the database can be swapped.
- SQLite is a library, not a server, which is why it suits an offline desktop application.

KEY TERMS

- **Table** — a named collection of rows sharing the same columns.
- **Primary key** — the column or columns uniquely identifying each row.
- **Index** — an auxiliary structure that speeds up lookups on particular columns.
- **Constraint** — a rule the database enforces, such as `NOT NULL`.
- **JDBC** — the standard Java API for relational database access.

CHAPTER 11

JDBC Architecture and Drivers

> The interfaces, the driver's job, the four driver types, and how a connection is actually established.

11.1 The two halves of JDBC

JDBC is deliberately split in two. One half is fixed and shipped with Java: the interfaces in `java.sql`. The other half is supplied by whoever makes the database: the driver, which implements those interfaces for one specific product.

Table 11.1 — *The JDBC interfaces and who implements them.*

Interface	Defined in	Implemented by	The application uses it to
`Driver`	`java.sql`	The database vendor	Accept a URL and decide whether it can handle it
`Connection`	`java.sql`	The vendor's driver	Hold a session with the database
`Statement`	`java.sql`	The vendor's driver	Send fixed SQL
`PreparedStatement`	`java.sql`	The vendor's driver	Send parameterized SQL, safely
`CallableStatement`	`java.sql`	The vendor's driver	Invoke a stored procedure
`ResultSet`	`java.sql`	The vendor's driver	Iterate over the rows a query returned
`SQLException`	`java.sql`	The vendor's driver	Report a database error
`DriverManager`	`java.sql`	The JDK	Choose a driver for a URL and hand back a connection

CORE CONCEPT

This is the single most important structural fact about JDBC, and it is worth stating plainly: **every method you call on a `Connection` is code you have never read**. The interface is in the JDK; the implementation is in the driver jar. That is why changing the driver can change behaviour, and why exact exception messages differ between databases.

11.2 What the driver actually does

A driver has four responsibilities.

1. **Recognise its own URLs.** Given `jdbc:sqlite:data/basinwatch.db`, the SQLite driver accepts it and other drivers decline.

2. **Open a connection.** For SQLite that means opening the file; for PostgreSQL it means a TCP connection, a handshake and authentication.
3. **Translate calls into the database's protocol.** A `PreparedStatement.setDouble(3, 12.4)` becomes whatever bytes that database expects.
4. **Translate results back.** A result row becomes a `ResultSet` positioned on that row, with typed getters.

VISUAL WALKTHROUGH

Follow Diagram 13 with those four responsibilities in mind. The application touches only the top band. `DriverManager` is the switchboard. The driver is the translator. Everything below it belongs to the database.

11.3 The four driver types

JDBC drivers are classified into four types by their architecture. The classification is a standard examination topic, and it is also genuinely useful for understanding why modern drivers are the way they are.

Table 11.2 — *The four JDBC driver types.*

Type	Name	How it works	Pros	Cons
1	JDBC–ODBC bridge	Java calls an ODBC driver through native code	Could reach any database with an ODBC driver	Deprecated and removed; required native code and a platform ODBC stack
2	Native-API driver	Java calls the vendor's native client library	Slightly faster than type 1	Requires the vendor's client library installed on every machine
3	Network-protocol driver	Java talks to a middleware server that talks to the database	One driver for many databases; useful behind a firewall	An extra server to install, configure and maintain
4	Thin driver	Pure Java, speaks the database's own protocol directly	**No native code, no middleware, one jar, portable**	One driver per database product

TECHNICAL CLARIFICATION

Type 1 is no longer available. The `jdbc-odbc` bridge was deprecated and then removed from the JDK; it does not exist on a modern Java runtime. If a lecture note presents it as a current option, the correct modern position is: it was type 1, it is historical, and types 3 and 4 are what remain in practice. Types 3 and 4 have largely converged, because modern databases provide efficient binary wire protocols that a thin driver can speak directly.

11.4 How a connection is established

Three things are needed: a URL, the driver on the classpath, and a call to `DriverManager`.

```
// the URL names the driver family, then whatever that driver needs
jdbc:sqlite:data/basinwatch.db
jdbc:postgresql://localhost:5432/basinwatch
jdbc:mysql://localhost:3306/basinwatch?useSSL=false

// opening a connection
String url = "jdbc:sqlite:data/basinwatch.db";
try (Connection connection = DriverManager.getConnection(url)) {
    // ... use the connection
}   // the connection is closed here
```

illustrative — the JDBC URL shape and the connection call

The URL is the only piece of configuration that names the database product, which is why changing databases is usually a change to one string and one dependency.

11.5 Driver registration

For `DriverManager` to find a driver, the driver class must be registered. There are three ways this can happen.

Table 11.3 — *Three ways a driver becomes available to `DriverManager`.*

Mechanism	How it works	When to use it
Service provider file	The driver jar contains `META-INF/services/java.sql.Driver` naming the class. The JDK loads it automatically.	**The normal case.** Modern drivers do this, so no code is needed.
`Class.forName(...)`	Explicitly loads the driver class, which registers itself in a static initialiser.	Legacy code, or a driver with no service file. Still seen in older textbooks.
`DriverManager.registerDriver(...)`	Registers a driver instance directly.	Unusual; used when constructing a driver manually.

COMMON MISTAKE

Writing `Class.forName("org.sqlite.JDBC")` in modern code. It is harmless but unnecessary: the SQLite driver ships a service file, so the JDK discovers it automatically. More importantly, the habit hides a real question — *is the driver actually on the classpath?* If it is not, `DriverManager` throws `SQLException: No suitable driver found for jdbc:sqlite:...`, and no amount of `forName` will help. The fix is a Maven dependency, not a line of code. Chapter 17 covers exactly that.

11.6 How BasinWatch manages connections

The project wraps connection handling in one small class, so no repository has to know the URL.

```
public final class DatabaseManager {
    private static final String DRIVER_CLASS = "org.sqlite.JDBC";
    private final String url;

    public DatabaseManager(Path databaseFile) {
        this.url = "jdbc:sqlite:" + databaseFile.toAbsolutePath();
    }

    public Connection openConnection() throws SQLException {
```

```
        Connection connection = DriverManager.getConnection(url);
        connection.setAutoCommit(true);            // each statement commits on its own
        return connection;
    }

    public boolean isAvailable() {
        try (Connection connection = openConnection()) {
            return connection.isValid(2);          // two-second validity check
        } catch (SQLException ex) {
            return false;                          // absence is reported, not thrown
        }
    }

    public String url() {
        return url;
    }
}
```

from BasinWatch, db/DatabaseManager.java — one place that knows the URL.

Three decisions in that class shape everything downstream.

The URL is built once, in the constructor. No other class contains the string `"jdbc:sqlite:"`.

`isAvailable` returns a boolean rather than throwing. This is what makes the database optional. The application asks "is history available?" and handles both answers, instead of being forced to handle an exception merely to find out.

Autocommit is set explicitly. It is the default, but stating it documents the decision — and Chapter 13 shows the one place where it is deliberately turned off.

CONNECTION TO ANOTHER CONCEPT

Notice that `isAvailable` catches `SQLException` and converts it into `false`. That is an exception-translation decision, and Chapter 14 explains why the rest of the application depends on it: an unavailable database must not prevent the application from running.

EXAM TIP

Two questions recur. "What does the driver do?" — answer with the four responsibilities: recognise the URL, open the connection, translate calls to the protocol, translate results back. "Which driver type does your project use?" — type 4, a pure-Java thin driver, which is why it needs only a jar on the classpath and no native library.

CHAPTER SUMMARY

- JDBC is split between fixed interfaces in `java.sql` and a vendor-supplied driver that implements them.
- A driver recognises its own URLs, opens connections, translates calls to the database protocol, and translates results back.
- The four driver types are the ODBC bridge (removed), native-API, network-protocol middleware, and the pure-Java thin driver.
- A connection needs a URL of the form `jdbc:product:details` and a driver on the classpath.
- Modern drivers register themselves through a service file, so `Class.forName` is no longer required.
- “No suitable driver found” means the driver jar is missing, not that registration code is missing.
- BasinWatch builds the URL in one place and reports database availability as a boolean rather than an exception.

KEY TERMS

- **JDBC URL** — a string of the form `jdbc:product:details` identifying a database to connect to.
- **Thin driver** — a pure-Java type 4 driver that needs no native code.
- **DriverManager** — the JDK class that selects a registered driver for a URL and returns a connection.
- **Service provider file** — `META-INF/services/java.sql.Driver`, which lets a driver register itself automatically.
- **Autocommit** — the mode in which each statement is its own transaction.

CHAPTER 12

SQL Execution Through JDBC

> Connection, Statement, PreparedStatement, CallableStatement, ResultSet, CRUD operations, and the complete execution lifecycle.

12.1 The objects in one query

Executing a query involves a small cast of objects, each with one job. It is worth meeting them before looking at any code.

Table 12.1 — *The objects involved in executing SQL.*

Object	Job	Created by	Closed by
`Connection`	A session with the database	`DriverManager.getConnection(url)`	try-with-resources
`Statement`	A container for fixed SQL	`connection.createStatement()`	try-with-resources
`PreparedStatement`	A pre-compiled statement with placeholders	`connection.prepareStatement(sql)`	try-with-resources
`CallableStatement`	A stored-procedure invocation	`connection.prepareCall(sql)`	try-with-resources
`ResultSet`	A cursor over the rows a query returned	Returned by `executeQuery()`	try-with-resources
`SQLException`	A database failure	Thrown by any of the above	Caught by the caller

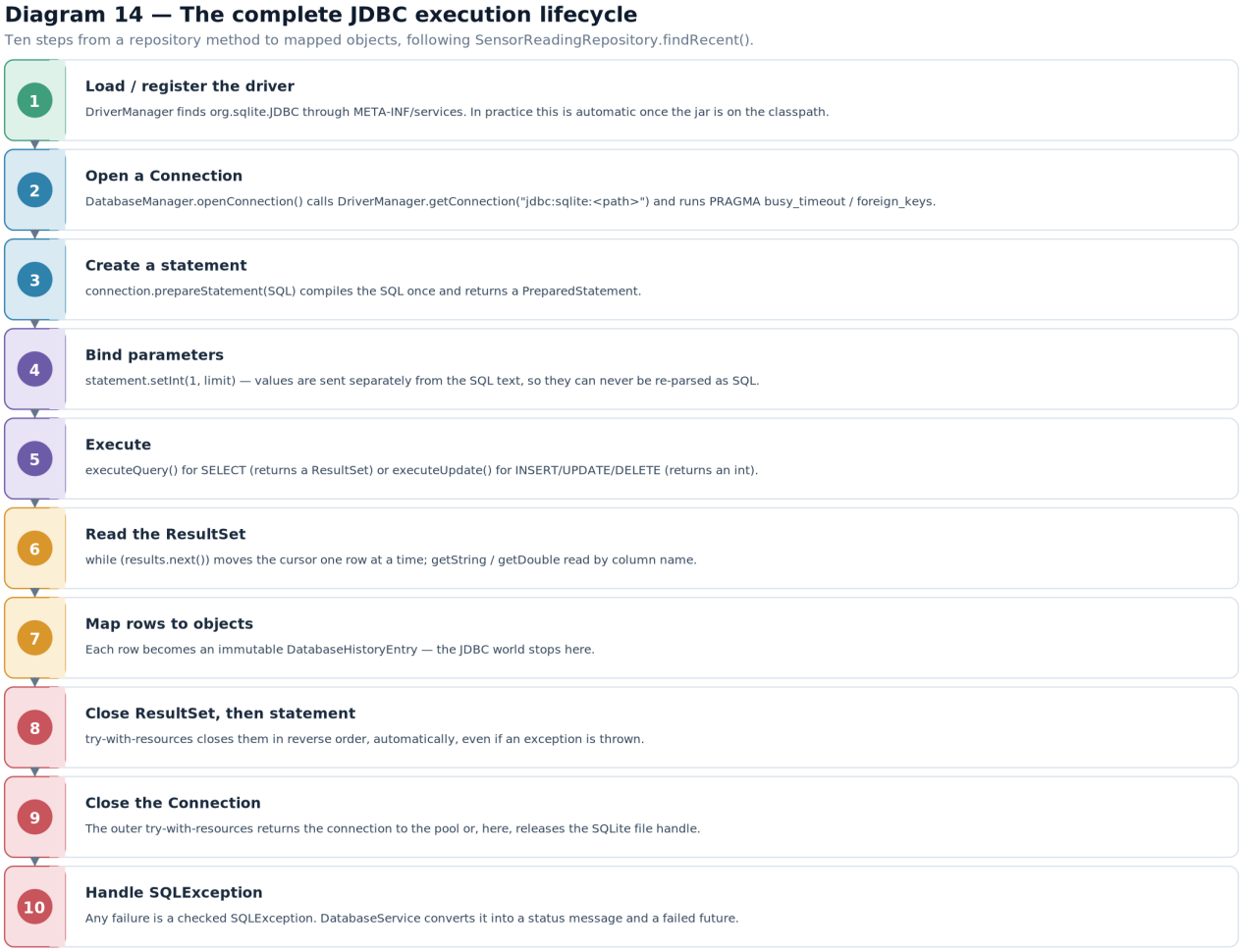


Diagram 14 — The SQL execution lifecycle. From obtaining a connection through preparing a statement, binding parameters, executing, iterating the result set, and closing every resource in reverse order. The two execution paths — query and update — diverge after execution and converge again at cleanup.

12.2 Three kinds of statement

All three statement types implement the same interface, but they exist for different reasons and one of them is the right default.

Table 12.2 — *The three statement types and when each is appropriate.*

	Statement	PreparedStatement	CallableStatement
SQL	Fixed, assembled in Java	Fixed template with ? placeholders	A stored procedure call
Parameters	None — values are concatenated into the string	Bound with typed setters	Bound as input, and registered as output
Pre-compiled by the database?	No — parsed on every execution	**Yes** — parsed once, executed many times	Yes
Safe against SQL injection?	**No**	**Yes**	**Yes**
Efficient in a loop?	No — reparsed each time	**Yes**	Yes

	Statement	PreparedStatement	CallableStatement
Typical use	A one-off DDL statement such as `CREATE TABLE`	Everything the application does repeatedly	Calling a procedure that returns values

Diagram 15 — Statement, PreparedStatement and CallableStatement

One interface family, three levels of capability.

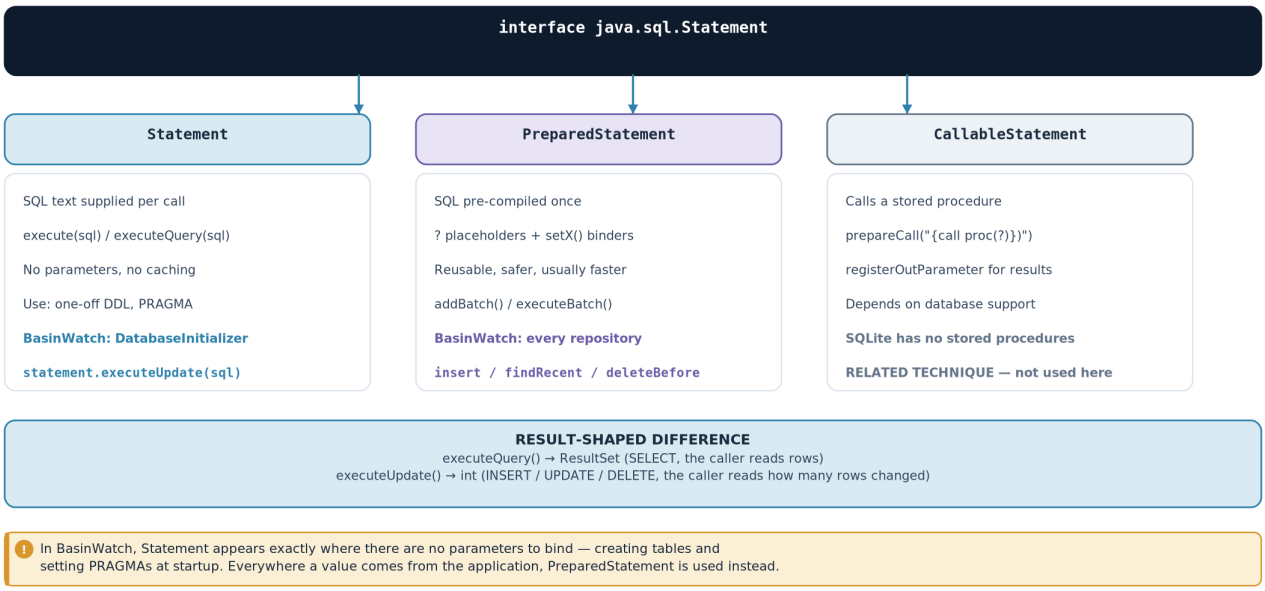


Diagram 15 — Statement, PreparedStatement and CallableStatement. One interface family, three levels of capability. Statement sends fixed SQL with no parameters; PreparedStatement adds numbered placeholders and pre-parsing; CallableStatement adds stored-procedure invocation with registered output parameters.

CORE CONCEPT

Use `PreparedStatement` by default. It is safer, it is faster when reused, and it makes the SQL readable by separating the statement from the values. `Statement` is for statements that genuinely have no parameters — schema creation, for example — and `CallableStatement` is for stored procedures.

12.3 Why PreparedStatement matters: SQL injection

This is the most important practical lesson in the chapter, and the demonstration is short.

```
// ---- DANGEROUS: values concatenated into the SQL text ----
String zone = userInput;                                    // e.g.  MIL' OR '1'='1
String sql = "SELECT * FROM sensor_readings WHERE zone_id = &#x27;" + zone + "'";
Statement statement = connection.createStatement();
ResultSet rs = statement.executeQuery(sql);

// the database actually receives:
//   SELECT * FROM sensor_readings WHERE zone_id = 'MIL' OR '1'='1'
// the OR clause is now part of the statement, so EVERY row matches.

// ---- SAFE: the value is bound, never parsed as SQL ----
String safeSql = "SELECT * FROM sensor_readings WHERE zone_id = ?";
```

```
try (PreparedStatement ps = connection.prepareStatement(safeSql)) {
    ps.setString(1, userInput);                          // bound as a VALUE
    try (ResultSet rs2 = ps.executeQuery()) {
        // the database receives a plan for the fixed text,
        // plus one parameter whose value is MIL' OR '1'='1 -- a literal string
    }
}
```

illustrative — the injection vulnerability and its fix

The difference is not about escaping characters. With a `PreparedStatement`, the SQL text is sent to the database and *parsed* before the parameter values are supplied. The parser has already decided what is a statement and what is a value, so no value can become part of the statement's structure. Quoting and escaping become irrelevant because the value never enters the parser's input.

> **IMPORTANT DISTINCTION**
>
> **Concatenating values into SQL is unsafe even when you escape them yourself.** Manual escaping has to handle quote characters, backslashes, multi-byte encodings and database-specific syntax, and getting it subtly wrong is easy. Binding parameters removes the problem entirely rather than mitigating it. BasinWatch uses `PreparedStatement` at every site, including the statements with no parameters.

12.4 Binding parameters

Placeholders are numbered from one, and each needs a typed setter matching the column's type.

Diagram 16 — PreparedStatement parameter binding

The SQL text and the data travel separately. That is both a correctness and a security property.

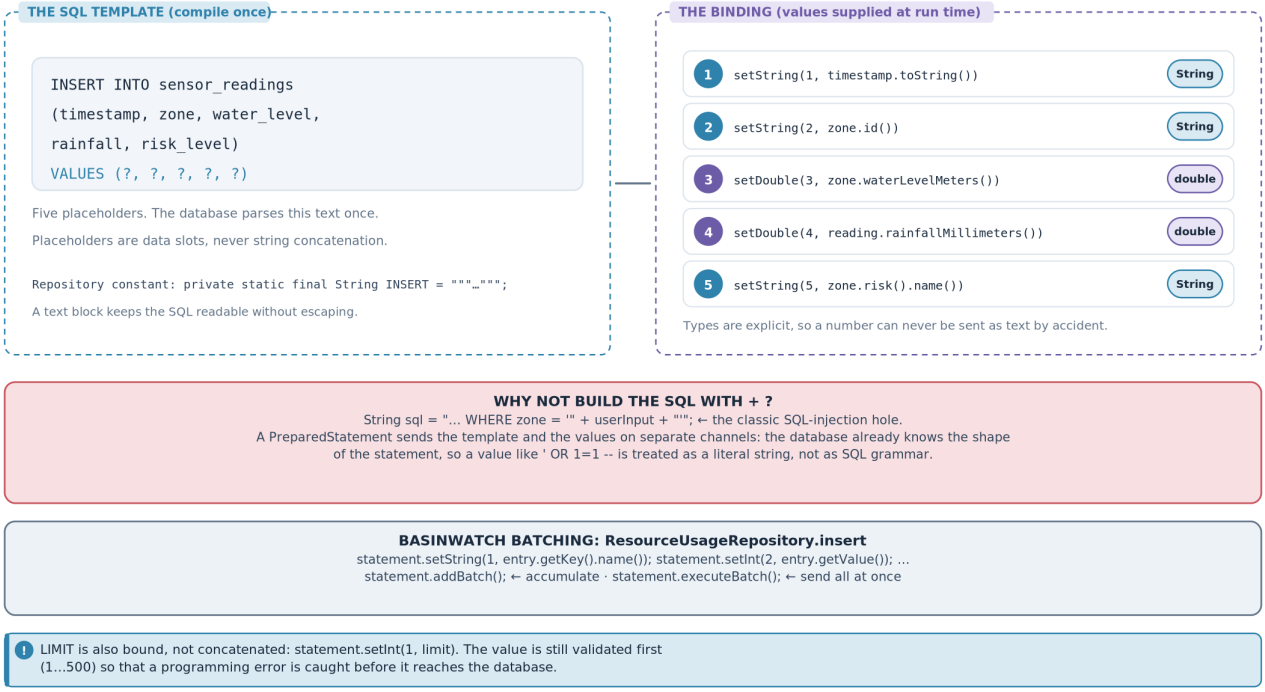


Diagram 16 — PreparedStatement parameter binding. An SQL template with numbered placeholders, the typed setter call for each, and the value the database receives. The order of the placeholders is what the index refers to — not the order of the setter calls.

```
String sql = """
        INSERT INTO sensor_readings (reading_tick, station_id, zone_id, rainfall_mm,
                                     rise_cm, water_level_m, risk_level,
                                     risk_escalated, recorded_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
        """;

try (PreparedStatement statement = connection.prepareStatement(sql)) {
    statement.setLong(1, reading.tick());                       // 1 -> reading_tick
    statement.setString(2, reading.stationId());                // 2 -> station_id
    statement.setString(3, reading.zoneId());                   // 3 -> zone_id
    statement.setDouble(4, reading.rainfallMillimeters());      // 4 -> rainfall_mm
    statement.setDouble(5, reading.observedRiseCentimeters());  // 5 -> rise_cm
    statement.setDouble(6, zone.waterLevelMeters());            // 6 -> water_level_m
    statement.setString(7, zone.risk().name());                 // 7 -> risk_level
    statement.setInt(8, riskEscalated ? 1 : 0);                 // 8 -> risk_escalated
    statement.setString(9, timestamp.toString());               // 9 -> recorded_at
    statement.executeUpdate();
}
```

from BasinWatch, db/SensorReadingRepository.java — nine placeholders, nine typed setters.

VISUAL WALKTHROUGH

Count the placeholders in that statement, then count the setter calls. There are nine of each, and the indices run 1 to 9 with no gaps. That is the check to perform whenever you read JDBC code: a missing setter, or an index that skips a number, means a parameter silently retains whatever value it had.

Table 12.3 — *The common setters and the SQL types they correspond to.*

SQL type	Setter	Java type
`INTEGER`	`setInt` / `setLong`	`int` / `long`
`REAL`	`setDouble` / `setFloat`	`double` / `float`
`TEXT`	`setString`	`String`
`BOOLEAN`	`setBoolean`	`boolean`
`NULL`	`setNull(index, java.sql.Types.X)`	any reference type
A date or timestamp	`setTimestamp` / `setDate`	`java.sql.Timestamp` / `java.sql.Date`

COMMON MISTAKE

Two binding mistakes appear constantly. First, off-by-one indices: placeholders are **1-based**, so the first setter is `setX(1, ...)`, not zero. Second, treating a boolean as a boolean in SQLite: the column here is declared `INTEGER`, so the code converts with `riskEscalated ? 1 : 0` rather than assuming a native boolean type exists.

12.5 Executing: query versus update

After binding, the two execution paths diverge and the choice is determined by whether the statement returns rows.

Table 12.4 — *Execution methods and what each returns.*

Method	Use for	Returns	Typical statement
`executeQuery()`	Statements that produce rows	A `ResultSet`	`SELECT`
`executeUpdate()`	Statements that change rows	An `int`: rows affected	`INSERT`, `UPDATE`, `DELETE`
`execute()`	When you do not know which will happen	A `boolean`: true if a result set is available	Dynamic or DDL statements
`executeBatch()`	Several statements at once	An `int[]` of counts	Bulk inserts

CORE CONCEPT

The return value of `executeUpdate()` is worth using rather than discarding. It is the number of rows the statement actually affected, which tells you whether your `WHERE` clause matched anything. An `UPDATE` returning zero almost always means the condition was wrong, and that is worth knowing.

12.6 Reading a ResultSet

A result set is a cursor. It starts *before* the first row, so the loop must advance before reading.

```
public List<DatabaseHistoryEntry> loadHistory(Connection connection, int limit)
        throws SQLException {
    String sql = """
            SELECT occurred_at, event_type, zone_id, detail
              FROM operational_events
             ORDER BY occurred_at DESC
             LIMIT ?
            """;
    ArrayList<DatabaseHistoryEntry> entries = new ArrayList<>();
    try (PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setInt(1, limit);
        try (ResultSet rows = statement.executeQuery()) {
            while (rows.next()) {                          // advance, then read
                entries.add(new DatabaseHistoryEntry(
                        DatabaseHistoryType.valueOf(rows.getString("event_type")),
                        rows.getString("zone_id"),
                        rows.getString("detail"),
                        Instant.parse(rows.getString("occurred_at"))));
            }
        }
    }
    return List.copyOf(entries);
}
```

from BasinWatch, db/OperationalEventRepository.java — the canonical read loop.

Four habits in that method are the whole of result-set handling.

The loop is `while (rows.next())`. `next()` moves the cursor and returns false when there are no more rows. A `do-while` would be wrong, because the cursor starts before the first row.

Columns are read by name. `rows.getString("event_type")` is clearer than `rows.getString(2)` and survives a change to the select list. The trade-off is a small lookup cost, which is irrelevant here.

Each row is converted into a domain object inside the loop. The method returns `List<DatabaseHistoryEntry>`, not a `ResultSet`. That is essential: a result set is tied to its statement and its

connection, and once they close it is unusable. Converting immediately means the caller receives ordinary values.

`List.copyOf` makes the result immutable. The same discipline as every other boundary in the project.

> **COMMON MISTAKE**
>
> Returning a `ResultSet` from a method that also closes the connection. The caller receives an object that looks usable and fails with `SQLException: ResultSet closed` on the first `next()`. The rule is that a result set never outlives the block that created it — map rows into your own objects before leaving.

12.7 SQLException and what it tells you

Every JDBC method that touches the database can throw `SQLException`. It is checked, and it carries more information than a plain message.

Table 12.5 — *What an `SQLException` carries.*

Method	Meaning
`getMessage()`	The driver's description, often including the database's own error text
`getSQLState()`	A five-character standard code, such as `23505` for a unique-constraint violation
`getErrorCode()`	A vendor-specific numeric code
`getNextException()`	A chained second failure, when one statement produces several errors
`getCause()`	The underlying exception, such as an `IOException` from the database file

12.8 The complete lifecycle, in order

Everything in this chapter reduces to one sequence. Learn it as a sequence, because examiners and reviewers both ask for it.

The eight steps of a parameterized query

1. **Obtain a connection.** `DriverManager.getConnection(url)`, inside try-with-resources.
2. **Prepare the statement.** `connection.prepareStatement(sql)` sends the SQL text to the database, which parses and plans it once.
3. **Bind every parameter.** One typed setter per placeholder, indices 1 to n, no gaps.
4. **Execute.** `executeQuery()` for a read, `executeUpdate()` for a write.
5. **Consume the result.** For a query, loop with `while (rows.next())` and map each row into a domain object. For an update, check the returned row count.
6. **Close the result set.** Handled by its own try-with-resources block.
7. **Close the statement.** Handled by the statement's try-with-resources block.
8. **Close the connection.** Handled by the outermost try-with-resources block — and this is where a transaction would be committed or rolled back.

CONNECTION TO ANOTHER CONCEPT

Step eight is where Chapter 13 begins. Everything above happens inside a transaction, whether you think about it or not: in autocommit mode each statement is its own transaction, and turning autocommit off lets several statements share one.

EXAM TIP

If asked to "write the code to insert a row safely", the marks are distributed roughly as follows: using `PreparedStatement` rather than concatenation; binding every parameter with the correct type and 1-based index; using `executeUpdate`; and closing all three resources. Mentioning `try-with-resources` explicitly is usually worth a mark on its own.

CHAPTER SUMMARY

- A query involves a Connection, a Statement of some kind, and possibly a ResultSet, all of which must be closed.
- PreparedStatement is the default choice: safer, pre-parsed, and reusable.
- Concatenating values into SQL permits injection; binding parameters makes the value unable to become part of the statement.
- Placeholders are numbered from one and each needs a typed setter whose Java type matches the SQL type.
- executeQuery returns a ResultSet; executeUpdate returns a row count; execute returns a boolean.
- A ResultSet cursor starts before the first row, so the loop must call next() before reading.
- Map each row into your own object inside the loop; never return a ResultSet from a method that closes its connection.
- SQLException carries a message, an SQLState, a vendor error code, and possibly chained exceptions.

KEY TERMS

- **Placeholder** — the `?` character in a prepared statement, replaced by a bound value.
- **SQL injection** — an attack in which user input becomes part of a statement's structure.
- **ResultSet cursor** — the position marker that starts before the first row.
- **Row count** — the number of rows an update affected, returned by `executeUpdate`.
- **SQLState** — the standard five-character error code carried by a `SQLException`.

CHAPTER 13

Database Transactions and Reliable Persistence

> Transaction boundaries, commit and rollback, and why several related changes must succeed or fail together.

13.1 The problem: a half-finished operation

Consider what a dispatch actually involves. A mission row must be created. Several resource counts must be decremented. An event must be recorded for the audit trail. These are three separate statements against three separate tables.

If the first succeeds and the second fails, the database now claims a mission is running with resources that were never deducted. That is not a small inconsistency: the resource board will be wrong for the rest of the session, and the history will not explain why.

DEFINITION

Transaction — a group of database operations treated as a single unit: either every operation in the group takes effect, or none of them does.

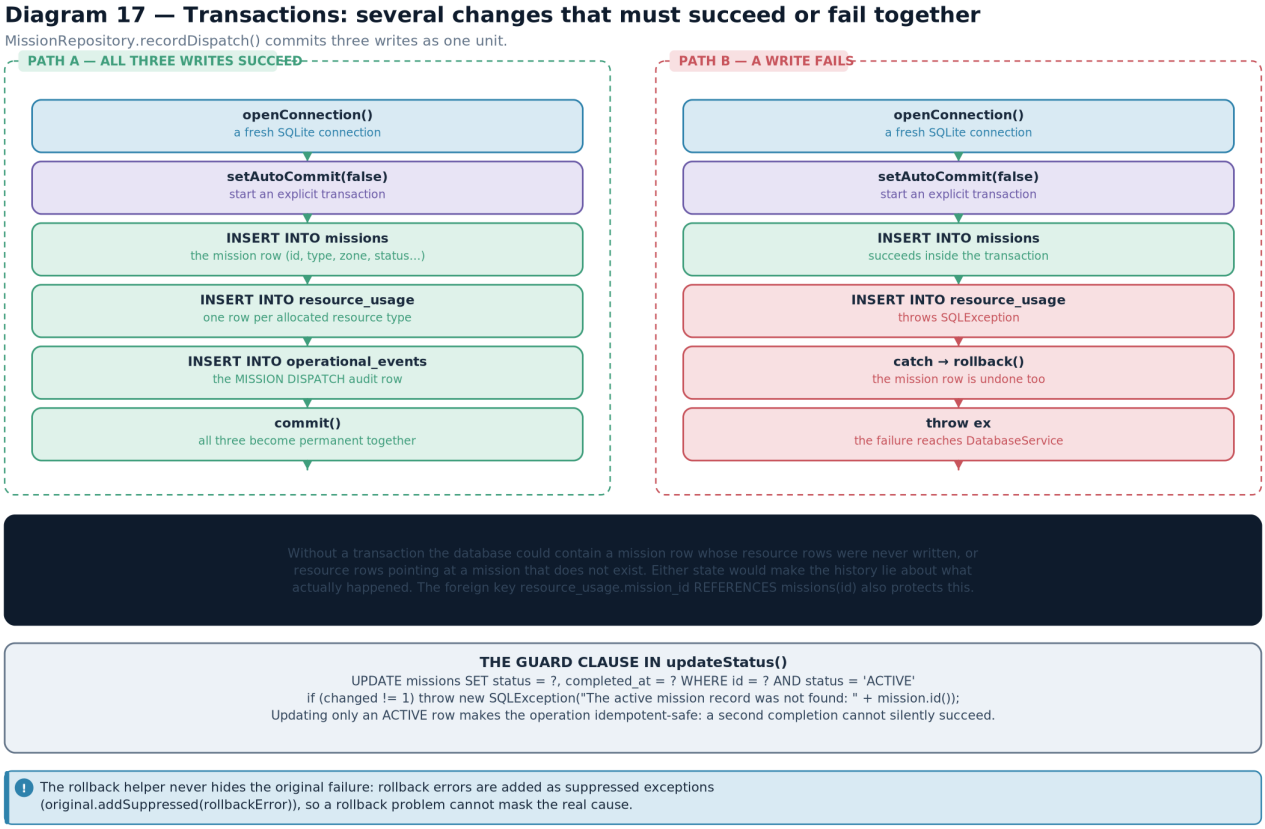


Diagram 17 — Transaction commit and rollback. Three statements inside one transaction. On success the connection commits and all three changes become visible together. On failure the connection rolls back and the database returns to its state before the transaction began — including the statements that had already succeeded.

13.2 The four guarantees

Transactions are described by four properties, known by the acronym ACID. Each is worth understanding in concrete terms rather than as a slogan.

Table 13.1 — *The ACID properties and what each means here.*

Property	Means	In this project
Atomicity	All operations take effect, or none do	A dispatch writes a mission, deducts resources and logs an event — or writes nothing
Consistency	The database moves from one valid state to another	`NOT NULL` and type constraints hold at commit time
Isolation	Concurrent transactions do not see each other's partial work	The database work runs on a single thread, so there is one writer
Durability	Once committed, the change survives a crash	SQLite flushes the committed transaction to the database file

13.3 Controlling a transaction in JDBC

By default a connection is in **autocommit** mode: each statement is committed immediately as its own transaction. Turning autocommit off gives you control.

```
connection.setAutoCommit(false);        // begin: nothing commits until you say so
try {
    // ... statement 1
    // ... statement 2
    // ... statement 3
    connection.commit();                // all three become visible together
} catch (SQLException ex) {
    connection.rollback();              // undo everything since setAutoCommit(false)
    throw ex;
} finally {
    connection.setAutoCommit(true);     // restore the default for the next caller
}
```

illustrative — the standard transaction skeleton

CORE CONCEPT

Notice the shape: turn autocommit off, do the work in a `try`, commit at the end of the try, roll back in the `catch`, and restore autocommit in a `finally`. The `finally` matters when the connection comes from a pool and will be reused by someone who does not expect an open transaction.

COMMON MISTAKE

Forgetting to roll back, or forgetting to restore autocommit. Both leave the connection in an unexpected state. A missing rollback means a later commit could persist work that was supposed to be discarded; a missing restore means the next user of a pooled connection accidentally joins your transaction. BasinWatch avoids the first problem by rolling back in the catch block and the second by closing the connection afterwards rather than pooling it.

13.4 How BasinWatch records a dispatch

The dispatch path is the project's one genuine multi-statement operation, and it is where the transaction is used.

```
public void recordDispatch(ResponseMission mission, ZoneSnapshot zone,
                           Map<ResourceType, Integer> allocation, Instant timestamp) {
    enqueue(() -> {
        try (Connection connection = database.openConnection()) {
            connection.setAutoCommit(false);              // begin the transaction
            try {
                missions.insert(connection, mission, timestamp);        // 1. the mission row
                for (Map.Entry<ResourceType, Integer> entry : allocation.entrySet()) {
                    resourceUsage.insert(connection, mission.id(), entry.getKey(),
                            entry.getValue(), timestamp);               // 2. one row per resource
                }
                operationalEvents.insert(connection, timestamp, "DISPATCH", zone.id(),
                        mission.type().label() + " allocated to " + zone.name());  // 3. the audit event
                connection.commit();                      // all three take effect together
            } catch (SQLException ex) {
                connection.rollback();                    // none of them take effect
                throw ex;
            }
        } catch (SQLException ex) {
```

```
            statusReporter.accept("Could not record dispatch: " + ex.getMessage());
        }
    });
}
```

from BasinWatch, db/DatabaseService.java — one transaction, three kinds of write.

VISUAL WALKTHROUGH

Follow the three numbered writes in that method. If the third throws, the rollback undoes the first two — so the database never contains a mission with no audit event, or resource deductions with no mission. That is what atomicity buys, and it is why the transaction boundary is drawn around all three rather than around each one.

13.5 Why the writes share one connection

This is the detail that most often goes wrong in student implementations, and it explains the package-private overloads from Chapter 2.

```
// Each repository can open its own connection — fine for a single write
public void insert(ResponseMission mission, Instant timestamp) throws SQLException {
    try (Connection connection = database.openConnection()) {
        insert(connection, mission, timestamp);
    }
}

// ...but a transaction must span ONE connection, so a shared overload is needed
void insert(Connection connection, ResponseMission mission, Instant timestamp)
        throws SQLException {
    try (PreparedStatement statement = connection.prepareStatement(INSERT_SQL)) {
        statement.setString(1, mission.id());
        // ... bind the rest, then executeUpdate()
    }
}
```

from BasinWatch, db/MissionRepository.java — the two overloads and why both exist.

IMPORTANT DISTINCTION

A transaction belongs to a connection, not to a statement. Calling `commit()` on one connection has no effect on work done on another. So if three repositories each opened their own connection, three separate transactions would exist and atomicity would be lost entirely — the code would look correct and silently not be. Passing one connection through every write is the mechanism that makes the transaction real.

13.6 Isolation, and why it is simple here

Isolation is the property that governs what one transaction can see of another's uncommitted work. Real systems offer several isolation levels, trading consistency against concurrency, and reasoning about them is one of the harder parts of database engineering.

BasinWatch sidesteps the whole problem in one design decision: all database work runs on a single dedicated thread. There is exactly one writer, so there is never a second transaction to interfere.

```
private void enqueue(DatabaseOperation operation) {
    if (!available) {
```

```
            return;                                  // database disabled: silently do nothing
        }
        try {
            executor.submit(() -> {
                try {
                    operation.run();
                } catch (SQLException ex) {
                    statusReporter.accept("Database operation failed: " + ex.getMessage());
                } catch (RuntimeException ex) {
                    statusReporter.accept("Unexpected database error: " + ex);
                }
            });
        } catch (RejectedExecutionException ex) {
            statusReporter.accept("Database queue is full; operation dropped.");
        }
    }
```

from BasinWatch, db/DatabaseService.java — one single-threaded executor is the whole concurrency policy.

That method is worth reading closely, because it is where several chapters meet.

The executor is single-threaded. Every database operation runs one after another on `"basin-database"`. No two transactions are ever concurrent, so no isolation anomaly is possible and no locking policy is needed.

An unavailable database is a no-op, not an error. The early return means every call site can invoke `recordSensor` or `recordDispatch` without checking availability first. The optional subsystem is optional at the call site too.

Failures are reported, not thrown. A database failure cannot propagate into the event processing path, because the work happens on a different thread. It is reported through the status listener and the application continues.

A full queue drops the operation rather than blocking. `RejectedExecutionException` is caught and reported. For an optional history feature, dropping a record is better than stalling the sensor pipeline.

> **CONNECTION TO ANOTHER CONCEPT**
>
> Compare this with the resource allocator in Chapter 3. That used a synchronized block to make an in-memory reservation atomic. Here the database provides atomicity through a transaction. Two different mechanisms, one shared goal: never let a partially complete change be observable.

> **EXAM TIP**
>
> A likely question is "give an example from your project where a transaction is needed, and explain what would go wrong without it." The dispatch path is the ideal answer: without a transaction, a failure after the mission insert would leave a running mission with resources that were never deducted, so the resource board would be permanently wrong and the audit trail would not explain it. Naming the three writes and the failure point is what makes the answer convincing.

CHAPTER SUMMARY

- A transaction groups several operations so that all of them take effect or none do.
- The ACID properties are atomicity, consistency, isolation and durability.
- JDBC connections are in autocommit mode by default; `setAutoCommit(false)` begins a transaction you control.
- The standard skeleton is: disable autocommit, work in a try, commit, roll back in the catch, restore autocommit in a finally.
- A transaction belongs to a connection, so every statement in it must use that same connection.
- BasinWatch's dispatch records a mission, its resource usage and an audit event in one transaction.
- All database work runs on one single-threaded executor, so isolation anomalies cannot arise and no locking policy is needed.
- A database failure is reported to the status listener rather than propagated into event processing.

KEY TERMS

- **ACID** — atomicity, consistency, isolation, durability.
- **Commit** — making a transaction's changes permanent and visible.
- **Rollback** — discarding a transaction's changes as though they never happened.
- **Autocommit** — the default mode in which each statement is its own transaction.
- **Isolation level** — the degree to which concurrent transactions are prevented from seeing each other's partial work.

CHAPTER 14

How BasinWatch Uses JDBC and SQLite

> The database architecture, the schema, the repositories, and a complete trace of a history record from an accepted observation to a table cell.

14.1 The four classes that make up the database layer

The whole database subsystem is four kinds of class, each with one responsibility.

Table 14.1 — *The four responsibilities of the database layer.*

Class	Responsibility	Knows about	Does not know about
`DatabaseManager`	Hold the URL and hand out connections	The driver, the URL	Tables, SQL, the domain
`DatabaseInitializer`	Create the tables and indexes if absent	The schema	Anything else
`DatabaseService`	Own the worker thread, wrap operations in transactions, report failures	Repositories, the status listener	Swing, files, SQL text
`*Repository` (four)	Turn domain values into SQL and rows back into domain values	Its own table and its SQL	Threads, transactions, other repositories

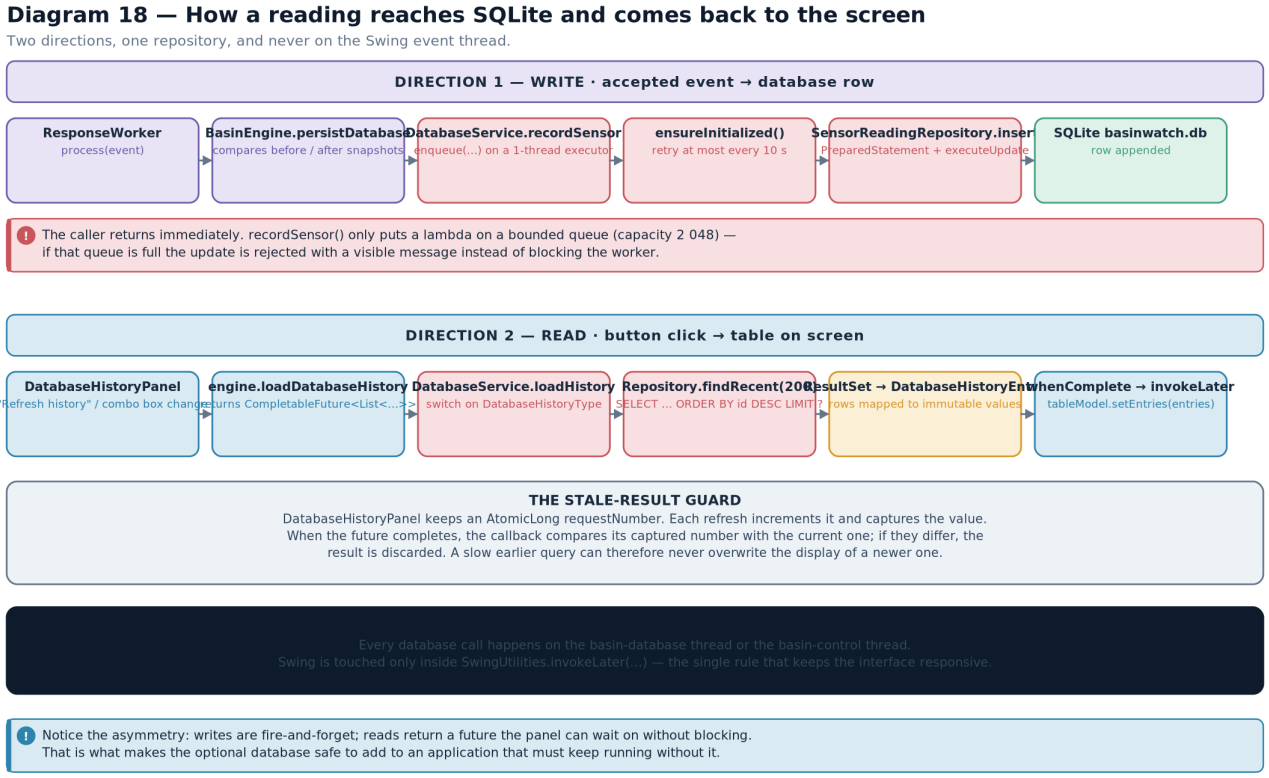


Diagram 18 — Database data flow. An accepted observation travels from the engine, through the database service's queue, onto the worker thread, into the repository, through a prepared statement into SQLite. A history read travels the reverse path as a future whose callback returns to the event-dispatch thread.

14.2 The schema

Four tables hold the history. Each corresponds to one kind of thing the application does.

```
CREATE TABLE IF NOT EXISTS sensor_readings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    reading_tick INTEGER NOT NULL,
    station_id TEXT NOT NULL,
    zone_id TEXT NOT NULL,
    rainfall_mm REAL NOT NULL,
    rise_cm REAL NOT NULL,
    water_level_m REAL NOT NULL,
    risk_level TEXT NOT NULL,
    risk_escalated INTEGER NOT NULL,
    recorded_at TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS missions (
    id TEXT PRIMARY KEY,
    mission_type TEXT NOT NULL,
    zone_id TEXT NOT NULL,
    started_tick INTEGER NOT NULL,
    completes_at_tick INTEGER NOT NULL,
    status TEXT NOT NULL,
    recorded_at TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS resource_usage (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    mission_id TEXT NOT NULL,
    resource_type TEXT NOT NULL,
    quantity INTEGER NOT NULL,
    recorded_at TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS operational_events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    occurred_at TEXT NOT NULL,
    event_type TEXT NOT NULL,
    zone_id TEXT,
    detail TEXT NOT NULL
);
```

from BasinWatch, db/DatabaseInitializer.java — the four tables.

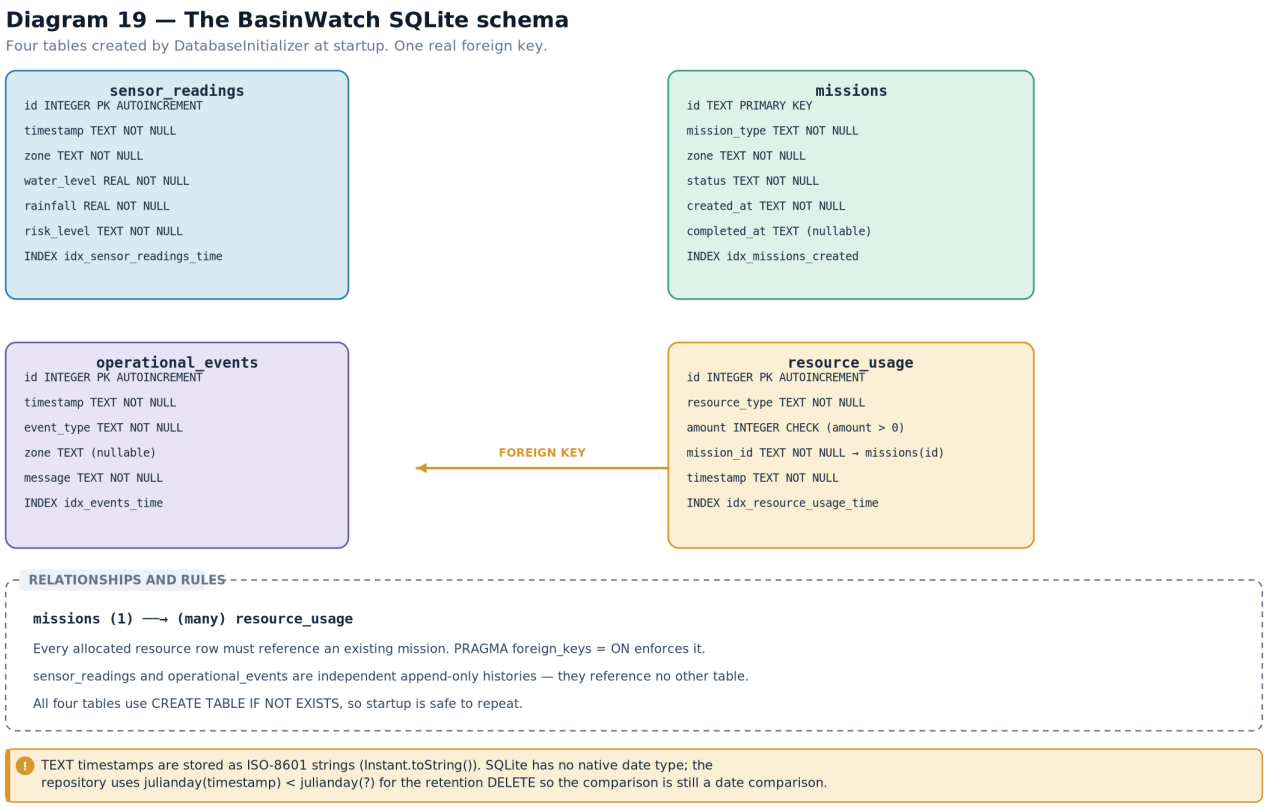


Diagram 19 — The database schema. Four tables and the relationships between them. Missions and resource usage are linked by mission id; sensor readings and operational events carry a zone id that names a zone in the model. Every table carries a timestamp so history can be ordered.

Three design decisions are visible in the schema.

Every table has a timestamp. History is only useful if it can be ordered and filtered by time, and `recorded_at` is stored as ISO-8601 text so that string ordering matches chronological ordering — the same choice the log file makes.

`missions.id` is the primary key and is text. The application already generates identifiers such as `BW-0001` — `"BW-"` followed by a four-digit zero-padded sequence — so the database uses them rather than inventing a second identity. This is what lets `resource_usage.mission_id` refer to a mission.

`operational_events.zone_id` is the one nullable column. Not every event belongs to a zone — a global warning does not — and `NOT NULL` would force a meaningless value.

RELATED TECHNIQUE — NOT USED IN BASINWATCH

The schema declares the mission relationship with a plain column rather than a `FOREIGN KEY ... REFERENCES` constraint. That is a deliberate simplification: SQLite enforces foreign keys only when `PRAGMA foreign_keys = ON` is set for the connection, so a constraint written in the schema could be silently ignored. A production schema would enable the pragma and declare the constraint, so that deleting a mission could not orphan its resource rows. BasinWatch's single-writer design means the application itself maintains the relationship.

14.3 Initialisation, and why it is idempotent

The initialiser runs at start-up and must be safe to run every time.

```
public void initialise() throws SQLException {
    try (Connection connection = database.openConnection();
         Statement statement = connection.createStatement()) {
        for (String ddl : SCHEMA) {
            statement.execute(ddl);            // every statement uses IF NOT EXISTS
        }
    }
}
```

from BasinWatch, db/DatabaseInitializer.java — creating the schema is idempotent.

The `IF NOT EXISTS` clause in every `CREATE` is what makes this safe. Without it, the second launch of the application would fail with "table already exists". With it, the first launch creates the tables and every later launch does nothing. This is the simplest possible migration strategy, and it is appropriate for a project with one schema version.

IMPORTANT DISTINCTION

`Statement` is used here, not `PreparedStatement`. That is correct and deliberate: these are fixed DDL statements with no parameters, so there is nothing to bind and nothing to inject. It is a useful example of the one case where `Statement` is the right choice — and it shows that "always use PreparedStatement" is really "always use it when there are values".

14.4 Tracing a reading from the sensor to the table

This is the central trace of the chapter. Follow one observation all the way to a stored row.

One accepted observation, from pipe to table

1. **The feed encodes it.** `SensorFeed` builds a `SensorReading` and writes `reading.encode()` plus a line separator into the `PipedWriter`.
2. **The intake parses it.** `SensorIntake` reads the line and calls `SensorReading.parse(line)`, which validates the field count and ranges.
3. **The intake queues it.** `eventBuffer.put(event)` hands it to the workers; the call blocks if the buffer is full.
4. **A worker processes it.** One of the three `ResponseWorker` threads takes the event and calls `model.process(event)`, which applies the reading to the zone and returns activity messages.
5. **The worker fans out.** The engine appends to the journal, submits to the archive worker, and — if the reading was accepted and changed the zone — calls `database.recordSensor(...)`.
6. **The service queues it.** `recordSensor` calls `enqueue`, which submits a lambda to the single-threaded executor and returns immediately.
7. **The worker thread runs it.** The lambda opens a connection, and `SensorReadingRepository.insert` prepares the nine-placeholder statement and binds every value.
8. **SQLite executes it.** The driver sends the statement and the bound values; the database inserts the row and commits it, because autocommit is on.
9. **If it was a risk escalation, a second insert follows.** An `operational_events` row is written with the type `RISK ESCALATION`.
10. **A failure is reported, not thrown.** Any `SQLException` is caught by `enqueue`'s lambda and passed to the status listener, which posts it to the interface.

BASINWATCH IN PRACTICE

Step five is where the acceptance test lives, and it is worth seeing exactly. The engine writes to the database only when the model actually changed:

```
if (event instanceof SensorReading reading) {
    ZoneSnapshot zone = model.zone(reading.zoneId());
    boolean riskEscalated = zone.risk().rank() > previousRiskRank;
    database.recordSensor(reading, zone, Instant.now(), riskEscalated);
}
```

Listing 14.1 — *from BasinWatch, `engine/BasinEngine.java`*

This is why Chapter 2's stale-reading guard matters. A reading older than the last one applied is rejected by `BasinZone.apply`, the model does not change, and no row is written. Without that guard, a duplicate or out-of-order observation would appear in the history as though it had taken effect.

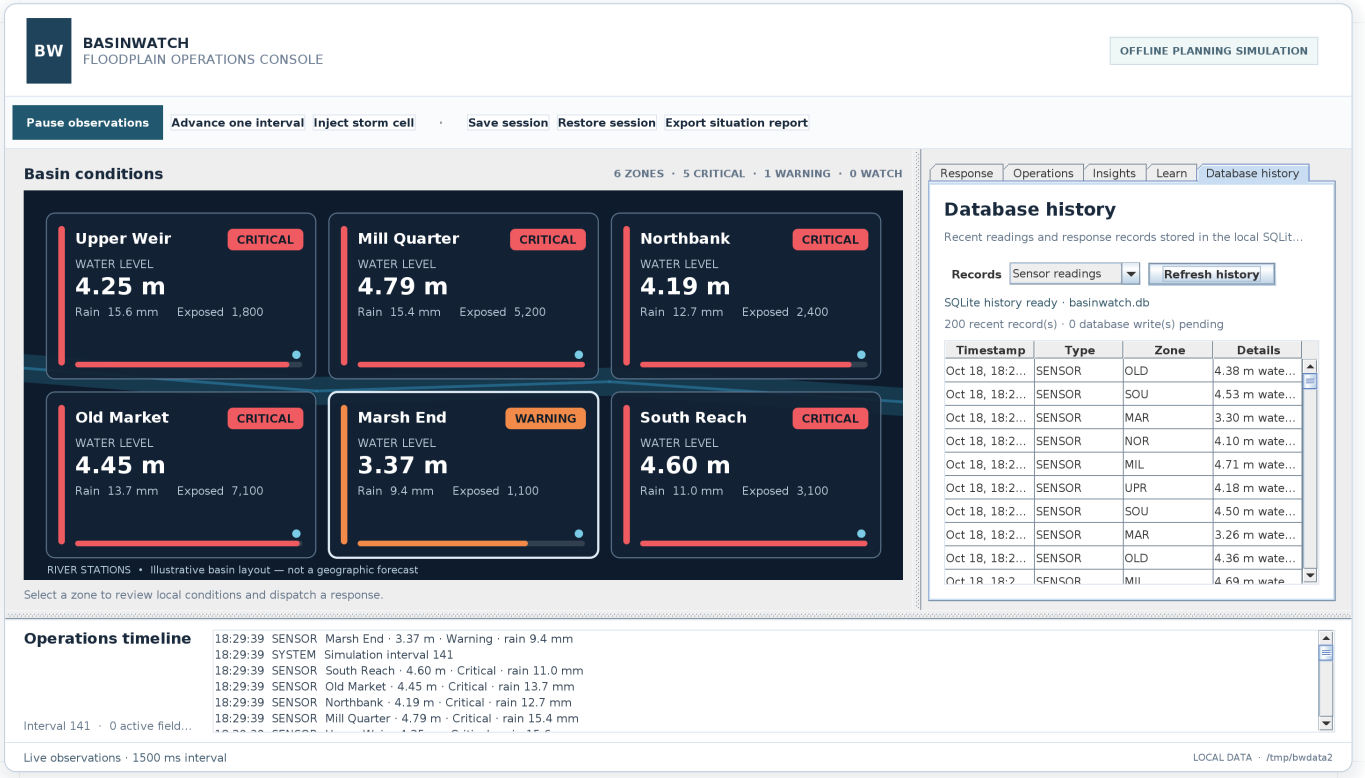
14.5 Reading history back into the interface

The read path is the reverse, and it uses a future so the interface never blocks.

```
public CompletableFuture<List<DatabaseHistoryEntry>> loadHistory(int limit) {
    if (!available) {
        return CompletableFuture.completedFuture(List.of());   // optional means empty, not error
    }
    CompletableFuture<List<DatabaseHistoryEntry>> future = new CompletableFuture<>();
    enqueue(() -> future.complete(history.loadRecent(limit)));
    return future;
}
```

from BasinWatch, db/DatabaseService.java — a read returns a future; an unavailable database returns an empty list.

Two details are worth naming. When the database is unavailable the future is completed *immediately* with an empty list, so the caller's callback runs normally and simply shows no history. And the actual query runs on the database thread, so the future completes on that thread — which is why the interface's callback must hop back to the event-dispatch thread.



Screenshot S2 — The Database history tab reading real rows from SQLite. The same query seen from the operator's side. The status line reads `200 recent record(s) · 0 database write(s) pending` and the table below it is populated with genuine rows: `SENSOR` entries with their timestamp, zone code and a rendered detail string such as `4.38 m water`. Each visible cell came out of a `ResultSet` via `OperationalEventRepository.loadRecent` and was copied into a `HistoryTableModel` on the event-dispatch thread. The category selector on the left chooses between sensor readings, response records and missions, which is why `DatabaseHistoryType` exists as an enum.

Nothing in that table is hard-coded for the screenshot. The rows were written by the running simulation, stored in `basinwatch.db`, read back through JDBC and mapped into the table model. If the SQLite file is deleted and the application restarted, the same panel reports that history is unavailable and shows an empty table — the rest of the console keeps working.

```
public List<DatabaseHistoryEntry> loadRecent(int limit) throws SQLException {
    String sql = """
            SELECT occurred_at AS at, event_type AS type, zone_id AS zone, detail AS detail
              FROM operational_events
             ORDER BY occurred_at DESC
             LIMIT ?
            """;
    // ... prepare, bind limit, iterate the result set, map each row
}
```

from BasinWatch, db/OperationalEventRepository.java — the query behind the history table.

14.6 Why the database is optional

The final question about this layer is the most important one: why does the application work without it?

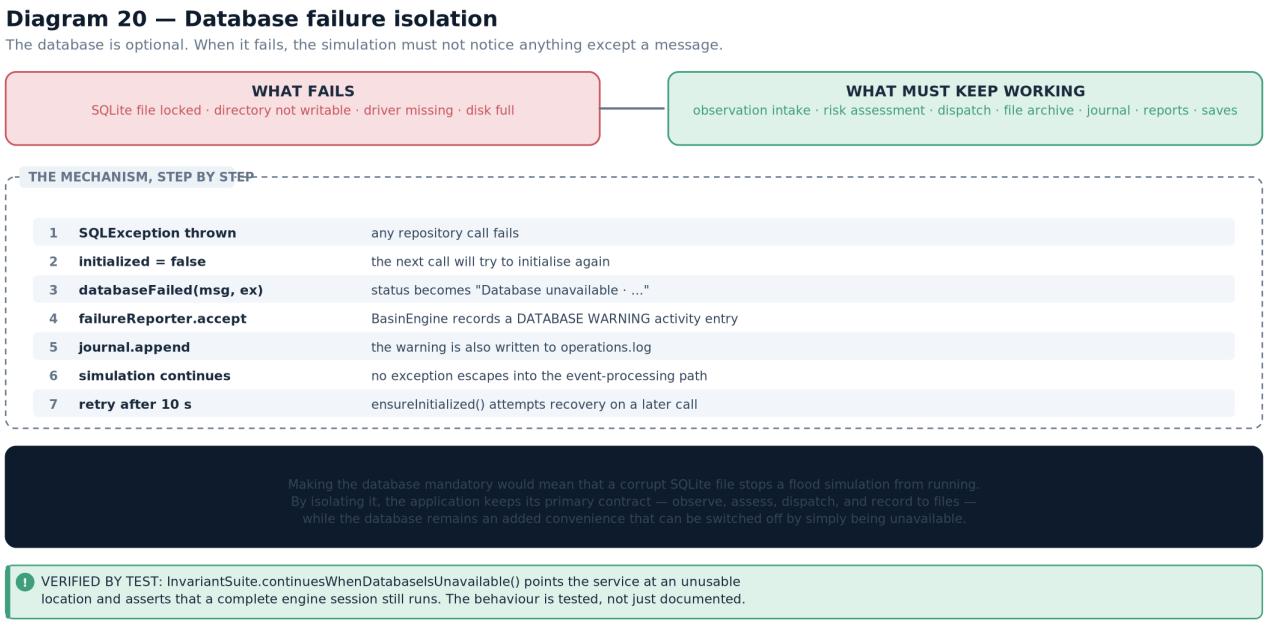


Diagram 20 — Database failure isolation. How a database failure is contained. The exception is caught on the database worker thread, converted into a status message, and posted to the interface. The sensor pipeline, the model, the files and the interface all continue unaffected.

Four mechanisms together produce that isolation, and each was a decision.

Availability is a boolean, checked once. `DatabaseManager.isAvailable` catches the exception and returns false, so the rest of the code tests a value rather than handling a failure.

Every write is a queued lambda. Because the work happens on another thread, an exception there cannot propagate into the caller's stack.

The queue wrapper catches everything. `enqueue` catches both `SQLException` and `RuntimeException` and routes them to the status listener.

A read on an unavailable database returns an empty list. The interface's code path is identical in both cases, so no branch is needed at the call site.

CORE CONCEPT

This is the design pattern worth taking away from Part III, and it generalises well beyond databases: **an optional subsystem should be detectable, isolated, and silent-on-absence.** Detectable through a boolean or a capability query. Isolated by running on its own thread so failures cannot propagate. Silent-on-absence by returning a sensible empty result rather than raising an error the caller must anticipate.

EXAM TIP

If asked "what happens if the database is unavailable?", the complete answer has four parts: the application still starts, because availability is checked rather than assumed; the history panel shows an empty list and a status message; all writes become no-ops; and the file layer is entirely unaffected, so the log, archive, snapshot and reports are still produced. Answering with only the first part is the usual mistake.

CHAPTER SUMMARY

- The database layer has four responsibilities: connection management, schema creation, service coordination, and per-table SQL.
- Four tables hold readings, missions, resource usage and operational events; every one carries a timestamp.
- Schema creation is idempotent through `CREATE TABLE IF NOT EXISTS`, so it is safe on every launch.
- Fixed DDL statements use `Statement`; everything with values uses `PreparedStatement`.
- A reading reaches the database only when the model actually changed, which is why the stale-reading guard matters.
- Reads return `CompletableFutures` so the interface never blocks on a query.
- The database is optional because availability is a boolean, writes are queued and isolated on their own thread, and reads on an unavailable database return an empty list.
- An optional subsystem should be detectable, isolated and silent on absence.

KEY TERMS

- **Repository** — a class that owns the SQL for one table and maps rows to and from domain objects.
- **Idempotent** — safe to perform more than once with the same result.
- **DDL** — data definition language: statements such as `CREATE TABLE`.
- **Single-threaded executor** — an executor with one worker, serialising all submitted tasks.
- **Capability query** — asking whether an optional subsystem is available rather than assuming it is.

PART IV

Swing and the User Interface

An interface is not a drawing of the model; it is a set of listeners, a component tree and a repaint loop, all of which must share exactly one thread. This part explains Swing from containers outward, walks through the lecture-note login application, and then shows how BasinWatch connects a click to a database query and back to a visible table.

15 Swing Fundamentals and Event-Driven Programming

16 Connecting Application Logic, JDBC and Swing

CHAPTER 15

Swing Fundamentals and Event-Driven Programming

> Containers, components, layouts, listeners and events — and the lecture-note login application explained line by line.

15.1 What makes a program event-driven?

A procedural program runs from top to bottom: it reads input, computes, prints output, and exits. A graphical program does none of that. It starts, builds a window, and then *waits*. Everything that happens afterwards happens because something triggered it — a click, a key press, a timer, a window being resized.

The structure that makes this work is a loop that never ends and a set of callbacks that the loop invokes. In Swing, that loop lives on a dedicated thread called the **event-dispatch thread**, and every listener you register runs on it.

DEFINITION

Event-driven programming — a style in which the program declares handlers for events and then waits, letting the framework invoke the appropriate handler when each event occurs, rather than controlling the sequence of execution itself.

CORE CONCEPT

Three consequences follow from the event-driven model, and they explain almost every Swing rule in this chapter.

1. **You do not control the order of execution.** The user does.
2. **All event handling and all painting happen on one thread.** So any long operation inside a handler freezes the entire interface.
3. **Components must be updated from that thread.** Touching a Swing component from another thread is unsafe.

15.2 Containers and components

Swing's class hierarchy separates things that hold other things from things that display something.

Table 15.1 — *The three categories of Swing class.*

Category	Base class	Examples	Role
Top-level container	`JFrame`, `JDialog`	`JFrame`	A real operating-system window with a title bar and close button
Intermediate container	`JPanel`, `JScrollPane`, `JSplitPane`	`JPanel`, `JScrollPane`	Groups and arranges other components; invisible itself

Category	Base class	Examples	Role
Simple component	`JComponent`	`JButton`, `JLabel`, `JTable`, `JComboBox`, `JSlider`	Displays something or accepts input

IMPORTANT DISTINCTION

A `JFrame` is a window; a `JPanel` is not. You cannot add a `JPanel` to another window's title bar, and a `JPanel` has no close button. Confusing the two is behind a large share of beginner layout problems: the top-level container is the window, and everything else is arranged *inside* its content pane.

Diagram 21 — Swing component containment and the event-dispatch thread

A Swing window is a tree of components. Only one thread is allowed to touch that tree.

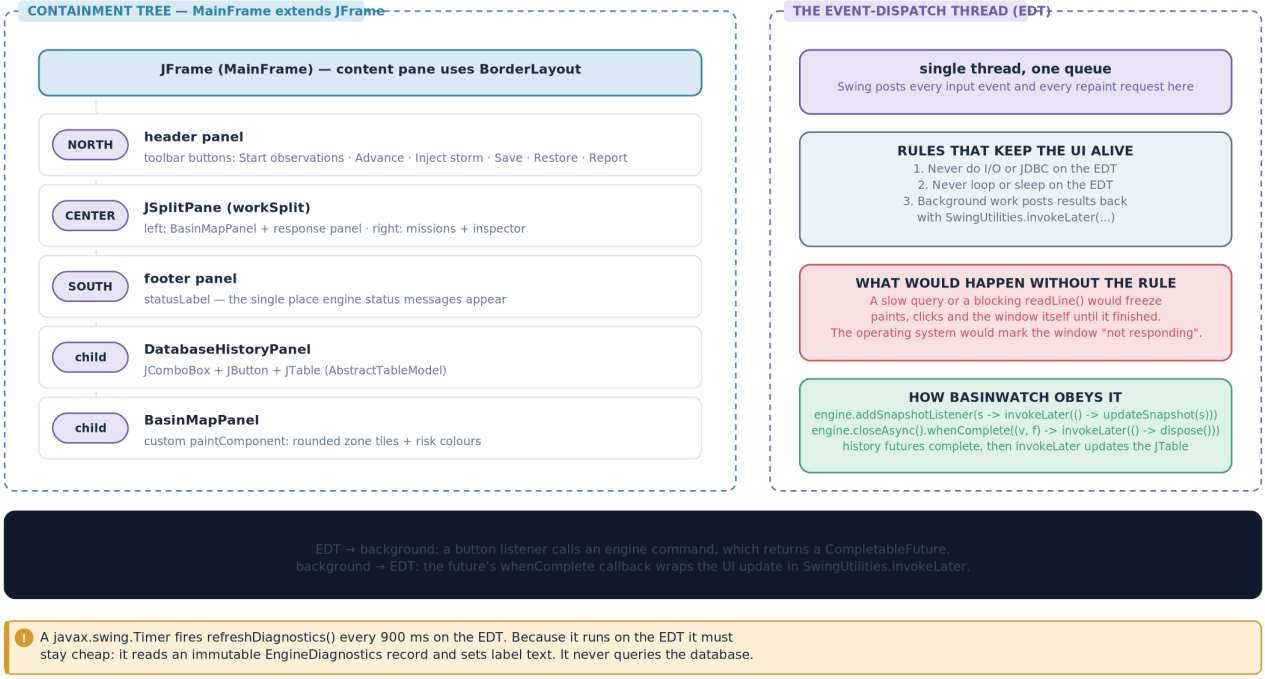


Diagram 21 — The Swing component tree. A JFrame containing a toolbar, a central split pane, and a status bar. The split pane holds the map panel on the left and a tabbed panel on the right. Each node is a real component class.

15.3 Layout managers

Swing does not use absolute coordinates. Instead, each container has a layout manager that decides where its children go. This is why a Swing window resizes sensibly rather than leaving controls stranded.

Table 15.2 — *The common layout managers.*

Layout	Arrangement	Best for	Characteristic
`BorderLayout`	Five regions: NORTH, SOUTH, EAST, WEST, CENTER	The overall frame — toolbar on top, status bar at the bottom, content in the centre	CENTER takes all remaining space; the other four take their preferred size

Layout	Arrangement	Best for	Characteristic
`FlowLayout`	Left to right, wrapping to the next line	A row of buttons or controls	The default for `JPanel`
`GridLayout`	A fixed grid of equal-sized cells	A keypad, or a uniform form	Every cell is the same size
`BoxLayout`	A single row or column	Stacking controls vertically	Use `Box.createVerticalStrut` for spacing
`GridBagLayout`	A grid with per-component constraints	Complex forms that must align precisely	Powerful and verbose; needs a `GridBagConstraints` per component

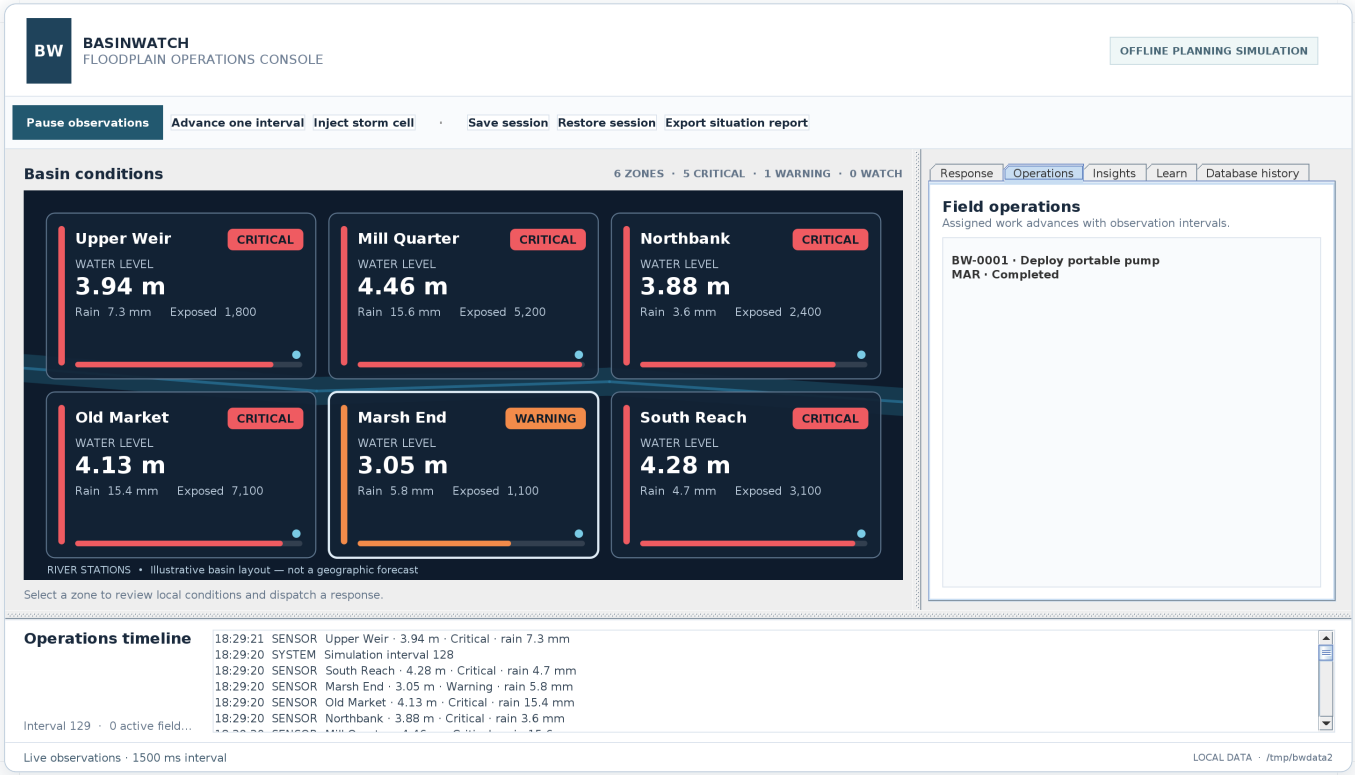
BASINWATCH IN PRACTICE

BasinWatch's main frame uses `BorderLayout` for the window and nested containers inside it, which is the standard composition:

```
setLayout(new BorderLayout(8, 8));
add(buildToolbar(), BorderLayout.NORTH);
add(buildStatusBar(), BorderLayout.SOUTH);
JSplitPane split = new JSplitPane(JSplitPane.HORIZONTAL,
        mapPanel, rightPanel);
split.setResizeWeight(0.45);
add(split, BorderLayout.CENTER);
```

Listing 15.1 — *from BasinWatch, `app/MainFrame.java`*

Read the composition: the window is a border layout, the toolbar and status bar take the fixed north and south regions, and the split pane fills the centre. The split pane divides the centre horizontally, giving 45 per cent of extra space to the map when the window is resized.



Screenshot S3 — The Operations tab, showing a completed field response. The panel on the right is a `JTabbedPane`; this is its second card. The heading reads *Field operations* with the note *Assigned work advances with observation intervals*, and the list below shows mission `BW-0001 · Deploy portable pump` against zone `MAR` with status *Completed*. That mission row is the end of a real chain: a click on the *Dispatch response* button became a `DispatchRequest` event, which the engine accepted, processed on the next tick, and wrote to SQLite as a single transaction across the `missions`, `resource_usage` and `operational_events` tables (Chapter 13). The zone cards on the left are unchanged by the tab switch — they belong to the map panel, which is outside the tabbed pane.

15.4 Events and listeners

An event is an object describing something that happened. A listener is an object that wants to be told. Registering a listener connects the two.

Table 15.3 — *The listeners you meet most often.*

Listener interface	Method	Triggered by	Event object carries
`ActionListener`	`actionPerformed(ActionEvent)`	A button click, a menu item, pressing Enter in a field	The source component and an action command string
`MouseListener`	`mouseClicked`, `mousePressed`, `mouseReleased`, `mouseEntered`, `mouseExited`	Mouse activity over a component	The x/y position and which button
`MouseMotionListener`	`mouseMoved`, `mouseDragged`	Movement, with no click	The x/y position
`KeyListener`	`keyTyped`, `keyPressed`, `keyReleased`	Keyboard input when the component has focus	The key code and character
`WindowListener`	`windowClosing`, `windowClosed`, …	Window lifecycle	The window and its new state
`ChangeListener`	`stateChanged(ChangeEvent)`	A slider or tab selection changing	The source component

Listener interface	Method	Triggered by	Event object carries
`ListSelectionListener`	`valueChanged(ListSelectionEvent)`	A row or list item being selected	Whether the change is still adjusting

Diagram 22 — Swing event handling: from a click to a changed screen

Following "Dispatch response" through listeners, futures and repaints.

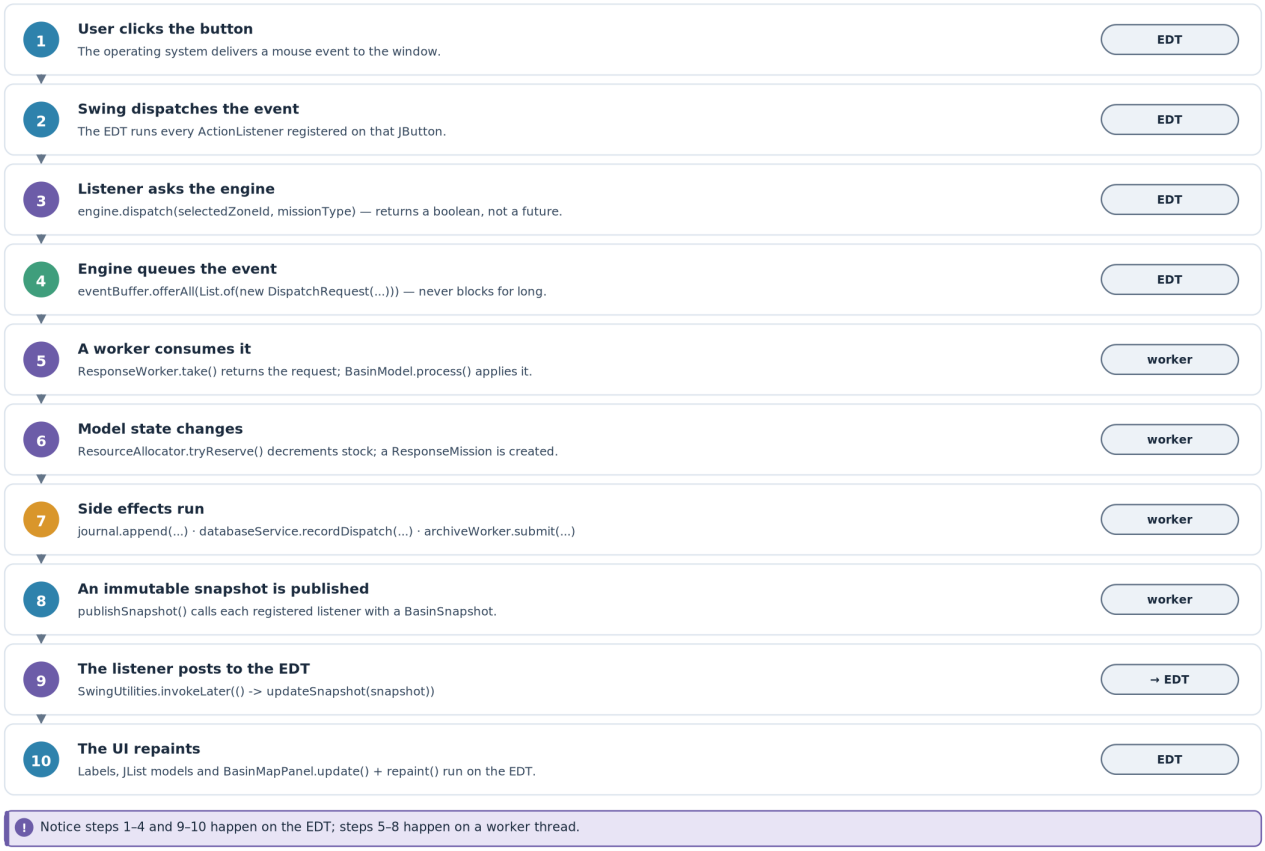


Diagram 22 — Swing event handling flow. The user's click is delivered to the event-dispatch thread, which dispatches it to every listener registered on the source component. The listener runs its handler on that same thread. Painting is also requested through the same thread, so the handler must return quickly.

15.5 The three ways to write a listener

Java gives you several idioms for the same thing, and each is appropriate in a different situation.

```
// (1) A separate named class — best when the handler is substantial or reused
final class ExportHandler implements ActionListener {
    private final BasinEngine engine;
    ExportHandler(BasinEngine engine) { this.engine = engine; }
    @Override public void actionPerformed(ActionEvent event) {
        engine.exportReport();
    }
}
button.addActionListener(new ExportHandler(engine));

// (2) An anonymous inner class — the classic style in older Swing code
button.addActionListener(new ActionListener() {
    @Override public void actionPerformed(ActionEvent event) {
        engine.exportReport();
    }
});
```

```
// (3) A lambda — correct when the interface has one abstract method
button.addActionListener(event -> engine.exportReport());

// (4) A method reference — when the handler is already a method
button.addActionListener(this::onExportRequested);
```

illustrative — four listener idioms

CORE CONCEPT

All four compile to the same thing. A lambda works because `ActionListener` is a **functional interface**: it declares exactly one abstract method, so the compiler can infer the implementation. This is why Swing code written after Java 8 is markedly shorter than older examples, and why the anonymous-class form you may see in lecture notes is now mostly historical.

15.6 The lecture-note login application

The lecture notes contain a Swing login screen. It is a small program, but it demonstrates most of the fundamentals at once, so it is worth walking through carefully.

Diagram 23 — The lecture-note Swing login application

A small, self-contained example: text fields, a button, an event listener and a database check.

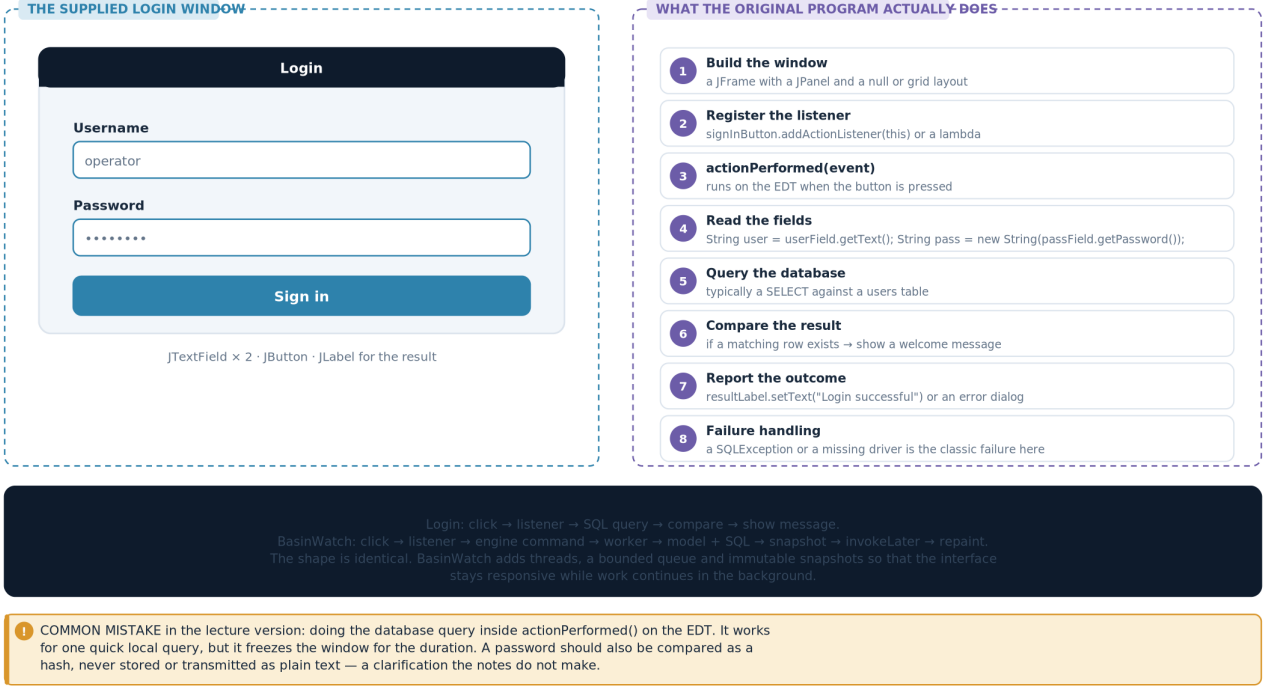


Diagram 23 — The lecture-note login application. The login frame: a username field, a password field, a login button and a status label, arranged in a grid inside a border layout. The annotated flow shows the button's ActionListener reading both fields and comparing them against the expected credentials.

```
import javax.swing.*;
import java.awt.*;

public class LoginApplication {
    private final JFrame frame = new JFrame("Login");
    private final JTextField usernameField = new JTextField(16);
```

```
    private final JPasswordField passwordField = new JPasswordField(16);
    private final JButton loginButton = new JButton("Login");
    private final JLabel statusLabel = new JLabel("Enter your credentials", SwingConstants.CENTER);

    public LoginApplication() {
        JPanel form = new JPanel(new GridLayout(2, 2, 8, 8));
        form.add(new JLabel("Username:"));
        form.add(usernameField);
        form.add(new JLabel("Password:"));
        form.add(passwordField);

        JPanel content = new JPanel(new BorderLayout(8, 8));
        content.setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));
        content.add(form, BorderLayout.CENTER);
        content.add(loginButton, BorderLayout.SOUTH);
        content.add(statusLabel, BorderLayout.NORTH);

        loginButton.addActionListener(event -> attemptLogin());

        frame.setContentPane(content);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.pack();
        frame.setLocationRelativeTo(null);          // centre on screen
        frame.setVisible(true);
    }

    private void attemptLogin() {
        String username = usernameField.getText().trim();
        char[] password = passwordField.getPassword();
        if (username.isEmpty() || password.length == 0) {
            statusLabel.setText("Both fields are required.");
            return;
        }
        if ("admin".equals(username) && "secret".equals(new String(password))) {
            statusLabel.setText("Welcome, " + username + ".");
        } else {
            statusLabel.setText("Incorrect username or password.");
            passwordField.setText("");              // clear the password on failure
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(LoginApplication::new);
    }
}
```

lecture-note style — the Swing login application, annotated

15.7 What the login program actually does

It builds a component tree. A `GridLayout` panel holds two label and field pairs. That panel sits in the centre of a border layout, with the button below and the status label above. Nothing is positioned by coordinates.

It registers one listener. The button's action listener calls `attemptLogin()`. That is the only connection between the user and the logic.

It validates before comparing. An empty username or password produces a specific message, which is more useful than "incorrect credentials". This is the same validate-at-the-boundary habit as `SensorReading.parse`.

It reads the password as a `char[]`, not a `String`. That is not stylistic. `JPasswordField.getPassword()` returns a character array precisely so that the caller can overwrite it when finished. A `String` is

immutable and its contents may persist in memory until garbage collection, so a password held in a `String` is readable from a memory dump for longer than necessary.

It clears the password on failure. A small detail with a real security purpose: the credential is not left on screen for someone walking past.

It starts on the event-dispatch thread. `main` does nothing but schedule the constructor with `SwingUtilities.invokeLater`. Every component is therefore created on the EDT, which is the rule.

TECHNICAL CLARIFICATION

This program checks a hard-coded credential in the client. That is fine for a demonstration of Swing mechanics and unacceptable for a real system, for three reasons. The password appears in the compiled code. There is no hashing, so anyone reading the class file learns the credential. And any check performed in the user interface can be bypassed by anyone who can run different code. A real login compares a *hash* of the submitted password against a stored hash, and it does so on a server the user does not control. The lecture example is a UI exercise, not a security pattern — and saying so is part of understanding it.

COMMON MISTAKE

Calling `new LoginApplication()` directly from `main` instead of through `SwingUtilities.invokeLater`. It often appears to work, because the components happen to be created before the window is shown. But Swing's internals assume single-threaded access, and creating components off the EDT can produce intermittent, hard-to-reproduce failures in layout or painting. The fix is one line, and it is not optional.

15.8 Custom painting

When a component must draw something Swing does not provide — a schematic map, for instance — you override `paintComponent`.

```
@Override
protected void paintComponent(Graphics graphics) {
    super.paintComponent(graphics);              // always first: clears and paints the background
    Graphics2D g = (Graphics2D) graphics;
    g.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);
    for (ZoneSnapshot zone : snapshot.zones()) {
        Point centre = positionOf(zone.id());
        g.setColor(colourFor(zone.risk()));
        g.fillOval(centre.x - RADIUS, centre.y - RADIUS, RADIUS * 2, RADIUS * 2);
        g.setColor(Color.WHITE);
        g.drawString(zone.id(), centre.x - 12, centre.y + 4);
    }
}
```

from BasinWatch, app/BasinMapPanel.java — custom painting, condensed.

Three rules govern `paintComponent`, and all three matter.

Call `super.paintComponent` first. It clears the component's area. Without it, previous frames are never erased and the drawing smears as the component is repainted.

Never perform slow work inside it. Painting runs on the event-dispatch thread, and it runs whenever the window is uncovered, resized or asked to repaint. Reading a file or running a query here would freeze the interface repeatedly.

Never call `repaint()` from inside it. That requests another paint, which requests another repaint, producing an unresponsive loop. To change what is drawn, change the data and then call `repaint()` from elsewhere.

CONNECTION TO ANOTHER CONCEPT

This is why the engine publishes immutable snapshots. The paint method reads `snapshot` on the event-dispatch thread while workers may be mutating the model on other threads. Because the snapshot is an immutable copy, the paint method sees a consistent picture with no locking at all.

EXAM TIP

A very common question asks why a long-running task in an `ActionListener` freezes the interface. The complete answer: the handler runs on the event-dispatch thread, and that same thread is responsible for delivering subsequent events and performing all painting. While the handler is busy, neither can happen, so the window appears frozen even though the application is running. The fix is to move the work to a background thread and post the result back — the subject of Chapter 16.

CHAPTER SUMMARY

- Swing is event-driven: the program declares handlers and then waits for the framework to call them.
- Components fall into top-level containers, intermediate containers and simple components.
- Layout is managed by a layout manager per container; BorderLayout for the frame with nested layouts inside it is the standard composition.
- Events are dispatched to registered listeners, and all event handling runs on the event-dispatch thread.
- A listener can be a named class, an anonymous class, a lambda or a method reference; lambdas work because the listener interfaces are functional.
- The lecture-note login application validates input, reads the password as a char array, clears it on failure, and starts on the EDT.
- Custom painting goes in paintComponent, must call super first, must be fast, and must never call repaint.
- A hard-coded credential check in a UI is a demonstration of mechanics, not a security design.

KEY TERMS

- **Event-dispatch thread (EDT)** — the single thread that runs Swing event handling and painting.
- **Listener** — an object registered to be notified when a particular event occurs.
- **Functional interface** — an interface with exactly one abstract method, so a lambda can implement it.
- **Layout manager** — the object that decides where a container's children are placed.
- **paintComponent** — the method overridden to draw a custom component.

CHAPTER 16

Connecting Application Logic, JDBC and Swing

> From a click to a command, through a background thread and a database query, back to a visible table — and the one thread rule that governs all of it.

16.1 The problem this chapter solves

Chapter 15 established that all Swing work happens on the event-dispatch thread. Chapter 12 established that a database query is a blocking operation. Put those two facts together and you have a conflict: running the query in a listener freezes the window.

This chapter is about resolving that conflict without introducing a worse one. The solution has four parts, and each is a rule rather than a trick.

CORE CONCEPT

1. **A listener does not do the work.** It asks the engine for an operation and returns.
2. **Slow work happens on a background thread.** The engine's executors own those threads.
3. **The result is delivered as a future.** A callback runs when the value arrives.
4. **The callback posts to the EDT before touching any component.** Every time, without exception.

16.2 Step one: the listener asks, and returns

BasinWatch's listeners are short by design. Each one translates a user gesture into an engine call and nothing else.

```
startButton.addActionListener(event -> {
    if (engine.isRunning()) {
        engine.pause();
    } else {
        engine.start();
    }
    refreshControlStates();
});

stepButton.addActionListener(event -> engine.step());
stormButton.addActionListener(event -> engine.injectStorm(1.35, 0.42));
reportButton.addActionListener(event -> engine.exportReport());
saveButton.addActionListener(event -> chooseAndSave());
refreshHistoryButton.addActionListener(event -> historyPanel.reload());
```

from BasinWatch, app/MainFrame.java and app/DatabaseHistoryPanel.java — six listeners, all of them short.

None of those handlers opens a file, runs a query or waits for anything. That is the point. A listener that returns quickly cannot freeze the interface, and the method names read like the operations the user is actually performing.

16.3 Step two: the engine runs the work elsewhere

The engine decides which thread performs an operation. Several executors and named threads are involved, and each exists for a reason.

Table 16.1 — *The threads in the application and why each is separate.*

Thread	Created by	Runs	Why it is separate
`"event-dispatch"`	Swing	Event handling and painting	Swing requires it
`"basin-sensor-feed"`	`SensorFeed extends Thread`	The emission loop and its timed sleep	It must be interruptible and joinable
`"basin-sensor-intake"`	The engine, from a `Runnable`	Reading and validating pipe lines	It blocks on `readLine()`
`"basin-response-1..3"`	A fixed thread pool	Applying events to the model	Processing is concurrent and CPU-bound
`"basin-sensor-archive"`	A single-thread executor	Appending typed records to the archive	Disk writes must be serialised
`"basin-control"`	A single-thread executor	Save, restore and report generation	Only one control operation at a time
`"basin-database"`	A single-thread executor	All JDBC work	One writer, so no isolation anomalies

VISUAL WALKTHROUGH

Count the threads and note that only one of them may touch Swing. Every other row in that table describes work that would freeze the window if it ran on the event-dispatch thread — a blocking read, a disk write, a database round trip, a computation over hundreds of events.

16.4 Step three: the result arrives as a future

An operation that produces a result returns a `CompletableFuture`. The caller attaches a callback and carries on.

```
public CompletableFuture<Path> exportReport() {
    return CompletableFuture.supplyAsync(() -> {
        try {
            BasinSnapshot snapshot = model.snapshot();
            Path report = ReportWriter.write(AppPaths.reportsDirectory(), snapshot);
            publishStatus("Report written to " + report.getFileName());
            return report;
        } catch (IOException ex) {
            publishStatus("Could not write the report: " + ex.getMessage());
            throw new CompletionException(ex);
        }
    }, controlExecutor);
}
```

from BasinWatch, engine/BasinEngine.java — the work is supplied to an executor; the caller gets a future.

Three properties of that method are the pattern.

The work is a supplier submitted to a named executor. `supplyAsync(..., controlExecutor)` runs on `"basin-control"`, not on the caller's thread and not on the common pool.

Failures are converted. `IOException` is checked, and a supplier cannot throw it, so it is wrapped in `CompletionException`. The future then completes exceptionally, and any `whenComplete` callback can inspect the cause.

The status is published from the worker thread. `publishStatus` is called inside the supplier, so it runs on the control thread. The listener that receives it must hop to the EDT — which brings us to step four.

16.5 Step four: the callback posts to the EDT

This is the rule that everything else depends on.

```
// Registering a listener: the engine may call it from ANY thread
engine.addStatusListener(message ->
        SwingUtilities.invokeLater(() -> statusLabel.setText(message)));

// Attaching a callback to a future: it runs on the completing thread
engine.loadHistory(200).whenComplete((entries, failure) ->
        SwingUtilities.invokeLater(() -> {
            if (failure != null) {
                historyStatus.setText("History unavailable: " + failure.getMessage());
                return;
            }
            historyTableModel.setEntries(entries);
            historyStatus.setText(entries.size() + " entries loaded.");
        }));
```

from BasinWatch, app/MainFrame.java and app/DatabaseHistoryPanel.java — every callback hops to the EDT.

CORE CONCEPT

Notice that `SwingUtilities.invokeLater` appears in both places, and in exactly the same position: *wrapping the entire body that touches a component*. That is the rule stated precisely. It is not enough to call it somewhere in the method — the component access must be inside the lambda that is posted.

IMPORTANT DISTINCTION

`invokeLater` and `invokeAndWait` are different, and one of them is dangerous. `invokeLater` queues the runnable on the EDT and returns immediately — the calling thread continues. `invokeAndWait` queues it and blocks until the EDT has run it, which is a deadlock if you call it *from* the EDT. BasinWatch uses `invokeLater` exclusively.

16.6 The complete path, from click to table cell

Now the two halves of the book can be joined. Here is the whole path for one user action: pressing "Refresh" in the history panel.

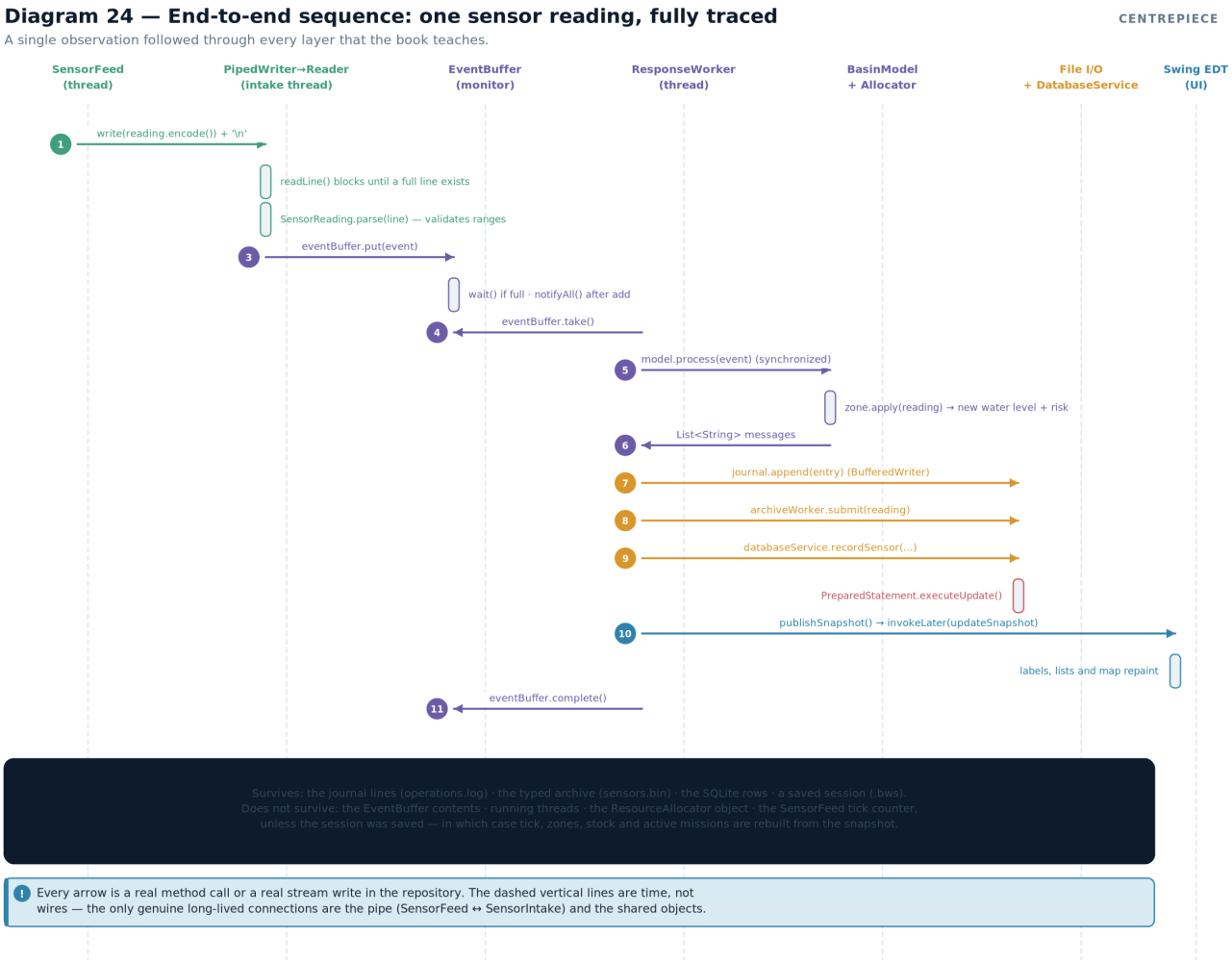


Diagram 24 — The complete path from a click to a table cell. Fourteen numbered steps crossing four threads: the event-dispatch thread dispatches the click, the database thread runs the query and completes the future, and the callback posts back to the event-dispatch thread to update the table model.

From a button press to a populated table

1. **The user clicks Refresh.** The operating system delivers a mouse event to the window.
2. **The EDT dispatches it.** Swing calls the `ActionListener` registered on that button.
3. **The listener calls one method.** `historyPanel.reload()` — and returns immediately.
4. **reload asks the engine for a future.** `engine.loadHistory(200)` returns a `CompletableFuture` at once.
5. **The engine checks availability.** If the database is unavailable it returns an already-completed future holding an empty list, so no error path is needed.
6. **The engine enqueues the query.** A lambda is submitted to the single-threaded database executor.
7. **The database thread opens a connection.** `DatabaseManager.openConnection()` on `basin-database`.
8. **The repository prepares and binds.** A `PreparedStatement` with one placeholder for the limit.
9. **SQLite executes.** The query is planned and the matching rows are returned in descending time order.
10. **The rows are mapped to objects.** Each row becomes a `DatabaseHistoryEntry`, inside the result-set loop.
11. **The future completes on the database thread.** `future.complete(entries)` runs the `whenComplete` callback on that same thread.
12. **The callback posts to the EDT.** `SwingUtilities.invokeLater` queues a runnable.
13. **The EDT runs the runnable.** The table model's entries are replaced and the status label is updated.
14. **Swing repaints.** The table asks its model for the new row count and redraws only the visible cells.

VISUAL WALKTHROUGH

Follow the thread changes in that list: EDT, then database thread, then EDT again. Steps one to four are EDT. Steps five to eleven are the database thread. Steps twelve to fourteen are EDT again. The two `invokeLater` boundaries are the only places the two threads touch.

16.7 Why the table model matters

A `JTable` does not store data. It asks a `TableModel` for it, and the model is where the application's data lives.

```
final class HistoryTableModel extends AbstractTableModel {
    private static final String[] COLUMNS = {"Time", "Type", "Zone", "Detail"};
    private List<DatabaseHistoryEntry> entries = List.of();

    void setEntries(List<DatabaseHistoryEntry> entries) {
        this.entries = List.copyOf(entries);       // defensive copy, immutable
        fireTableDataChanged();                    // tell the table to ask again
    }

    @Override public int getRowCount() { return entries.size(); }
    @Override public int getColumnCount() { return COLUMNS.length; }
    @Override public String getColumnName(int column) { return COLUMNS[column]; }

    @Override public Object getValueAt(int row, int column) {
        DatabaseHistoryEntry entry = entries.get(row);
        return switch (column) {
            case 0 -> entry.at();
            case 1 -> entry.type();
            case 2 -> entry.zoneId() == null ? "\u2014" : entry.zoneId();
            default -> entry.detail();
        };
    }
}
```

from BasinWatch, app/DatabaseHistoryPanel.java — a table model over immutable rows.

Three things in that class are worth noting.

The model holds the data, and the table renders it. Replacing the entries and calling `fireTableDataChanged()` is what makes the table update. Without the notification, the table would keep displaying stale rows, because it has no way to know the model changed.

`setEntries` takes an immutable copy. The list came from the database thread; copying it removes any possibility of concurrent modification while the EDT is reading.

The nullable zone is handled in the view. `operational_events.zone_id` is nullable by design, and the model renders null as a dash rather than the text "null".

CONNECTION TO ANOTHER CONCEPT

Compare this with the login application in Chapter 15. That program put its logic directly in a private method of the frame class. BasinWatch puts the logic in the engine and gives the panel only a table model. Both are Swing programs; the difference is that one is a demonstration of mechanics and the other is an application with a persistence layer behind it. Neither is wrong — but the second is what you need when the data comes from a database.

RELATED TECHNIQUE — NOT USED IN BASINWATCH

Some applications use `SwingWorker` instead of futures. It is a JDK class with `doInBackground()` on a worker thread and `process()` and `done()` on the EDT, and it solves the same problem. BasinWatch does not use it, for one reason: the engine must work without a user interface, so its operations cannot depend on a Swing type. Returning `CompletableFuture` keeps the engine independent of Swing, and the interface chooses how to consume the result. If you are asked which is better, the honest answer is that `SwingWorker` is more convenient inside a UI and `CompletableFuture` is more flexible outside one.

EXAM TIP

The highest-value question in this part is "trace the path of a user action that reads from the database." Answer it as a thread journey, not a class list: the EDT handles the click, the call returns a future immediately, the query runs on the database thread, the future completes there, and the callback posts back to the EDT to update the model. Mention `invokeLater` explicitly, and mention that the interface never blocks.

CHAPTER SUMMARY

- A listener must return quickly, so it asks the engine for an operation rather than performing one.
- The engine owns several executors and named threads, each with a specific job.
- Operations that produce a result return `CompletableFutures`, so the caller never blocks.
- Any code touching a Swing component must be wrapped in `SwingUtilities.invokeLater`, including callbacks from futures.
- `invokeLater` queues and returns; `invokeAndWait` blocks and can deadlock if called from the EDT.
- A complete click-to-table path crosses the EDT, the database thread, and the EDT again.
- A `JTable` renders whatever its `TableModel` provides, and the model must call `fireTableDataChanged` to trigger an update.
- The engine returns futures rather than using `SwingWorker`, so that it remains independent of Swing.

KEY TERMS

- **invokeLater** — queueing a runnable to run on the event-dispatch thread, without waiting.
- **CompletionException** — the wrapper used when a checked exception must be thrown from an asynchronous supplier.
- **TableModel** — the object supplying data to a `JTable`.
- **fireTableDataChanged** — the notification that tells a table its model's contents have changed.
- **Thread confinement** — the rule that a particular object may only be accessed from one thread; Swing components are confined to the EDT.

PART V

Configuration and Related Techniques

An application is more than its source files. This part explains how dependencies reach the classpath, how a build produces a runnable artefact, what XML, schemas, namespaces and annotations are for, and how Spring's JdbcTemplate relates to the raw JDBC you have just studied.

17 Maven, pom.xml and Project Dependencies

18 XML, Schemas, Namespaces, Annotations and JavaBeans

19 Raw JDBC versus Spring JdbcTemplate

CHAPTER 17

Maven, pom.xml and Project Dependencies

> How a Java project declares what it needs, how the build turns source into a runnable artefact, and why the JDBC driver is on the classpath at all.

17.1 The problem a build tool solves

Compiling a single file is easy: `javac Hello.java`. Compiling a project is not. A real project has hundreds of source files, depends on third-party libraries, needs those libraries' own dependencies, must be compiled in the right order, and must be packaged into something distributable. Doing all of that by hand does not scale.

A build tool automates four things: resolving dependencies, compiling, testing, and packaging. Maven is one such tool, and it is the one BasinWatch uses.

DEFINITION

Build tool — a program that reads a project description, downloads and resolves the libraries the project depends on, compiles the sources in the correct order, runs tests, and produces a packaged artefact.

Diagram 25 — Maven: from pom.xml to a runnable jar

Dependencies, plugins and build phases for the actual BasinWatch pom.xml.

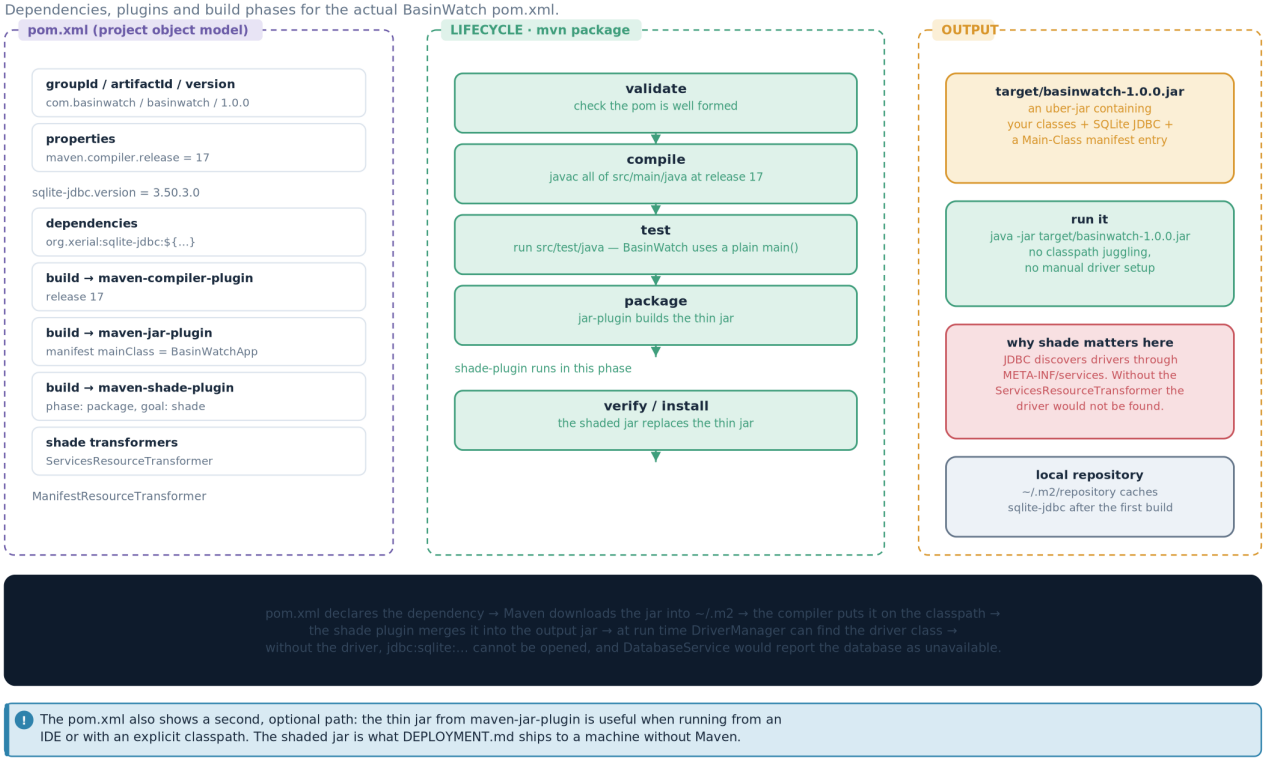


Diagram 25 — The Maven build workflow. The pom.xml declares coordinates, dependencies and plugins. Maven resolves the dependency tree from a repository into the local cache, compiles the sources against that classpath, runs the tests, and packages the result. The shade plugin then merges the dependency classes into one runnable jar.

17.2 Coordinates: how a dependency is named

Maven identifies every library by three values, and it identifies your project the same way.

Table 17.1 — *Maven coordinates, for this project and for its one dependency.*

Coordinate	Meaning	BasinWatch's project	The SQLite driver
`groupId`	The organisation or namespace	`com.basinwatch`	`org.xerial`
`artifactId`	The project or library name	`basinwatch`	`sqlite-jdbc`
`version`	The release identifier	`1.0.0`	`3.50.3.0`

CORE CONCEPT

Those three values are the complete address of a jar in a repository. Given `org.xerial:sqlite-jdbc:3.50.3.0`, Maven can find, download and cache the exact file — and it does the same for every library that library itself needs. That transitive resolution is the main reason build tools replaced hand-managed jars.

17.3 The pom.xml, section by section

The Project Object Model is the project's description. BasinWatch's is short, and every section has a purpose.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.basinwatch</groupId>
    <artifactId>basinwatch</artifactId>
    <version>1.0.0</version>
    <name>BasinWatch</name>
    <description>Offline floodplain operations console and planning simulation</description>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>17</maven.compiler.release>
        <sqlite-jdbc.version>3.50.3.0</sqlite-jdbc.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.xerial</groupId>
            <artifactId>sqlite-jdbc</artifactId>
            <version>${sqlite-jdbc.version}</version>
        </dependency>
    </dependencies>
    <!-- build section follows -->
</project>
```

from BasinWatch, pom.xml — coordinates, properties and the single dependency.

Three things are worth pointing out.

There is exactly one dependency. The project uses only the JDK, Swing and the SQLite driver. Everything else — streams, threads, collections, futures — is standard library. A short dependency list is a real advantage: fewer things to update, fewer vulnerabilities, fewer surprises.

The version is a property, not a literal. `<sqlite-jdbc.version>` is declared once in `<properties>` and referenced as `${sqlite-jdbc.version}`. Upgrading the driver means changing one line.

The source encoding is declared. UTF-8 is stated explicitly, so the compiler reads the source files the same way on every machine. This is the build-file equivalent of the explicit charsets in Chapter 5 — the same discipline applied to a different artefact.

17.4 Plugins and the build lifecycle

Maven's build is a fixed sequence of **phases**. Running a later phase runs every earlier one.

Table 17.2 — *The Maven lifecycle phases and what each produces.*

Phase	What happens	Plugins involved
`validate`	The project description is checked	Core
`compile`	Sources in `src/main/java` are compiled to `target/classes`	`maven-compiler-plugin`
`test`	Tests in `src/test/java` are compiled and run	`maven-surefire-plugin`
`package`	Compiled classes are packaged into a jar in `target/`	`maven-jar-plugin`, then `maven-shade-plugin`
`verify`	Additional checks run	Any verification plugin
`install`	The artefact is copied into the local repository	Core
`deploy`	The artefact is uploaded to a remote repository	Core

So `mvn package` compiles, tests and packages. The commands you will actually type are short.

```
mvn compile            # compile only
mvn test               # compile and run the test suite
mvn package            # compile, test, and produce target/basinwatch-1.0.0.jar
mvn clean package      # delete target/ first, then do all of the above
mvn dependency:tree    # print the resolved dependency tree
```

the four commands you need in practice

17.5 Where the JDBC driver comes from

Chapter 11 asked why the driver is on the classpath and answered "a Maven dependency". Here is the mechanism.

How the SQLite driver reaches the running program

1. **Declared.** `pom.xml` names `org.xerial:sqlite-jdbc:3.50.3.0`.
2. **Resolved.** Maven downloads the jar from a repository into the local cache, typically `~/.m2/repository`.
3. **Added to the compile classpath.** The compiler can now resolve any driver-specific types referenced in the source.
4. **Added to the runtime classpath.** When the program runs, the jar is on the class path, so `org.sqlite.JDBC` is loadable.
5. **Discovered automatically.** The jar contains `META-INF/services/java.sql.Driver`, so the JDK registers the driver without any code.
6. **Selected by URL.** `DriverManager.getConnection("jdbc:sqlite:…")` asks each registered driver whether it accepts the URL; the SQLite driver says yes.

COMMON MISTAKE

Forgetting that the driver must be on the *runtime* classpath, not just the compile classpath. This is the classic cause of `SQLException: No suitable driver found` in a program that compiles perfectly. The dependency is present when the compiler runs and absent when the program runs — usually because the jar was built without it, which is the next section's subject.

17.6 Packaging a runnable jar

A plain jar contains only your own compiled classes. Run it and the driver is missing. Maven's `shade` plugin solves this by producing an "uber jar": your classes plus the dependency classes, merged into one file.

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-shade-plugin</artifactId>
    <version>3.6.0</version>
    <executions>
        <execution>
            <phase>package</phase>
            <goals><goal>shade</goal></goals>
            <configuration>
                <createDependencyReducedPom>false</createDependencyReducedPom>
                <transformers>
                    <!-- CRITICAL: merge META-INF/services files from every jar -->
                    <transformer
implementation="org.apache.maven.plugins.shade.resource.ServicesResourceTransformer"/>
                    <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
                        <mainClass>com.basinwatch.app.BasinWatchApp</mainClass>
                    </transformer>
                </transformers>
            </configuration>
        </execution>
    </executions>
</plugin>
```

from BasinWatch, pom.xml — the shade plugin and the transformer that matters.

CORE CONCEPT

The `ServicesResourceTransformer` is not optional. Without it, the shading process would keep only one `META-INF/services/java.sql.Driver` file and discard the others. The merged jar would then contain the SQLite driver's classes but no registration, and the application would fail with "No suitable driver found" — even though the driver is physically inside the jar. This is one of the most confusing failures in Java packaging, and the transformer is the fix.

The `ManifestResourceTransformer` sets `Main-Class` in the jar's manifest, which is what makes `java -jar basinwatch-1.0.0.jar` start the application instead of printing an error.

VISUAL WALKTHROUGH

Diagram 25 shows both jars Maven produces. The thin jar in `target/` contains only the project's own classes and is not runnable alone. The shaded jar contains the project plus the driver's classes plus the merged service files, and it is the artefact you distribute.

17.7 Why this matters to the rest of the book

The build file is not administrative overhead. It is the answer to three questions the other chapters raise.

- **Why does the JDBC code compile?** Because `java.sql` is in the JDK and the driver is on the compile classpath.
- **Why does the JDBC code run?** Because the driver is on the runtime classpath and registers itself through a service file.
- **Why does the application start with one command?** Because the shade plugin merged everything into one jar and set `Main-Class`.

Remove any one of those and the application stops working, even though no Java source changed. That is the lesson of this chapter: the build is part of the program.

EXAM TIP

A likely question is "how does your application get its database driver?" The complete answer has three parts: it is declared as a Maven dependency with groupId, artifactId and version; Maven resolves it and places it on the classpath; and the driver registers itself through `META-INF/services`, which the shade plugin must be configured to merge. That last clause is what distinguishes an answer that has read the pom from one that has not.

CHAPTER SUMMARY

- A build tool resolves dependencies, compiles, tests and packages; Maven is the one this project uses.
- Every library is identified by groupId, artifactId and version, and Maven resolves their transitive dependencies automatically.
- The pom.xml declares coordinates, properties, dependencies and build plugins.
- BasinWatch has exactly one dependency — the SQLite JDBC driver — because everything else is in the JDK.
- Maven's lifecycle is a fixed sequence of phases, and a later phase runs every earlier one.
- The JDBC driver reaches the runtime classpath as a Maven dependency and registers itself through a service file.
- The shade plugin builds an uber jar containing the dependency classes, and its ServicesResourceTransformer is required for driver registration to survive.
- The build configuration is part of the program: change it and the application's behaviour changes without any Java source changing.

KEY TERMS

- **pom.xml** — the Project Object Model, Maven's description of a project.
- **Coordinate** — the groupId, artifactId and version triple identifying a library.
- **Transitive dependency** — a library required by a library you depend on directly.
- **Build lifecycle** — Maven's ordered sequence of phases.
- **Uber jar** — a jar containing a project's classes plus all its dependencies' classes.
- **ServicesResourceTransformer** — the shade transformer that merges service provider files so driver registration survives packaging.

CHAPTER 18

XML, Schemas, Namespaces, Annotations and JavaBeans

> Structured text, the rules that constrain it, and the two ways Java records metadata about code — with a clear statement of what this project does and does not use.

18.1 XML: a tree of labelled text

XML is a way of writing structured data as text. Everything in it is either an **element** with a name, or **text** inside an element. Elements nest, forming a tree.

```
<?xml version="1.0" encoding="UTF-8"?>
<basin name="BasinWatch" revision="3">
    <zone id="UPR" waterLevelMeters="5.44">
        <name>Upper Weir</name>
        <residents>1800</residents>
    </zone>
    <zone id="MIL" waterLevelMeters="5.83">
        <name>Mill Quarter</name>
        <residents>5200</residents>
    </zone>
</basin>
```

illustrative — the real zone identifiers and exposed populations the project holds in Java (domain/BasinModel.java), expressed as XML. BasinWatch itself does not read or write XML; this shows the data in a second notation.

Table 18.1 — *XML terminology.*

Term	Meaning	In the example
Element	A labelled node with a start tag and an end tag	`<zone>…</zone>`
Attribute	A name-value pair inside a start tag	`id="UPR"`
Text content	Characters between tags	`Upper Weir`
Root element	The single outermost element	`<basin>`
Well-formed	Every tag closed, correctly nested, one root, quoted attributes	The example is well-formed
Valid	Well-formed *and* conforming to a schema	A separate question — see below

IMPORTANT DISTINCTION

Well-formed and valid are different things. A document is *well-formed* if it follows XML's syntax rules: tags match, nesting is correct, there is exactly one root. It is *valid* if it also conforms to a schema that says which elements may appear, in what order, with what types. Every valid document is well-formed; a well-formed document need not be valid.

18.2 Namespaces

A namespace is a prefix that qualifies element and attribute names, so that two vocabularies can be mixed in one document without collision.

Diagram 26 — XML, namespaces and XML Schema

Markup for configuration and data, with names to avoid collisions and a schema to define the rules.

XML · a tree of labelled elements

```
<?xml version="1.0" encoding="UTF-8"?>
<basinwatch xmlns="http://example.org/basinwatch">
<station id="G-MIL" zone="MIL">
<rainfall unit="mm">17.5</rainfall>
<rise unit="cm">3.2</rise>
</station>
</basinwatch>
```

An element has a name, attributes and content. Order and nesting carry meaning.

NAMESPACES · two vocabularies, one document

```
<app:station xmlns:app="http://example.org/basinwatch"
xmlns:geo="http://example.org/geo">
<app:rainfall unit="mm">17.5</app:rainfall>
<geo:location lat="51.5" lon="-0.12"/>
</app:station>
```

app:rainfall and geo:location are different names from different vocabularies — no collision even if both vocabularies define a rainfall element.

XML SCHEMA (XSD) · rules for a document

```
<xs:element name="station">
<xs:complexType>
<xs:sequence>
<xs:element name="rainfall" type="xs:decimal"/>
<xs:element name="rise" type="xs:decimal"/>
</xs:sequence>
<xs:attribute name="id" type="xs:string" use="required"/>
</xs:complexType>
</xs:element>
```

A schema states which elements may appear, in what order, how many times, and which attributes are required. It turns "looks like XML" into "is valid for this vocabulary".

XSD files are themselves XML documents, so they use namespaces too.

WHERE THIS BELONGS IN A JAVA PROJECT

- **Maven pom.xml** — the one XML file every Maven project really needs
- **Spring XML configuration** — beans, data sources and wiring declared in markup
- **Web descriptors** — web.xml, deployment descriptors, SOAP/WSDL contracts
- **Data interchange** — XML payloads validated against an XSD
- **Annotations vs XML** — the same intent expressed two ways — see Chapter 18
- **BasinWatch** — uses pom.xml only; no Spring XML and no custom XML data

IMPORTANT DISTINCTION: pom.xml is XML that Maven reads at build time. It is not read by the application at run time. BasinWatch reads no XML when it runs, so the XML and XSD material is examinable background and a RELATED TECHNIQUE — NOT USED IN BASINWATCH, except for the pom.xml itself.

Diagram 26 — Namespaces, schemas and validation. A document declaring two namespaces, each bound to a URI that identifies a vocabulary. A schema in the xsd namespace describes the elements and types allowed in the document namespace. Validation checks the document against that description.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
</project>
```

from BasinWatch, pom.xml — the namespace declarations at the top of the file you already met.

That header is worth decoding, because it is the same pattern everywhere.

`xmlns="http://maven.apache.org/POM/4.0.0"` declares the **default namespace**. Every unprefixed element — `project`, `modelVersion`, `dependencies` — belongs to it.

`xmlns:xsi="…XMLSchema-instance"` binds the prefix `xsi` to the XML Schema instance namespace, which defines two attributes used for validation: `schemaLocation` and `noNamespaceSchemaLocation`.

`xsi:schemaLocation` pairs a namespace URI with the URL of the schema that describes it. A validating parser can therefore find the schema and check the document.

CORE CONCEPT

A namespace URI is an identifier, not an address. `http://maven.apache.org/POM/4.0.0` does not have to be fetchable. Its purpose is to be a globally unique string that everyone agrees means the same vocabulary. That is why the same URI appears unchanged in every Maven project, and why it is a common examination point.

18.3 XML Schema

A schema is itself an XML document that describes which elements and attributes another XML document may contain. Two schema languages exist: DTD and XSD. XSD is the modern one and the one the syllabus material covers.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="basin">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="zone" maxOccurs="unbounded">
                    <xs:complexType>
                        <xs:sequence>
                            <xs:element name="name" type="xs:string"/>
                            <xs:element name="residents" type="xs:nonNegativeInteger"/>
                        </xs:sequence>
                        <xs:attribute name="id" type="xs:string" use="required"/>
                        <xs:attribute name="waterLevelMeters" type="xs:decimal" use="required"/>
                    </xs:complexType>
                </xs:element>
            </xs:sequence>
            <xs:attribute name="name" type="xs:string"/>
        </xs:complexType>
    </xs:element>
</xs:schema>
```

illustrative — a schema describing the basin document

A schema buys you four things, and each corresponds to something you would otherwise have to check in code.

Table 18.2 — *What a schema expresses, and the Java construct that expresses the same idea.*

Schema feature	What it constrains	Java equivalent
Element and attribute declarations	Which names may appear	The fields of a class
`type`	What values are allowed	The declared type of a field
`use="required"`	Whether something must be present	A constructor that rejects null
`maxOccurs="unbounded"`	How many times something may repeat	A `List` field
`xs:sequence`	The required order of children	The parameter order of a constructor
`xs:nonNegativeInteger`	A value range	A range check in a compact constructor

CONNECTION TO ANOTHER CONCEPT

That comparison is the useful way to think about schemas. A schema is validation declared outside the program, in a language a validator can read. BasinWatch does the same work in Java: `SensorReading`'s compact constructor rejects out-of-range values, `SessionSnapshot.validate` re-checks restored state, and the database's `NOT NULL` constraints reject missing columns. Three mechanisms, one purpose.

RELATED TECHNIQUE — NOT USED IN BASINWATCH

BasinWatch does not read or write any XML of its own. Its only XML file is `pom.xml`, which Maven reads. The project has no configuration XML, no schema, and no XML parser on its classpath. The XML material in this chapter is syllabus content and ecosystem knowledge — it is what you need to understand the build file and to recognise the technique when you meet it in a framework. It is not part of the application's runtime design, and saying so is more honest than inventing a use for it.

18.4 Annotations

An annotation is metadata attached to a declaration. It does not change what the code does by itself; it records information that a compiler or a framework can read.

```
@Override                                   // read by the compiler
public String toString() { return name; }

@Deprecated                                 // read by the compiler and by tools
public void oldDispatch() { }

@FunctionalInterface                        // read by the compiler
public interface BasinEvent { /* ... */ }

@SuppressWarnings("unchecked")              // read by the compiler
void copy(List<?> source) { }

// An annotation with members — this is the shape frameworks use
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Component {
    String value() default "";
}
```

illustrative — standard annotations and one framework-style annotation

Table 18.3 — *The three retention policies, and why the third is the one frameworks need.*

Retention	Available at	Used for
`SOURCE`	Compile time only; not in the class file	`@Override`, `@SuppressWarnings`
`CLASS`	In the class file, but not readable at runtime by default	Some build-tool and analysis annotations
`RUNTIME`	In the class file and readable by reflection at runtime	Framework annotations such as `@Component`, `@Autowired`

CORE CONCEPT

That table explains why annotations matter to frameworks. If an annotation's retention is `RUNTIME`, a framework can discover it by reflection and act on it — that is the entire mechanism behind Spring's component scanning, JPA's entity mapping and JUnit's test discovery. Without runtime retention, an annotation is only a message to the compiler.

BASINWATCH IN PRACTICE

BasinWatch uses standard annotations only, and each one is doing exactly what its retention promises:

```
@Override public String toString() { ... }        // SOURCE — compiler checks the signature
@FunctionalInterface public interface BasinEvent { ... }  // SOURCE — compiler enforces one method
@SuppressWarnings("unchecked") ...                          // SOURCE — silences one warning
```

Listing 18.1 — *from BasinWatch — standard annotations, no framework annotations.*

There is no `@Component`, no `@Autowired` and no `@Entity` anywhere in the project, because there is no framework to read them.

18.5 POJO, JavaBean and the wiring they enable

Chapter 2 introduced the distinction. Here is why it exists, which is the part that matters when you meet a framework.

Diagram 27 — Annotations, JavaBeans and dependency wiring

How a framework learns what your classes mean without reading a configuration file.

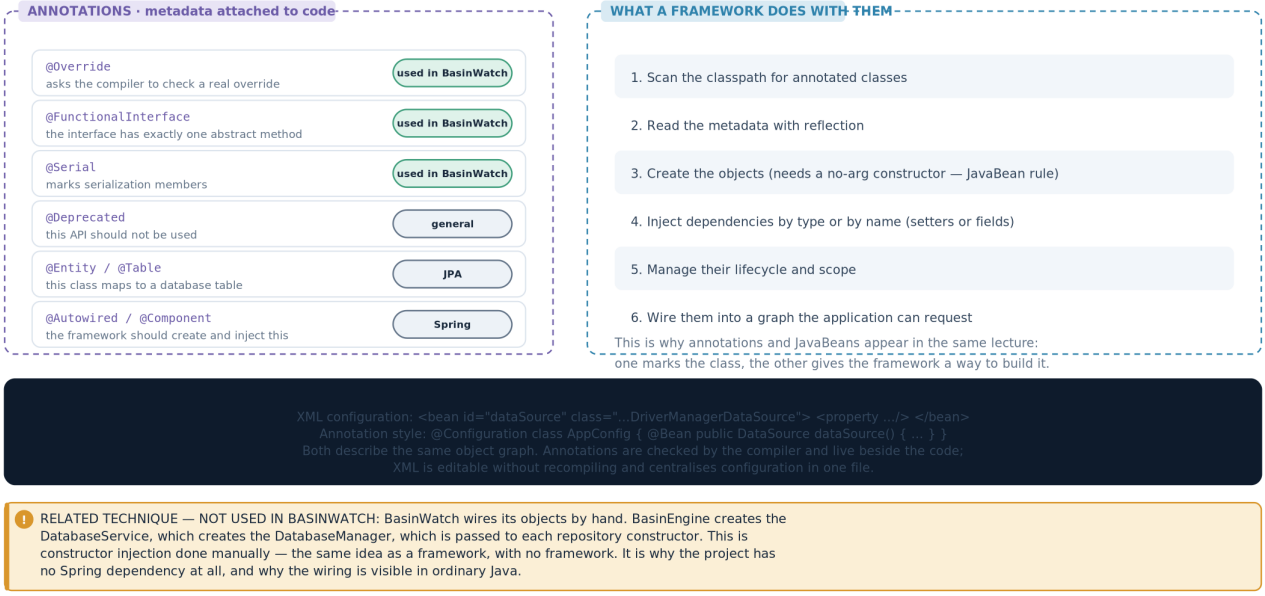


Diagram 27 — Hand-written wiring versus framework wiring. Left: BasinWatch constructs its services explicitly, passing each dependency through a constructor. Right: a framework scans annotated classes, instantiates JavaBeans reflectively, and injects dependencies by matching types. Both produce the same object graph.

Table 18.4 — *Two ways to assemble an object graph.*

Aspect	BasinWatch (hand-written wiring)	A framework such as Spring
How objects are created	Explicit `new` calls in constructors	Reflection, driven by annotations or XML
How dependencies arrive	Constructor arguments	Injected fields, setters or constructors
What the classes must look like	Anything; records and final classes are fine	Usually a JavaBean: no-argument constructor and setters, or an annotated constructor
Where the wiring lives	In the source, readable with the class	In annotations plus the framework's scan configuration
Compile-time safety	A missing dependency is a compile error	A missing bean is a runtime error
Discoverability	Follow the constructor	Search for the annotation

RELATED TECHNIQUE — NOT USED IN BASINWATCH

BasinWatch wires by hand. `BasinEngine` constructs the `DatabaseService`, which constructs the `DatabaseManager` and passes it to each repository's constructor. That is constructor injection, performed manually and visibly. It is a legitimate choice for an application of this size: there are six packages and roughly two dozen classes, so a framework would add a dependency, a start-up scan and a class of runtime errors in exchange for saving perhaps thirty lines. A larger application with hundreds of components would make the opposite trade, which is the subject of Chapter 19.

EXAM TIP

If asked "what is the difference between a POJO and a JavaBean?", give the convention first — no-argument constructor, private fields, public accessors, often Serializable — and then the reason: a framework needs to instantiate and populate the object without knowing anything about it. Adding the reason is what turns a definition into an explanation. Then note that every JavaBean is a POJO but not the reverse, and that a record can never be a JavaBean.

CHAPTER SUMMARY

- XML represents structured data as a tree of elements, attributes and text.
- A document is well-formed if it follows XML syntax; it is valid if it also conforms to a schema.
- A namespace URI is a unique identifier for a vocabulary, not necessarily a fetchable address.
- An XSD schema declares which elements and attributes may appear, in what order and with what types.
- An annotation is metadata; its retention policy determines whether it is available at runtime.
- Runtime retention is what allows frameworks to discover annotations by reflection.
- BasinWatch uses standard annotations only, and contains no XML of its own beyond pom.xml.
- A JavaBean's conventions exist so a framework can instantiate and populate it reflectively; BasinWatch instead wires its objects by hand.

KEY TERMS

- **Well-formed** — following XML's syntax rules.
- **Valid** — well-formed and conforming to a schema.
- **Namespace** — a URI-qualified vocabulary that prevents name collisions in a document.
- **XSD** — XML Schema Definition, a schema language written in XML itself.
- **Annotation retention** — how long an annotation survives: source, class or runtime.
- **Reflection** — inspecting and using classes and members at runtime, which is how frameworks read annotations.

CHAPTER 19

Raw JDBC versus Spring JdbcTemplate

> The repetitive pattern that raw JDBC forces you to write, the abstraction that removes it, and an honest comparison with what this project actually does.

19.1 The repetition in raw JDBC

Look back at any repository method in Chapter 12 and count the lines that are about the query rather than about the resource handling.

```
public List<DatabaseHistoryEntry> loadRecent(int limit) throws SQLException {
    String sql = "SELECT ... FROM operational_events ORDER BY occurred_at DESC LIMIT ?";
    ArrayList<DatabaseHistoryEntry> entries = new ArrayList<>();
    try (Connection connection = database.openConnection()) {                // boilerplate
        try (PreparedStatement statement = connection.prepareStatement(sql)) { // boilerplate
            statement.setInt(1, limit);                                      // the actual work
            try (ResultSet rows = statement.executeQuery()) {                // boilerplate
                while (rows.next()) {                                        // boilerplate
                    entries.add(new DatabaseHistoryEntry(                    // the actual work
                            DatabaseHistoryType.valueOf(rows.getString("event_type")),
                            rows.getString("zone_id"),
                            rows.getString("detail"),
                            Instant.parse(rows.getString("occurred_at"))));
                }
            }
        }
    }
    return List.copyOf(entries);
}
```

from BasinWatch, db/OperationalEventRepository.java — three lines of intent inside ten lines of resource handling.

The structure is identical in every method: open a connection, prepare a statement, bind parameters, execute, iterate, close everything. Only the SQL and the row mapping change. That is the definition of code that wants to be abstracted.

CORE CONCEPT

Repetition like this is not merely ugly — it is a source of bugs. Every place that must remember to close a connection is a place that might forget. Every place that must bind parameters in the right order is a place that might get the order wrong. Raw JDBC in BasinWatch is safe because it is written carefully in a handful of repository classes; in a larger codebase, the same pattern repeated two hundred times would not stay safe.

19.2 What Spring JdbcTemplate does

Spring's `JdbcTemplate` is a thin wrapper over the JDBC interfaces you already know. It performs exactly the steps in that method, and asks you to supply only the two things that vary: the SQL, and the code that turns a row into an object.

```
// ---- the same query, with JdbcTemplate ----
String sql = "SELECT occurred_at, event_type, zone_id, detail "
           + "FROM operational_events ORDER BY occurred_at DESC LIMIT ?";
```

```
List<DatabaseHistoryEntry> entries = jdbcTemplate.query(sql,
        (rows, rowNumber) -> new DatabaseHistoryEntry(
                DatabaseHistoryType.valueOf(rows.getString("event_type")),
                rows.getString("zone_id"),
                rows.getString("detail"),
                Instant.parse(rows.getString("occurred_at"))),
        limit);
```

lecture-note style — the same query expressed with Spring JdbcTemplate

Compare the two listings. The intent is identical and the SQL is identical. What has disappeared is every line about connections, statements, result sets and closing — because `JdbcTemplate` does all of it. The lambda in the middle is the **RowMapper**: given a row, produce an object.

Table 19.1 — *What JdbcTemplate takes over, and what remains your responsibility.*

Raw JDBC step	Who does it with JdbcTemplate	What you still supply
Obtain a connection	`JdbcTemplate`, from its `DataSource`	Nothing
Prepare the statement	`JdbcTemplate`	The SQL text with `?` placeholders
Bind parameters	`JdbcTemplate`, from the argument list	The values, in order
Execute	`JdbcTemplate`	Nothing
Iterate the result set	`JdbcTemplate`	The `RowMapper` lambda
Close the result set, statement and connection	`JdbcTemplate`	Nothing
Translate `SQLException`	`JdbcTemplate`, into a `DataAccessException` hierarchy	Nothing

19.3 The DataSource, and DriverManagerDataSource

`JdbcTemplate` does not call `DriverManager` itself. It asks a `DataSource` for connections.

DEFINITION

DataSource — an interface representing a source of database connections. Code asks it for a connection; how that connection is created — a direct call, a pool, a JNDI lookup — is the DataSource's business.

```
// ---- a DataSource configured by hand, as the lecture notes show ----
@Bean
public DataSource dataSource() {
    DriverManagerDataSource dataSource = new DriverManagerDataSource();
    dataSource.setDriverClassName("org.sqlite.JDBC");
    dataSource.setUrl("jdbc:sqlite:data/basinwatch.db");
    return dataSource;
}

@Bean
public JdbcTemplate jdbcTemplate(DataSource dataSource) {
    return new JdbcTemplate(dataSource);
}
```

lecture-note style — DriverManagerDataSource and JdbcTemplate declared as beans

`DriverManagerDataSource` is a simple `DataSource` implementation that creates a brand-new connection on every call and never pools them. It is useful for teaching and for tests, and it is not what you would use in a production application — there you would use a pooling implementation such as HikariCP, because opening a physical connection per query is expensive.

TECHNICAL CLARIFICATION

The lecture material configures these as Spring beans, using either `@Bean` methods in a configuration class or XML `<bean>` elements. Both are the same idea expressed two ways: a container creates and holds the objects, and injects them where needed. The `DriverManagerDataSource` class itself lives in `org.springframework.jdbc.datasource` and is a teaching-friendly convenience rather than a production choice — its own documentation says so.

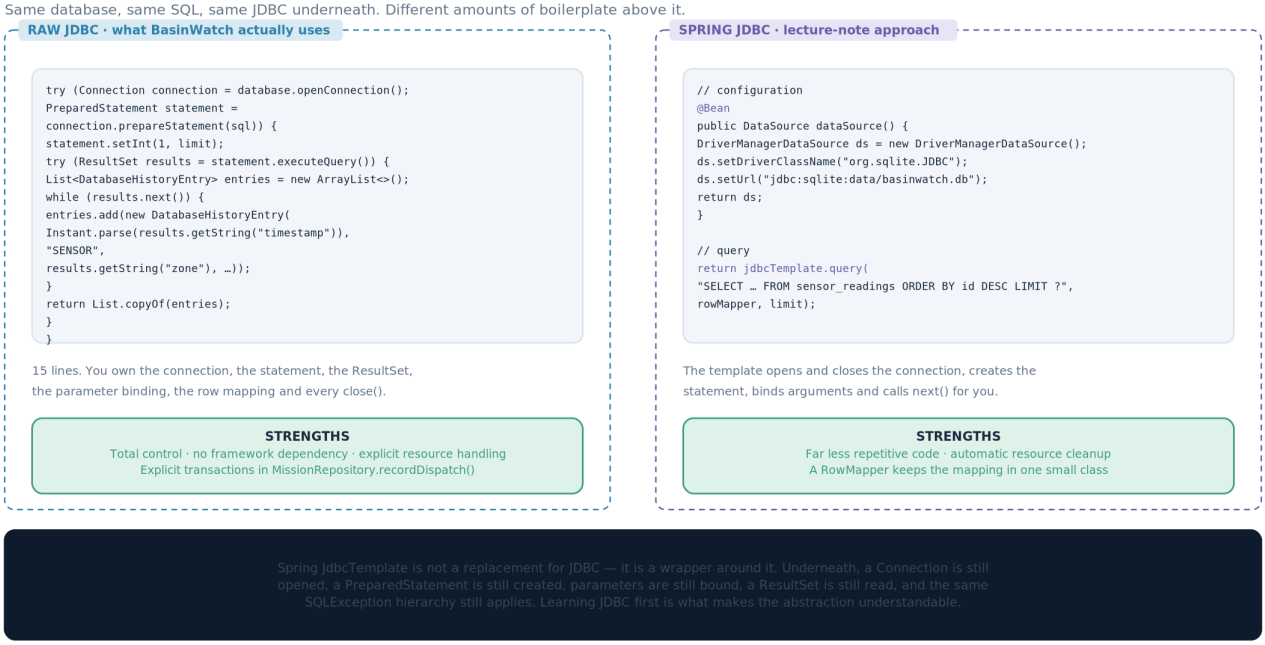
RELATED TECHNIQUE — NOT USED IN BASINWATCH

BasinWatch does not use JdbcTemplate, Spring, or a DataSource. It calls `DriverManager.getConnection` directly inside `DatabaseManager` and passes a `Connection` to each repository. That is raw JDBC. Nothing in this chapter should be read as a description of the project's implementation — it is the alternative approach the syllabus teaches, presented so you can compare it with what the project does.

19.4 Side by side

Diagram 28 — Raw JDBC versus Spring JdbcTemplate

Same database, same SQL, same JDBC underneath. Different amounts of boilerplate above it.



	Raw JDBC (BasinWatch)	JdbcTemplate (lecture notes)
Boilerplate	high — you write the try-with-resources	low — the template handles it
Control	complete, including transactions	good, with TransactionTemplate for explicit control
Dependencies	JDK only	Spring JDBC plus a DataSource implementation

Diagram 28 — Raw JDBC and Spring JdbcTemplate. Left: the project's raw path, where the application opens a connection through DriverManager and manages every resource itself. Right: the JdbcTemplate path, where a DataSource supplies connections and the template manages resources, leaving only SQL and row mapping to the application.

Table 19.2 — *The two approaches compared, with the project's actual choice stated.*

Question	Raw JDBC (BasinWatch)	Spring JdbcTemplate
Who opens the connection?	`DatabaseManager.openConnection()`	The `DataSource` behind the template
Who closes it?	try-with-resources in each repository method	`JdbcTemplate`
Who binds parameters?	Typed setters, written by hand	`JdbcTemplate`, from the argument list
Who maps rows?	A loop inside the method	The `RowMapper` you supply
Exception type	`SQLException`	`DataAccessException`
Transactions	`setAutoCommit(false)` and `commit()` written by hand	`@Transactional` or a `TransactionTemplate`
Extra dependencies	The driver only	Spring Core, Spring JDBC, plus the driver
Lines per query	Roughly fifteen	Roughly five
Compile-time checking of the SQL	None	None

Question	Raw JDBC (BasinWatch)	Spring JdbcTemplate
Used in BasinWatch?	**Yes**	**No**

CORE CONCEPT

Two conclusions follow from that table, and both are worth holding together.

JdbcTemplate is not a replacement for JDBC. It is a class that calls the same `Connection`, `PreparedStatement` and `ResultSet` interfaces you studied in Chapter 12. Everything you learned about parameter binding, result-set iteration and transactions still applies — you have simply delegated the resource handling.

Neither approach is wrong. For an application with one database and a dozen queries, raw JDBC in a few repository classes is perfectly reasonable and keeps the dependency list to one jar. For an application with hundreds of queries, pooling, declarative transactions and several data sources, the framework earns its place.

EXAM TIP

A very likely question is "why does Spring JdbcTemplate not eliminate the need for JDBC?" The answer is that it is built *on* JDBC: it obtains connections from a `DataSource`, creates `PreparedStatements`, binds parameters, iterates `ResultSets` and closes everything — all through the same `java.sql` interfaces. It removes repetition, not the underlying protocol. Saying that plainly is usually worth full marks.

CHAPTER SUMMARY

- Raw JDBC forces the same resource-handling structure into every method, with only the SQL and row mapping varying.
- JdbcTemplate performs that structure for you and asks only for the SQL and a RowMapper.
- A DataSource supplies connections; JdbcTemplate never calls DriverManager directly.
- DriverManagerDataSource creates a new connection per call and does not pool; it is for teaching and tests.
- JdbcTemplate translates SQLException into Spring's DataAccessException hierarchy.
- JdbcTemplate is built on the same java.sql interfaces, so it removes repetition rather than JDBC itself.
- BasinWatch uses raw JDBC exclusively; the JdbcTemplate material is a taught alternative, not a description of the project.

KEY TERMS

- **JdbcTemplate** — a Spring class that executes SQL and manages JDBC resources, leaving SQL and row mapping to the caller.
- **RowMapper** — a callback that converts one result-set row into an object.
- **DataSource** — an abstraction representing a source of database connections.
- **DriverManagerDataSource** — a simple non-pooling DataSource implementation.
- **DataAccessException** — Spring's unchecked exception hierarchy for data access failures.

PART VI

Putting Everything Together

This part returns to the whole system. Four real scenarios are traced from their origin to their visible result, crossing every thread and every layer the book has described. Then the final exercise gives you the eight questions that turn knowing the parts into explaining the whole.

20 End-to-End BasinWatch Architecture Walkthrough

21 Final Integration Exercise

CHAPTER 20

End-to-End BasinWatch Architecture Walkthrough

> Four scenarios traced through every layer, each following a genuinely different route. This is the integrative centerpiece of the book.

20.1 Why four scenarios and not one

It is tempting to trace a single "typical" request through the system and declare the architecture explained. That would be misleading, because BasinWatch has four genuinely different data paths and forcing them into one diagram would misrepresent at least three of them.

Table 20.1 — *The four scenarios and how each differs.*

Scenario	Origin	Threads crossed	Persists to	Visible result
A — an observation	Sensor feed thread	Feed, intake, worker, database	Log, archive, database	Map colour and activity timeline
B — a dispatch	Operator click	EDT, worker, control, database	Log, database (transaction)	Resource board and mission list
C — save and restore	Operator click, then next launch	EDT, control	Session snapshot file	The whole restored model
D — a history read	Operator click	EDT, database, EDT	Nothing — it is a read	A populated history table

CORE CONCEPT

Read the table's second and third columns together. Scenario A is the only one that starts without a user. Scenario D is the only one that writes nothing. Scenario B is the only one that needs a transaction. Scenario C is the only one whose result is visible on a *later* run. Those differences are why one trace would not be enough.

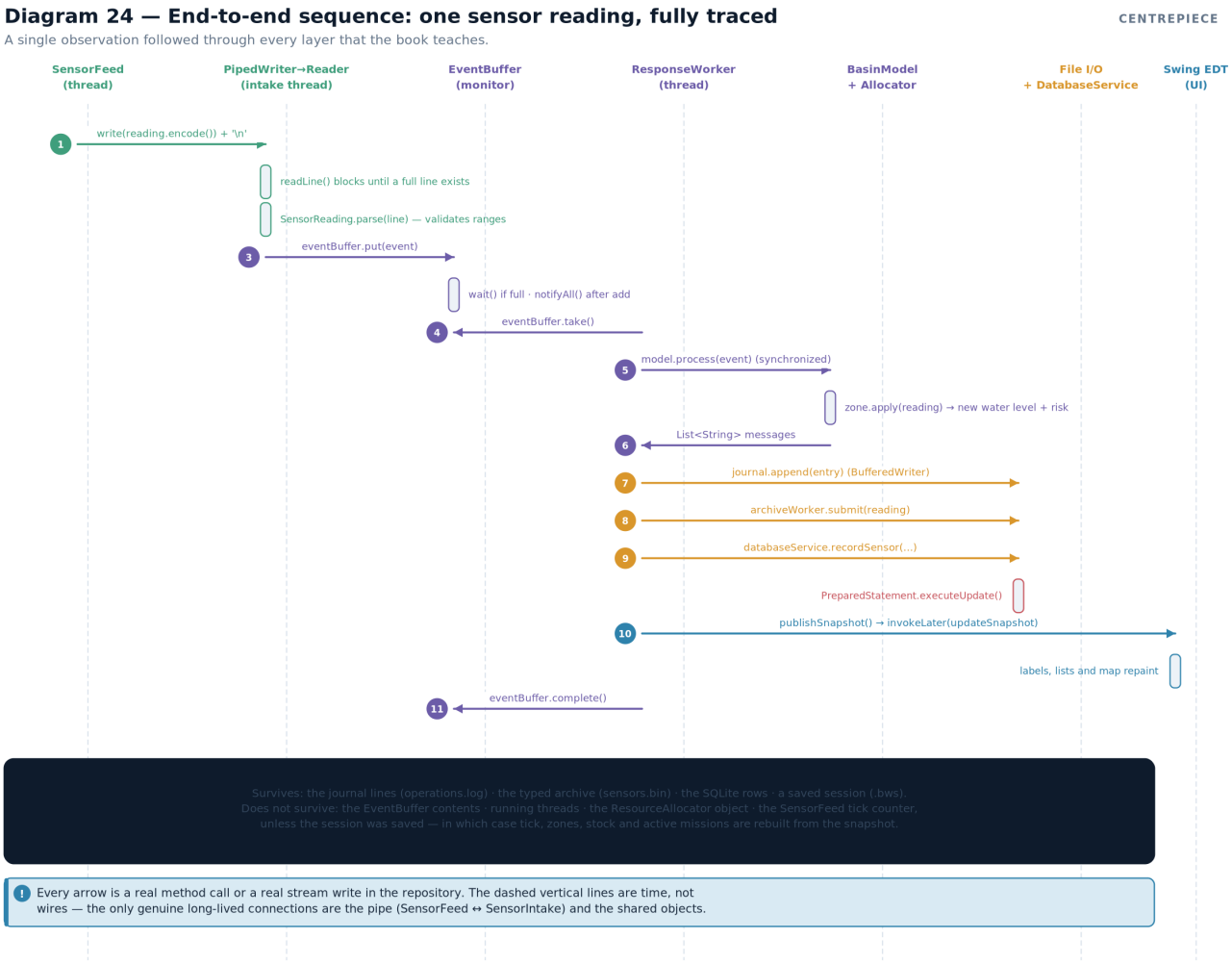


Diagram 24 — The full sequence diagram, with all four scenarios. Repeated from Diagram 24 in Chapter 16 so the trace can be read beside the scenarios. The complete sequence across the operator, the event-dispatch thread, the engine, the sensor path, the workers, the file layer, the database thread and the SQLite file. Scenarios A, B, C and D are marked so you can follow each separately.

20.2 Scenario A: an observation, fully traced

This is the system's heartbeat. Nothing is clicked; the application produces its own data.

Scenario A — from the feed loop to a coloured zone on the map

1. **1. The feed loop wakes.** `SensorFeed.run` is in its timed loop on `basin-sensor-feed`. It increments the shared tick counter and begins a new interval.
2. **2. Six readings are generated.** For each of the six zones, a `SensorReading` is constructed with randomised rainfall and rise values within the configured bounds. The compact constructor validates the ranges as it is built.
3. **3. Each reading is encoded and written.** `reading.encode()` produces a pipe-delimited line such as `READ|214|MIL-ST-2|MIL|12.400|3.100`, which is written into the `PipedWriter` along with a line separator.
4. **4. The pipe transfers and applies back-pressure.** Characters move into the pipe's 8,192-character buffer. If intake has fallen behind and the buffer is full, the feed's write blocks here — the producer waits rather than overrunning memory.
5. **5. Intake reads a line.** On `basin-sensor-intake`, `SensorIntake.run` is blocked in `source.readLine()`. It wakes, receives the line, and returns it without the terminator.
6. **6. The record is parsed and validated.** `SensorReading.parse` splits on the pipe character and checks that there are exactly six fields beginning with `READ`. A malformed line is reported and skipped; the loop continues.
7. **7. The event is queued.** `eventBuffer.put(event)` adds it to the bounded deque. If the buffer is full, this call waits on the monitor; if it is closed, the method returns false and intake stops.
8. **8. A worker takes it.** One of the three `ResponseWorker` threads is waiting in `eventBuffer.take()`. It wakes, removes the event, and calls `model.process(event)`.
9. **9. The model applies it.** `applyReading` finds the zone, calls `BasinZone.apply(reading)`, and — crucially — receives false if the reading is stale. The risk level is recomputed from the new water level.
10. **10. Escalation is detected.** The new risk rank is compared with the previous one. If it increased, the activity message is prefixed with “Risk increased:” and a boolean is prepared for the database.
11. **11. Activity is recorded and the snapshot is published.** An `ActivityEntry` is added to the model's ring buffer, and `model.snapshot()` produces a new immutable `BasinSnapshot`.
12. **12. The listener chain fires.** `BasinEngine` calls every snapshot listener on the worker thread. Each listener's first action is `SwingUtilities.invokeLater`.
13. **13. The EDT updates the interface.** The map panel's snapshot field is replaced and `repaint()` is requested; the activity list and labels are updated. The zone's circle changes colour.
14. **14. Persistence runs in parallel.** The engine appends each activity message to the journal, submits the reading to the archive worker, and — only if the model changed — calls `database.recordSensor(...)`.
15. **15. The database thread writes.** On `basin-database`, the repository prepares the nine-placeholder insert, binds every value, and executes it. If the risk escalated, a second row is written to `operational_events`.

VISUAL WALKTHROUGH

Follow the thread changes in Scenario A: feed, intake, worker, then EDT *and* database in parallel. Steps one to three are the feed thread. Steps five to seven are intake. Steps eight to twelve are a worker. Steps thirteen and fifteen happen concurrently on two different threads. Only step thirteen touches Swing.

BASINWATCH IN PRACTICE

Step nine is worth pausing on, because it is where the architecture earns its keep. The model decides whether a reading counts, and everything downstream obeys that decision:

```
// BasinZone.apply returns false for a stale reading
boolean changed = zone.apply(reading);
if (!changed) {{
    // no activity entry, no archive record, no database row
    return;
}}
```

One boolean, decided in the domain, determines whether three separate persistence mechanisms write anything at all. That is what "the model is the source of truth" means in practice.

20.3 Scenario B: a dispatch, and its transaction

This scenario starts with a person and ends with a changed resource board. It is also the only scenario that needs a database transaction.

Scenario B — from a dispatch click to a changed resource board

1. **1. The operator selects a mission type and a zone, then clicks Dispatch.** Two combo boxes and a button, all on the EDT.
2. **2. The listener calls the engine.** `engine.dispatch(zoneId, missionType)` — one call, returning immediately.
3. **3. The engine submits the work.** The request is submitted to `basin-control` and the caller receives a `CompletableFuture`.
4. **4. The control thread reserves resources.** `ResourceAllocator.tryReserve(requirements)` runs under the allocator's monitor: it checks every requirement, and only if all are satisfied does it decrement any of them.
5. **5. A failed reservation stops here.** If any resource is short, the method returns false, no state changes, and the future completes with a message the interface displays. Nothing is persisted, because nothing happened.
6. **6. A successful reservation creates a mission.** `BasinModel` constructs a `ResponseMission` with an identifier, a start tick and a completion tick derived from the mission type's duration.
7. **7. The model publishes a new snapshot.** The resource board now shows the deduction and the mission list shows the new operation. The listener chain posts both to the EDT.
8. **8. Persistence begins.** The engine appends an ALLOCATED entry to the journal and calls `database.recordDispatch(...)`.
9. **9. The database transaction opens.** On `basin-database`, a connection is obtained and `setAutoCommit(false)` is called.
10. **10. Three writes execute on that one connection.** The mission row, one resource-usage row per required resource, and an audit event — all through package-private repository overloads that accept the shared connection.
11. **11. The transaction commits.** All three rows become visible together, or none of them does.
12. **12. A failure rolls back.** If any of the three throws, `rollback()` discards all of them, and the status listener reports the failure to the interface.
13. **13. Completion is scheduled.** In the model, the mission's completion tick is in the future. When the tick arrives, a later pulse completes the mission, releases the resources, and records a second transaction.

VISUAL WALKTHROUGH

Notice step four and step eleven are the same idea implemented twice. Step four makes an *in-memory* change atomic using a monitor; step eleven makes a *database* change atomic using a transaction. Both exist to prevent a partially completed change from being observable. Chapter 13 makes this comparison explicitly.

COMMON MISTAKE

A common wrong design puts the resource reservation *after* the database write. Then a failed reservation leaves a mission row claiming resources that were never deducted, and the history contradicts the live state. The order matters: reserve first, because the in-memory reservation is the decision, and the database records the decision.

20.4 Scenario C: save, exit, and restore

This scenario is the only one whose result is visible on a later run, and it is the reason the snapshot format is versioned and validated.

Scenario C — from a save click to a restored basin on the next launch

1. **1. The operator clicks Save.** On the EDT, the listener opens a file chooser and obtains a target path.
2. **2. The listener calls the engine.** `engine.save(target)`, returning a future.
3. **3. The control thread takes a snapshot.** `model.snapshot()` produces an immutable `BasinSnapshot` while holding the model's monitor, so the copy is consistent.
4. **4. The snapshot is converted to a DTO.** `SessionSnapshot.from(snapshot)` copies zones, resources, missions and recent activity into serializable plain classes.
5. **5. A temporary file is written.** `SnapshotStore.save` opens `name.bws.tmp`, writes the format version, then serializes the DTO.
6. **6. The temporary file is moved into place.** `Files.move(..., ATOMIC_MOVE)` replaces the real file in one step, so an interrupted save cannot corrupt an existing snapshot.
7. **7. The status is reported.** The future completes with the path, and the callback posts the message to the EDT.
8. **8. Later, the application exits cleanly.** `closeEngine()` stops the feed, closes the pipe, joins intake, drains the archive queue, flushes the journal, and writes `last-session.bws` automatically.
9. **9. On the next launch, the engine checks for a restore point.** It looks for `last-session.bws` in the saves directory.
10. **10. Four gates are applied on load.** The file size is bounded, an `ObjectInputFilter` approves only `com.basinwatch`, `java.lang`, `java.util` and `java.time` classes, the format version is compared, and the DTO is validated.
11. **11. The model is rebuilt.** Zones are restored with their water levels, the resource allocator is refilled, missions are reinstated with their remaining durations, and the activity ring buffer is repopulated.
12. **12. The interface shows the restored state.** The map, the resource board, the mission list and the activity timeline all reflect the saved session, and the tick counter resumes.

VISUAL WALKTHROUGH

Count the differences between the save path and the restore path. Saving is straightforward: copy, convert, write, move. Restoring is defensive: four gates before a single value is trusted. That asymmetry is deliberate and it is the single most important thing to understand about serialization in an application — writing is your own data, reading is unverified input.

20.5 Scenario D: a history read

This scenario writes nothing. It is the shortest of the four, and it is the one that most directly connects JDBC to Swing.

Scenario D — from a Refresh click to a populated table

1. **1. The operator clicks Refresh.** Delivered to the EDT and dispatched to the button's listener.
2. **2. The listener calls `historyPanel.reload()` and returns.** Nothing blocks.
3. **3. The panel asks the engine.** `engine.loadHistory(200)` returns a future immediately.
4. **4. Availability is checked.** If the database is unavailable, an already-completed future holding an empty list is returned and steps five to eight are skipped.
5. **5. The query is enqueued.** A lambda is submitted to `basin-database`.
6. **6. The repository prepares and binds.** One placeholder for the limit, bound with `setInt(1, limit)`.
7. **7. SQLite executes and returns rows.** The index on `operational_events` serves the descending time order.
8. **8. Rows are mapped and the future completes.** Each row becomes a `DatabaseHistoryEntry`; `future.complete(entries)` runs the callback on the database thread.
9. **9. The callback posts to the EDT.** `SwingUtilities.invokeLater`.
10. **10. The table model is updated.** `setEntries(entries)` copies the list and calls `fireTableDataChanged()`.
11. **11. The table repaints.** It asks the model for the row count and redraws the visible rows.

20.6 Where the four scenarios differ, and why that matters

Placing the four traces side by side shows the architecture's real shape more clearly than any single diagram.

Table 20.2 — *The four scenarios compared across the dimensions that matter.*

	A — observation	B — dispatch	C — save/restore	D — history read
Initiated by	A timer	A click	A click, then a later launch	A click
First thread	Feed	EDT	EDT	EDT
Crosses the pipe?	**Yes**	No	No	No
Needs a transaction?	No — single-row inserts	**Yes** — three related writes	No	No — read only
Writes files?	Log and archive	Log	**Snapshot**	No
Writes the database?	Yes, if the model changed	Yes, transactionally	No	No
Result visible immediately?	Yes, on the map	Yes, on the resource board	No — on the next launch	Yes, in the table
Can fail silently?	Yes — a malformed record is skipped	No — a failed reservation is reported	No — a failed save is reported	Yes — an unavailable database shows empty history

20.7 The dependency graph

Finally, the static picture: who depends on whom. Dependencies point one way, and that direction is what makes the layers testable and replaceable.

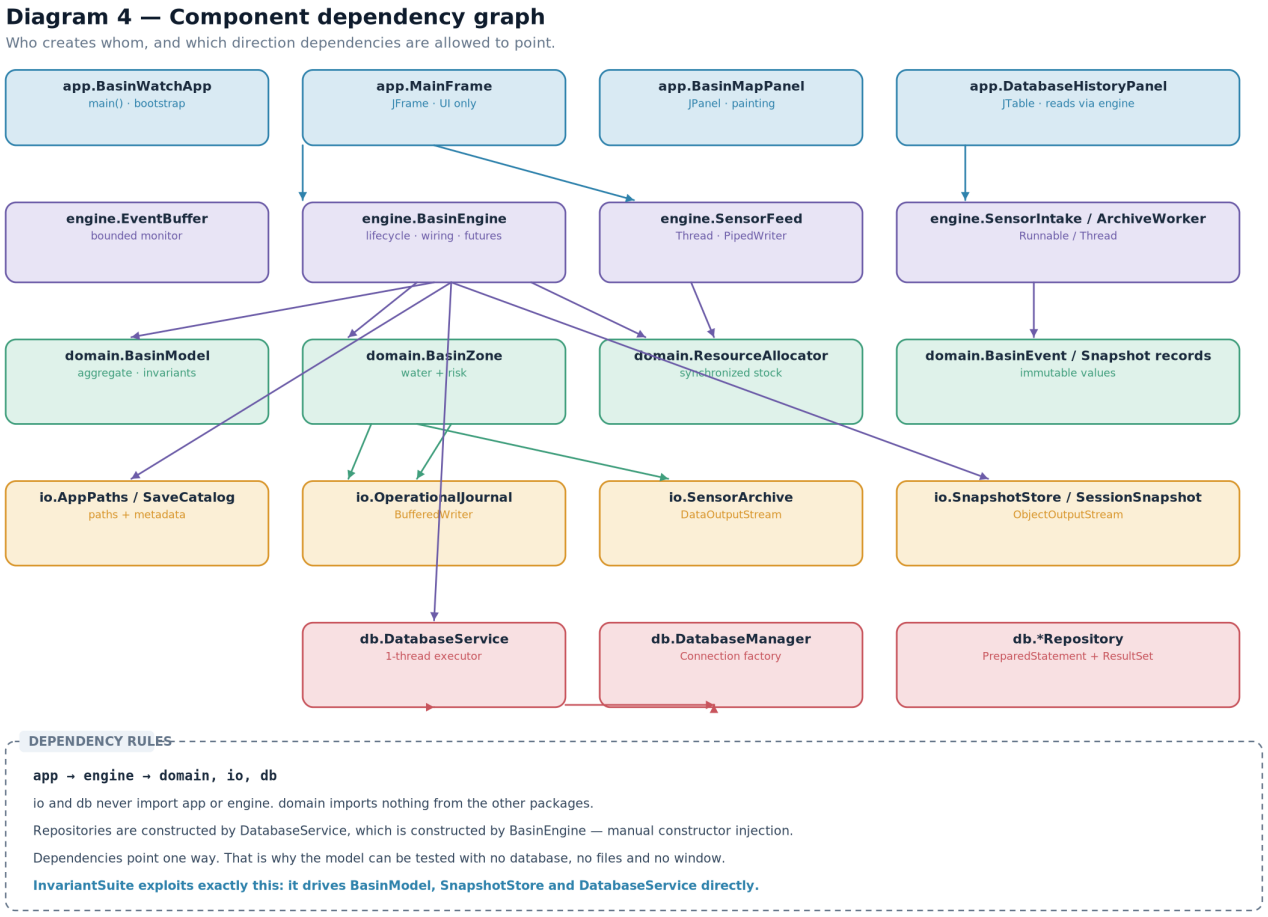


Diagram 4 — The component dependency graph, revisited. Repeated from Diagram 4 in Chapter 3, now read as a design argument. Arrows point from a component to what it depends on. The app package depends on the engine, the engine depends on the domain, io and db packages, and none of those depends back on the app. The domain depends on nothing.

Three properties of that graph are worth naming.

The domain is at the bottom and depends on nothing. No file, no database, no Swing. That is why the model can be tested in a unit test with no setup at all, and why the same model could be driven by a different interface.

There are no cycles. Nothing the engine depends on depends on the engine. A cycle would make the layers impossible to test in isolation and impossible to replace.

The interface is at the top and depends on the engine, not the reverse. The engine has no import of `javax.swing` anywhere, which is what allows the whole application to run headless in a test.

> **CONNECTION TO ANOTHER CONCEPT**
>
> This chapter has traced four paths through that graph. If you can redraw Diagram 4 from memory, name the thread for each box, and describe one scenario that crosses it, you can answer almost any architecture question about this application.

EXAM TIP

In a viva, the strongest thing you can do is offer to trace a scenario and then choose the one that best matches the question. If the question is about persistence, trace C. If it is about transactions, trace B. If it is about threading, trace A. If it is about the interface and the database together, trace D. Choosing the scenario is itself evidence that you understand the architecture.

CHAPTER SUMMARY

- Four genuinely different paths exist: an observation, a dispatch, a save-and-restore, and a history read.
- Scenario A crosses the pipe and fans out to three persistence mechanisms, but writes to the database only when the model changed.
- Scenario B is the only transactional path, and it reserves resources before recording anything.
- Scenario C is defensive on the way in and straightforward on the way out, because reading is unverified input.
- Scenario D writes nothing and is the clearest example of the JDBC-to-Swing path.
- The dependency graph is acyclic, with the domain at the bottom and the interface at the top.
- The engine has no dependency on Swing, which is what makes the application testable without a window.

KEY TERMS

- **Fan-out** — one decision causing several independent downstream operations.
- **Restore point** — a saved state the application checks for at start-up.
- **Acyclic dependency graph** — a component graph with no cycles, so each layer can be tested and replaced independently.
- **Source of truth** — the single component whose decision everything else obeys; here, the domain model.

CHAPTER 21

Final Integration Exercise

> A guided walkthrough of the real code, the eight questions that turn knowledge of parts into an explanation of the whole, and where every syllabus concept lives.

21.1 How to use this chapter

This is not a summary. It is an exercise. Read the guided walkthrough below, then answer the eight questions from memory, using the code as a reference only to check yourself. If you can answer all eight about any class in the project, you can explain the application.

21.2 Guided walkthrough: following one class completely

Take `SensorIntake`. It is a small class — a single `run` method — but it sits at the junction of streams, threads, validation and queueing, so it is the ideal subject for the exercise.

```
public final class SensorIntake implements Runnable {
    private final BufferedReader source;
    private final EventBuffer buffer;
    private final Consumer<String> warningReporter;
    private final Consumer<String> statusReporter;

    public SensorIntake(BufferedReader source, EventBuffer buffer,
                        Consumer<String> warningReporter, Consumer<String> statusReporter) {
        this.source = source;
        this.buffer = buffer;
        this.warningReporter = warningReporter;
        this.statusReporter = statusReporter;
    }

    @Override
    public void run() {
        statusReporter.accept("Sensor intake online");
        try (source) {
            String line;
            while ((line = source.readLine()) != null) {
                BasinEvent event;
                try {
                    if (line.startsWith("PULSE|")) {
                        event = new SimulationPulse(Long.parseLong(line.substring(6)));
                    } else {
                        event = SensorReading.parse(line);
                    }
                } catch (IllegalArgumentException ex) {
                    warningReporter.accept("Discarded malformed sensor record: " + ex.getMessage());
                    continue;
                }
                if (!buffer.put(event)) {
                    warningReporter.accept("Event buffer closed; intake stopping.");
                    break;
                }
            }
        } catch (IOException ex) {
            warningReporter.accept("Sensor intake ended with an I/O error: " + ex.getMessage());
        } catch (InterruptedException ex) {
            Thread.currentThread().interrupt();
        } finally {
```

```
            statusReporter.accept("Sensor intake offline");
        }
    }
}
```

from BasinWatch, engine/SensorIntake.java — the complete class, unedited.

Now apply the eight questions to it.

1. Why is this class needed?

Because reading and validating the observation stream is a distinct job with its own failure modes. It blocks on input, it must tolerate malformed data without stopping, and it must hand off to a queue that can be full. Putting that in `SensorFeed` would make the producer responsible for the consumer's timing; putting it in the workers would make every worker re-validate the same stream. As its own task, it can be reasoned about — and tested — alone.

2. How does it communicate with other classes?

Through four constructor arguments, each a narrow contract. A `BufferedReader` (a stream), an `EventBuffer` (a queue), and two `Consumer<String>` callbacks (events). It has no reference to the feed, the model, the engine or any interface class. That is why it can be tested by handing it a `StringReader` and a fresh buffer.

3. Which data does it receive?

Lines of text, one per event. Each line is either a pipe-delimited observation beginning with `READ|`, or a pulse beginning with `PULSE|`. It receives nothing else — no objects, no configuration, no state.

4. Does it use a stream, an event, a method call, or a database query?

All three of the first, and none of the last. A **stream** supplies the characters. A **method call** performs the parse, which validates. An **event** — a callback — reports warnings and status, and the queue hand-off is a fourth mechanism, condition waiting. There is no database interaction at all, which is correct: persistence is a decision for the engine, made after the model has processed the event.

5. How does its result affect the application?

Indirectly and only through the queue. For every accepted line it places one event in the buffer; a worker then takes it. For every rejected line it produces a warning and continues. It never returns a value, because it is a long-running task rather than a function — its effect is the stream of events it delivers.

6. What survives application shutdown?

Nothing that this class owns. The buffer's contents are drained and processed before shutdown; the pipe is closed; the reader is closed by try-with-resources. The durable record is created by the engine — a journal line, an archive record and possibly a database row — none of which this class touches. That separation is deliberate.

7. What happens if the relevant operation fails?

Three distinct outcomes, and the differences are the design.

- **A malformed line** produces a warning and the loop continues. One bad record cannot stop the stream.
- **An `IOException`** ends the loop and reports. The channel is broken; there is nothing left to read.

- **A closed buffer** makes `put` return false, so intake reports and stops cleanly rather than waiting forever for a consumer that will never come.

8. Which design alternative is taught in the lecture notes?

Reading from a file with `FileReader` and a `BufferedReader` in a `main` method. That is the same read loop with a different source and no threading. The lecture version demonstrates the stream API; this version demonstrates what changes when the source is another thread and the consumer is concurrent — which is why the try-with-resources, the two failure categories and the closed-buffer check all matter.

CORE CONCEPT

Notice what those eight answers produced between them. They covered streams, threads, queues, callbacks, validation, exception strategy, persistence boundaries and the relationship to the lecture material — for one small class. That is the exercise's purpose: the same eight questions work on `BasinZone`, on `SensorReadingRepository`, on `SnapshotStore` and on `DatabaseHistoryPanel`.

21.3 Where every syllabus concept lives

This table is the book's index of concepts against their homes in the system. Use it to check coverage and to find the example for a topic you are revising.

Table 21.1 — *Every syllabus concept, where it is taught, and where it lives in the project.*

Syllabus concept	Where it is taught	Where it lives in BasinWatch
Classes, objects, encapsulation	Ch. 2	`BasinZone` — private state, package-private behaviour
Interfaces	Ch. 2	`BasinEvent`, implemented by three records
Polymorphism	Ch. 2	`BasinModel.process` pattern matching
Inheritance	Ch. 2	`SensorFeed extends Thread`, Swing panels
Enums	Ch. 2	`RiskLevel`, `MissionType`, `ResourceType`
Records	Ch. 2	`SensorReading`, `ZoneSnapshot`, `DatabaseHistoryEntry`
POJO versus JavaBean	Ch. 2, 18	Records and serializable DTOs; no framework beans
Threads and Runnable	Ch. 3, 7	`SensorFeed`, `SensorIntake`, three workers
Synchronization and monitors	Ch. 3	`ResourceAllocator.tryReserve`, `BasinModel.process`
wait / notifyAll	Ch. 3, 7	`EventBuffer.put` and `take`
Producer–consumer	Ch. 3, 7	Feed produces, intake consumes, workers process
Interruption and joining	Ch. 7	`closeEngine` shutdown sequence

Syllabus concept	Where it is taught	Where it lives in BasinWatch
Byte streams	Ch. 4, 6	Sensor archive via `DataOutputStream`
Character streams	Ch. 4, 5	Journal, reports, and the pipe
FileInputStream / FileOutputStream	Ch. 5	The classic API, contrasted with `Files.newXxx`
FileReader / FileWriter	Ch. 5	The classic character API, contrasted with `Files.newBufferedReader`
ByteArrayInputStream / ByteArrayOutputStream	Ch. 7	Demonstrated as the in-memory case
Piped streams	Ch. 7	`PipedWriter` / `PipedReader` between feed and intake
Buffered streams	Ch. 6	Every file stream in the project is wrapped
DataInputStream / DataOutputStream	Ch. 6	`SensorArchive`
Serialization	Ch. 8	`SnapshotStore` with `ObjectOutputStream`
Deserialization safety	Ch. 8	Size bound, class filter, version check, validation
File metadata and exceptions	Ch. 5	`SaveCatalog.list`, `EOFException` handling
JDBC architecture	Ch. 11	`DatabaseManager` and the driver
Driver types	Ch. 11	SQLite's pure-Java type 4 driver
Connection	Ch. 12	`DatabaseManager.openConnection`
Statement	Ch. 12	`DatabaseInitializer` DDL
PreparedStatement	Ch. 12	Every repository insert and query
CallableStatement	Ch. 12	Taught; not used — no stored procedures
ResultSet	Ch. 12	`OperationalEventRepository.loadRecent`
CRUD	Ch. 12	Insert, select and update; delete is deliberately absent
Transactions	Ch. 13	`DatabaseService.recordDispatch`
SQLException handling	Ch. 12, 14	Caught in `enqueue`, reported to the UI
Swing components and layouts	Ch. 15	`MainFrame`, `BasinMapPanel`
Events and listeners	Ch. 15	Action listeners throughout the frame
Custom painting	Ch. 15	`BasinMapPanel.paintComponent`

Syllabus concept	Where it is taught	Where it lives in BasinWatch
Event-dispatch thread	Ch. 15, 16	Every listener and every `invokeLater` call
JDBC to Swing	Ch. 16	`DatabaseHistoryPanel` and its table model
Maven and pom.xml	Ch. 17	`pom.xml` and the shade plugin
XML and namespaces	Ch. 18	`pom.xml` header; taught, not otherwise used
XML Schema	Ch. 18	Taught; not used in the project
Annotations	Ch. 18	`@Override`, `@FunctionalInterface`, `@SuppressWarnings`
JavaBeans	Ch. 18	Taught; the project wires by hand
Spring JdbcTemplate	Ch. 19	Taught as an alternative; the project uses raw JDBC
DriverManagerDataSource	Ch. 19	Taught; the project uses `DriverManager` directly

21.4 The eight questions

Work through these for a class of your choosing. If you can answer all eight without looking, you understand that class and its place in the system.

Exercise 1 · Why is this class needed?

State the responsibility in one sentence, then say what would go wrong if the class did not exist. The second part is the one that shows understanding: “the model would have to validate input” or “the interface would have to know SQL” is a real answer; “it is a class that does X” is not.

Exercise 2 · How does it communicate with other classes?

List the constructor arguments and the methods it calls on them. Name the mechanism for each — method call, stream, callback, queue, query. Note the classes it does *not* reference, because those absences are also design decisions.

Exercise 3 · Which data does it receive?

Give the types and the shape. “A line of pipe-delimited text with six fields” is a better answer than “a string”, because it explains why the parse validates what it validates.

Exercise 4 · Does it use a stream, an event, a method call, or a database query?

Answer with all that apply, and say why each is the right mechanism. If the answer is “none of them”, explain what mechanism it does use instead — an in-memory collection, a monitor, a future.

Exercise 5 · How does its result affect the application?

Trace forward to the visible effect. A method that returns a boolean may ultimately change a colour on a map, and being able to say so is the difference between knowing a class and knowing a system.

Exercise 6 · What survives application shutdown?

Answer for each artefact: nothing, an audit record, a typed measurement, or a restorable snapshot. Then say which component is responsible for writing it — it will rarely be the class you started with.

Exercise 7 · What happens if the relevant operation fails?

Distinguish recoverable from unrecoverable, and reported from propagated. A class that skips a bad record and one that aborts the loop are making different assumptions about what failure means.

Exercise 8 · Which design alternative is taught in the lecture notes?

Name the textbook version and say precisely what differs. “The lecture version reads a file in main; this version reads a pipe on a background thread” is the shape of a good answer, because it explains why the extra machinery exists.

21.5 Three design alternatives worth being able to argue

An examiner who wants to know whether you understand a design will ask what you would do differently. These are the three most likely, with both sides of the argument.

Table 21.2 — *Three design choices, and the case for each side.*

Choice in BasinWatch	The alternative	Argument for the alternative	Argument for what the project does
Raw JDBC with hand-written repositories	Spring JdbcTemplate with a DataSource	Less boilerplate per query; declarative transactions; pooling available	One dependency instead of four; the JDBC mechanics remain visible; appropriate for a dozen queries
One file per artefact, no database by default	Store everything in the database	One mechanism, one query language, consistent access	The application works with no database at all; the log is readable without tooling; the snapshot restores state that a row store would need replay for
Hand-written constructor wiring	A dependency-injection framework	Wiring is declarative; large graphs become manageable; lifecycles are managed	Wiring is visible in the source; a missing dependency is a compile error rather than a runtime one; no scanning, no reflection, no framework start-up

CORE CONCEPT

The point of that table is not that one column is right. It is that a design choice can be defended with reasons. In a viva, saying “I used raw JDBC because the project has one database and a dozen queries, and adding Spring would add four dependencies to save perhaps forty lines” is a far stronger answer than “the lecture notes used JDBC”.

21.6 What you should now be able to do

- **Draw the architecture from memory**, with the layers, the threads and the two persistence columns.
- **Name the mechanism** for any interaction in the system: method call, monitor, stream, queue, callback or query.
- **Trace four scenarios** end to end, saying which thread runs each step.
- **Choose a stream** for a task and justify the layers, including the charset and the buffering.
- **Explain why serialization needs four defences**, and name them.
- **Write a parameterized query** with correct 1-based binding and complete resource handling, and say why binding beats concatenation.
- **Say what a transaction guarantees** and identify the one place the project needs one.
- **Keep the event-dispatch thread free**, and explain what happens when you do not.
- **Explain how the JDBC driver reaches the classpath**, and why the shade transformer matters.
- **Distinguish raw JDBC from JdbcTemplate** without claiming the project uses both.
- **Answer the eight questions** about any class in the project.

CONNECTION TO ANOTHER CONCEPT

The revision section that follows turns each of those capabilities into comparison tables, viva questions and tracing exercises. Work through Chapter 21 first; then use the revision material to check what you missed.

CHAPTER SUMMARY

- The eight questions — purpose, communication, data received, mechanism, effect, survival, failure, and the lecture alternative — explain any class in the system.
- Every syllabus concept has a home in the project; Table 21.1 maps all of them.
- The three most defensible design alternatives are raw JDBC versus JdbcTemplate, files versus a database, and hand wiring versus a DI framework.
- A design choice is worth defending with reasons, not with a reference to the lecture notes.

KEY TERMS

- **Design alternative** — a different but defensible way of achieving the same requirement.
- **Traceability** — the ability to follow a piece of data from its origin to its visible effect.
- **Responsibility** — the single job a class exists to do.

REVISION

Comparison Tables and Chapter Summaries

> The twelve comparisons that carry the most marks, followed by a one-page summary of every chapter in the book.

R.1 The twelve essential comparisons

Each of these is a question an examiner can ask directly. Learn the table, but more importantly learn the *axis* of each comparison — that is what the question is really testing.

1. Byte streams versus character streams

Table R.1 — *The byte/character decision.*

	Byte stream	Character stream
Roots	`InputStream`, `OutputStream`	`Reader`, `Writer`
Unit	8-bit byte	16-bit character
Encoding	None	A charset converts characters to bytes
Line methods	None	`readLine()`, `newLine()`
Use for	Binary data, typed values, serialized objects	Text a human will read
Project example	`SensorArchive`, `SnapshotStore`	`OperationalJournal`, `ReportWriter`

2. The file and memory stream classes

Table R.2 — *The four-by-four grid of stream classes.*

Purpose	Byte input	Byte output	Character input	Character output
A file	`FileInputStream`	`FileOutputStream`	`FileReader`	`FileWriter`
Memory	`ByteArrayInputStream`	`ByteArrayOutputStream`	`StringReader`	`StringWriter`
Another thread	`PipedInputStream`	`PipedOutputStream`	`PipedReader`	`PipedWriter`
Adding a capability	`FilterInputStream`	`FilterOutputStream`	`FilterReader`	`FilterWriter`

3. Buffered versus unbuffered streams

Table R.3 — *Buffering, and why it is not optional.*

	Unbuffered	Buffered
Reads per 8 KB	One system call per `read()`	One system call per buffer refill

	Unbuffered	Buffered
Line methods	Not available	`readLine()` and `newLine()`
`flush()` matters?	No — data goes straight out	Yes — data may sit in the array
Risk if not closed	Minor	Silent data loss of buffered content
Project use	Never used alone for files	Every file stream in the project

4. Data streams versus object serialization

Table R.4 — *Two ways to persist structured data.*

	DataInputStream / DataOutputStream	ObjectInputStream / ObjectOutputStream
What is written	Primitives and strings, in a fixed order	An object graph, with class descriptors
Format defined by	You — the sequence of calls	The JDK
Readable if the class changes?	Only if the order is unchanged	Yes, with an explicit `serialVersionUID`
Constructor called on read?	Not applicable	**No** — which is why validation must be explicit
Security concern	Low — no classes are named in the stream	**High** — the stream names classes to instantiate
Project example	`SensorArchive`	`SnapshotStore`

5. The six communication mechanisms

Table R.5 — *Choosing a communication mechanism.*

Mechanism	Direction	Blocks?	Crosses threads?	Project example
Method call	two-way	briefly	no	`model.process(event)`
Monitor	two-way	on the lock	yes	`ResourceAllocator.tryReserve`
Stream	one-way	when full or empty	usually	The pipe; all file access
Queue	one-way	when full or empty	yes	`EventBuffer`
Callback	inverted	no	may arrive anywhere	Snapshot, status and database listeners
Database operation	request/response	yes unless offloaded	yes	Writes queued, reads as futures

6. File persistence versus database persistence

Table R.6 — *The two persistence mechanisms and what each is good at.*

	File	Database
Structure	Whatever format you invent	Tables, columns and declared types
Selective read	Read and filter everything	`WHERE` plus an index
Aggregation	Application code	SQL aggregate functions
Concurrency	Effectively one writer	Locking and isolation levels
Atomic multi-part change	Write to a temporary file and rename	A transaction
Readable without tooling	Yes, for text formats	No — needs a client
Restores full application state	Yes, via a snapshot	Only by replaying or reconstructing
Project use	Log, archive, snapshot, reports	Optional history in SQLite

7. Statement, PreparedStatement and CallableStatement

Table R.7 — *The three statement types.*

	Statement	PreparedStatement	CallableStatement
Values	Concatenated into the SQL text	Bound with typed setters	Bound as input or registered as output
Pre-parsed	No	**Yes**	Yes
Injection-safe	**No**	**Yes**	**Yes**
Reusable efficiently	No	**Yes**	Yes
Project use	Schema DDL in `DatabaseInitializer`	Every repository method	Not used — no stored procedures

8. The four JDBC driver types

Table R.8 — *The four driver types.*

Type	Approach	Status today
1 — JDBC–ODBC bridge	Native ODBC driver called through Java	**Removed** from the JDK
2 — Native-API	Calls the vendor's native client library	Rarely used; needs native libraries
3 — Network protocol	Talks to a middleware server	Largely converged with type 4
4 — Thin driver	Pure Java, speaks the database protocol directly	**The modern default**; SQLite's driver is type 4

9. Execution methods and their results

Table R.9 — *The four execution methods.*

Method	Returns	Use for
`executeQuery()`	`ResultSet`	`SELECT`

Method	Returns	Use for
`executeUpdate()`	`int` — rows affected	`INSERT`, `UPDATE`, `DELETE`, DDL
`execute()`	`boolean` — is a ResultSet available	When the kind of statement is unknown
`executeBatch()`	`int[]` — counts per statement	Bulk operations

10. Raw JDBC versus Spring JdbcTemplate

Table R.10 — *Two levels of JDBC abstraction.*

	Raw JDBC	JdbcTemplate
Connection handling	Written by hand	Delegated to the template
Resource closing	try-with-resources at every site	Automatic
Parameter binding	Typed setters, hand-written	From the argument list
Row mapping	A loop in the method	A `RowMapper` lambda
Exceptions	`SQLException` (checked)	`DataAccessException` (unchecked)
Dependencies	The driver only	Spring Core, Spring JDBC, the driver
Used in BasinWatch?	**Yes**	**No**

11. POJO versus JavaBean

Table R.11 — *POJO and JavaBean.*

	POJO	JavaBean
Convention	None	No-argument constructor, private fields, public get/set accessors
Serializable	Not required	Conventionally yes
Can be a record?	Yes	**No** — records have no no-argument constructor or setters
Purpose of the convention	None needed	Allows a framework to create and populate by reflection
Project example	`SensorReading`, `ZoneSnapshot`	None — no framework is used

12. Swing event handling essentials

Table R.12 — *The twelve comparisons, and the last one.*

Concept	Meaning	Project example
Event-dispatch thread	The single thread for events and painting	Every listener in `MainFrame`
Listener	An object registered to receive an event	`ActionListener` on each button

Concept	Meaning	Project example
Functional interface	One abstract method, so a lambda works	`ActionListener`, `Runnable`
Layout manager	Decides where children go	`BorderLayout` and `JSplitPane`
Custom painting	Override `paintComponent`	`BasinMapPanel`
Off-EDT hand-off	`SwingUtilities.invokeLater`	Every engine callback
Table model	Supplies data to a `JTable`	`HistoryTableModel`

R.2 One-page chapter summaries

Each entry is the smallest complete statement of what a chapter established. If any line is not familiar, that is the chapter to reread.

Table R.13 — *Every chapter in one paragraph.*

Ch.	In one paragraph
1	BasinWatch is a single-process offline floodplain console in six layers: operator and interface, engine commands, domain and engine, sensor input path, file persistence, and an optional JDBC layer. Six threads do background work and publish immutable snapshots to the event-dispatch thread. Technologies meet at exactly two boundaries: the accepted event and the published snapshot.
2	Encapsulation is enforced with access modifiers; one interface unifies three event types; polymorphism is expressed as pattern matching; inheritance is used only where a framework contract requires it; enums carry their own data; records provide validated immutable values; a JavaBean adds conventions a framework needs and a record cannot have.
3	Components communicate in six distinct ways: method call, shared object under a monitor, stream, queue with condition waiting, listener callback, and database operation. Each has different blocking and threading behaviour. A method call must never block on external work; condition waits must be loops that hold the monitor and use `notifyAll`.
4	A stream is an ordered one-directional sequence consumed incrementally. Four abstract roots: `InputStream`, `OutputStream`, `Reader`, `Writer`. Bytes preserve data exactly; characters pass through a charset and are right for text. `read()` returns `int` so that -1 can mean end of stream.
5	A `Path` names a file; only a stream reads it. Opening for output truncates unless append is requested. A complete write is create, open, write, flush, close; a complete read is check, open with an explicit charset, loop on

Ch.	In one paragraph
	`readLine() != null`, close. `flush()` pushes data onward; `close()` flushes and releases.
6	A buffer turns many small operations into few large ones. Wrapper classes add one capability each by taking another stream in their constructor. `readLine()` exists only on `BufferedReader`. Data streams write primitives in a fixed order that *is* the schema, so the read order must match exactly; a usable binary format has a magic number, a version and self-describing strings.
7	Byte-array streams use the whole stream API with no device. A pipe is a matched writer/reader pair with a bounded buffer, intended for two threads; it provides back-pressure and a natural end-of-stream signal. Shutdown order is stop the producer, close the writer, join the consumer, close the queue.
8	Serialization writes an object graph; deserialization reconstructs it without calling constructors. `transient` and `static` fields are skipped; an explicit `serialVersionUID` prevents unrelated recompiles from invalidating saves. Because constructors are bypassed, a reader needs four defences: bound the size, filter the classes, check the version, validate the result. BasinWatch writes to a temporary file and moves it into place.
9	Four file mechanisms answer four questions: the log records what happened, the archive records exact measurements, the snapshot restores state, the report communicates. All paths derive from `AppPaths`. None of the four can substitute for another, because the format determines which questions the file can answer.
10	Data lives in memory, in files, or in a database; the decisive difference is selective access through indexes. BasinWatch uses all three because different questions need different mechanisms. A table declares columns, types and constraints; an index makes a query fast; JDBC is the interface layer that lets Java talk to any database.
11	JDBC splits into fixed interfaces in `java.sql` and a vendor driver that implements them. A driver recognises URLs, opens connections, translates calls and translates results back. There are four driver types; the pure-Java thin driver is the modern default. BasinWatch builds its URL in one place and reports availability as a boolean.
12	A query involves a Connection, a Statement and possibly a ResultSet, all closed. `PreparedStatement` is the default: safer against injection, pre-parsed and reusable. Placeholders are 1-based and each needs a typed setter. `executeQuery` returns rows, `executeUpdate` returns a count. Map rows into your own objects inside the loop.
13	A transaction groups operations so all take effect or none do; the ACID properties are atomicity, consistency,

Ch.	In one paragraph
	isolation and durability. JDBC is in autocommit mode by default; `setAutoCommit(false)` begins a transaction you control. A transaction belongs to a connection, so every statement in it must share that connection. BasinWatch records a dispatch in one transaction, and runs all database work on one thread so isolation anomalies cannot arise.
14	Four classes form the database layer: connection management, schema creation, service coordination, and per-table SQL. Four tables carry readings, missions, resource usage and events, each with a timestamp. Schema creation is idempotent. The database is optional because availability is a boolean, writes are queued on their own thread, and reads on an unavailable database return an empty list.
15	Swing is event-driven: handlers are declared and the framework calls them. Components are containers or simple components, arranged by layout managers. All event handling and painting run on the event-dispatch thread, which is why a slow handler freezes the window. The lecture-note login application validates input, reads the password as a char array and starts on the EDT; its hard-coded credential is a UI exercise, not a security design.
16	A listener asks the engine and returns; slow work runs on a background thread; the result arrives as a future; and the callback posts to the EDT before touching any component. A complete click-to-table path crosses the EDT, the database thread and the EDT again. A `JTable` renders whatever its `TableModel` supplies, and the model must fire a change event.
17	A build tool resolves dependencies, compiles, tests and packages. Maven identifies libraries by groupId, artifactId and version. BasinWatch has exactly one dependency. The driver reaches the runtime classpath as a Maven dependency and registers itself through a service file, which the shade plugin's `ServicesResourceTransformer` must preserve in the uber jar.
18	XML is a tree of elements, attributes and text; a document is well-formed if it follows the syntax and valid if it conforms to a schema. A namespace URI is an identifier, not an address. Annotations are metadata whose retention policy decides whether a framework can read them at runtime. A JavaBean's conventions exist so a framework can instantiate and populate it reflectively; BasinWatch wires by hand instead.
19	Raw JDBC forces the same resource handling into every method; `JdbcTemplate` performs it for you and asks only for SQL and a `RowMapper`. A `DataSource` supplies connections; `DriverManagerDataSource` does not pool and is for teaching and tests. JdbcTemplate is built on the

Ch.	In one paragraph
	same `java.sql` interfaces, so it removes repetition rather than JDBC. BasinWatch uses raw JDBC only.
20	Four scenarios take genuinely different routes: an observation crosses the pipe and fans out to three persistence mechanisms; a dispatch is the only transactional path; a save-and-restore is defensive on the way in; a history read writes nothing. The dependency graph is acyclic, with the domain at the bottom and the interface at the top.
21	Eight questions explain any class: purpose, communication, data received, mechanism, effect, survival, failure, and the lecture alternative. Every syllabus concept has a home in the project, and the three most defensible design alternatives are raw JDBC versus JdbcTemplate, files versus a database, and hand wiring versus a dependency-injection framework.

VIVA AND THEORY

Viva and Theory Questions

> Fifty questions with answers, grouped from definitions through mechanisms to integration. The last group is the one that separates understanding from memorisation.

V.1 Definitions and distinctions

Q1. What is a stream?

A one-directional, ordered sequence of data with a source or a destination, consumed or produced incrementally rather than all at once. The incrementality is the point: it lets you handle data larger than memory, and data that has no end yet.

Q2. State the difference between a byte stream and a character stream.

A byte stream carries 8-bit units and applies no interpretation, so it preserves data exactly. A character stream carries 16-bit characters and converts them through a charset, so it is correct for text a human will read. Using a character stream on binary data corrupts it, because byte sequences invalid in the charset are replaced.

Q3. Why does `read()` return an `int` rather than a `byte`?

Because the end of a stream needs a marker distinct from every legal data value. All 256 byte values are legal data, so widening the return type to `int` makes room for `-1` to mean end of stream.

Q4. What is the difference between `flush()` and `close()`?

`flush()` pushes buffered data to the destination and leaves the stream usable. `close()` flushes and then releases the underlying resource; further writes after closing throw. Calling `close()` twice is safe.

Q5. What is a filter stream?

A stream that wraps another stream, taking it as a constructor argument, and adds one capability — buffering, typed reads, object serialization or character decoding. It is the decorator pattern applied to I/O, and it is what makes stream chains composable.

Q6. Define serialization and deserialization.

Serialization converts an object into a byte sequence containing its class identity and its non-transient field values, recursing into referenced objects. Deserialization reverses the process, reconstructing the object graph.

Q7. What do `transient` and `static` mean for serialization?

Neither is written to the stream. A `transient` field is skipped deliberately and restores to its default value; a `static` field belongs to the class rather than the instance, so it is not instance state and is not serialized.

Q8. What is `serialVersionUID` for?

It is the explicit version identity of a serializable class. Without it, the compiler derives one from the class's structure, so adding a method or a field can change it and make previously saved files unreadable with `InvalidClassException`. Declaring it makes compatibility deliberate.

Q9. What is a JavaBean, and how does it differ from a POJO?

A JavaBean is a POJO that follows conventions — a public no-argument constructor, private fields, public `get`/`set` accessors and usually `Serializable` — so a framework can create and populate it by reflection. Every JavaBean is a POJO; not every POJO is a JavaBean. A record can never be a JavaBean, because it has no no-argument constructor and no setters.

Q10. What is JDBC?

The Java Database Connectivity API: a set of interfaces in `java.sql` plus a driver-registration mechanism, letting Java code open connections, send SQL and read results without depending on a particular database product.

Q11. What is a primary key?

The column or columns that uniquely identify each row in a table. It prevents duplicate identities and is the target of references from other tables.

Q12. What is an index and why does it matter?

An auxiliary structure that lets the database find rows by particular columns without examining every row. It is the reason a selective query is cheap, and it is the main capability a database has that a plain file does not.

Q13. What is a transaction?

A group of database operations treated as one unit: either all of them take effect or none does. It is what prevents a partially completed change from being visible.

Q14. What do the ACID properties mean?

Atomicity: all or nothing. Consistency: the database moves between valid states, honouring its constraints. Isolation: concurrent transactions do not see each other's partial work. Durability: a committed change survives a failure.

Q15. What is the event-dispatch thread?

The single thread on which Swing performs event handling and painting. All component access must happen there, which is why a long-running handler freezes the interface and why callbacks from other threads must post their work to it.

V.2 Mechanisms and code

Q16. Why does an application need both file-based logs and a database?

Because they answer different questions. A log records what happened, in order, and is readable with a text editor but must be scanned to be queried. A database stores structured rows that can be filtered, ordered and aggregated cheaply through indexes. The log is an audit trail; the database is an index over the same events.

Q17. What is the role of a JDBC driver?

Four things: recognise its own URLs, open a connection to the database, translate JDBC calls into the database's own protocol, and translate results back into `ResultSet` rows. Everything you call on a `Connection` is implemented by the driver, not by the JDK.

Q18. Which JDBC driver type does BasinWatch use, and why does that matter?

A type 4 thin driver, `org.xerial:sqlite-jdbc`. It is pure Java, so it needs no native library and no middleware — just a jar on the classpath. That is why the project's entire database configuration is one Maven dependency and one URL string.

Q19. How does `PreparedStatement` bind values, and why is that safe?

Placeholders written as `?` are numbered from one, and each is filled by a typed setter such as `setDouble(4, value)`. The SQL text is sent to the database and parsed before any value is supplied, so the parser has already decided what is a statement and what is a value. No value can therefore become part of the statement's structure, which is what defeats SQL injection.

Q20. What is the difference between `executeQuery()` and `executeUpdate()`?

`executeQuery()` runs a statement that produces rows and returns a `ResultSet`. `executeUpdate()` runs a statement that changes rows and returns the number of rows affected as an `int`.

Q21. Why must a result set be consumed inside the block that created it?

Because it is tied to its statement and its connection, and both are closed at the end of that block. Returning a `ResultSet` from a method that closes its connection produces an object that fails on the first `next()`. The fix is to map each row into your own object inside the loop.

Q22. How does a character stream differ from a byte stream in terms of the data it can represent?

Neither represents more data — both are sequences of units. The difference is interpretation: a byte stream preserves exact 8-bit values with no mapping, while a character stream applies a charset, so the same character may occupy one or several bytes on disk. Character streams are therefore lossless for text and destructive for binary data.

Q23. How does a piped stream relate to multithreading?

A pipe is a matched writer/reader pair intended for two different threads: one writes characters, the other reads them, and the pipe's bounded buffer transfers between them. It gives a one-directional contract with back-pressure and a natural end-of-stream signal when the writer closes. Using both ends in one thread is a deadlock, because the reader waits for data only that same thread could write.

Q24. What is a condition wait, and what are the four rules for writing one correctly?

A condition wait is `wait()` used to pause a thread until a predicate becomes true. The four rules: wait inside a loop that re-tests the predicate, because a woken thread is not guaranteed its condition now holds; hold the object's monitor, since `wait()` requires it; use `notifyAll()` rather than `notify()`, because waiters may have different predicates; and release the monitor while waiting, which `wait()` does and `sleep()` does not.

Q25. Why does BasinWatch use a pipe between the feed and the intake rather than a shared list?

Because the pipe is a much narrower contract. A shared list would give both threads access to every element and to the list's own methods, so a bug in either could corrupt it. The pipe gives each thread one operation — write a character sequence, read a character sequence — and provides back-pressure for free, since the writer blocks when the buffer is full.

Q26. What is the difference between `writeUTF` and `writeChars`?

`writeUTF` writes a two-byte length followed by modified UTF-8 bytes, so the string is self-describing and can be read back without knowing its length. `writeChars` writes two bytes per character with no length prefix, so the reader must already know how many characters to read.

Q27. Why does a data stream require the read order to match the write order?

Because the format contains no field metadata: the sequence of calls *is* the schema. A `readDouble()` where a `readInt()` was written does not throw — it consumes eight bytes that were never a double and returns a plausible but wrong value. Versioning the format explicitly is the defence.

Q28. What four defences does BasinWatch apply when restoring a session?

Bound the input size before opening the stream; install an `ObjectInputFilter` that approves only the project's own packages and a small set of JDK packages; check the format version written in the header; and validate the restored domain state, because deserialization bypasses constructors.

Q29. Why is the session snapshot written to a temporary file and then moved?

So that an interrupted save cannot corrupt an existing snapshot. The temporary file is written completely, then moved into place with `ATOMIC_MOVE`, which replaces the target in one step. A crash mid-save leaves the previous good file intact and an orphaned temporary file.

Q30. Why does `DatabaseManager.isAvailable` return a boolean rather than throwing?

Because the database is optional. Returning a boolean lets every caller test a value rather than handling an exception to discover availability, and it lets a read return an empty list on an unavailable database so the interface's code path is identical either way.

V.3 Integration and architecture

Q31. How do database results reach a GUI?

In four steps. A listener on the event-dispatch thread asks the engine for an operation and returns immediately with a `CompletableFuture`. The query runs on the database thread. The future completes there, running its callback on that same thread. The callback wraps its component access in `SwingUtilities.invokeLater`, so the table model is updated on the event-dispatch thread.

Q32. Why does Spring JdbcTemplate not eliminate the need for JDBC itself?

Because it is built on JDBC. It obtains connections from a `DataSource`, creates `PreparedStatements`, binds parameters, iterates `ResultSets` and closes everything — all through the same `java.sql` interfaces. It removes the repetition, not the protocol, and everything you know about parameter binding, result-set iteration and transactions still applies.

Q33. How do Maven dependencies relate to database-driver availability?

A driver is declared in `pom.xml` by groupId, artifactId and version. Maven resolves it and places it on the compile and runtime classpaths. The driver registers itself through `META-INF/services/java.sql.Driver`, and the shade plugin must be configured with `ServicesResourceTransformer` so that file survives being merged into an uber jar. Without it the classes are present but the registration is lost, and `DriverManager` reports no suitable driver.

Q34. Why is it a mistake to let the interface do the work?

Three consequences. The event-dispatch thread is blocked for the duration, so the window freezes. The logic cannot be tested without constructing a window. And SQL or file handling spreads across the interface code, so a change to the schema or the file format touches many classes. The project's rule — the interface asks, the engine decides, a worker executes, a snapshot reports — avoids all three.

Q35. How does the model stay consistent while workers mutate it and the interface reads it?

By publishing immutable snapshots. `BasinSnapshot` and its parts are records whose constructors copy every collection with `List.copyOf` and `Map.copyOf`. The interface therefore reads a value that cannot change underneath it, so no locking is needed on the event-dispatch thread.

Q36. Why does the domain package depend on nothing?

Because that is what makes it testable and reusable. With no import of Swing, files, SQL or threads, the model can be exercised in a plain unit test with no setup, and the same model could be driven by a different interface or by an automated harness. It is also why the dependency graph has no cycles.

Q37. Where exactly do the I/O, JDBC and Swing technologies meet in this application?

At two boundaries. The first is the accepted event: after the model processes an event, the engine fans out to the journal, the archive worker and the database queue. The second is the published snapshot: the engine hands an immutable value to every registered listener, whose first action is to post to the event-dispatch thread. Everywhere else the layers are deliberately ignorant of one another.

Q38. Why does the application need a transaction when recording a dispatch?

Because three related writes must succeed or fail together: the mission row, the resource-usage rows and the audit event. Without a transaction, a failure after the mission insert would leave a running mission with resources that were never deducted, so the resource board would be permanently wrong and the history would not explain why.

Q39. Why must every write in a transaction use the same connection?

Because a transaction belongs to a connection, not to a statement. Calling `commit()` on one connection has no effect on work done on another, so three repositories each opening their own connection would create three separate transactions and atomicity would be lost — while the code still looked correct.

Q40. Why is isolation not a problem in this application?

Because all database work runs on a single-threaded executor. There is exactly one writer, so there is never a second transaction to interfere with the first, and no isolation level or locking policy needs to be reasoned about.

Q41. What happens to the application if the database is unavailable?

Four things, and all four matter. The application still starts, because availability is checked rather than assumed. The history panel shows an empty list and a status message. Every write becomes a no-op because `enqueue` returns early. And the file layer is entirely unaffected, so the log, archive, snapshot and reports are still produced.

Q42. Why does a reading reach the database only when the model changed?

Because `BasinZone.apply` rejects a reading whose tick is not newer than the last one applied, returning false. The engine writes to the database only when the model changed, so a duplicate or out-of-order observation is not recorded as though it had taken effect. The stale guard in the domain is therefore what makes the history accurate.

Q43. Explain the shutdown sequence and why the order matters.

Stop the producer, close the writer, join the intake thread, close the queue. Closing the writer is what makes the consumer's `readLine()` return null and its loop end. Joining the consumer before closing the queue ensures accepted events are drained rather than discarded. Closing the writer before stopping the producer would make the producer throw a spurious error, and never closing the writer would leave the consumer blocked forever.

Q44. Why is inheritance used so little in this project?

Because composition is more flexible and less coupled. The two uses are both framework contracts: `SensorFeed extends Thread` because a feed genuinely owns a thread's lifecycle and needs a name and priority, and the Swing panels extend `JPanel` and `JFrame` because Swing requires it. Everywhere else, objects are composed — `BasinEngine` has an executor, a buffer and a service rather than being one.

Q45. What is the difference between `invokeLater` and `invokeAndWait`?

`invokeLater` queues a runnable on the event-dispatch thread and returns immediately, so the calling thread continues. `invokeAndWait` queues it and blocks until the event-dispatch thread has run it — which is a deadlock if called from the event-dispatch thread itself. BasinWatch uses `invokeLater` exclusively.

Q46. Why does the project not use `SwingWorker`?

Because the engine must work without a user interface, so its operations cannot depend on a Swing type. Returning `CompletableFuture` keeps the engine independent of Swing and lets the interface choose how to consume the result. `SwingWorker` is more convenient inside a UI; a future is more flexible outside one.

Q47. Is a database always better than a file?

No. For a single-user desktop application a database adds a dependency, a schema and a failure mode. BasinWatch treats its database as optional precisely because the file layer already satisfies the essential requirements. The database earns its place by enabling selective queries and aggregation, not by being more modern.

Q48. Why does BasinWatch not use Spring?

Because for an application of this size the trade is unfavourable. There are six packages and roughly two dozen classes, wired by hand through constructors. Spring would add several dependencies, a component scan and a class of runtime errors in exchange for saving perhaps thirty lines of wiring. It would also make the database layer depend on a framework, when the project's aim is to show raw JDBC.

Q49. What is the single most important design decision in the project, and why?

The publication of immutable snapshots. It is what allows background threads to mutate the model while the event-dispatch thread renders it, with no locking in the interface and no possibility of a torn read. It is also what keeps the interface's job small — render a value, collect intent, report an outcome.

Q50. If you had to change one thing about this architecture, what would it be?

A defensible answer names a real trade-off rather than a fault. For example: the database schema declares the mission relationship as a plain column rather than a foreign key, which means a deleted mission could orphan its resource rows; enabling `PRAGMA foreign_keys` and declaring the constraint would fix it. Or: history is retained indefinitely, so a retention or archival policy would be needed before the database grew without bound. Naming a specific consequence is what makes the answer credible.

EXERCISES

Programming and Tracing Exercises

> Thirty exercises in four groups: predicting output, repairing code, tracing data through the system, and interpreting the architecture. Answers are given so you can check yourself, but attempt each one first.

E.1 Output prediction

Exercise 1 · What does this print?

```
try (ByteArrayOutputStream buffer = new ByteArrayOutputStream();
     DataOutputStream out = new DataOutputStream(buffer)) {
    out.writeInt(4);
    out.writeUTF("AB");
    out.writeDouble(1.0);
    out.flush();
    System.out.println(buffer.size());
}
```

Answer. The size is **16** bytes. `writeInt` contributes 4, `writeUTF("AB")` contributes 2 length bytes plus 2 character bytes, and `writeDouble` contributes 8. The lesson is to count from the format rather than guess: a data stream has no field metadata, so these sizes are the only thing that makes the layout predictable.

Exercise 2 · What does this print?

```
String line;
int count = 0;
try (BufferedReader reader = new BufferedReader(new StringReader("a\nb\nc"))) {
    while ((line = reader.readLine()) != null) {
        count++;
    }
}
System.out.println(count);
```

Answer. 3. The string contains two newline characters and therefore three lines: `a`, `b` and `c`. The final line has no trailing newline, and `readLine()` still returns it — the fourth call then returns null.

Exercise 3 · What does this print, and why is it surprising?

```
PipedWriter writer = new PipedWriter();
PipedReader reader = new PipedReader(writer);
writer.write("hello");
writer.close();
System.out.println(reader.read());
System.out.println((char) reader.read());
```

Answer. It prints `104` then `e`. The surprise is that `read()` on a character stream returns the character's numeric code, not the character — which is exactly why the return type is `int`. Casting to `char` is what makes it readable. Also note that both ends are used in the same thread here, which works only because the data was written before it was read; a pipe used concurrently in one thread would deadlock.

Exercise 4 · What does this print?

```
System.out.println(Math.min(8.0, Math.max(0.0, 5.0 + (10.0 * 0.0014 + 4.0 / 100.0) * 0.58)));
```

Answer. 5.03132. The rainfall contribution is 10.0 × 0.0014 = 0.014; the observed rise is 4.0 / 100 = 0.04; their sum is 0.054; multiplied by the reinforced-levee factor 0.58 gives 0.03132; added to 5.0 gives 5.03132, which is already inside the 0 to 8 clamp. This is exactly the arithmetic in `BasinZone.apply`.

Exercise 5 · What does this print?

```
ObjectInputFilter filter = info -> {
    Class type = info.serialClass();
    if (type == null) return ObjectInputFilter.Status.ALLOWED;
    return type.getName().startsWith("com.basinwatch.")
            ? ObjectInputFilter.Status.ALLOWED
            : ObjectInputFilter.Status.REJECTED;
};
System.out.println(filter.checkInput(new ObjectInputFilter.FilterInfo() {
    public Class serialClass() { return java.util.ArrayList.class; }
    public long arrayLength() { return 0; }
    public long depth() { return 0; }
    public long references() { return 0; }
    public long streamBytes() { return 0; }
}));
```

Answer. REJECTED. `java.util.ArrayList` does not start with `com.basinwatch.`, so the filter refuses it. This is the point of the exercise: a filter that permits only one package would break ordinary serialization of collections, which is why BasinWatch's real filter also allows `java.lang`, `java.util` and `java.time`.

Exercise 6 · How many statements execute, and how many commits?

```
connection.setAutoCommit(false);
try (PreparedStatement a = connection.prepareStatement("INSERT ...");
     PreparedStatement b = connection.prepareStatement("UPDATE ...")) {
    a.executeUpdate();
    b.executeUpdate();
    connection.commit();
}
```

Answer. Two statements execute and **one** commit occurs. With autocommit off, neither statement is durable until `commit()` is called; both become visible together at that moment. If the second statement threw, a rollback would discard the first as well — which is the whole purpose of the transaction.

Exercise 7 · What is wrong with this loop, and what does it print?

```
try (PreparedStatement ps = connection.prepareStatement(
        "SELECT zone_id FROM sensor_readings LIMIT ?")) {
    ps.setString(1, "10");
    try (ResultSet rows = ps.executeQuery()) {
        while (rows.next()) {
            System.out.println(rows.getString(1));
        }
    }
}
```

Answer. The parameter is bound with `setString` for a `LIMIT`, which expects a number. Depending on the driver this either fails or produces an implicit conversion; the correct call is `setInt(1, 10)`. This is the value of typed setters: the type is part of the statement's contract, and using the wrong setter hides a mistake that a numeric type would have caught.

E.2 Debugging and repair

Exercise 8 · Repair this method.

```
public String readFirstLine(Path path) {
    BufferedReader reader = new BufferedReader(new FileReader(path));
    return reader.readLine();
}
```

Answer. Three problems: the reader is never closed, the charset is the platform default, and the method declares no checked exception. A corrected version:

```
public String readFirstLine(Path path) throws IOException {
    try (BufferedReader reader = Files.newBufferedReader(path, StandardCharsets.UTF_8)) {
        return reader.readLine();
    }
}
```

Note that returning from inside a try-with-resources still closes the resource.

Exercise 9 · Repair this loop.

```
String line = reader.readLine();
while (line != null) {
    process(line);
    line = reader.readLine();
}
```

Answer. This is actually correct — it reads before the loop and again at the end, which is the classic pre-read form. The repair is not about correctness but about clarity and duplication:

```
String line;
while ((line = reader.readLine()) != null) {
    process(line);
}
```

The single-expression form makes it impossible for the two read sites to diverge, which is the real risk in the original.

Exercise 10 · Repair this queue method.

```
public synchronized BasinEvent take() {
    if (events.isEmpty()) {
        wait();
    }
    return events.removeFirst();
}
```

Answer. Three defects: `wait()` is not inside a loop, so a spurious wake-up or a competing consumer can empty the deque again; the method does not declare `InterruptedException`; and a closed empty buffer would wait forever. Corrected:

```
public synchronized BasinEvent take() throws InterruptedException {
    while (events.isEmpty() && !closed) {
        waitingConsumers++;
        try {
            wait();
        } finally {
            waitingConsumers--;
        }
    }
    if (events.isEmpty()) {
        return null;                // closed and drained
    }
    BasinEvent event = events.removeFirst();
    notifyAll();
    return event;
}
```

Exercise 11 · Repair this shutdown.

```
public void close() throws IOException {
    eventBuffer.close();
    feedWriter.close();
    sensorFeed.requestStop();
}
```

Answer. The order is wrong on two counts. Closing the queue before the consumer has drained it discards events already read from the pipe, and closing the writer before stopping the producer makes the producer throw a spurious `IOException` on its next write. Corrected order: `requestStop()`, then `feedWriter.close()`, then join the intake thread, then `eventBuffer.close()`.

Exercise 12 · Repair this SQL.

```
String sql = "SELECT * FROM operational_events WHERE zone_id = '"
           + zoneId + "' ORDER BY occurred_at DESC";
```

Answer. Two problems. The value is concatenated into the statement, so it can alter the statement's structure — SQL injection. And `SELECT *` makes the code depend on the column order, which a schema change can silently break. Corrected:

```
String sql = "SELECT occurred_at, event_type, zone_id, detail "
           + "FROM operational_events WHERE zone_id = ? "
           + "ORDER BY occurred_at DESC";
try (PreparedStatement ps = connection.prepareStatement(sql)) {
    ps.setString(1, zoneId);
    try (ResultSet rows = ps.executeQuery()) { ... }
}
```

Exercise 13 · Repair this transaction.

```
public void record(Connection a, ResponseMission mission) throws SQLException {
    try (Connection b = database.openConnection()) {
        b.setAutoCommit(false);
        missions.insert(b, mission, Instant.now());
        resourceUsage.insert(b, mission.id(), ResourceType.PUMP, 1, Instant.now());
        b.commit();
    }
}
```

Answer. The method accepts a connection `a` and then ignores it, opening a second connection `b`. If the caller intended to enlist this work in an existing transaction on `a`, atomicity is lost — the work is committed independently. The repair is to use the connection that was passed in, and not to commit if the caller owns the transaction boundary.

Exercise 14 · Repair this Swing update.

```
engine.addSnapshotListener(snapshot -> {
    mapPanel.setSnapshot(snapshot);
    mapPanel.repaint();
});
```

Answer. The listener may be invoked from a worker thread, and `setSnapshot` and `repaint` both touch a Swing component. The repair wraps the whole body:

```
engine.addSnapshotListener(snapshot ->
        SwingUtilities.invokeLater(() -> {
            mapPanel.setSnapshot(snapshot);
            mapPanel.repaint();
        }));
```

Note that the wrapper goes around *both* statements, not just the repaint.

Exercise 15 · Repair this table update.

```
void setEntries(List<DatabaseHistoryEntry> entries) {
    this.entries = entries;
}
```

Answer. Two problems. The list is stored by reference, so a later change on the database thread could be visible to the event-dispatch thread mid-paint. And the table is never told the data changed, so it would keep displaying the old rows. Corrected:

```
void setEntries(List<DatabaseHistoryEntry> entries) {
    this.entries = List.copyOf(entries);
    fireTableDataChanged();
}
```

E.3 Tracing and data flow

Exercise 16 · Trace a malformed record.

A line arrives from the pipe reading `READ|abc|MIL-ST-2|MIL|12.4|3.1`. Describe every consequence.

Answer. `SensorReading.parse` splits the line into six fields and finds the `READ` marker, so the field-count check passes. `Long.parseLong("abc")` then throws `NumberFormatException`, which the method catches and rethrows as `IllegalArgumentException`. `SensorIntake` catches that, reports a warning through the warning consumer, and executes `continue`. The event never enters the buffer, no worker sees it, no activity is recorded, no archive record is written, no database row exists. The intake loop continues with the next line.

Exercise 17 · Trace a stale reading.

Zone MIL has already applied a reading with tick 214. A duplicate arrives with tick 214. Describe every consequence.

Answer. `BasinZone.apply` compares `reading.tick() <= lastReadingTick` and returns `false` without changing any field. The model produces no activity message, so the journal writes nothing for it; the engine's changed test fails, so no archive record and no database row are written. The interface is unchanged. The only trace is that the intake accepted and queued an event that the model declined — which is correct, because the intake's job is to validate the format, not the ordering.

Exercise 18 · Trace a dispatch that fails its reservation.

The operator requests a pump deployment for zone NOR, but all four pumps are already committed. Describe every consequence.

Answer. `ResourceAllocator.tryReserve` checks every requirement and finds `PUMP` unavailable, so it returns `false` having changed nothing — no resource was partially deducted. `BasinModel` does not create a mission, no snapshot changes, and the interface's resource board is unchanged. The engine reports a failure message, which the listener posts to the event-dispatch thread and the status bar displays. Nothing is written to the log or the database, because nothing happened. This is the all-or-nothing property in action.

Exercise 19 · Trace a dispatch whose third database write fails.

In `recordDispatch`, the mission insert succeeds, the resource rows succeed, and the audit event insert throws `SQLException`. Describe every consequence.

Answer. The catch block calls `connection.rollback()`, which discards the mission row and the resource-usage rows as well. The exception is then rethrown and caught by the outer catch, which reports it through the status listener. The database therefore contains none of the three writes, so it agrees with the in-memory model, which *did* record the dispatch. That divergence is a real limitation worth naming honestly: the interface and the log show the dispatch, the database does not, and the status message is what tells the operator. A production system would reconcile on the next start.

Exercise 20 · Trace a session restore with a truncated file.

The file `last-session.bws` is 2,000 bytes but was truncated mid-object. Describe every consequence.

Answer. The size gate passes, because 2,000 bytes is below the 4 MB bound. The class filter approves the classes it sees. The version check passes, because the header was written first and is intact. `readObject` then throws `EOFException` — a subclass of `IOException` — because the stream ends inside an object. The load fails, the exception propagates to the caller, and the engine reports it. The application starts with a fresh model rather than a corrupt one, which is the correct outcome: a partial session is worse than none.

Exercise 21 · Trace a history read when the database file is deleted between launches.

Describe the sequence from start-up to an empty history table.

> **Answer.** `DatabaseManager.isAvailable` calls `openConnection`, and because SQLite creates a missing database file on demand, the connection succeeds — so availability is actually true, and the initialiser recreates the four empty tables. `loadHistory` therefore returns an empty list from a real query rather than from the unavailable branch. The table shows no rows and the status reports zero entries. The distinction matters: an empty database and an unavailable database produce the same visible result through different code paths.

Exercise 22 · Trace a shutdown during a save.

The operator clicks Save and immediately closes the window. Describe what survives.

> **Answer.** Two save operations may be in flight. The explicit save writes to `name.bws.tmp`; the shutdown's automatic save writes to `last-session.bws` through its own temporary file. Both run on `basin-control`, which is single-threaded, so they are serialised rather than concurrent. `closeEngine` stops the feed, closes the pipe, joins intake, drains the archive queue and flushes the journal. If the process is killed before either save completes, the temporary files are incomplete but the previous `last-session.bws` is intact, because the move had not happened. That is the purpose of the atomic replace.

E.4 Architecture interpretation

Exercise 23 · Why is the archive written on a separate thread from the journal?

The journal is a character stream that the engine appends to directly, and it flushes on a policy. The archive is a byte stream with a header that must be written exactly once and typed records that must not interleave. Writing both on the worker thread would make every event wait for two disk operations. Putting the archive behind a single-threaded executor means the worker's cost is a queue insertion, while the header-once and no-interleaving guarantees are still upheld because there is only one archive writer.

Exercise 24 · Why does the engine publish snapshots rather than letting the interface read the model?

Because the model is mutated by three worker threads while the event-dispatch thread renders. Letting the interface read live objects would require the model's monitor to be held during painting, which would block every worker for the duration of a repaint and could deadlock if a listener ever called back into the model. An immutable snapshot removes the shared state entirely: the interface reads a value that cannot change, so no locking is needed and no torn read is possible.

Exercise 25 · Why does the database layer have both a service and repositories?

The service owns concerns that are not about any one table: the worker thread, the availability flag, the queue, the transaction boundaries and the failure reporting. The repositories own concerns that are about exactly one table: its SQL and its row mapping. Merging them would put transaction logic in every repository and SQL in the service; separating them means a repository can be tested against a plain connection with no threads involved, and the service's policy can change without touching any SQL.

Exercise 26 · Why is the sensor feed a `Thread` subclass while the intake is a `Runnable`?

The feed owns a lifecycle: it is started, it sleeps between intervals, it must be interruptible, and it holds a blocking pipe endpoint that must be closed. Extending `Thread` lets it carry a name, a priority and a state that diagnostics report. The intake is a task — a unit of work — with no lifecycle of its own; the engine creates a plain thread, names it, and hands it the task. This is the standard advice to prefer `Runnable` unless you must own the thread, applied where it genuinely differs.

Exercise 27 · What would break if the model were not thread-safe?

Three workers call `model.process` concurrently, and the event-dispatch thread reads snapshots. Without the monitor on `process` and the immutable snapshot boundary, two workers could apply readings to the same zone simultaneously, producing a lost update — one water level overwriting the other. The resource allocator could also over-allocate, because two dispatches could both pass the check loop before either decremented. Both failures are intermittent and load-dependent, which is exactly why the design avoids them structurally rather than by testing.

Exercise 28 · Why is the resource allocator's reservation all-or-nothing rather than incremental?

Because a mission requires several resources, and a partial reservation is a state the system cannot act on: the mission cannot start without all of them, and the deducted resources cannot be used by anything else. Incremental deduction would require a compensating rollback if a later requirement failed, and that rollback is itself a place where a bug could leave the inventory wrong. Checking everything before changing anything means no intermediate state exists to be cleaned up.

Exercise 29 · Why does the engine have three response workers but one archive worker and one database worker?

Because the three kinds of work have different constraints. Applying an event to the model is CPU-bound and independent per event, so concurrency helps. Appending to the archive must be serialised, because the header is written once and records must not interleave. Database work must be serialised for two reasons: SQLite permits one writer, and a single writer means no transaction can interfere with another, so no isolation policy is needed. The thread counts follow from the constraints, not from a guess about performance.

Exercise 30 · If you replaced SQLite with PostgreSQL, what would change?

Five things, and none of them is in the domain or the interface. The Maven dependency changes from `org.xerial:sqlite-jdbc` to the PostgreSQL driver. The URL changes from `jdbc:sqlite:path` to `jdbc:postgresql://host:port/database` with credentials. The DDL changes, because `AUTOINCREMENT` is SQLite syntax and the corresponding PostgreSQL type is different. Connections would need pooling, because a server round trip is far more expensive than opening a local file. And isolation would become a real concern, because a server database allows concurrent writers — so the single-writer guarantee the current design relies on would need replacing with explicit transaction handling. Everything else — the repositories' SQL shape, the prepared statements, the row mapping, the service, the interface — is unchanged, which is the payoff of programming to the JDBC interfaces.

REFERENCE

High-Yield Revision, Glossary and Reference

> The material to read the night before an examination: essential definitions, the classes and signatures worth memorising, the mistakes that cost marks, a glossary, and the source coverage matrix.

G.1 Twenty-five definitions to know exactly

Table G.1 — *The definitions worth being able to state word for word.*

Term	Definition
Stream	An ordered, one-directional sequence of data with a source or destination, consumed or produced incrementally.
Byte stream	A stream carrying 8-bit units with no interpretation applied.
Character stream	A stream carrying 16-bit characters converted through a charset.
Charset	The reversible mapping between characters and bytes, such as UTF-8.
Filter stream	A stream that wraps another stream and adds one capability.
Buffering	An intermediate array that turns many small operations into few large ones.
Serialization	Converting an object graph into a byte sequence including class descriptors.
Deserialization	Reconstructing an object graph from such a byte sequence, without invoking constructors.
`transient`	A modifier excluding a field from serialization.
`serialVersionUID`	The explicit version identity of a serializable class.
Pipe	A matched writer/reader pair with a bounded buffer, used between two threads.
Back-pressure	A bounded channel making a fast producer wait rather than buffer without limit.
Monitor	The lock associated with an object, entered by `synchronized`.
Condition wait	A `wait()` inside a loop that re-tests a predicate while holding a monitor.
Producer–consumer	

Term	Definition
	A pattern in which one thread produces items into a shared channel and another consumes them.
Database table	A named collection of rows sharing the same columns and types.
Primary key	The column or columns uniquely identifying each row.
Index	An auxiliary structure that speeds up lookups on particular columns.
JDBC	The standard Java API for relational database access, comprising interfaces plus a driver mechanism.
JDBC driver	The vendor-supplied implementation that translates JDBC calls into a database's protocol.
PreparedStatement	A pre-parsed SQL statement with numbered placeholders filled by typed setters.
ResultSet	A cursor over the rows a query returned, positioned before the first row initially.
Transaction	A group of database operations that all take effect or none do.
ACID	Atomicity, consistency, isolation and durability — the four transaction guarantees.
Event-dispatch thread	The single thread on which Swing performs event handling and painting.

G.2 Classes and method signatures worth memorising

Table G.2 — *The API surface most likely to be asked about.*

Class	Key methods	Notes
`InputStream`	`read()`, `read(byte[])`, `close()`	`read()` returns `int`; -1 means end
`OutputStream`	`write(int)`, `write(byte[])`, `flush()`, `close()`	Abstract
`Reader`	`read()`, `read(char[])`, `close()`	No `readLine()` here
`Writer`	`write(int)`, `write(String)`, `flush()`, `close()`	Abstract
`BufferedReader`	`readLine()`	Returns `null` at end
`BufferedWriter`	`newLine()`	Platform line separator
`DataOutputStream`	`writeInt` (4), `writeLong` (8), `writeDouble` (8), `writeUTF` (2+n)	Big-endian; the order *is* the schema
`ObjectOutputStream`	`writeObject(Object)`	Requires `Serializable`
`ObjectInputStream`	`readObject()`, `setObjectInputFilter(...)`	Constructors are not called

Class	Key methods	Notes
`Files`	`newBufferedReader`, `newBufferedWriter`, `newInputStream`, `newOutputStream`, `size`, `move`, `createDirectories`	Prefer these to the classic classes
`DriverManager`	`getConnection(String url)`	Selects a registered driver
`Connection`	`prepareStatement`, `createStatement`, `setAutoCommit`, `commit`, `rollback`, `close`	A transaction belongs to a connection
`PreparedStatement`	`setInt`, `setLong`, `setDouble`, `setString`, `executeQuery`, `executeUpdate`	Placeholders are 1-based
`ResultSet`	`next()`, `getString`, `getInt`, `getDouble`, `close`	Cursor starts before the first row
`SwingUtilities`	`invokeLater(Runnable)`	The only safe way to touch Swing from another thread
`CompletableFuture`	`supplyAsync`, `whenComplete`, `completedFuture`	The callback runs on the completing thread

G.3 Ten mistakes that cost marks

1. **Concatenating values into SQL.** Use `PreparedStatement` with placeholders. Always.
2. **Assuming deserialization validates.** It bypasses constructors, so the rules you wrote in them do not run. Validate the restored state explicitly.
3. **Forgetting the append flag.** Opening a file for output truncates it. A journal that should grow will silently restart.
4. **Omitting the charset.** The platform default differs between machines, so a file written on one may be unreadable on another.
5. **Mixing up the end-of-stream signals.** `read()` returns -1; `readLine()` returns `null`.
6. **Using `wait()` in an `if`.** It must be a loop, because a woken thread is not guaranteed its condition holds.
7. **Confusing `wait()` with `sleep()`.** `wait()` releases the monitor; `sleep()` does not.
8. **Returning a `ResultSet` from a method that closes its connection.** Map rows into your own objects before leaving the block.
9. **Touching Swing from a background thread.** Wrap the whole component-accessing body in `invokeLater`.
10. **Spreading several statements across different connections and calling it a transaction.** A transaction belongs to one connection.

G.4 Glossary

Table G.3 — *Glossary.*

Term	Meaning
ACID	Atomicity, consistency, isolation, durability — the four transaction guarantees.

Term	Meaning
Aggregate root	The single object that owns a group of related state and enforces its invariants; here, `BasinModel`.
Annotation	Metadata attached to a declaration; its retention policy determines whether it survives to runtime.
Append mode	Opening an output stream so new writes follow existing content.
Atomic move	Replacing a file in a single step, so no partial state is ever observable.
Audit trail	An ordered record of what happened, for later inspection.
Autocommit	The JDBC default in which each statement is its own transaction.
Back-pressure	A bounded channel making a fast producer wait rather than buffer without limit.
Bean (JavaBean)	A POJO following the conventions a framework needs: no-argument constructor, private fields, public accessors.
Bounded buffer	A queue with a maximum size that makes producers wait when full.
Build lifecycle	Maven's ordered sequence of phases, where a later phase runs every earlier one.
Byte stream	A stream of 8-bit units with no interpretation applied.
Callback	A function supplied to be invoked later, typically on another thread.
Charset	The reversible mapping between characters and bytes.
Commit	Making a transaction's changes permanent and visible.
Condition wait	A `wait()` inside a loop that re-tests a predicate while holding a monitor.
Constructor injection	Supplying an object's dependencies as constructor arguments.
Coordinate (Maven)	The groupId, artifactId and version triple identifying a library.
DataAccessException	Spring's unchecked exception hierarchy for data access failures.
DataSource	An abstraction representing a source of database connections.
DDL	Data definition language: statements such as `CREATE TABLE`.
Decorator pattern	Adding behaviour by wrapping an object in another of the same interface.

Term	Meaning
Deserialization	Reconstructing an object graph from bytes, without invoking constructors.
Durability	The property that a committed change survives a failure.
EDT (event-dispatch thread)	The single thread on which Swing handles events and paints.
Encapsulation	Keeping state private so behaviour is the only route in.
Enum	A type whose values come from a fixed, closed set; may carry data.
Fan-out	One decision causing several independent downstream operations.
Filter stream	A stream that wraps another stream and adds one capability.
Fire-and-forget	A call that queues work and returns immediately, reporting failure out of band.
Flush	Forcing buffered data out to the destination without closing.
Idempotent	Safe to perform more than once with the same result.
Index (database)	An auxiliary structure speeding up lookups on particular columns.
Inversion of control	A component declaring interest and being called, rather than polling.
Isolation	The property that concurrent transactions do not see each other's partial work.
JDBC	The standard Java API for relational database access.
JdbcTemplate	A Spring class that executes SQL and manages JDBC resources, leaving SQL and row mapping to the caller.
Layout manager	The object deciding where a container's children are placed.
Magic number	A constant at the start of a file identifying its format.
Metadata (file)	Information about a file — name, size, modification time — as opposed to its contents.
Monitor	The lock associated with an object, entered by `synchronized`.
Namespace (XML)	A URI-qualified vocabulary that prevents name collisions in a document.
`ObjectInputFilter`	A hook approving or rejecting each class a deserialization stream tries to construct.
Pattern matching	Testing and binding an object's type in one expression.

Term	Meaning
Pipe	A matched writer/reader pair with a bounded buffer, used between two threads.
Placeholder	The `?` in a prepared statement, replaced by a bound value.
POJO	A Plain Old Java Object: any ordinary class with no framework convention.
Primary key	The column or columns uniquely identifying each row.
Producer–consumer	A pattern in which one thread produces into a shared channel and another consumes.
Record	A class declaring immutable data, with a compiler-generated constructor and accessors.
Reflection	Inspecting and using classes and members at runtime.
Repository	A class owning the SQL for one table and mapping rows to and from domain objects.
ResultSet cursor	The position marker over a query's rows, starting before the first.
Rollback	Discarding a transaction's changes as though they never happened.
RowMapper	A callback converting one result-set row into an object.
Serialization	Converting an object graph into bytes including class descriptors.
`serialVersionUID`	The explicit version identity of a serializable class.
Service provider file	`META-INF/services/java.sql.Driver`, which lets a driver register itself automatically.
Snapshot (immutable)	A value object copied out of the model so another thread can read it safely.
SQL injection	An attack in which user input becomes part of a statement's structure.
SQLState	The standard five-character error code carried by a `SQLException`.
Stale guard	A check that rejects input older than the state already applied.
Stream	An ordered, one-directional sequence of data, consumed incrementally.
Thin driver	A pure-Java type 4 JDBC driver needing no native code.
Thread confinement	The rule that an object may only be accessed from one thread.
Transaction	A group of database operations that all take effect or none do.

Term	Meaning
Transient	A modifier excluding a field from serialization.
try-with-resources	A statement that closes every declared resource automatically, in reverse order.
Uber jar	A jar containing a project's classes plus all its dependencies' classes.
Well-formed (XML)	Following XML's syntax rules, as distinct from conforming to a schema.
XSD	XML Schema Definition, a schema language written in XML itself.

G.5 Key workflows on one page

```
READ A TEXT FILE
  Path -> Files.newBufferedReader(path, UTF_8) -> loop on readLine() != null -> close (try-with-resources)

WRITE A TEXT FILE
  Path -> createDirectories(parent) -> Files.newBufferedWriter(path, UTF_8[, CREATE, APPEND])
       -> write -> flush -> close

READ TYPED RECORDS
  Files.newInputStream -> BufferedInputStream -> DataInputStream
  -> readLong, readUTF, readDouble, ... until EOFException

WRITE TYPED RECORDS
  Files.newOutputStream -> BufferedOutputStream -> DataOutputStream
  -> writeLong, writeUTF, writeDouble, ... -> flush -> close

SERIALIZE AN OBJECT GRAPH
  ObjectOutputStream( BufferedOutputStream( Files.newOutputStream(temp) )
  -> writeInt(version) -> writeObject(snapshot) -> flush -> close -> atomic move into place

DESERIALIZE SAFELY
  check size -> ObjectInputStream -> setObjectInputFilter -> readInt(version)
  -> readObject -> cast -> validate

PIPE BETWEEN THREADS
  PipedWriter w = new PipedWriter();  PipedReader r = new PipedReader(w, 8192);
  feed thread writes + flush; intake thread reads readLine() until null; writer.close() ends the loop

BOUNDED QUEUE
  producer: synchronized put -> while (full && !closed) wait(); add; notifyAll();
  consumer: synchronized take -> while (empty && !closed) wait(); remove; notifyAll();

SHUTDOWN ORDER
  stop producer -> close writer -> join consumer -> close queue -> drain archive -> flush journal -> save

JDBC QUERY
  DriverManager.getConnection(url) -> prepareStatement(sql) -> setX(1..n)
  -> executeQuery() -> while (rows.next()) map row -> close rows, statement, connection

JDBC UPDATE
  ... -> executeUpdate() -> inspect the returned row count -> close

JDBC TRANSACTION
  setAutoCommit(false) -> statement 1 -> statement 2 -> commit()
  catch: rollback()   finally: setAutoCommit(true)
```

```
SWING OFF-THREAD UPDATE
  worker thread -> SwingUtilities.invokeLater(() -> { update components; repaint(); })

SWING TABLE UPDATE
  model.setEntries(List.copyOf(rows)) -> fireTableDataChanged()
```

the twelve workflows in the book, as checklists

G.6 Source coverage matrix

Every topic area from the source lecture material, mapped to where it is taught and whether it is verified against the project's code.

Table G.4 — *Source coverage matrix.*

Source topic	Book chapter	Project evidence	Status
Java I/O package overview	4	Every stream in the `io` package	Verified
InputStream / OutputStream	4	Basis of `SensorArchive`	Verified
FileInputStream / FileOutputStream	5	Contrasted with `Files.newXxx`	Verified (modern form used)
Reader / Writer	4	`OperationalJournal`, `ReportWriter`	Verified
FileReader / FileWriter	5	Contrasted with the modern API	Verified (modern form used)
Buffered streams	6	Every file stream is buffered	Verified
DataInputStream / DataOutputStream	6	`SensorArchive`	Verified
ByteArrayInputStream / ByteArrayOutputStream	7	Illustrative example	Taught; not used
PipedInputStream / PipedOutputStream	7	Character variants used instead	Related
PipedReader / PipedWriter	7	`SensorFeed` to `SensorIntake`	Verified
File handling and metadata	5, 9	`AppPaths`, `SaveCatalog`	Verified
Exceptions and resource management	4, 5	try-with-resources, suppressed exceptions	Verified
Object serialization	8	`SnapshotStore`	Verified
Threading and the producer–consumer model	3, 7	Feed, intake, buffer, workers	Verified
JDBC fundamentals	10, 11	`DatabaseManager`	Verified
JDBC architecture and drivers	11	SQLite type 4 driver	Verified

Source topic	Book chapter	Project evidence	Status
Connection	12	`DatabaseManager.openConnection`	Verified
Statement	12	Schema DDL in `DatabaseInitializer`	Verified
PreparedStatement	12	Every repository method	Verified
CallableStatement	12	No stored procedures exist	Taught; not used
ResultSet and CRUD	12	All four repositories	Verified
SQL parameters	12	Nine-placeholder insert	Verified
Transactions	13	`recordDispatch`	Verified
SQLException and cleanup	12, 14	`enqueue` catches and reports	Verified
Spring JdbcTemplate	19	Not used	Taught as alternative
DriverManagerDataSource	19	Not used; `DriverManager` used directly	Taught as alternative
Maven and pom.xml	17	`pom.xml` with one dependency	Verified
Dependency management	17	sqlite-jdbc version property	Verified
XML and namespaces	18	`pom.xml` header only	Verified (build file only)
XML Schema	18	Not used	Taught; not used
Annotations	18	`@Override`, `@FunctionalInterface`, `@SuppressWarnings`	Verified
POJO versus JavaBean	2, 18	Records and DTOs; no beans	Verified
Swing components and layouts	15	`MainFrame`, `BasinMapPanel`	Verified
Event handling	15	Action listeners throughout	Verified
The login application	15	Walked through as lecture material	Lecture-note style
Swing and JDBC together	16	`DatabaseHistoryPanel`	Verified

G.7 The final visual summary

One diagram to hold in mind: the whole system, the concepts that make each part work, and the two boundaries where the technologies meet.

Diagram 29 — Final visual summary of the complete Java system

The whole book on one page: from an incoming byte to a restored session.

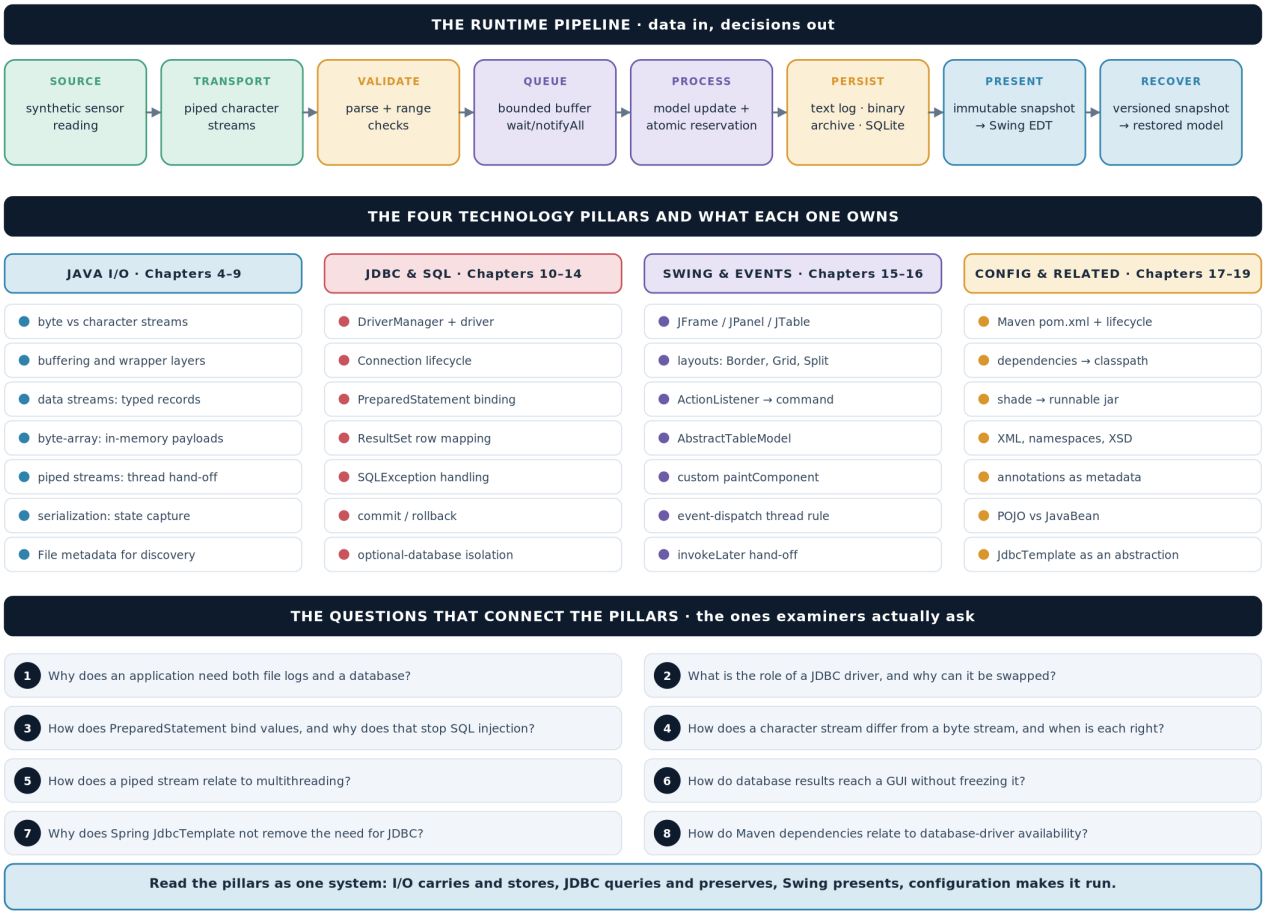


Diagram 29 — The complete Java system, revisited. The architecture from Diagram 1 with the syllabus concepts annotated onto each layer, plus the two interaction boundaries and the build that puts the driver on the classpath.

THE LAST WORD

- **Architecture first.** Name the layers, the threads and the two boundaries before any API.
- **Three properties describe every stream.** Direction, unit, and source or destination.
- **Four defences protect deserialization.** Size, filter, version, validation.
- **One rule governs Swing.** Only the event-dispatch thread touches components.
- **One connection owns a transaction.** Everything inside it must share that connection.
- **Two persistence mechanisms, two questions.** A file records; a database answers.
- **Explain connections, not definitions.** That is what this book has been for.